zCFDguide Documentation

Release v2025.11.13331-307-gef05f

Zenotech Ltd

Sep 11, 2026

CONTENTS

CHAPTER

ONE

VERSION: V2025.11.13331-307-GEF05F

1.1 Overview

zCFD is a GPU-accelerated high-performance computational fluid dynamics (CFD) solver.

zCFD is designed to make highly effective use of modern computer architectures. It is structured as a high performance core, executed via a flexible Python based wrapper. zCFD will run on unstructured meshes from a variety of sources, and will export data in VTK HDF format (.vtkhdf) for efficient post-processing with tools such as ParaView. zCFD offers an easier route to coupling multi-disciplinary capabilities and integrating third-party data sources.

Features include both compressible and incompressible finite-volume formulations, overset capability and fluid-structure interaction (FSI) running on both CPUs and GPUs. zCFD is capable of performing both steady-state and time accurate simulations, with a range of time marching schemes. The numerical schemes available have been selected for accuracy and robustness. The code provides automatic scaling of turbulent wall functions; automatic determination of inflow / outflow conditions and pre-conditioning to allow for low and high Mach number flow.

zCFD makes use of the MPI protocol to manage execution over a set of compute nodes. The execution process is completely parallel from the start to finish including all the input/output functionality. To support this scalable implementation the mesh input is provided in hdf5 format which is portable and self describing. Filters are provided to convert the most frequently used formats to this internal format.

zCFD uses a single control dictionary to specify all the parameters. This is a Python file containing a python dictionary with certain mandatory keys. The use of Python for the control dictionary enables the user to add custom functions that populate the dictionary at runtime.

The zCFD distribution also ships with zm3 or zMesh-Cubed, our cartesian cut cell immersed boundary mesh generator. This allows users to rapidly generate meshes for CFD from STL files, with minimal user cleanup effort required.

1.2 Installation

1.2.1 System Requirements

zCFD is supported on the following Linux distributions:

- RHEL 7+
- CentOS 7+
- Ubuntu 20.04+
- Windows 10/11 via WSL2

By default zCFD is compiled for x86_64 but is available for ARM and IBM Power on request.

GPU support

zCFD supports running on NVidia GPUs with a compute capability of 6.1 and above (See Compute Capability).

An AMD GPU (HIP) based version is available on request.

1.2.2 Linux / Unix

If you are running Linux/Unix zCFD is provided as a stand-alone installer containing all the packages required to get started. The installer is available for download from the zCFD website. Once the zCFD installer has been downloaded run the install script and follow the prompts.

```
$ bash zCFD-icx-sse-impi-<INSTALL_VERSION>-Linux-64bit.sh
```

Then copy the license file provided over email into the lic folder in the install directory.

```
$ cp zcfd.lic /<path to zCFD install>/lic/
```

Once zCFD has been installed the zCFD environment can be activated from a bash shell:

```
$ source /<path to zCFD install>/bin/activate
(zCFD)$
```

1.2.3 Windows

zCFD can be run locally on windows machines if Windows Subsystem for Linux 2 (WSL2) is installed. To install WSL2 simply follow the Microsoft instructions which can be found here.

The chosen Linux distribution shouldn't have an effect on how the code runs, but the Linux distributions listed above are supported.

In order to access your Linux filesystem on Windows it can be useful to add a quick access link in file explorer. To do this, in the address in file explorer enter

```
\\wsl$\
```

This will show you show you the root folders for your installed Linux distributions.

The installer is available for download from the zCFD website. For WSL, download the standard Linux distribution. Once the zCFD installer has been downloaded run the install script from within the WSL2 environment and follow the prompts.

```
$ bash zCFD-icx-sse-impi-<INSTALL_VERSION>-Linux-64bit.sh
```

Then copy the license file provided over email into the lic folder in the install directory.

```
$ cp zcfd.lic /<path to zCFD install>/lic/
```

Once zCFD has been installed the zCFD environment can be activated:

```
$ source /<path to zCFD install>/bin/activate
(zCFD)$
```

1.2.4 Licence

zCFD requires a license. If you do not already have a license, please contact zcfd@zenotech.com to obtain one. When zCFD starts it will look in the following locations for a licence file:

1. 'zcfd.lic' in the directory alongside the input files
2. 'zcfd.lic' in the lic directory of the zCFD installation. i.e. /<path to zCFD install>/lic/

3. RLM_LICENSE environment variable

The RLM_LICENSE environment variable can be defined as below.

```
export RLM_LICENSE=/path/to/license/file
```

or, where port is the port number in the license file, and host is the hostname in the license file

```
export RLM_LICENSE=port@server
```

In the case that the system running the simulation cannot access the internet a local license server will be required. See http://www.reprisesoftware.com/admin/software-licensing.php for details.

Licence Error codes

Below is a list of some of the error codes that you may encounter due to licensing errors. If the error code you receive is not listed, please see RLM's documentation a https://reprisesoftware.com/docs/isv/appendix/appendix-b-rlm-status-values.html or contact us.

Error Code	Description	Possible Remedy
-3	License has expired	Contact us to renew your license
-6	Requested version is not supported	Your license is for an older product version; contact us to upgrade
-8	checkout request for too many licenses	Reduce the number of ranks that you are running the solver on or stop any existing running processes.
-17	Error communicating with server	There was a connectivity issue connecting to the license server. Check that you have connectivity to the license server; check firewall settings and any endpoint security software you may be running
-21	No heartbeat	The license heartbeat was not received; check that the license server is still running and that have connectivity to it
-22	All licenses in use	All licenses are currently in use; stop any running processes to free up a license.
-37	License start date has not been reached	Your license has not yet become valid. Contact us if you believe this to be in error
-102	Can't read license data	Check the permissions on the license file on the server
-103	Network error	Check that you have connectivity to the license server; this may involve opening a port on your firewall or connecting to the internet
-104	Error writing to network	Check your connectivity to the license server; check firewall settings and any endpoint security software you may be running
-105	Error reading from network	Check your connectivity to the license server; check firewall settings and any endpoint security software you may be running
-106	Unexpected response from license server	Check that your license configuration is pointing to the correct server
-107	HELLO message for wrong server	Check that your license configuration is pointing to the correct server
-108	Error in private key	There is an issue with your server side license file - contact us to resolve this issue
-109	Error signing authorization	There is an issue with your server side license file - contact us to resolve this issue

continues on next page

Table 1.1 – continued from previous page

Error Code	Description	Possible Remedy
-110	Internal error	An error occurred in the license server - contact us if this error repeats itself
-111	Connection refused at server	Check your firewall settings and that you are pointing at the correct license server
-112	No server to connect to	Check that the server is running and that you are pointing at the correct license server
-113	Bad Communication handshake	Check that the server is running and that you are pointing at the correct license server
-114	Can't get ethernet address	Check that the user that you are running as has correct permissions to query the network card

1.3 Running zCFD

zCFD is primarily run from the command line, you can run on your local machine or submit it to a batch scheduler for parallel execution. The following sections give information on how to execute zCFD.

1.3.1 Running zCFD

zCFD is designed to exploit parallelism in the target computing hardware, making use of multiple processes (MPI), multiple threads per CPU process (OpenMP) and / or many threads per GPU. zCFD will handle most parallel functions automatically, so the user just needs to specify how much, and what type, of resource to use for a given run.

Table of Contents

Command Line Execution

zCFD includes an in-built command-line environment, which should be activated either when run directly or via a cluster management and job scheduling system, such as Slurm.

```
source /INSTALL_LOCATION/zCFD-version/bin/activate
```

This sets up the environment to enable execution of the specific version. When within the command-line environment, the command prompt will show a zCFD prefix:

```
(zCFD) >
```

To deactivate the command line environment, returning the environment to the previous state use:

```
deactivate
```

Your command prompt should return to its normal appearance.

To run zCFD on the command-line:

```
run_zcfd -n <num_tasks> -d <device_type> -p <mesh_name> -c <case_name>
```

where:

Parameter	Value	Description
num_tasks	Integer	The number of partitions (one per device socket - CPU or GPU)
device_type	CPU or GPU	Specifies whether or not to use GPU(s) if present
mesh_name	String	The name of the mesh file (<mesh>.h5)
case_name	String	The name of the control dictionary (<control_dict>.py)

For example, to run zCFD from the command-line environment on a desktop computer with a single 12-core CPU processor and no GPU:

```
run_zcfd -n 1 -d cpu -p <mesh_name> -c <case_name>
```

By default, zCFD will use all 12 cores via OpenMP unless a specific number is required. To run the above case using only 6 cores (there will be one thread per CPU core):

```
run_zcfd -n 1 -d cpu -o 6 -p <mesh_name> -c <case_name>
```

or

```
export OMP_NUM_THREADS=6; run_zcfd -n 1 -d cpu -p <mesh_name> -c <case_name>
```

To run zCFD on a desktop computer with a single CPU and two GPUs:

```
run_zcfd -n 2 -d gpu -p <mesh_name> -c <case_name>
```

Input Validation

zCFD provides an input validation script which can be used to check the solver control dictionary before run time reducing the likelihood of a job spending a long time queuing only to fail at start up.

The script can be executed from the zCFD environment as follows:

```
validate_input <case_name> [-m <mesh_name>]
```

If the -m option is given with a mesh file the script will check whether any zones specified as lists to boundary conditions, transforms or reports in the input exist in the mesh.

Multiple Meshes and Overset Cases

A case with more than one mesh (see *overset*) is run from a single control dictionary: each mesh is a named entry under the top-level `model` key, run together with the same `run_zcfd -c <case_name>` invocation used for a single-mesh case — there is no separate `-p`/mesh argument or override file for these cases.

An example control dictionary for a background domain with a single rotating turbine:

```
parameters = {
    "config_version": 2,
    "solver": {....},
    "model": {
        "background": {"mesh": "background.h5", ....},
        "turbine": {"mesh": "turbine.h5", ....},
    },
}
```

```
run_zcfd -n 2 -d gpu -c <case_name>
```

Batch Queue Submission

To run zCFD on a compute cluster with a queuing system (such as Slurm), the command-line environment activation is included within the submission script:

Example Slurm submission script "run_zcfd.sub":

```
#!/bin/bash
#SBATCH -J account_name
#SBATCH --output zcfd.out
#SBATCH --nodes 2
#SBATCH --ntasks 4
#SBATCH --exclusive
#SBATCH --time=10:00:00
#SBATCH --gres=gpu:2
#SBATCH --cpus-per-task=16
source /INSTALL_LOCATION/zCFD-version/bin/activate
run_zcfd -n 4 -d gpu -p <mesh_name> -c <case_name>
```

In this system we have 2 nodes, each with 2 GPUs. We allocate one task per GPU, giving a total of 4 tasks. Note that num_tasks, in this case 4, on the last line should match Slurm "ntasks" on line 5. The case is run in "exclusive" mode which means that zCFD has uncontested use of the devices. In this case we have a 64-core CPU, giving 16 CPU cores for each of the 4 tasks. The "gres" line tells zCFD that there are 2 GPUs available on each node.

The submission script would be submitted from the command-line:

```
sbatch run_zcfd.sub
```

The job can then be managed using the standard Slurm commands.

> **Note**
>
> Users unfamiliar with the batch submission parameters in a specific system should consult the system administrator.

1.3.2 Parallel Guide

Concepts

zCFD uses Intel MPI (Message Passing Interface) to enable running over multiple machines or multiple GPUs, either in one machine or across multiple. A paid license is required for this functionality. This is achieved by decomposing the domain into parallel regions that are distributed to each MPI rank. MPI can leverage high bandwidth network interconnects such as infiniband or AWS' Elastic Fabric adaptor,

There are two main strategies for running the code in parallel:

Hybrid MPI/OpenMP runs one MPI process per socket, reducing the number of MPI ranks at larger core counts. Running in hybrid mode assigns multiple cores to each MPI rank, allowing it to reduce the communication overheads associated with running in parallel.

Full MPI runs one MPI process per core and is typically the best strategy for low core count runs or GPU based runs. For GPU runs, you can run 1 MPI rank per GPU available on the node, or multiple MPI ranks per GPU. When running multiple ranks per GPU, the ranks will share the GPU resources. This can be useful for smaller problem sizes, testing and development, or workloads with low GPU utilization.

To run in parallel you add the -n option to the run_zcfd script. The provided argument is the number of MPI processes to launch.

Hybrid MPI/OpenMP

This is the default mode for zCFD. It will auto detect the number of sockets on each node and set the number of OpenMP threads appropriately. The –tpn option to run_zcfd can be used to set the number of MPI processes to launch on each node. The number of OpenMP threads will be set automatically by dividing the total number of cores between all processes.

For example, if you had two hosts with 2 x 12 core CPUs per host and you wanted to run Hybrid MPI/OpenMP you would request 1 MPI rank per CPU socket and zCFD will automatically set the number of OpenMP threads per rank to match the number of cores per socket (in our case 12).

```
run_zcfd -m <mesh_name> -c <case_name> -n 4 -tpn 2
```

The number of OpenMP threads assigned to each rank can be overridden by setting **OMP_NUM_THREADS** in the calling environment.

Full MPI

This is achieved by running zCFD with **OMP_NUM_THREADS** set to 1 and passing -n <tasks> where tasks is the number of MPI ranks.

For example, if you had two hosts with 2 x 12 core CPUs per host and wanted to run full MPI with 1 MPI rank per core you could do the following:

```
export OMP_NUM_THREADS=1
run_zcfd -p <mesh_name> -c <case_name> -n 48 -tpn 24
```

GPU Parallel Execution

zCFD automatically assigns GPUs to MPI ranks when running on GPU-enabled nodes.

Single Rank per GPU: This is the default and recommended configuration for most workloads. Each MPI rank gets exclusive access to one GPU.

```
# For a node with 4 GPUs, run 4 ranks (1 per GPU)
run_zcfd -p <mesh_name> -c <case_name> -n 4
```

Multiple Ranks per GPU (Experimental): To enable multiple MPI ranks sharing GPUs, set the `ZCFD_DEV_OVERSUBSCRIBE_GPU` environment variable. This is an experimental feature primarily intended for debugging and testing. When enabled, ranks will be assigned to GPUs using modulo arithmetic (rank % num_gpus).

> **Warning**
>
> GPU oversubscription is an experimental feature. It may result in reduced performance and is primarily intended for debugging purposes with limited GPU resources.

This can be useful for:

- Testing and development with limited GPU resources
- Debugging with smaller problem sizes
- Workloads with low GPU compute utilization

```
# For a node with 4 GPUs, run 8 ranks (2 per GPU) - experimental
export ZCFD_DEV_OVERSUBSCRIBE_GPU=1
run_zcfd -p <mesh_name> -c <case_name> -n 8
```

Example assignments with oversubscription enabled:

- 4 GPUs, 4 ranks: GPU assignment [0,1,2,3]

- 4 GPUs, 8 ranks: GPU assignment [0,1,2,3,0,1,2,3]
- 4 GPUs, 6 ranks: GPU assignment [0,1,2,3,0,1]

Running across multiple machines with a cluster scheduler

There are many cluster scheduling systems but commonly used ones are Slurm, PBS Pro or GridEngine. Please refer to either the cluster scheduling software or your local HPC systems' documentation for how to submit jobs to the appropriate nodes on your system as configuration varies between machines.

Intel MPI has tight integration to most commonly used scheduling systems and so will typically autodetect the nodes to run on.

Running across multiple machines without a cluster scheduler

If your cluster does not have a scheduling system then **I_MPI_HYDRA_HOST_FILE** can be set in the environment to point at a file containing the list of hostnames that zCFD will launch on. For example:

```
export I_MPI_HYDRA_HOST_FILE=~/hosts.file
```

Note

For this to run you need to be able to ssh without a password to each of the machines listed.

An example contents of a hosts file is shown below:

```
Compute01:2
compute02:2
Compute03:1
```

This file specifies three hosts, compute01, compute02, compute03 where 2 ranks will be launched on each of compute01 and compute02 and 1 rank will be launched on compute03.

Alternatively this could be specified as which would launch processes round robin through the list of machines listed in the file.

```
Compute01
compute02
Compute03
```

Libfabric provider selection

zCFD will attempt to autodetect the correct provider to use for MPI communication. It tries the following providers in order:

```
efa
psm2
mlx
verbs
tcp
sockets
```

If FI_PROVIDER is set then the autodetection is skipped.

Using cluster provided libfabric

By default zCFD will use the version of libfabric that ships with Intel MPI. For some advanced use cases you may wish to make use of preinstalled libfabric. This may be if you have a custom network driver or a specific configuration on your cluster. To enable this then set the **I_MPI_OFI_LIBRARY_INTERNAL** environment variable to 0

```
export I_MPI_OFI_LIBRARY_INTERNAL=0
```

Pre launch script

On some systems the defaults setup by zcfd may not be optimal and so a custom script to modify the environment can be injected by setting the environment variable **ZCFD_PRE_LAUNCH** to the path to a script. This script will be sourced from within a bash environment just before the solver is launched.

An example use case for this is to bind a GPU to a specific network interface on a multi-homed network system.

OpenMP affinity

By default zCFD binds its OpenMP threads to cores using the **OMP_PROC_BIND** and **OMP_PLACES** environment variables.

Other MPI implementations

On platforms where Intel MPI doesn't work then we can build zCFD against a specific implementation of MPI. This would be a custom deployment for your facility so get in touch with us if you need this.

Running at large scale

As zCFD is partly written in python, the install consists of a large quantity of small files, which on a large scale cluster can have a negative impact on the cluster filesystem during startup. In order to mitigate this it is possible to stage the code into either local storage or memory on the compute nodes using MPI to broadcast the data.

Contact us if you want to run in this mode.

1.3.3 Using Jupyter

We also recommend the use of Jupyter Notebooks for post-processing zCFD runs. This is one of the easiest ways to use Python, via an interactive notebook on your local computer. The notebook is a locally hosted server that you can access via a web-browser. Jupyter is included in the zCFD distribution and notebooks for visualising the residual data will automatically be created when you run a zCFD case.

To automatically start up notebook server on your computer within the zCFD environment run:

```
start_lab
```

The output of the command will include the URL to Jupyter is running on, simply copy and paste this into your browser to connect to Jupyter. To stop Jupyter use *Ctrl* + *c* in the terminal where Jupyter is running.

1.3.4 zCFD Environment Variables

The following table gives some of the environment variables that can be set before running zCFD. These are generally for advanced users can can impact the way zCFD runs on you system.

Variable	Default	Description
ZCFD_GPUDIREC	unset	Set if you are using NVIDIA GPU direct (custom builds only) with a CUDA aware mpi implementation.
ZCFD_PRE_LAUN	unset	This option allows a script to be run just before the code is launched so that the user can override any environment that has been automatically setup.
CUDA_VISIBLE_D	unset	This option restricts the list of GPU devices that zCFD will be able to use. Typically this can be used to remove an incompatible GPU or the display GPU from the list of candidate GPUs.
ZCFD_DISABLE_P	unset	When set this causes the GPU memory allocated to be page-able and not pinned memory. This should always be used on Windows in WSL as CUDA inside WSL has a fixed low threshold for the maximum amount of allowed pinned memory.
ZCFD_NCCL	unset	Set to 1 to make use of the NVIDIA NCCL library during parallel exchanges.
PMPI_CUDA	unset	Set if you are running with platform MPI (custom builds only) to enable GPU aware MPI memory allocations
MPICH_GPU_SUF	unset	Set to 1 if you are running with Cray MPI (custom builds only) to enable GPU aware MPI memory allocations
ZCFD_DISPATCH_	unset	Makes all kernel launches synchronous.
ZCFD_DEV_OVER	unset	**Experimental feature.** When set, allows multiple MPI ranks to share the same GPU. Without this variable set, the default behavior assigns one GPU per MPI rank. This is primarily intended for debugging and testing with limited GPU resources.

1.4 Getting Started

The getting started guide is here to teach the fundamentals of using zCFD, starting with a simple case and building up the complexity of the simulation. These should provide you a good starting point for setting up your own simulations. All of the input files required for the guide are available on line.

1.4.1 Tutorial 1: Simple Aerofoil

Table of Contents

What is in this tutorial?

The NACA0012 is a symmetrical aerofoil which is commonly used for code validation. In this configuration it is a quick and simple simulation to run. By downloading this zip file and following the instructions below, you should learn to run, post-process and visualise a simple zCFD simulation. This is a small simulation which most users should be able to run on a laptop in ~5 minutes.

What you need?

To run the tutorial you will need an installation of zCFD and a valid licence (the free licence is sufficient for this tutorial). For instructions on installing zCFD please see *here*.

The tutorial assumes that you are running all commands using a bash shell on Linux.

Step 1. Download the files

The tutorial zip file can be downloaded here. The zip file contains a mesh file (*naca0012.h5*) and a zCFD control file (*naca0012.py*).

This tutorial will assume you unzip the file in a directory called "zcfd_tutorial_1" in your users home directory.

```
cd ~
mkdir zcfd_tutorial_1
wget https://zcfd.zenotech.com/tutorials/1/naca0012.zip
unzip naca0012.zip
```

Step 2. Review Control file

naca0012.py is the zCFD control file. When we run zCFD, zCFD reads the control file and uses the Python dictionary `parameters` to set the solver settings. Control files declare `"config_version": 2`, and nest their settings under a top-level `solver` key (numerics, physics and output settings shared by the whole simulation) and a `model` key (one entry per mesh, holding that mesh's boundary conditions). A brief overview of the parameters dictionary for the simulation is given below; for a more detailed description of the control file structure, see the *reference guide*.

```
parameters = {
    "config_version": 2,
    "solver": {
        "equations": {
            "type": "rans",
            "turbulence": {"model": "sst"},
        },
        "numerical scheme": {
```

```
            "order": "euler_second",
            "inviscid flux scheme": "roe",
        },
        "fluid properties": {
            "material": "air",
            "gamma": 1.4,
            "gas constant": 287.0,
            "sutherlands const": 110.4,
            "prandtl no": 0.72,
            "turbulent prandtl no": 0.9,
        },
        "reference conditions": {
            "IC_1": {
                "temperature": 300,
                "pressure": 101325.0,
                "v": {"vector": [1.0, 0.0, 0.0], "mach": 0.15},
                "viscosity": 1.02145e-05,
                "turbulence intensity": 5.2e-2,
                "eddy viscosity ratio": 1.0,
            },
        },
        "solver settings": {"type": "compressible"},
        "initialisation": {"initial conditions": "IC_1", "restart": False},
        "time settings": {"type": "steady"},
        "convergence control": {
            "scheme": {"name": "implicit euler"},
            "cfl": {
                "cfl": 50,
                "cfl ramp": [{"type": "exponential", "factor": 1.1, "initial": 1.0}],
            },
            "cycles": 1500,
        },
        "output settings": {
            "report": {
                "frequency": 10,
                "forces": {
                    "FR_1": {"name": "wall", "zones": [4]},
                },
            },
            "solution": {
                "surface variables": ["V","p","mach","cp","cf","pressureforce",
↪"frictionforce"],
                "volume variables":  ["V", "p", "T", "rho", "mach", "cp", "eddy"],
            },
        },
    },
    "model": {
        "naca0012": {
            "mesh": "naca0012.h5",
            "boundary conditions": {
                "BC_1": {
                    "zones": [1],
                    "type": "farfield",
                    "condition": "IC_1",
                    "kind": "riemann",
                },
                "BC_2": {
                    "zones": [4],
```

```
                    "type": "wall",
                    "kind": "no slip",
                },
                "BC_3": {
                    "zones": [2, 3],
                    "type": "symmetry",
                },
            },
        },
    },
}
```

Time marching

The physical time layout and the pacing of the iterative solve are described by two sibling keys under *solver*: *time_settings*, which describes the physical time layout (steady or unsteady), and *convergence_control*, which paces the iterative solve. Keeping them separate means each one only has to say one thing: *time_settings* says *what kind of run this is*, and *convergence_control* says *how quickly to march towards convergence*.

`"time settings": {"type": "steady"}` declares a steady simulation outright, as an explicit, first-class declaration. Steady simulations assume that the flow we are simulating does not vary with time. Most real flows are not truly 'steady', but steady simulations tend to be much easier and cheaper to run than unsteady simulations. For an attached, streamlined flow like this one using a steady simulation is reasonable when time-resolved data is not required.

CFD simulations work by iteration - we start with a very rough estimate of the flow $f(x,0)$, then during the first 'iteration cycle' the solver uses $f(x,0)$ to create a better approximation of the flow, $f(x,1)$. This is repeated many times - we use $f(x,1)$ to calculate $f(x,2)$, $f(x,2)$ to calculate $f(x,3)$ and so on, until eventually we reach an estimate of the flow we are happy with.

The flow will generally change very quickly in the first few iterations, and then converge asymptotically on the 'right' answer. The *cycles* parameter, part of *convergence_control*, states how many iterations should be used in the simulation - the higher it is the more accurate the solution is likely to be, but given the convergence is asymptotic setting it too high will lead to lots of computational effort being used for very little change in the solution towards the end.

The *scheme* defines the numerical scheme used by the solver to get from $f(x,n)$ to $f(x,n+1)$. The *implicit euler* scheme is a sensible choice to use for this type of simulation.

The *CFL* essentially sets how much the flow is allowed to change between $f(x,n)$ and $f(x,n+1)$ approximations of the flow field - if it is set too high the simulation will not converge, and if it is set too low the simulation will take many cycles reach the appropriate flow. *cfl ramp* is a list of ramps, run in order; the single one here grows the CFL each cycle, from *initial* by a factor of *factor*, so early cycles - where the flow field is a poor approximation of the true solution - take a cautiously small step, while later cycles converge faster.

Equations

The *equations* block sets the type of flow we are simulating - there are a number of different options depending on the assumptions we make about the flow. *rans* is used when we are simulating a compressible flow, and want to simulate the effects of turbulence on the mean flow by use of a turbulence equation. The *turbulence* sub-dictionary sets the turbulence model used.

How the flow is spatially discretised - its *order* of accuracy and its inviscid flux scheme - is a separate concern from *what* is being solved, so those settings live in their own *numerical_scheme* block, alongside *equations*.

IC_1

Now that the numerical settings have been defined, the flow needs to have boundary conditions. Boundary conditions define what happens at exterior faces of the mesh. In this case, there are three boundary conditions.

The first boundary condition we consider is that of the freestream air, which is at a temperature of 300 K, a pressure of 101325 Pa and flows parallel to the mesh's x axis at a Mach number of 0.15. The *IC_1* entry in

reference_conditions defines a flow at these conditions.

Because the RANS equations include the effect of viscosity, we also need to define the viscosity at the *IC_1* flow condition (in this case, we set it to be 1.02145×10^{-5} kg m^{-1} s^{-1} at 300 K). This corresponds to a chord based Reynolds number of 6 million. Because the RANS equations include turbulence modelling, we also need to set the *turbulence_intensity* and *eddy_viscosity_ratio* - these are used as boundary conditions for the SST turbulence model equations. The values given are sensible values for flows without significant freestream turbulence.

IC_1 also doubles as the initial guess for the flow field: `"initialisation": {"initial conditions": "IC_1"}` tells the solver that at 0 cycles the entirety of the flow should be initialised to the *IC_1* conditions. Stating this explicitly starts to matter once more than one reference condition is in play (see *Tutorial 2*).

BC_1

The *BC_1* entry assigns the *IC_1* flow condition to all mesh faces contained in the mesh zone *1* (note the plural *zones* key - a boundary condition can span more than one mesh zone). Mesh zones are defined during the meshing process, and in this case mesh zone *1* contains all the mesh faces which are on the farfield exterior surface of the mesh. The *farfield* type means that flow can pass freely through the specified mesh faces, and the *riemann* kind means that flow can pass both in and out of the mesh via the specified mesh faces.

BC_2

The *BC_2* entry assigns a no-slip wall boundary condition to all mesh faces contained in the mesh zone *4*. Mesh zone *4*, in this case, contains all faces on the aerofoil surface. Note the *kind* is written *"no slip"*, with a space.

BC_3

The *BC_3* entry assigns a symmetry boundary condition to all mesh faces contained in mesh zones *2* and *3*, which are the +ve z and -ve z faces of the mesh. This boundary condition type is used here so that we can simulate an aerofoil of effectively infinite span, but with a low mesh resolution in the z axis to reduce simulation cost.

Report

The *report* sub-dictionary, nested under *output_settings*, defines which flow statistics zCFD should output to the user during the simulation, and how often. The 'frequency' key here defines that zCFD should report flow statistics to the user every 10 iteration cycles during the simulation. Exactly which flow statistics are outputted can be changed by the user, but the defaults should be enough for us to successfully monitor the convergence of the solution. The 'forces' sub-dictionary allows us to monitor the forces on specific mesh zones - here we are outputting the forces on mesh zone 4, which is the aerofoil surface. Some of the flow statistics (e.g. force on the aerofoil wall) are non-dimensionalised (see *here*) against the reference condition named in *initialisation*'s *reference* key. That key defaults to whichever condition *initial_conditions* names when it is left unset, which for us is *IC_1*, the freestream state - so there is nothing extra to write here.

Write Output

The final part of *output_settings*, the *solution* sub-dictionary, defines which variables should be outputted into VTK HDF format (.vtkhdf) for visualisation by the user at the end of the simulation.

Step 3. Running zCFD

To run zCFD you first need to source the "activate" script and then use the "run_zcfd" path.

1. Activating your environment:

```
source ./<ZCFD_INSTALL_PATH>/bin/activate
```

2. Run zCFD validate input to check the control dictionary for errors:

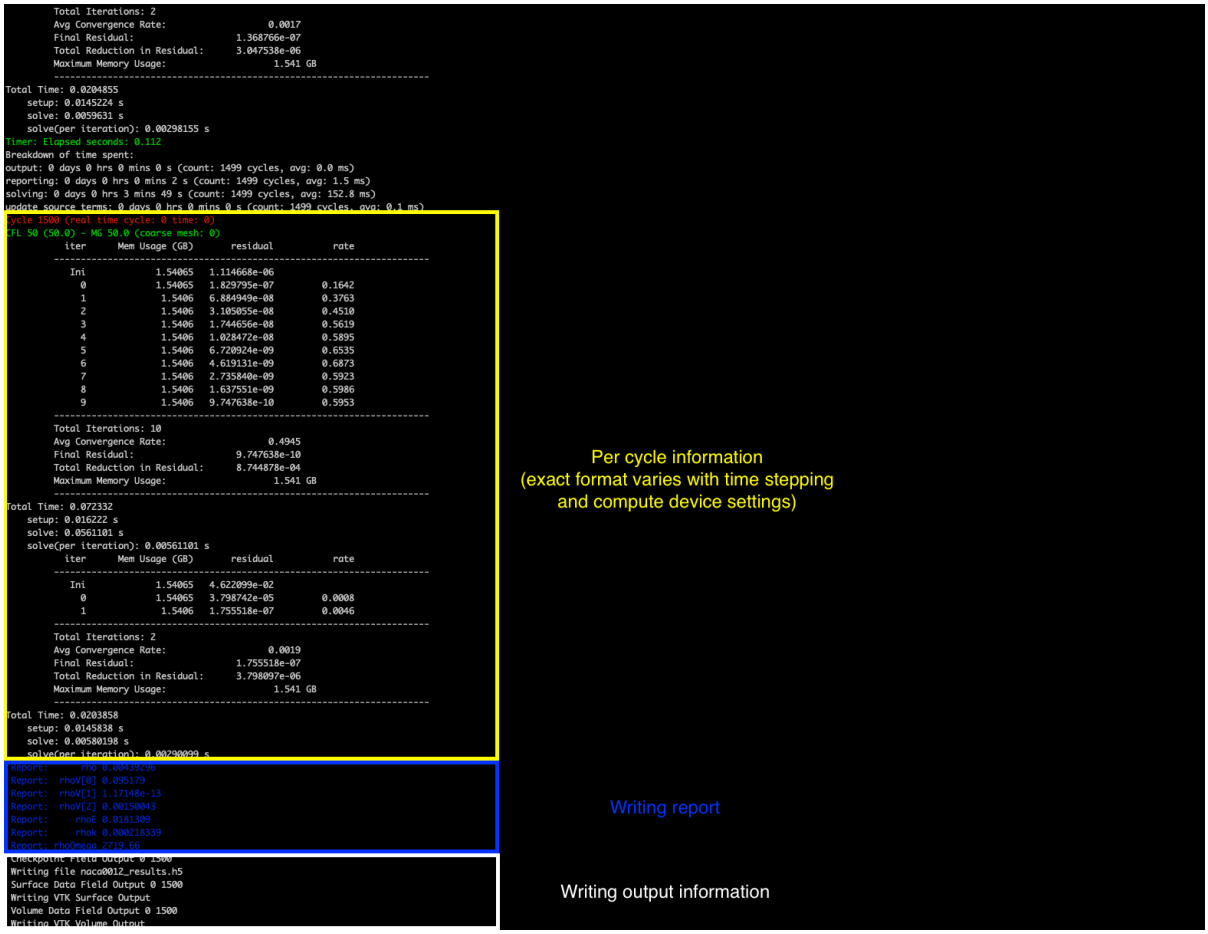
```
cd ~/zcfd_tutorial_1/
validate_input naca0012.py -m naca0012.h5
```

3. Run zCFD in the case directory:

```
run_zcfd -m naca0012.h5 -c naca0012.py
```

zCFD will attempt to detect the number of CPU cores and any GPUs present and make use of them. If you want to manually configure this see *here*.

As the solver is running it will output information about the current CFL, Multigrid level, timing, reporting information and file I/O. The output from zCFD will be written to the screen and also to a log file "naca0012.log".



zCFD will run for the 1500 *cycles* specified in the control dictionary, if you want to stop the solver early then you can either press CTRL+C to kill the process or update the *cycles* keyword to be a value smaller than the current cycle number, zCFD will detect this change and stop the solver.

The solver should take a few minutes to complete on a modern GPU, and a bit longer on a CPU.

Step 4. Review the output

A successful zCFD run will result in the following files and directories being created. In this case, since our control file is called *naca0012.py* the output files will also be created with that name. The _OUTPUT folder also includes an indication of the number of parallel partitions used, this tutorial assumes a single partition and so the output will be "naca0012_P**1**_OUTPUT" if we ran on two partitions it would be "naca0012_P**2**_OUTPUT"

File/Directory	Description
naca0012.log	This is simply a copy of the output to screen (see above) which happened during the run.
naca0012_report.csv	A .csv format table which shows the variation of flow statistics with cycle number
naca0012_report.ipynb	An auto-generated Jupyter notebook which can be used to read the naca0012_report.csv file and easily plot the flow statistics varying with cycle number.
naca0012_results.h5	The flow solution calculated by zCFD, in zCFD format. This can be used for visualisation, but it is generally only used for restarting zCFD simulations from
naca0012_status.txt	A .json format file outputted by zCFD. Among other things, it records when the simulation was last run, which mesh was used and which control file was used.
naca0012_P1_OUTPUT/	A directory which contains the flow solution calculated by zCFD, in VTK HDF format (.vtkhdf). This is generally more suitable for visualising the data with than the zCFD proprietary format.
naca0012_P1_OUTPUT/naca0(	A VTK HDF file containing the volume data from the simulation.
naca0012_P1_OUTPUT/naca0(naca0012_wall.vtkhdf etc	A series of VTK HDF files (one for each boundary condition type) containing the surface data for that boundary condition.
naca0012_P1_OUTPUT/LOGGI	Contains a separate copy of the naca0012.log file for each MPI partition. Used for debugging.

Step 5. Looking at residuals with notebook

The naca0012_report.ipynb file can be loaded using a variety of methods, but here we will load it using a browser. The commands below should open Jupyter Notebook within a browser window. If the browser does not open automatically then the notebook URL should be visible in the command output.

1. Activate your environment:

```
. ./<ZCFD_INSTALL_PATH>/bin/activate
```

2. Start the Jupyter Notebook:

```
cd ~/zcfd_tutorial_1/
start_lab
```

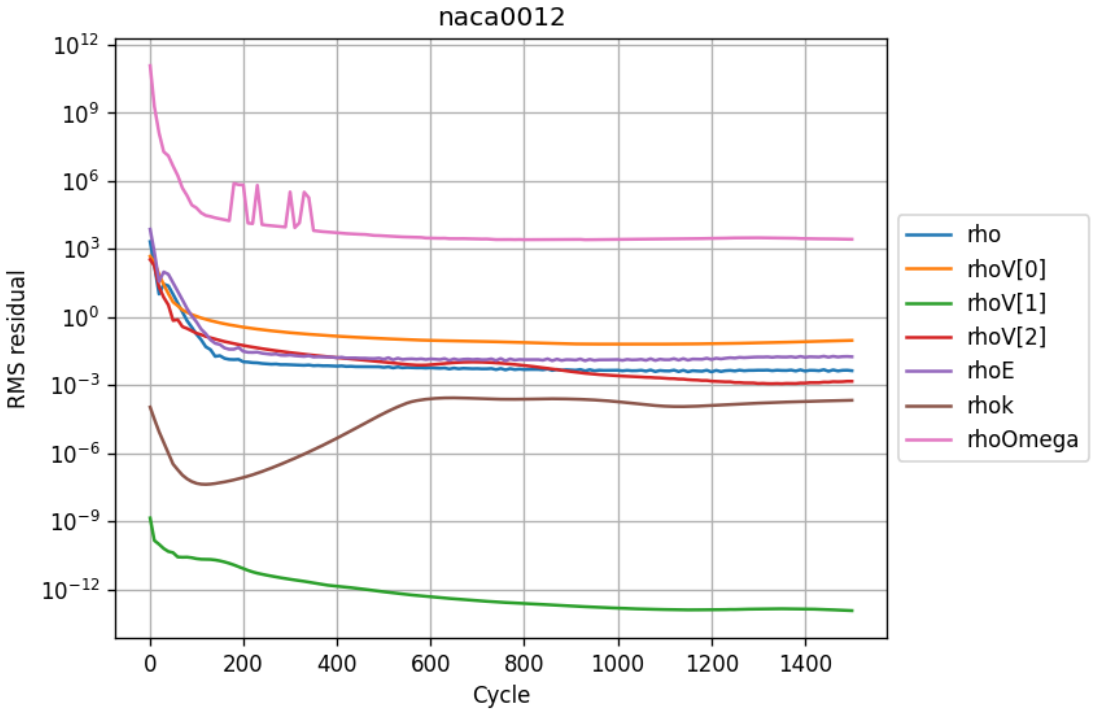
Load the naca0012_report.ipynb in Jupyter, then click `run all`.

Plotting residuals

The naca0012_report.ipynb makes use of the *zutil.plot.zcfd_plots* function to read and plot the zCFD report data. Calling the *zcfd_plots('naca0012.py')* function reads the zcfd control file, finds all the output files, and returns a *zutil.post.Report* object, which provides plotting functionality . To plot the convergence of residuals, use the *plot_residuals()* method:

```
from zutil.plot import zcfd_plots
r = zcfd_plots('naca0012.py')
r.plot_residuals()
```

The residual variables essentially measure how close to a stable solution of the chosen flow equations (e.g. the RANS equations) the simulation is. There is a residual for each flow variable, and the lower the residual, the more converged the flow solution is.



As would be expected (note the logarithmic y axis), the residuals drop very quickly in the first couple of hundred iterations as the flow quickly develops from the initial condition (which in this case, is the freestream condition everywhere). After that, the flow residuals (*rho*, *rhoV[0]*, *rhoV[1]*, *rhoV[2]* and *rhoE*) residuals continue to drop, while the turbulence residuals (*rhok* and *rhoOmega*) drop initially before flatlining / increasing.

This behaviour is common - the turbulence equations can take a while to react to changes in the flow field, and the initial guess for the turbulence equations is often poorer than the initial conditions for the flow field. A combination of the fact that the flow residuals have dropped several orders of magnitude over the simulation history and the fact that the residuals are in effect flat from 20,000 cycles onwards mean we can be confident that the simulation is sufficiently converged.

Plotting forces

Next, we can use the Jupyter notebook to plot the variation of the force coefficients on the aerofoil surface with cycle number.

```
r.plot_forces()
```

Check the *wall_Fx* and *wall_Fz* tickboxes - given this is a symmetrical aerofoil run at 0 angle of attack, the x force corresponds to drag and the z force corresponds to lift. We would expect zero lift and a non-zero, positive drag.

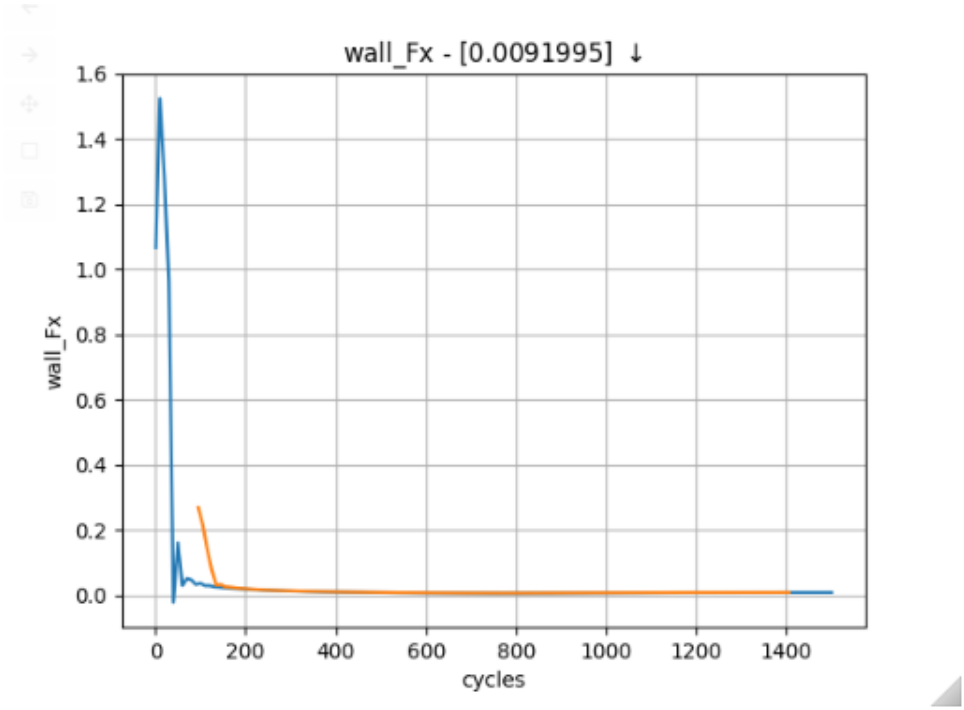
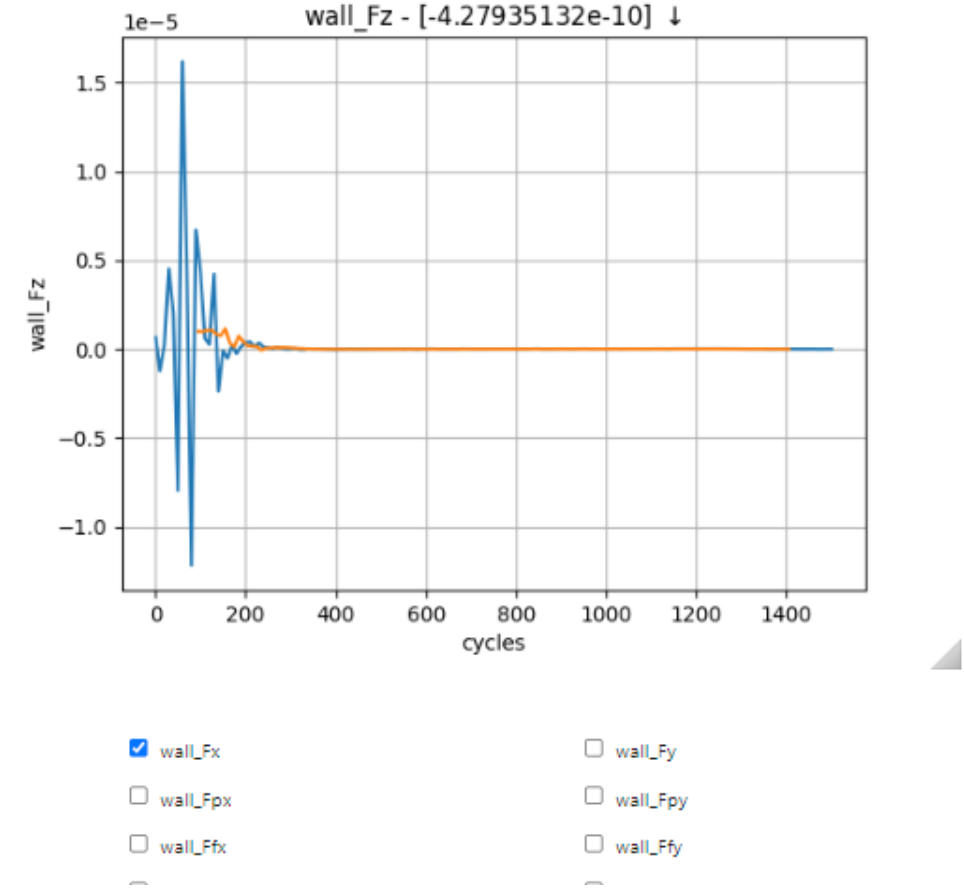


Figure 6



The force plots tell a similar story to the residuals - there are initially large oscillations as the flow settles down, before the force traces essentially flatline from 1200 cycles onwards.

Plotting performance

```
r.plot_performance()
```

Finally, we can plot the time used per cycle using the `plot_performance()` method. The last cycle is particularly long because it is the only cycle where the flow solution is written to a file. If an implicit solver is used, *plot_performance()* will also attempt to plot the memory usage per cycle.

Step 6. Looking at results with ParaView

ParaView is a general purpose visualisation tool that is the recommended way of post processing zCFD results. Please see ParaView.org for download and installation instructions.

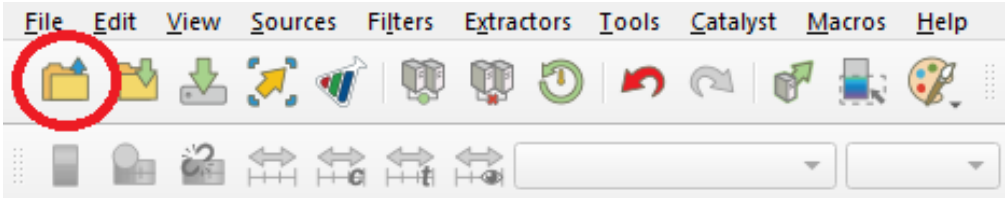
Note

The .vtkhdf output format requires ParaView 6.1.1 or later.

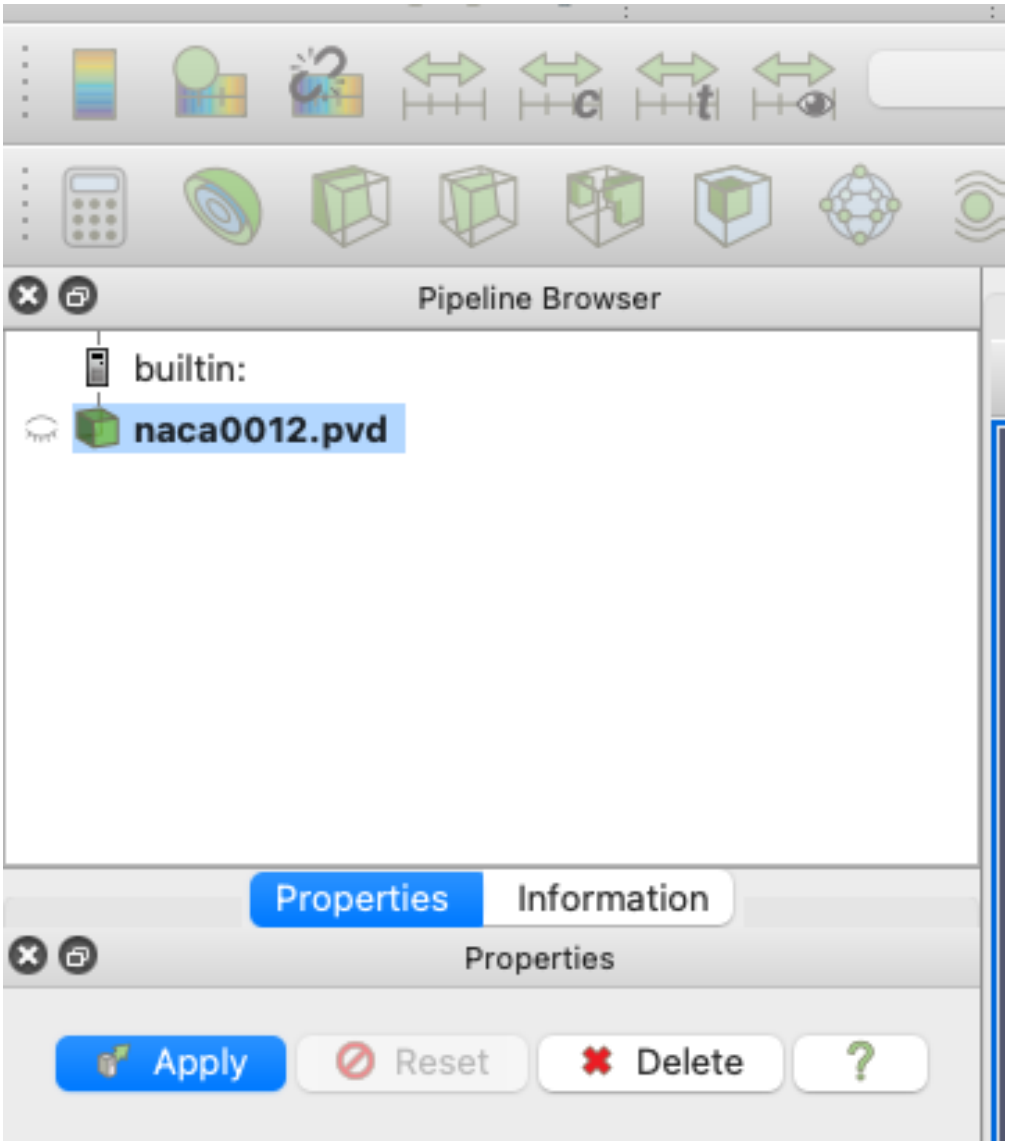
Loading your data

As mentioned above a "naca0012_P1_OUTPUT" folder has been generated, and contains a ".vtkhdf" file (naca0012.vtkhdf). This is a single VTK HDF file containing all of the volume output data, making it easy to load into ParaView.

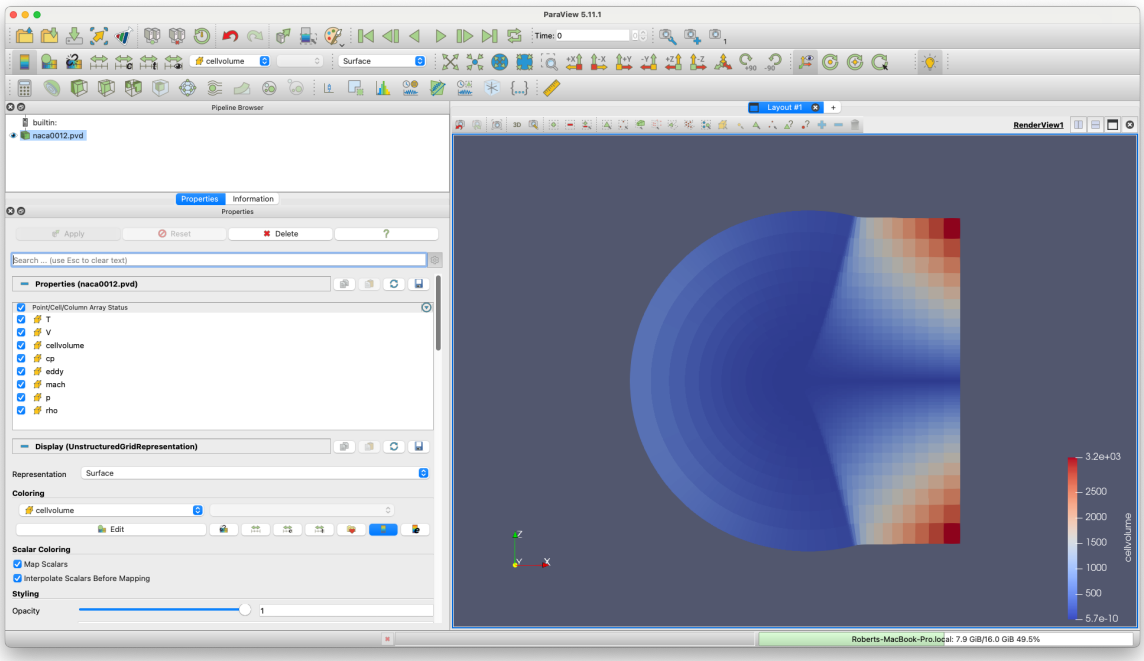
After opening ParaView, simply click on the "Open" icon shown below or go into File>Open or press Ctrl+O. Locate the directory in which you have stored your data and open naca0012.vtkhdf.



Once the file is correctly loaded in ParaView, the "Pipeline Browser" on the left of your screen should look like this:



Now you need to click “Apply”, which will lead to the following layout:

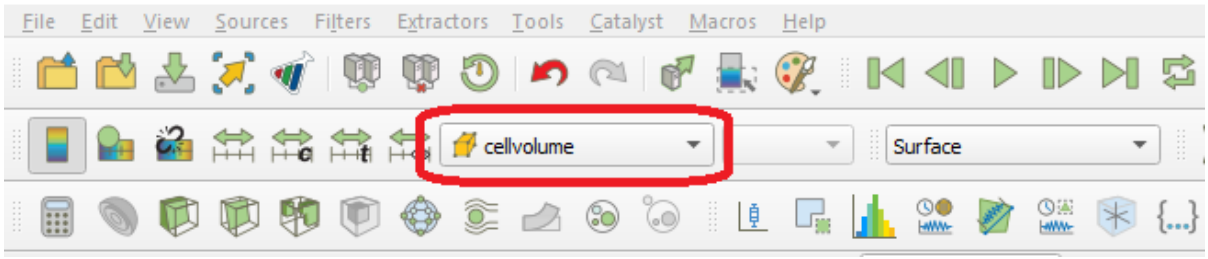


You may need to use the 'set view direction to +ve y' button:

Visualising Variables

By default, ParaView shows the "cellvolume" variable. This allows to see that the mesh is refined near the center of the mesh, where the airfoil is. This is common practice, as capturing the flow around a body is more demanding than doing so in the free-stream. Naturally, having a really fine mesh over the entire fluid domain would make the problem really computationally expensive.

Other variables can be selected and shown using the following scrolling menu:

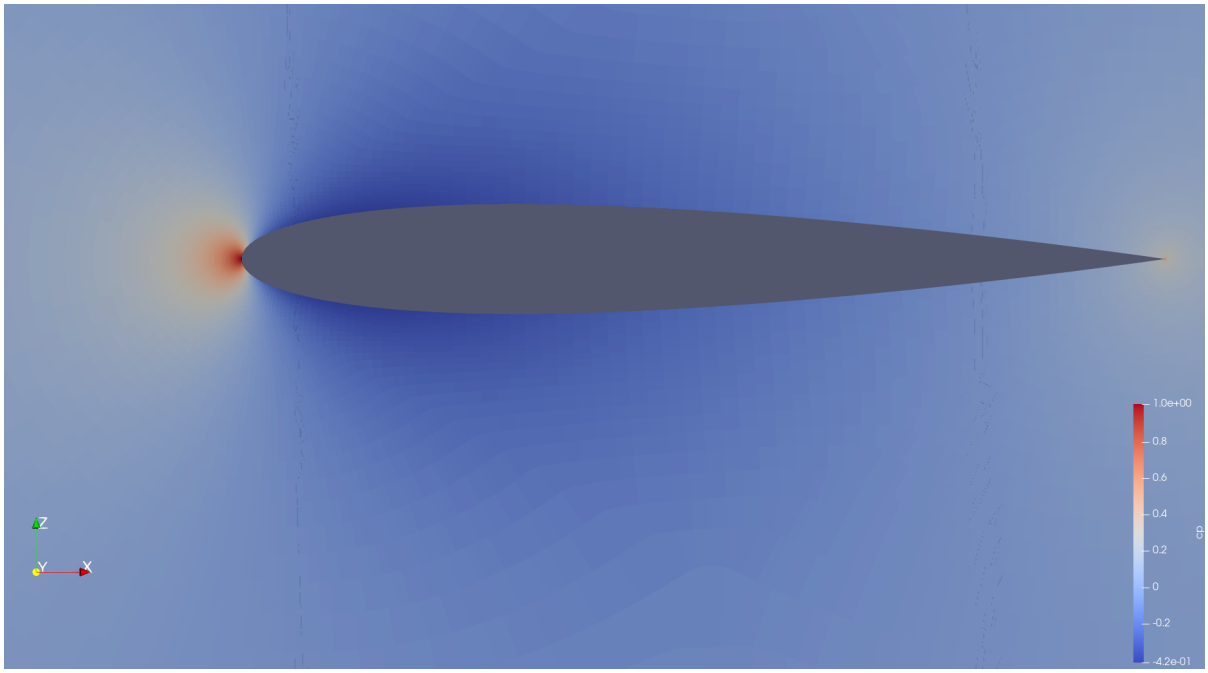


Let's observe the pressure coefficient around the airfoil. To do so, use the scrolling menu and select the "cp" variable.

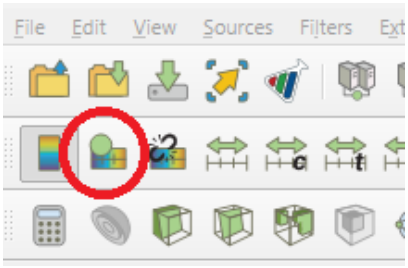
Note

The pressure coefficient is given by the following formula $C_p = \frac{p - p_\infty}{\frac{1}{2}\rho_\infty V_\infty^2}$

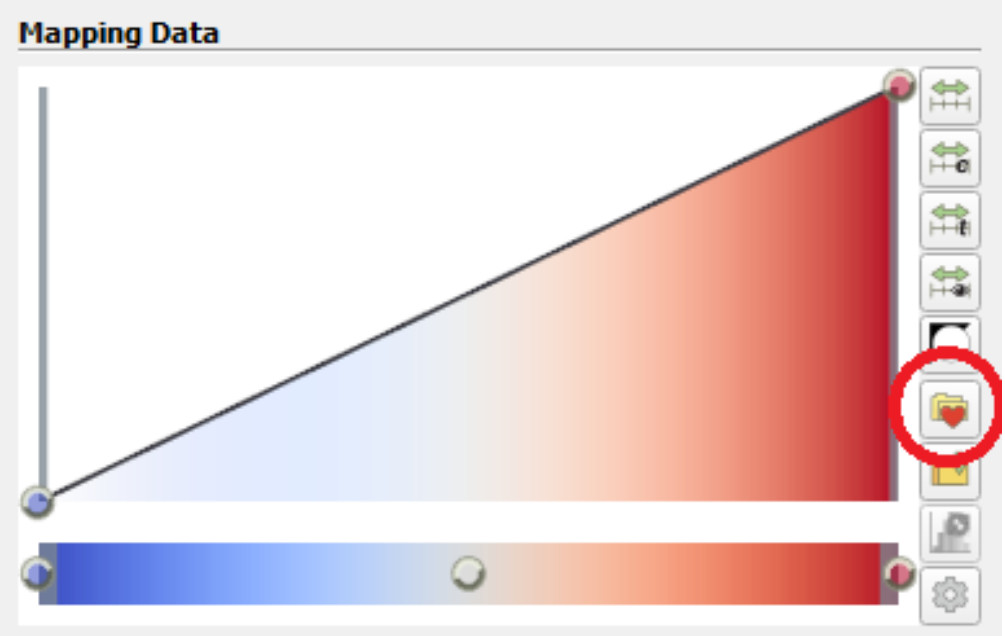
This gives the following (you will need to zoom in substantially to the centre of the mesh before you can clearly see the airfoil):



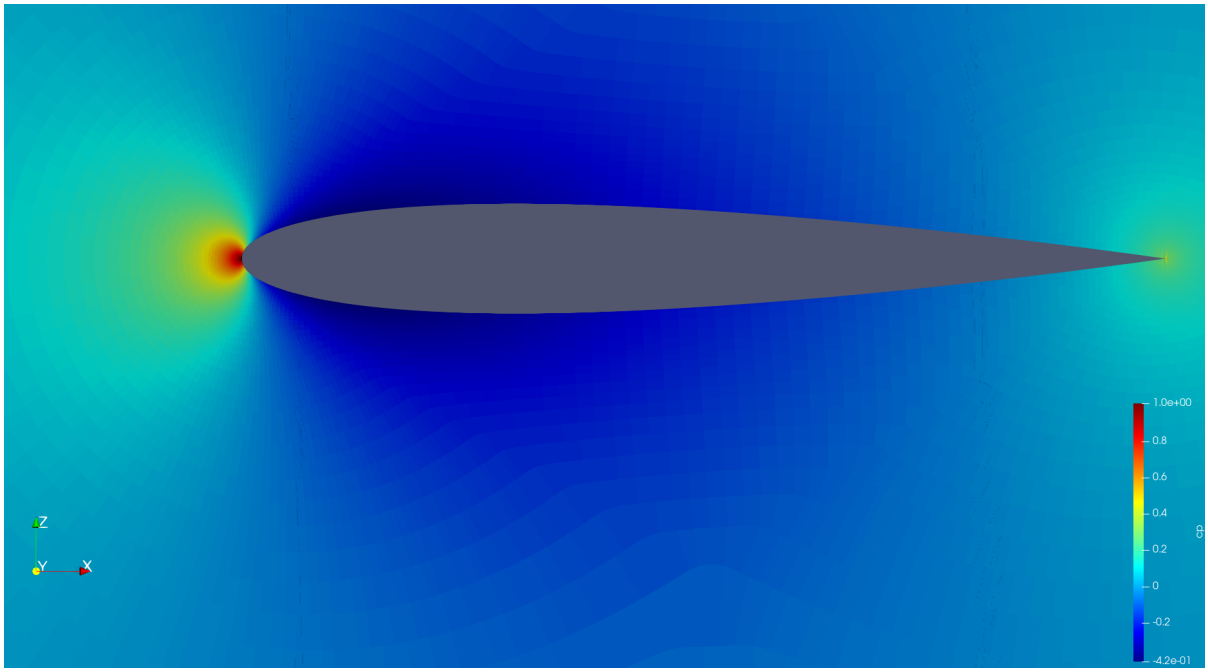
Note that you can change the colormap used by ParaView by clicking on the "Edit Color Map" icon:

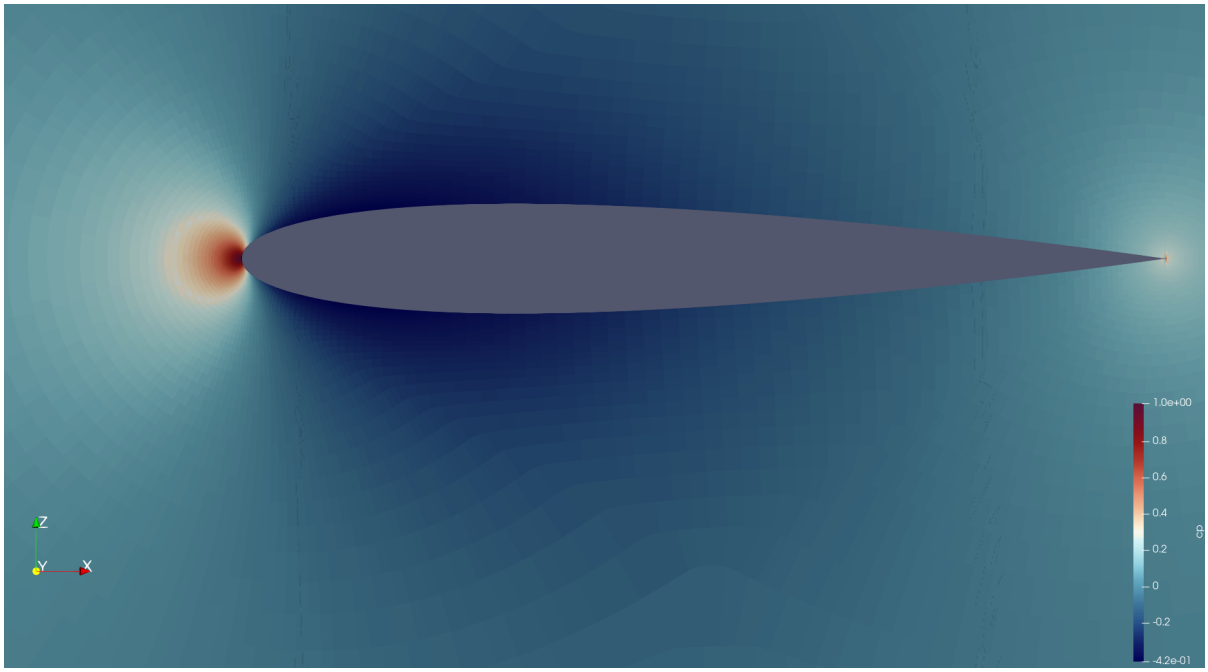


and then the "Choose Preset" icon:



Here are the same results shown using different colormaps:

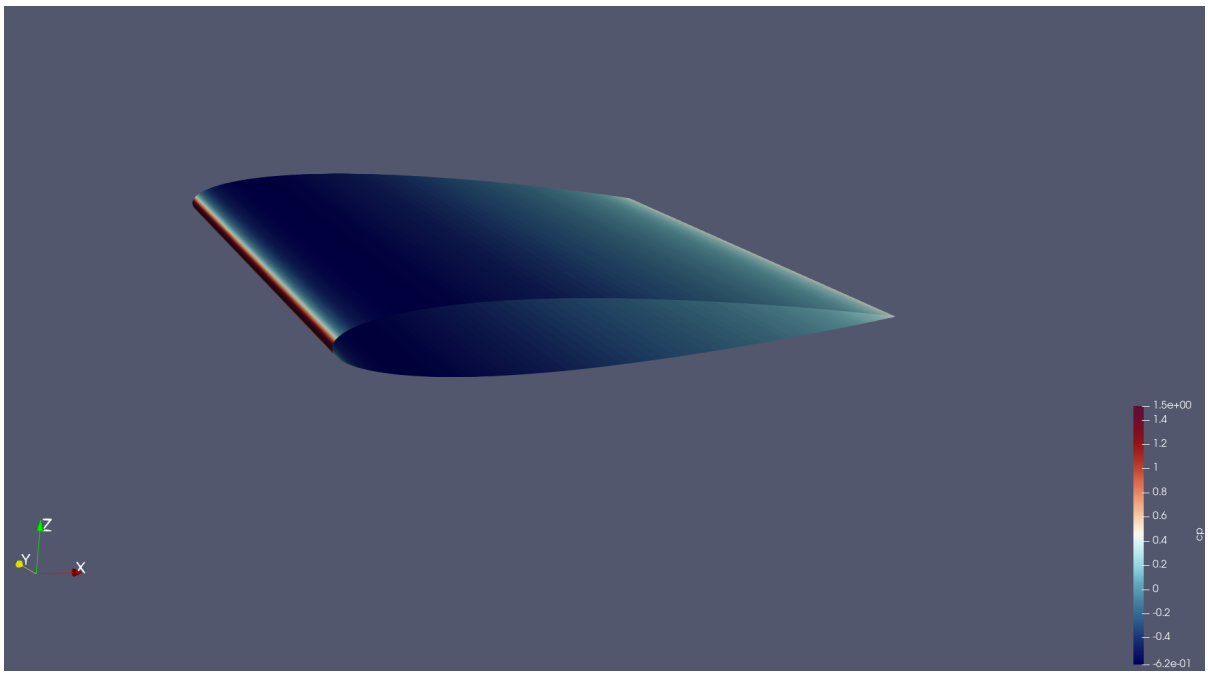




Those simple steps allow to qualitatively inspect the solution given by our CFD simulation. More advanced analysis is also feasible. A common result in aerodynamics, especially around an airfoil, is the plot of the pressure coefficient along the chord.

Pressure Coefficient Plot

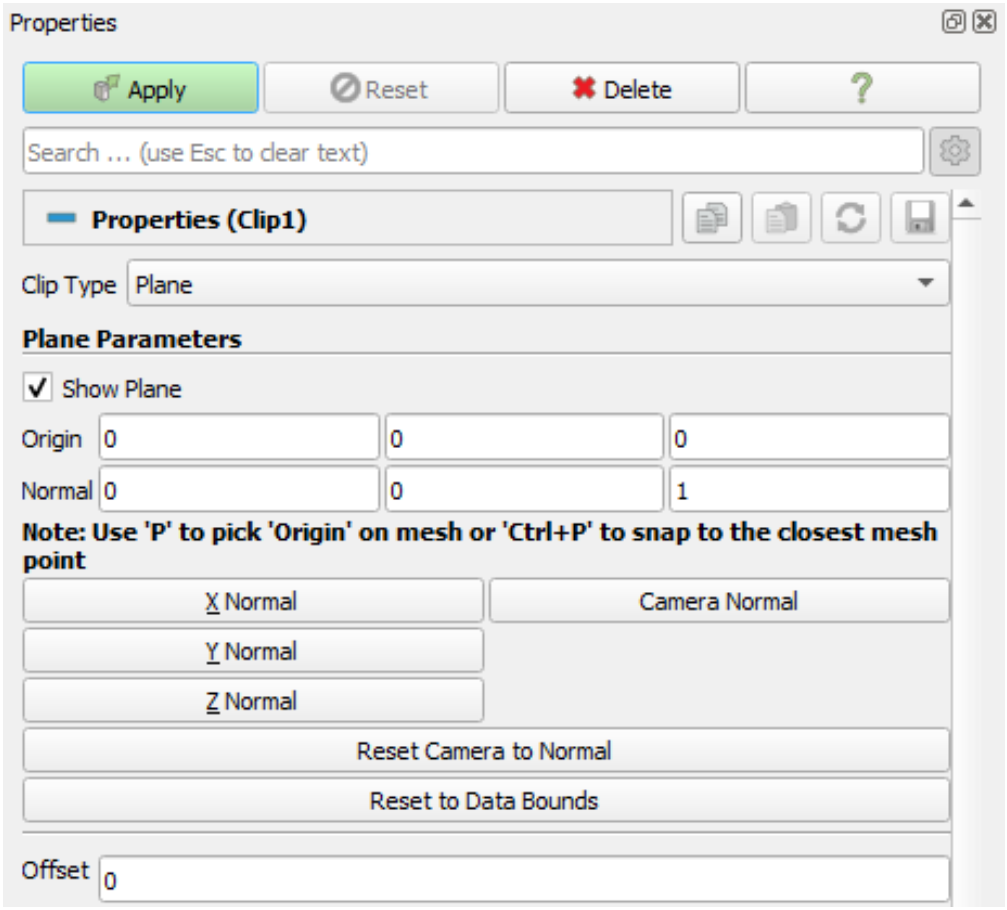
Open the "naca0012_wall.vtkhdf" file. Naturally, the resulting image looks different from the previous as this only contains the zones where a "wall" boundary condition has been defined, which in this case happens to just be the airfoil itself.



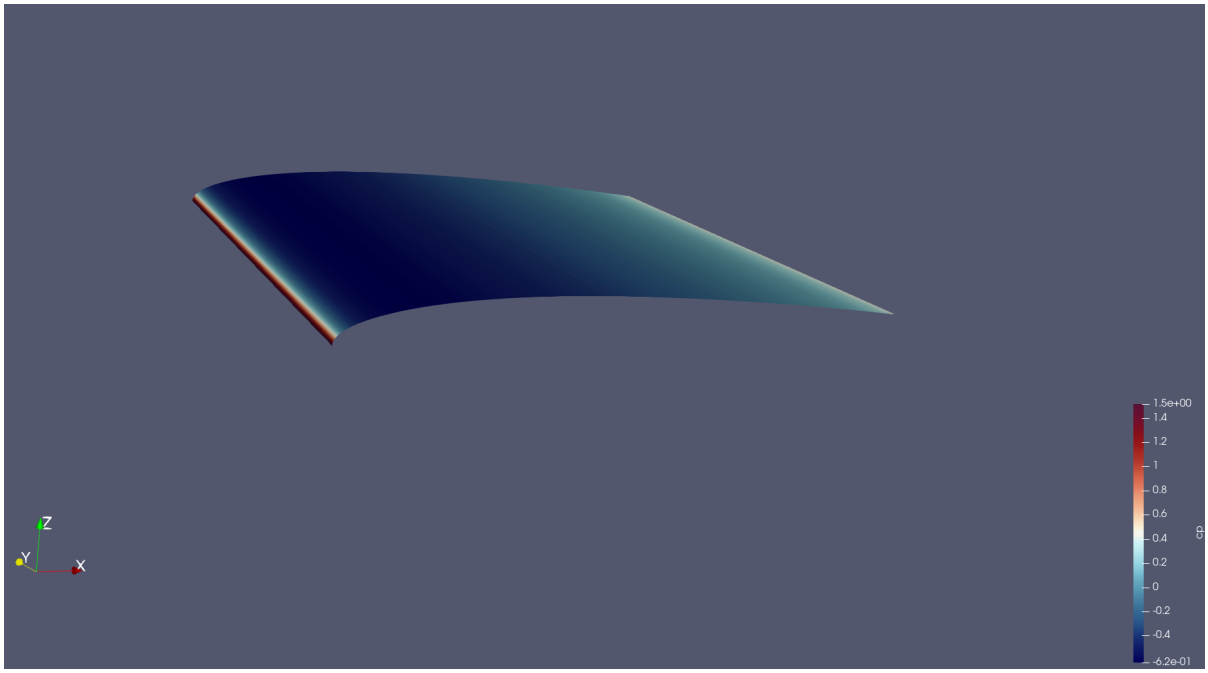
It is usually of interest to have a plot showing the pressure coefficient on the upper and the lower surface, therefore it is necessary to extract the data separately for each face of the airfoil. The upper and lower surfaces of the airfoil then need to be separated.

To this end, the "Clip" filter has to be used. To use it, right-click on the "naca0012_wall.vtkhdf" object in the Pipeline Browser, select Add Filter -> Alphabetical -> Clip. The properties of the "Clip" filter are displayed on

the left. Firstly, untick the “Invert” option and then you should set them to the following combination:



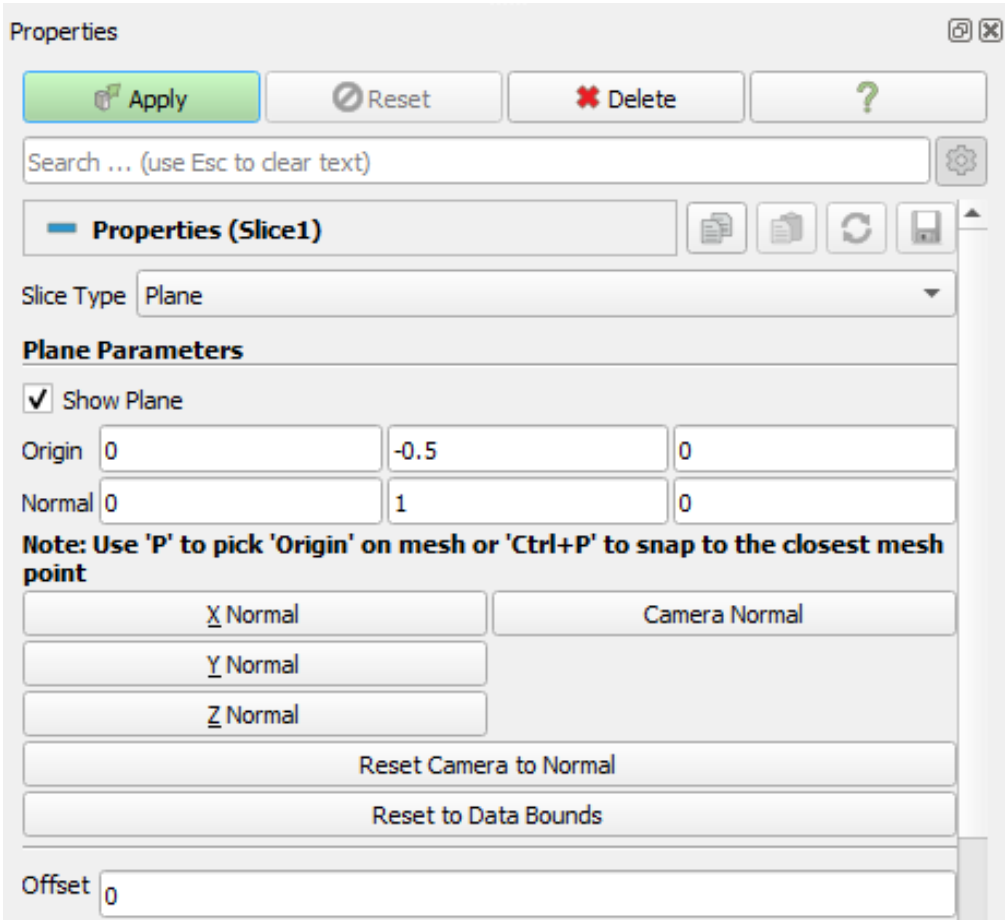
By default, this will “remove” the lower surface of the airfoil, leaving the following shape:



Note

To remove the upper surface instead, just tick the “Invert” option in the “Clip” filter’s properties.

Now, we want to take a “Slice” of the remaining surface. To do so, follow the same procedure as for the “Clip” filter but select the “Slice” filter instead. When prompted, set the properties of the filter to be as follows:

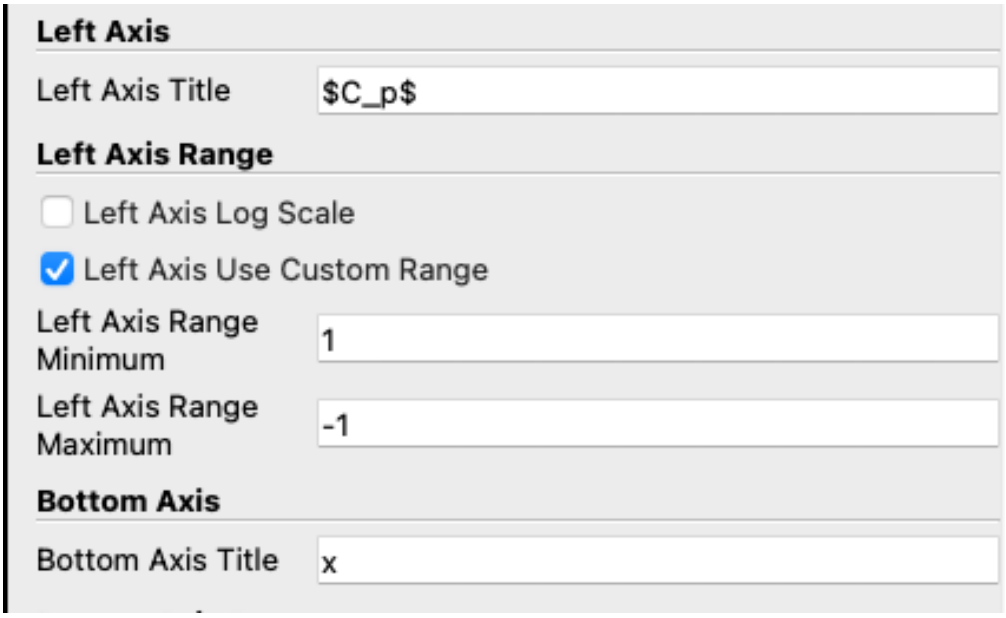


You should end up with this:

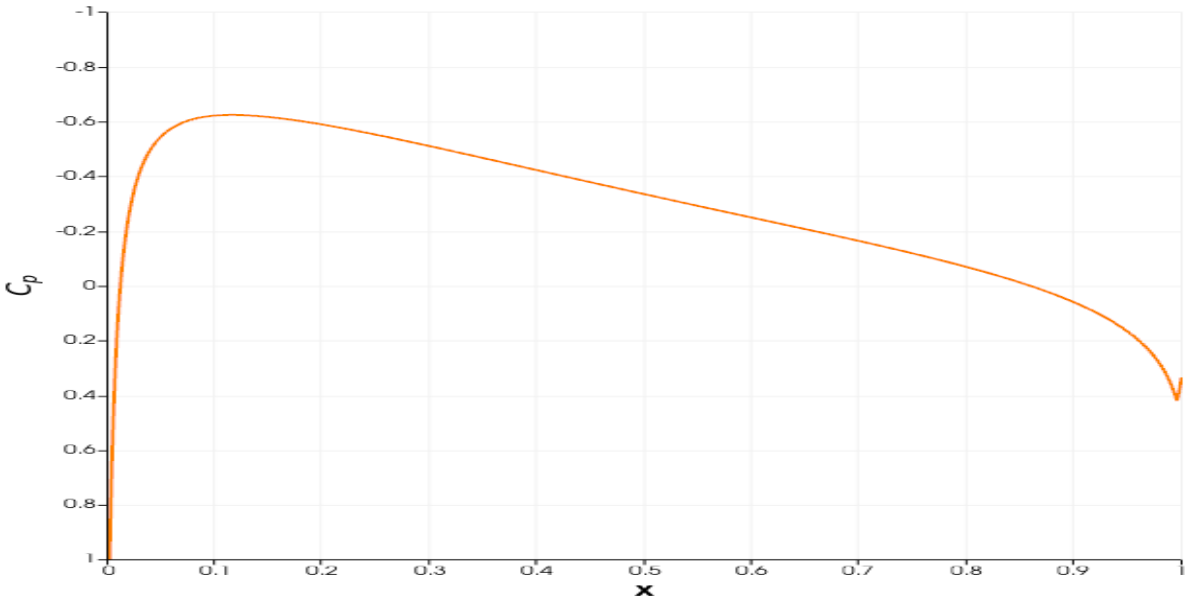
Then the data should be converted from “cell data” to “point data”. This is done by using the “CellDatatoPoint-Data” filter. Access by right-clicking on the “Slice” object selecting “Add filters”.

Once this is done, a plot can be produced in ParaView using the “PlotOnSortedLines” filter. Right-click on

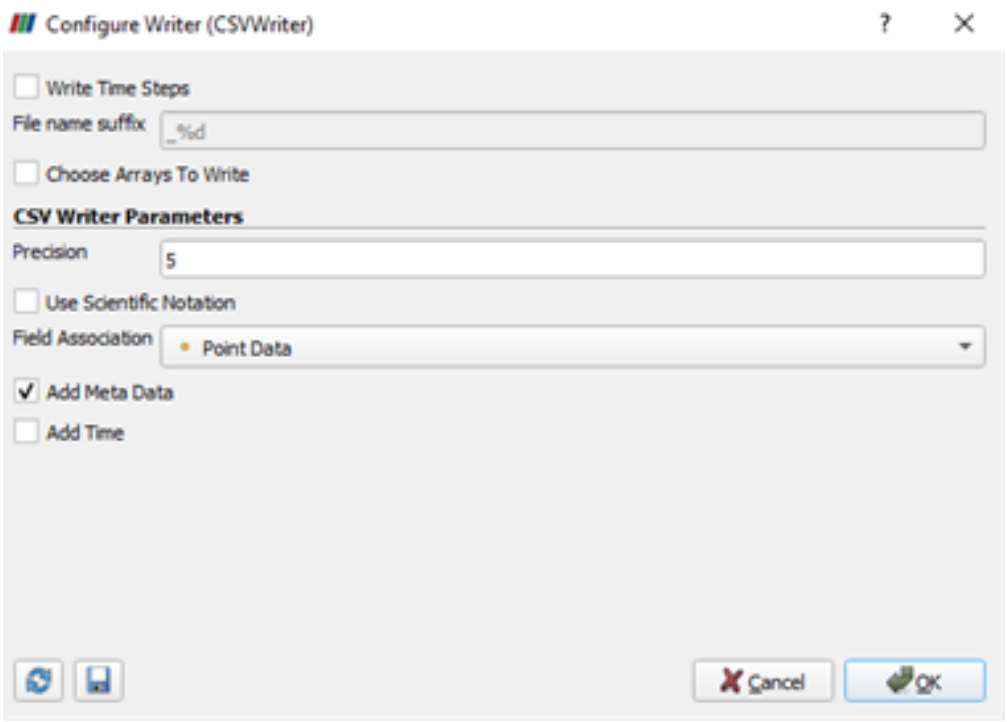
the “CellDatatoPointData1” object created previously and add the “PlotOnSortedLines” filter. A plot should appear on the right. You can select the variables to plot in the “Properties -> Series Parameters” window, as well as changing the settings of the axis. Select the “cp” variable in “Series Parameters’, and use ‘Points_X’ as the ‘X Array Name’. Untick ‘Show Legend’, set ‘Left Axis Title’ to ‘C_p’ and set the ‘Bottom Axis Title’ to ‘x’. You can also use the “Left Axis Use Custom Range” option to change the y-axis range:



Note that setting the maximum as a negative number will “invert” the y-axis and the negative numbers will be shown “at the top”.



The data can be exported as a .csv file, which can then be manipulated in Python, Excel, MATLAB or any other software. To do so, go to File -> Save data -> *select a folder and give a name to the file.* The following window should appear:



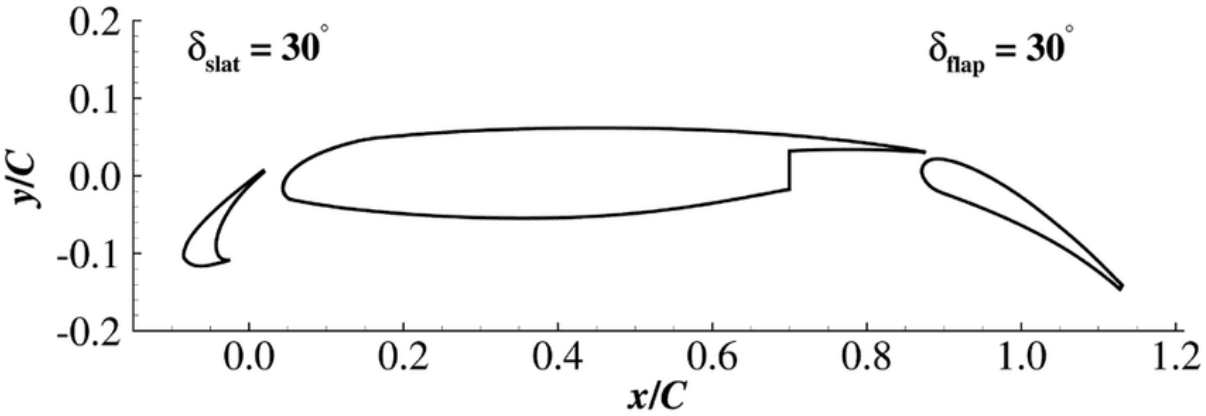
By default, ParaView writes all the variables into the .csv file. However, if only a few of them are of interest, you can use the "Choose Arrays To Write" option.

Now, this procedure can be repeated for the lower surface of the airfoil. A pressure coefficient plot can then be made for validation purposes.

1.4.2 Tutorial 2: Steady State RANS

Table of Contents

In Tutorial 1, we learnt how to run a case in zCFD and visualise the results. We will now use a more complex test case to showcase some of the features of the solver. The case we will look at is the 30P30N aerofoil, which is a 2D multi element aerofoil, developed as an acoustic benchmark case [*1*]. The geometry of this case can be seen below:



Initially we will begin by obtaining a steady state RANS solution.

The required meshes and control dictionary can be found here. The case has 64,000 cells, and will run in less than 10 minutes on an NVIDIA RTX 3060 GPU.

This case is run as a steady state simulation, using an implicit time marching scheme. The Reynolds Averaged Navier-Stokes equations are solved using Menter's SST turbulence model with wall functions.

Control Dictionary Breakdown

The setup for this case is very similar to the previous NACA0012 cases, but the control dictionary does feature some extra terms which can be useful for a more streamlined workflow. As in *Tutorial 1*, settings are nested under a top-level *solver* key and a *model* key, one entry per mesh.

Reference variables

At the top of the control file, the key variables for the case- Reynolds number, Mach number, temperature etc... are all hard coded, and then referred to later in the script. In cases where you might want to run many simulations with slightly different values to change, this can be an effective way to ensure consistency with variables. Additionally this allows implicit values such as the reference velocity, and freestream Mach to be calculated automatically in the script.

```
# create variables to assign main experimental parameters
reynolds = 1.71e6
mach = 0.17
T = 295.56
p = 101325
R = 287.6
gamma = 1.4
reference_length = 0.457
alpha = 5.5

# calculate implicit values
rho = p / (R * T)
U = math.sqrt(gamma * R * T) * mach

# assign each mesh zone to a boundary
wall = [6]
symmetry = [4,5]
farfield =[7]
```

Scale

In this case a scaling is applied to convert the size of the mesh. The .h5 mesh file was generated with units of inches, whereas the flow conditions are specified in SI units, therefore a scaling of 0.0254 is applied to convert the mesh into metres. The z (spanwise) direction has been scaled to be 1 metre wide. The mesh has only a single cell in the z direction so no spanwise flow is expected - making the z extent of the mesh 1 metre simply means that the forces and moments reported by zCFD will already be per unit span.

Mesh scaling is a property of a specific mesh, so it is set under that mesh's own entry in *model*, in a *transforms* sub-dictionary.

```
# scale the mesh to match the experimental data
"transforms": {"scale": [0.0254, 0.0254, 0.0254 / 0.127]},
```

Inflow vector

In this case we want to simulate the aerofoil operating at an angle of attack of 4 degrees. Rather than rotating the mesh, it is easier to rotate the inflow vector relative to the mesh, an example of a Galilean transformation. The `zutil` function `vector_from_angle()` calculates the inflow vector given an x-z angle of attack, x-y angle of attack and flow speed.

```
"IC_1": {
    "temperature": T,
    "pressure": p,
    "v": {
        # calculate the inflow vector based on the angle of attack
        "vector": zutil.vector_from_angle(0.0, alpha, U),
    },
    "reynolds no": reynolds,
    "reference length": reference_length,
    "turbulence intensity": 0.01,
    "eddy viscosity ratio": 0.1,
    "ambient turbulence intensity": 1e-20,
    "ambient eddy viscosity ratio": 1e-20,
},
```

Transform

The lift and drag forces acting on a body are defined relative to the freestream flow, lift normal and drag parallel. In this case, where we have rotated the relative inflow vector to a specific angle of attack, it is also useful to rotate the force report by the same vector. The transformed forces will appear as Ft_ terms in the report file.

```
# define a function to rotate the output forces by the angle of attack
def my_transform(x,y,z):
    v = [x,y,z]
    v =  zutil.rotate_vector(v,0.0,alpha)
    return {'v1' : v[0], 'v2' : v[1], 'v3' : v[2]}
```

Running the case

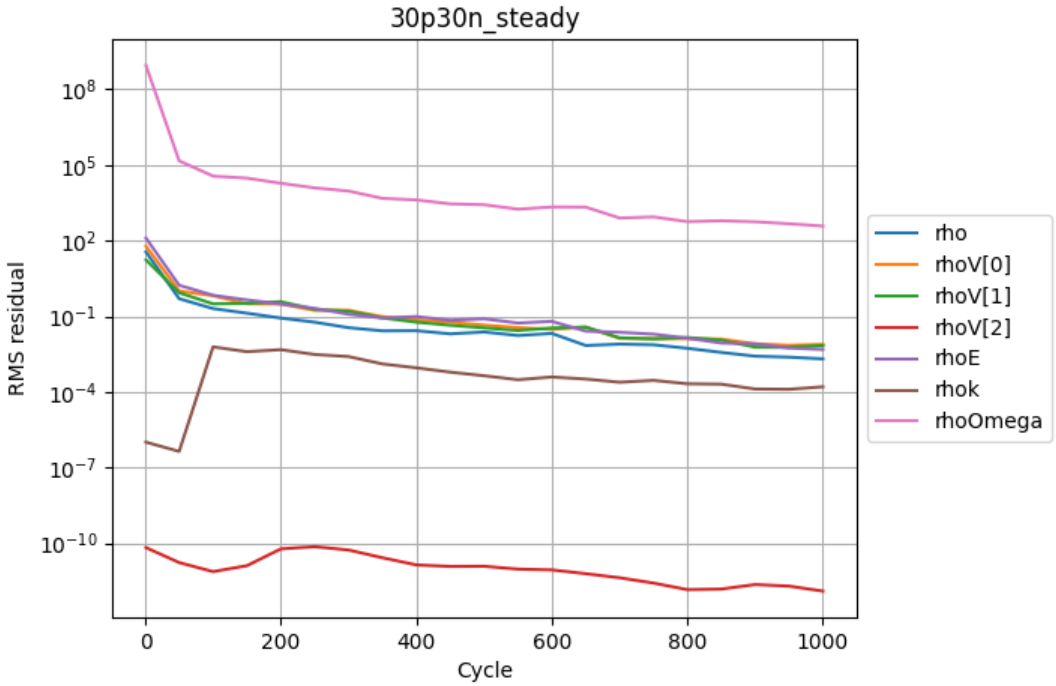
You can run the case as you did for Tutorial 1, but with the following **run_zcfd** command:

```
run_zcfd -m 30p30n_coarse.h5 -c 30p30n_steady.py
```

Monitoring convergence

Start by launching a Jupyter server to examine the residuals in the 30p30n_steady_report.csv file. Running the first cell from the 30p30n_steady_report.ipynb notebook will plot the residuals for the continuity, momentum, and energy equations, as well as the residuals from the turbulence model.

If the case is still running you can update the plot by rerunning the cell, this provides a way to monitor the progress of a running simulation.



You will notice that case is set to run for 1000 cycles but looking at the plots, it looks like the residuals are still converging. So we will now restart the solver from where we finished and run on for another 1000 cycles.

Performing a restart

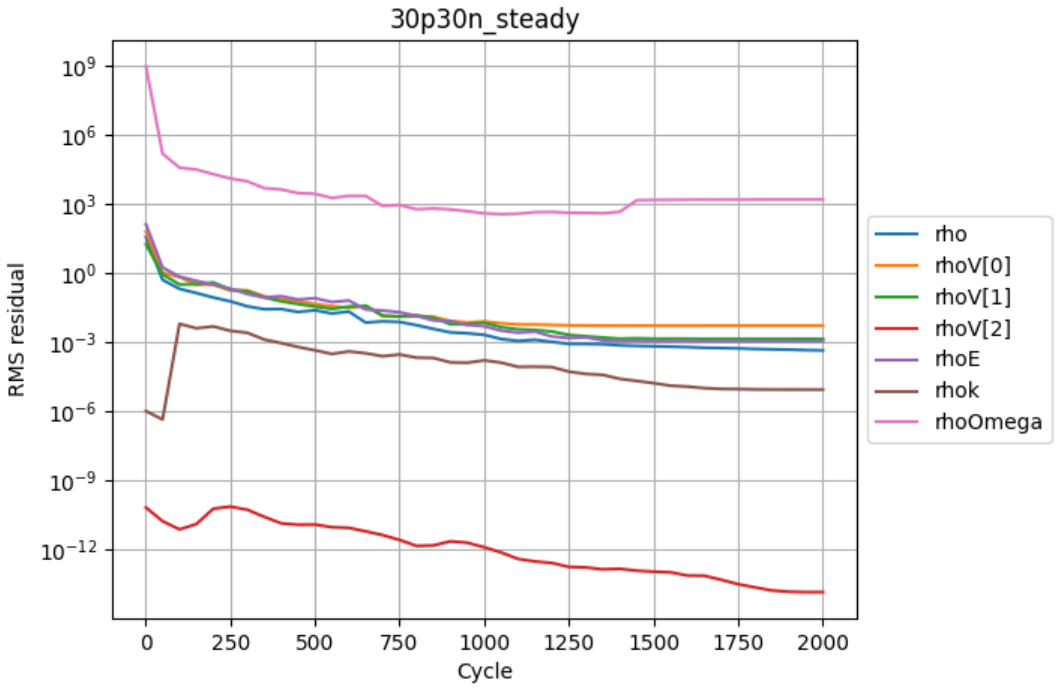
To restart the solver we need to update the *'restart'* parameter, which lives under *solver.initialisation* in the control dictionary. If this is set to *False* the solver will always perform a fresh start, if it is *True* then a results file will be read in and the solve will start based on the data in that file. To enable the restart edit *30p30n_steady.py* and change restart to *True*. You will also need to update *'cycles'* to 2000 - since this is a steady simulation, *cycles* lives under *solver.convergence_control*.

```
parameters["solver"]["initialisation"]["restart"] = True
parameters["solver"]["convergence control"]["cycles"] = 2000
```

By default zCFD will look for a results file with the same name as the current case file, appended with '_results.h5'. So in our case it will look for "30p30n_steady_results.h5". When you have edited the control dictionary go ahead and restart the simulation again using the same **run_zcfd** command:

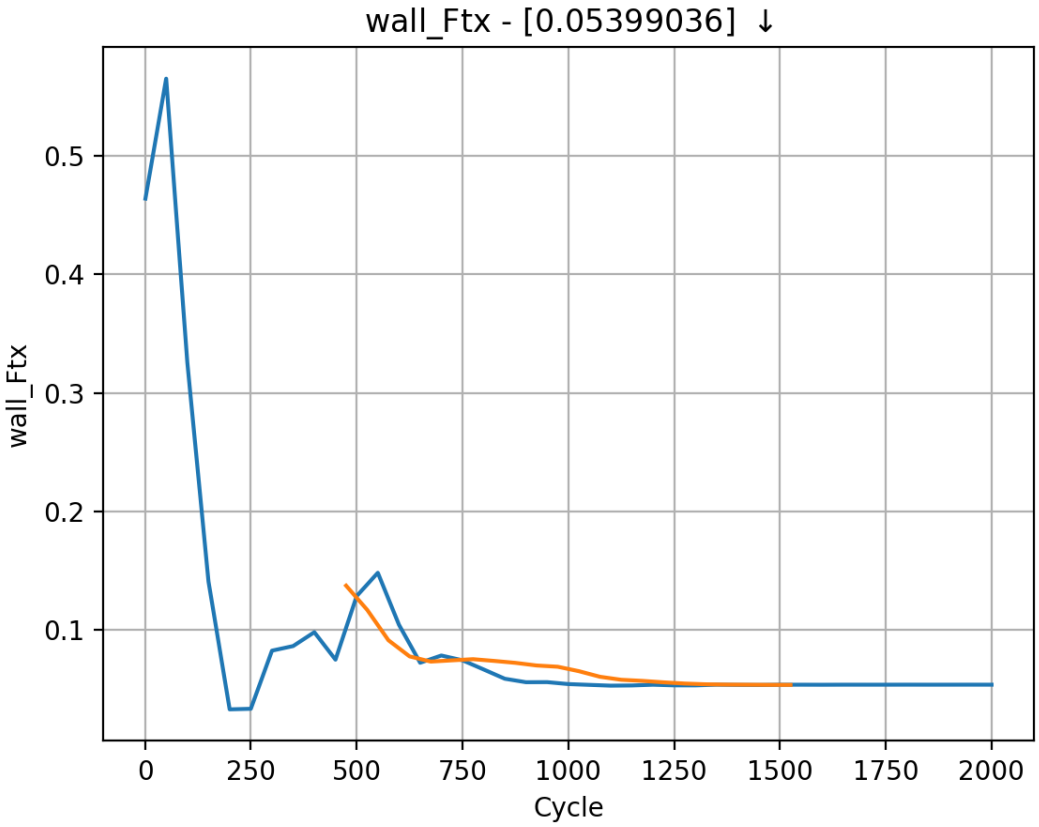
```
run_zcfd -m 30p30n_coarse.h5 -c 30p30n_steady.py
```

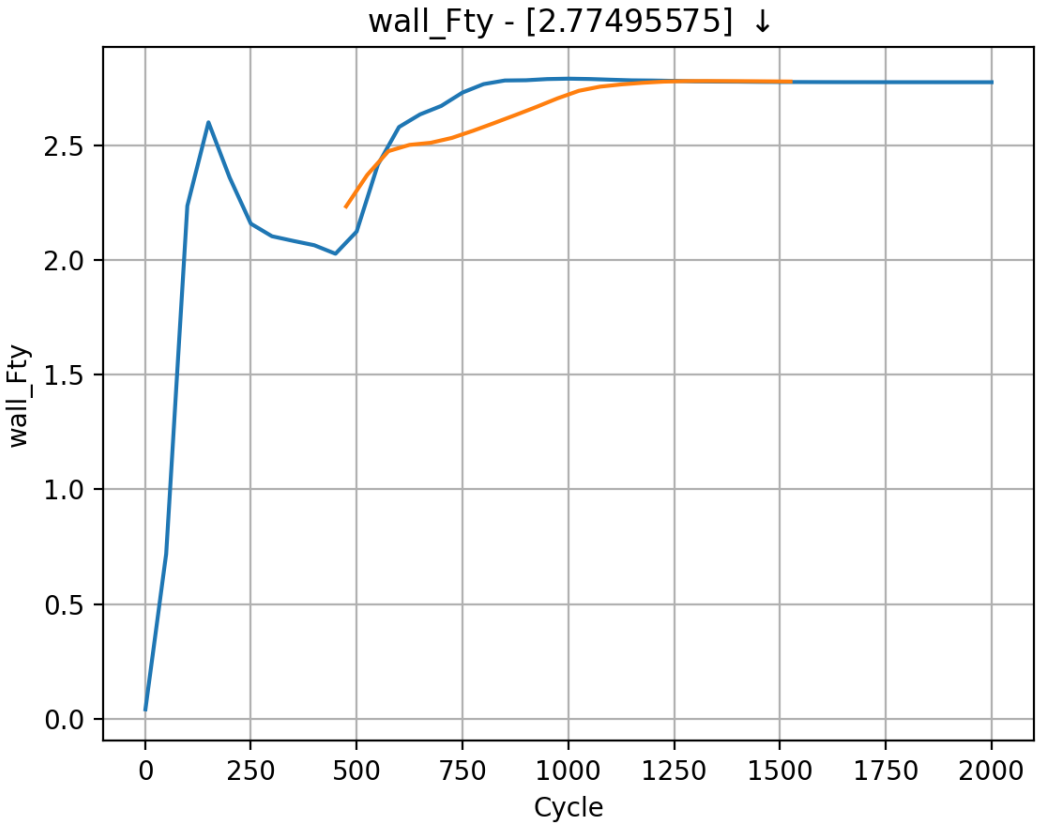
Once the solver has finished rerun the cell in the notebook to examine the residuals again. The residuals now look like they have converged:



Plotting force convergence

Examining the force convergence history next, we are interested in the x and y forces, but transformed by the inflow vector, therefore the wall_Ftx and wall_Fty plots are of interest.

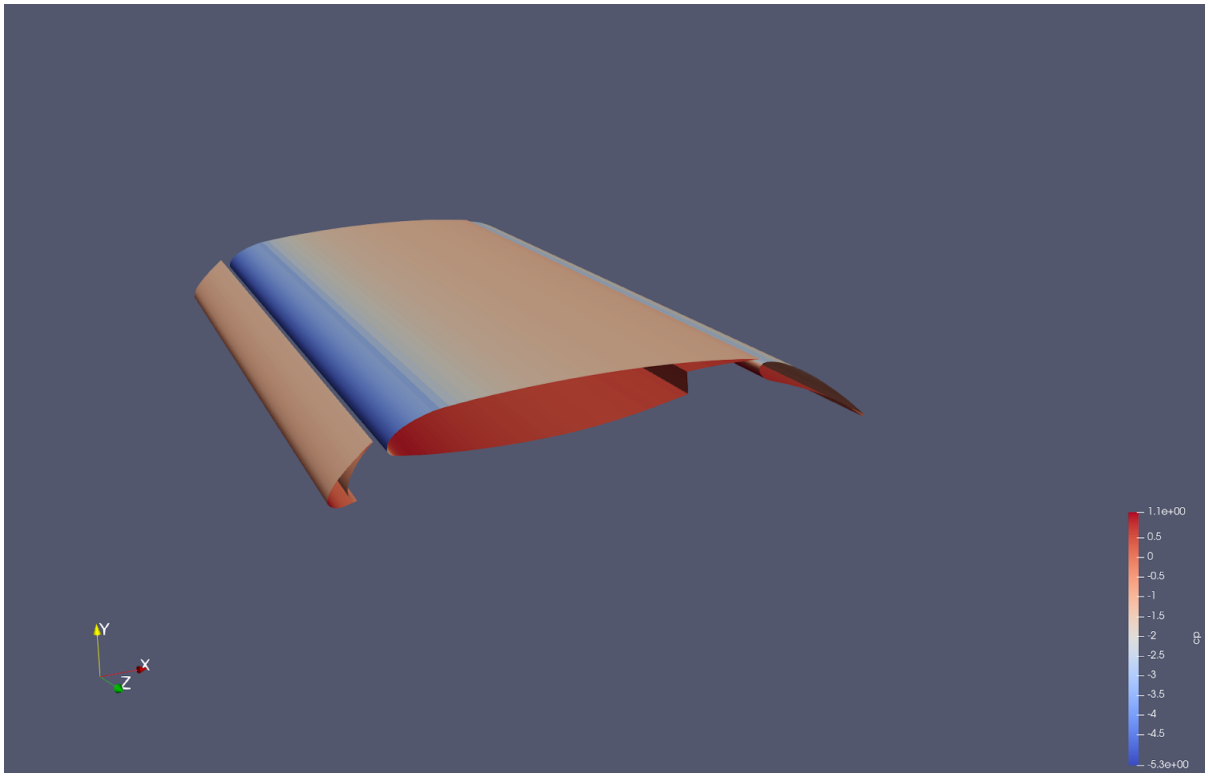




Which show reasonable agreement with experimental results, and additionally that the forces are at least nearing convergence. The grid for this case is still extremely coarse, accounting for the differences in lift and drag coefficients.

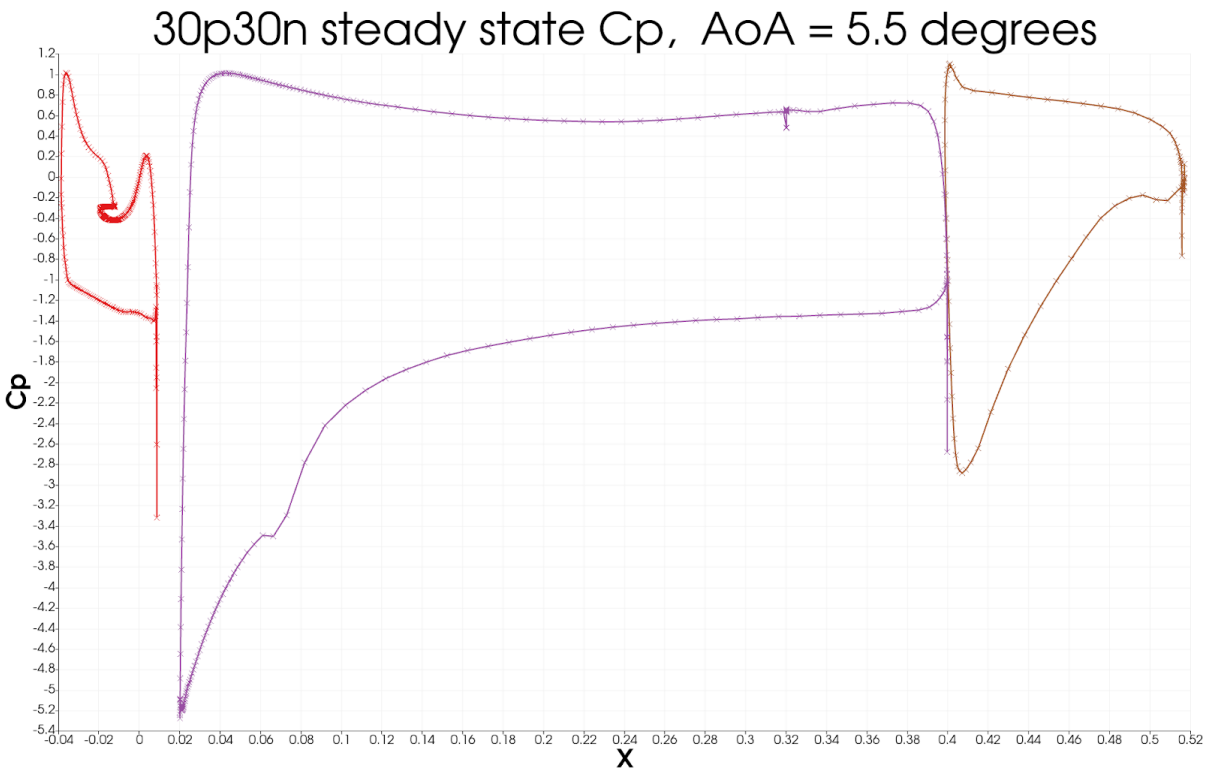
ParaView

Opening ParaView, and loading in the *30p30n_steady_wall.vtkhdf* results will allow you to view the aerofoil surface results:



To get a continuous plot of cp against x/c you need to use a plot over sorted lines filter. To do this:

1. On the surface data apply a *'cell data to point data'* filter.
2. On the *cell data to point data* apply a *'slice'* filter, ensuring the slice uses a z normal, and is centred through the middle of the aerofoil
3. On the *slice* data apply a *'plot on sorted lines'* filter and click apply. This will then bring up a new *'line graph'* view.
4. Make sure to select all 3 segments in the composite data set dialogue box, then ensure only the cp variables are selected in the series parameters. Finally for the X Array name, select *'Points_X'*
5. Modify the style and markers until you are happy with the result.



Citations

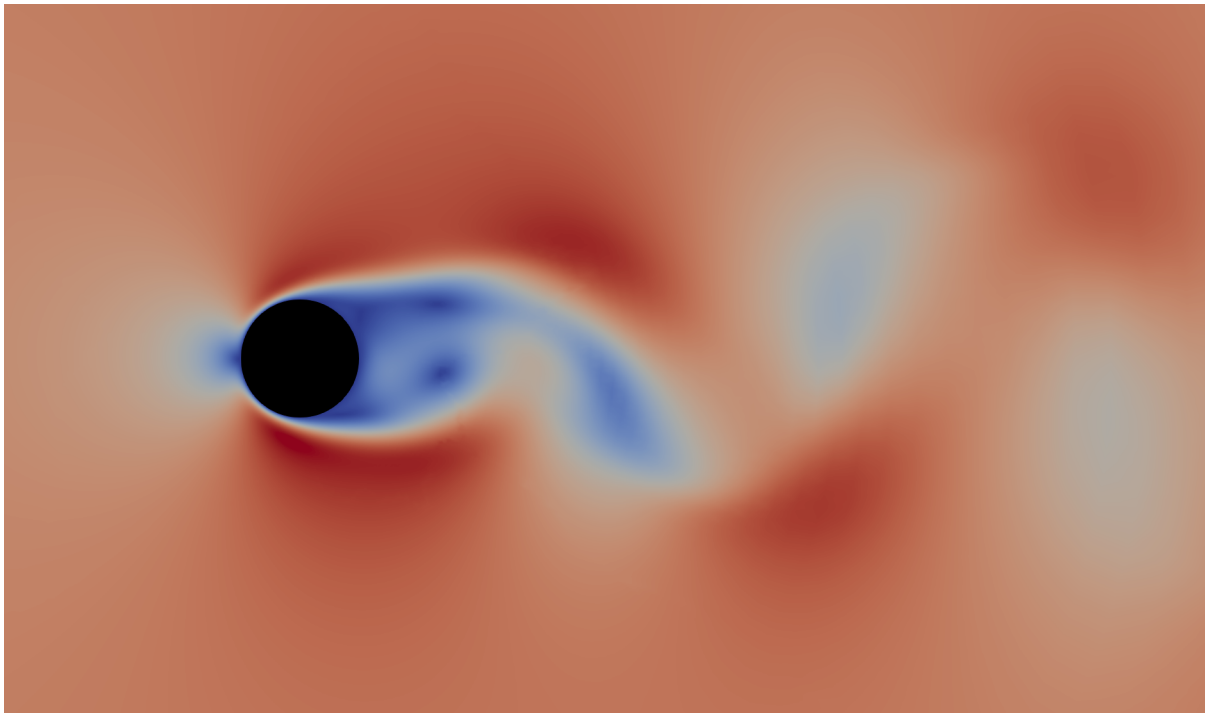
1. Zhang, Yufei & Chen, Haixin & Wang, Kan & Wang, Meng. (2017). Aeroacoustic Prediction of a Multi-Element Airfoil Using Wall-Modeled Large-Eddy Simulation. AIAA Journal. 55. 1-15. 10.2514/1.J055853.

1.4.3 Tutorial 3: Time accurate solution

Table of Contents

Introduction

Following on from executing the steady state simulation of *Tutorial 2*, this tutorial will explore the generation of a time accurate result for the unsteady laminar shedding from a 2D cylinder.



You can download the required files here.

Control Dictionary Breakdown

This case uses a similar control dictionary as the previous 30p30n steady state simulation, with the main difference being the time marching, and output regions of the file.

Time Marching

To switch to a time accurate simulation we need to update the *time_settings* and *convergence_control* blocks. In the steady state *tutorial 2* case, *time_settings* was simply `{"type": "steady"}`, and all of the pacing lived in *convergence_control*.

An unsteady, time accurate run instead declares a physical time layout on *time_settings*: *'total_time'* is set to 1.2, meaning the simulation will be run for a total of 1.2 seconds, and *'time_step'* is set to 0.002 seconds, the interval at which the solver steps time forwards. That gives a total of 1.2 / 0.002 = 600 unsteady timesteps to perform. The *'kind'* key says *how* the solver advances between those timesteps - here *"dual time stepping"* - and it is this *kind* that decides whether *convergence_control* is used at all, since only dual time stepping (and steady runs) have an inner pseudo-time loop to pace.

```
"time settings": {
    "type": "unsteady",
    "kind": "dual time stepping",
    "total time": 1.2,
    "time step": 0.002,
    "order": "second",
    "start": 0,
},
"convergence control": {
    "scheme": {"name": "implicit euler"},
    "cfl": {
        "cfl": 200,
        "cfl viscous factor": 1.0e-6,
        "cfl ramp": [{"type": "exponential", "factor": 1.05, "initial": 0.1}],
    },
    "cycles": 5,
```

(continues on next page)

(continued from previous page)

```
},
```

zCFD uses dual time stepping to advance in real time by first iterating to convergence on an inner steady state 'pseudo time' loop, before advancing the real time cycle. In this case the inner loop is performed using implicit euler time marching, the number of cycles this inner loop performs is controlled by *'cycles'*. So the in the example above the inner loop runs at a CFL of 200 for 5 cycles.

The choice of timestep size, and number of cycles per inner loop will have a significant effect on the quality and cost of the final solution. Using too large of a real time step will mean higher frequency unsteady effects are lost, similarly too few inner cycles will result in unsteady effects being lost. A rule of thumb is to ensure the residuals converge by at least two orders of magnitude over an inner cycle. On the other hand if too many cycles or timesteps are performed, the cost of the simulation will quickly increase. Finally the number of cycles to reach convergence in a pseudo time loop also depends on the step size. Meaning in some cases it is actually beneficial to run a smaller timestep, in order to require fewer inner loop iterations.

Output

The *solution* dictionary, nested under *output_settings*, is a bit more complicated here than in *tutorial 1* or *tutorial 2*.

```
"solution": {
    "format": "vtk",
    "surface variables": ["V", "p", "T", "rho", "cp"],
    "volume variables": ["V", "p", "T", "rho", "m", "cp"],
    "frequency": {
        "volume data": 5,
        "surface data": 5,
        "checkpoint": 20,
    },
},
```

First of all, the *'frequency'* parameter takes on a different meaning in a time-accurate simulation (*tutorial 1* and *tutorial 2* are both steady-state simulations).

In a steady-state simulation, the *'frequency'* parameter controls how the number of pseudo-time *'cycles'* between solver outputs. Since we have no real interest in any data apart from the converged data at the end for a steady-state simulation, every time the solver outputs data, it over-writes the previous output data.

We run time-accurate simulations when we're interested in the variation of the flow over time (e.g. due to vortex shedding), so in the time-accurate solver data outputs are not over-written, and the *'frequency'* variable relates to the number of real time steps between outputs, instead of number of pseudo-time cycles between outputs.

While we can just use a single number for *'frequency'* (e.g. `"frequency": 5`), we can also specify *'frequency'* as a dictionary for more fine grained control. This is especially relevant in time-accurate simulations, where frequent data output can not only slow down the simulation but also quickly use up disk space. Generally the volume data takes longest to output, so for large cases it can be a good idea to output volume data output infrequently, and instead rely on surface data, *monitor points* and *volume intepolated data* for high frequency data collection.

In this, a relatively small case, we have chosen to output surface and volume data every 5 real time-steps. Since the *'time step'* was set to 0.002 seconds, the volume and surface data will therefore be written out every 0.01 seconds of simulated time. We have chosen to output *checkpoint* data (see *here*) less frequently.

Monitor Points

In order to capture the shedding frequencies of vortices behind the cylinder, we will place a monitor point in this region to append data to the report file. A monitor point will simply report the requested output variables, at the point defined within the flow. These results can be visualised alongside the force report in the output notebook. Monitor points are keyed on the report itself, so they nest under *output_settings.report.monitor*.

```
"report": {
    "frequency": 5,
    "monitor": {
        "MR_1": {
            "point": [0.25, 1.07, 0.313],
            "variables": ["cp"],
        },
    },
},
```

Running the case

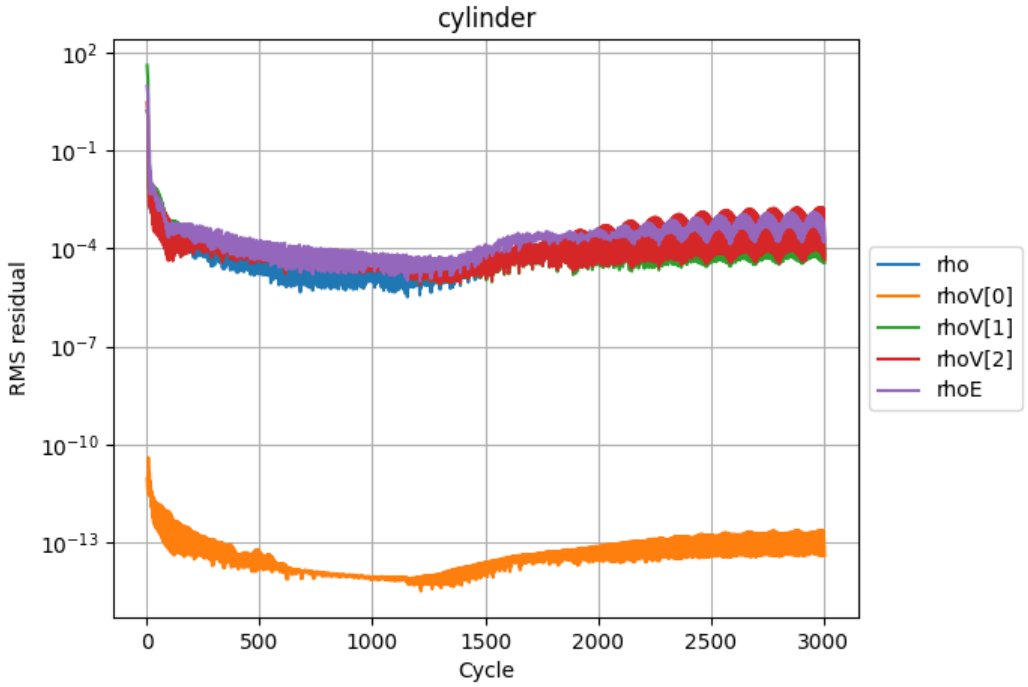
You can run the case as you did for Tutorial 1, but with the following **run_zcfd** command:

```
run_zcfd -m cylinder.h5 -c cylinder.py
```

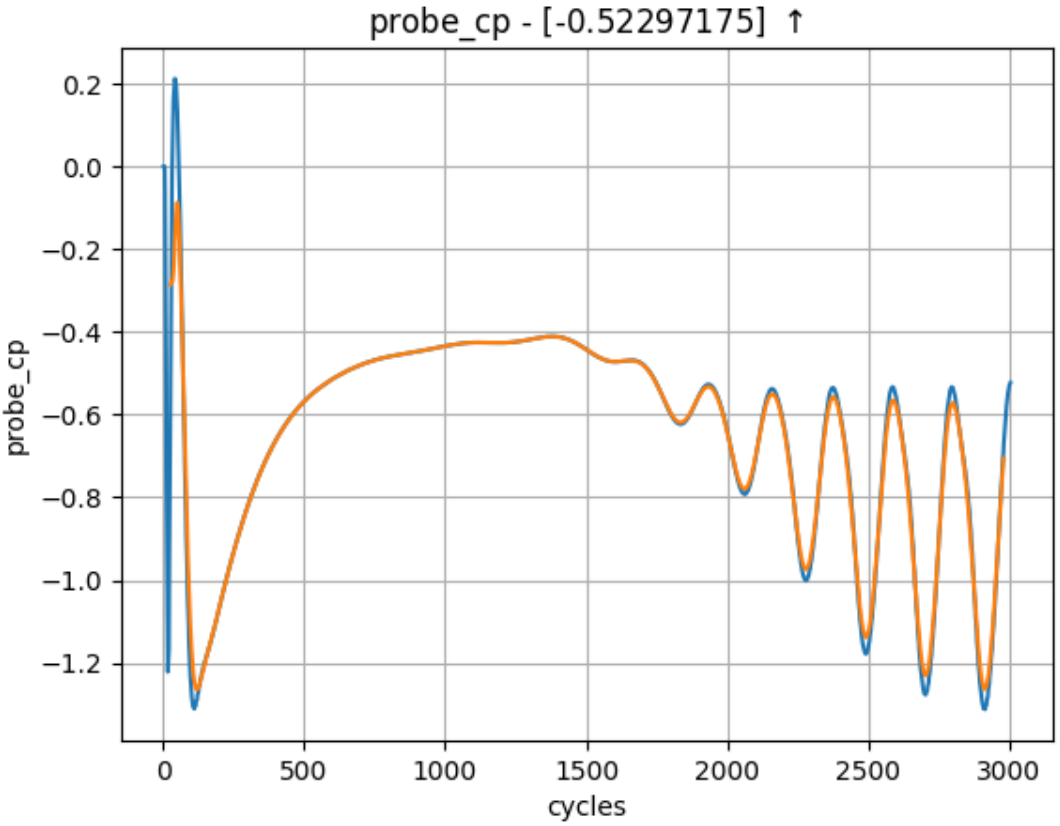
Post Processing

Output report

Start and connect to a Jupyter server, using **'start_lab'** as outlined in the previous sections. Go ahead and run all cells.



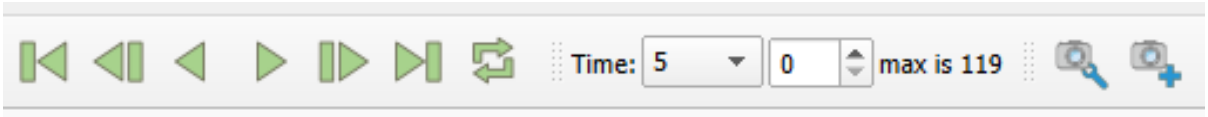
In the "plot_forces" cell, you should notice additional data points available provided by the monitor point. Visualising the pressure trace [*probe_cp*] should reveal an oscillatory pattern, indicating that the predicted vortex shedding is indeed occurring. This trace can then be used to calculate the corresponding shedding frequency.



Flow field

We set the output frequency to 5 when running the case and so the solver will have written visualisation data every 5 real time steps. All timesteps are stored within a single `cylinder.vtkhdf` file in the *'cylinder_P1_OUTPUT/'* directory — no separate per-timestep files are created.

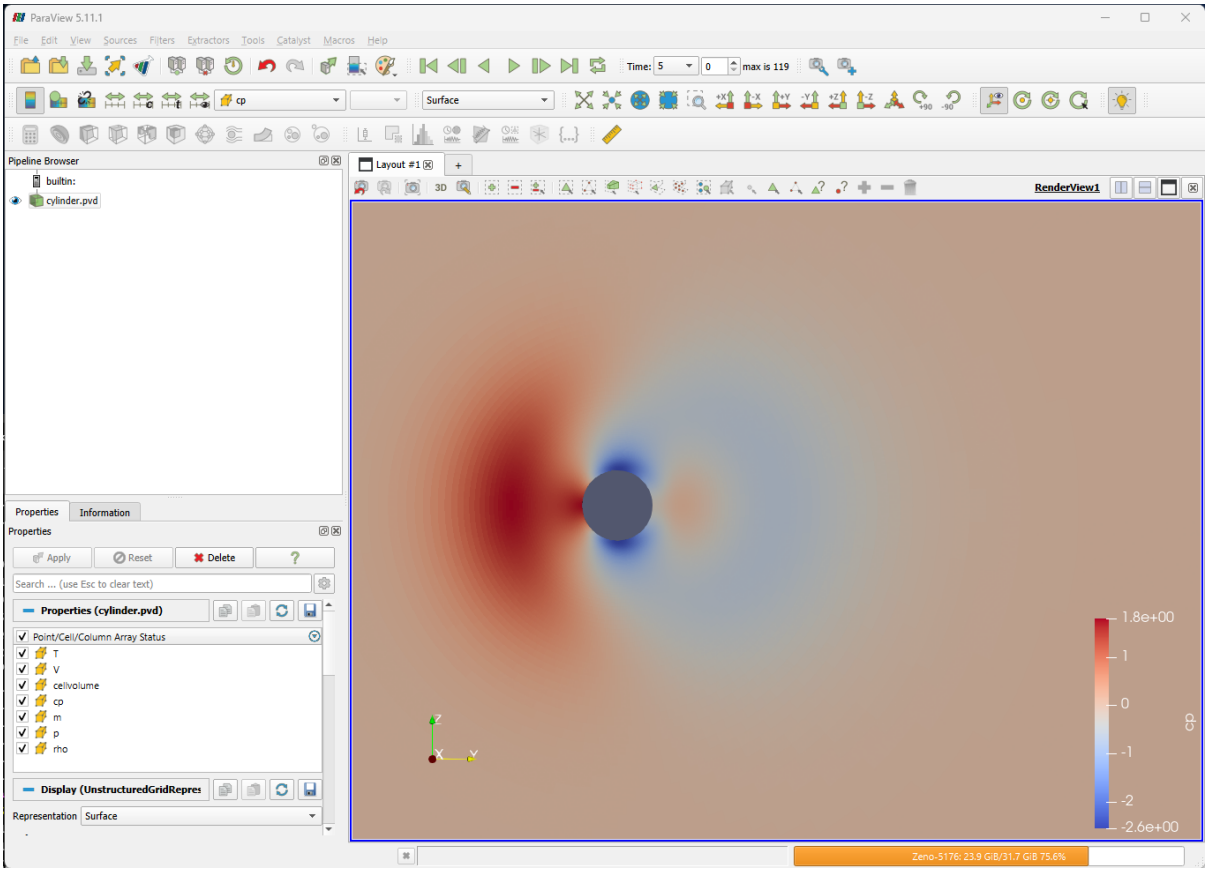
Launch ParaView and load *'cylinder_P1_OUTPUT/cylinder.vtkhdf'*. ParaView will render a single solution time step at a time, and time series data can be navigated using the icons at the top of the screen:



Animation

For unsteady data it is often useful to output an animation of the solution. This can be done in ParaView by either exporting an image per timestep and then using a third party tool to stitch the images into a video or by saving the animation directly. The advantage of the first approach is that it can easily be scripted and is more suitable for large cases. Since this case is quite small we will export an mp4 video directly from ParaView. The steps to do this are:

1. Launch ParaView and load *'cylinder_P1_OUTPUT/cylinder.vtkhdf'*.
2. Set the view up to shade the output by *"cp"* and zoom/move the image until you are happy with the view:



3. Select *'File->Save Animation'* in ParaView. To export as video select your preferred format, the format will depend on your Operating System but for Windows you should be able to select "MP4 files" and on Linux "FFMPEG AVI". Give the file a name and click OK.

4. On the next screen you can set the target resolution, frame rate and the timesteps to export. Set the Image resolution to 1920x1080 (FHD) and the frame rate to 24. If you just want to export the end of the simulation (where the oscillation is established) you can set the Frame Window from 300 to 600.

5. Click OK to export the animation. This will take some time.

You should now have an exported video of the unsteady simulation.

1.4.4 Tutorial 4: FWH Solver

A *Ffowcs-Williams Hawkings (FWH) solver*, which is used for prediction of farfield noise, is included in the zCFD distribution. In this section, we will use the zCFD FWH solver to calculate the sound experienced by two 'observer microphones' as a result of the aerodynamic noise generated by the flow around the cylinder.

In order to predict noise at observer microphone locations, an FWH solver requires an initial unsteady CFD solution. We will base our example on the CFD simulation setup from *Tutorial 3* with some additional modifications to record the flow variables on an 'FWH surface' in the near-field, noise generating region. We then use the FWH solver to propagate the noise from the near-field FWH surface to the observer microphones.

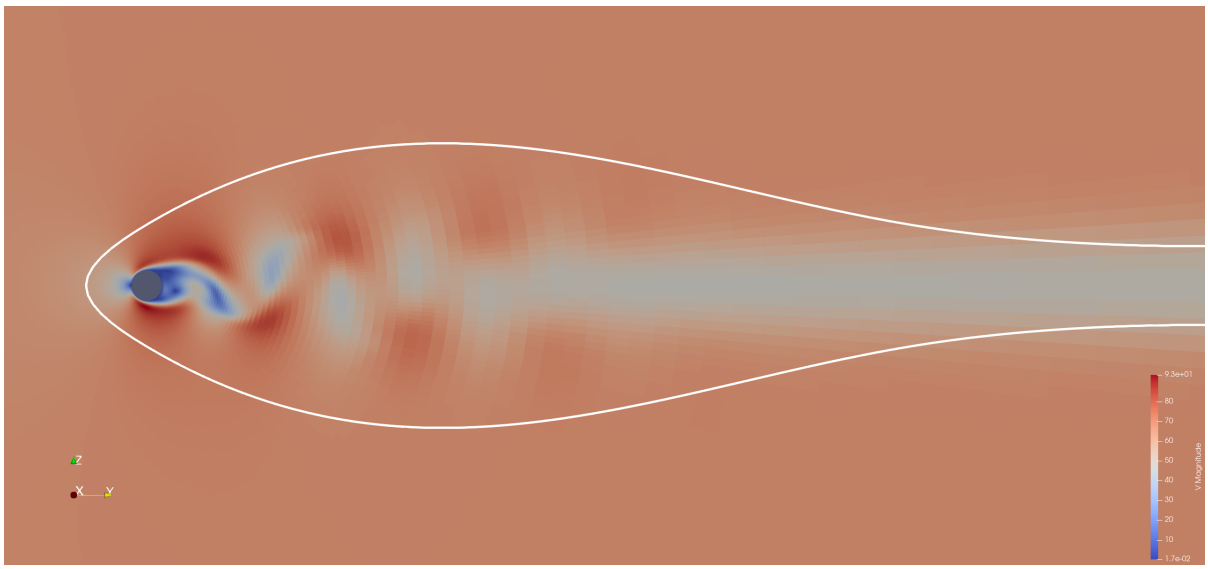
The FWH solver can be run in two ways - the *run_fwh utility* and *fwh python module*. In this tutorial, the simpler *run_fwh utility* is introduced first, followed by the *fwh python module* and postprocessing using a *jupyter notebook*.

Running zCFD with FWH output

The flow we're going to be running the FWH solver on is the shedding cylinder. To get the required inputs to our FWH solver, we need to define an *FWH surface*, and collect data on this surface during a CFD simulation.

The files required for the fwh tutorial can be downloaded here. In the zip file there's a FWH surface definition file, called *fwh_surface1.stl*. Given we've already run the cylinder case in *Tutorial 3*, we can view the FWH

surface in ParaView alongside the cylinder CFD volume data (see *here*), as below. You may find the surface easier to see if you set the *view style* to be *Feature Edges* instead of *Surface* for the FWH surface, and increase the *Line Width* in the *Geometry Representation*.



The FWH surface encloses the noise generating near-field region, but does not intersect the wake, in order to avoid spurious pseudo-sound [*1*, *2*] .

There's also a prepared control dictionary, *cylinder_fwh.py*. This is identical to tutorials 3's *cylinder.py* apart from the output section, which is detailed below.

```
"output settings": {
    "solution": {
        "format": "vtk",
        "surface variables": ["V", "p", "T", "rho", "cp"],
        "volume variables": ["V", "p", "T", "rho", "m", "cp"],
        "fwh interpolate": ["fwh_surface1.stl"],
        "frequency": {
            "fwh interpolate": 1,
            "volume data": 1000,
            "surface data": 1000,
        },
    },
},
```

This `output_settings.solution` block sits under the `solver` key of the control dictionary, alongside `equations`, `convergence_control` and the other *solver settings* inherited from *cylinder.py*. In this case, we have chosen to output *fwh_interpolate* data every single timestep, using the *fwh_surface1.stl* FWH surface definition file. Volume and surface data is much less frequent, since we already have all this data from when we ran *cylinder.py* *above*.

For the FWH tutorial we have also updated the `solver.time_settings` block to run the simulation for 4 seconds, rather than 1.2. We run the CFD simulation in the normal way, using the command below.

```
run_zcfd -m cylinder.h5 -c cylinder_fwh.py
```

When we've run the CFD simulation of the cylinder, we should be able to see the FWH interpolated data in *'cylinder_fwh_OUTPUT/ACOUSTIC_DATA/fwh_surface1_FWHData.h5'*. There is also a corresponding *xdmf* file, *'cylinder_fwh_OUTPUT/ACOUSTIC_DATA/fwh_surface1_FWHData.xdmf'*, which allows us to open the FWH surface in ParaView and see the pressure, velocity and density at each cell centre varying with simulation time. Use the ParaView *Time* controls to adjust the simulation time being viewed. You may find that changing the *view style* to *Point Gaussian* and increasing the *Gaussian Radius* in the *Properties* sidebar makes the cell values easier to see.

Running the zCFD FWH solver

Using the run_fwh utility

We're going to use the *run_fwh* utility to calculate noise at an observer *Obs1*. *'Obs1'* is an observer that is a fixed distance away from the cylinder - if we consider the cylinder in a wind tunnel, we could consider *'Obs1'* as a microphone attached to the wind tunnel wall. Since *Obs1* is static relative to the FWH surface, it is in the same *translational reference frame* as the FWH surface and we can therefore use the *run_fwh* utility.

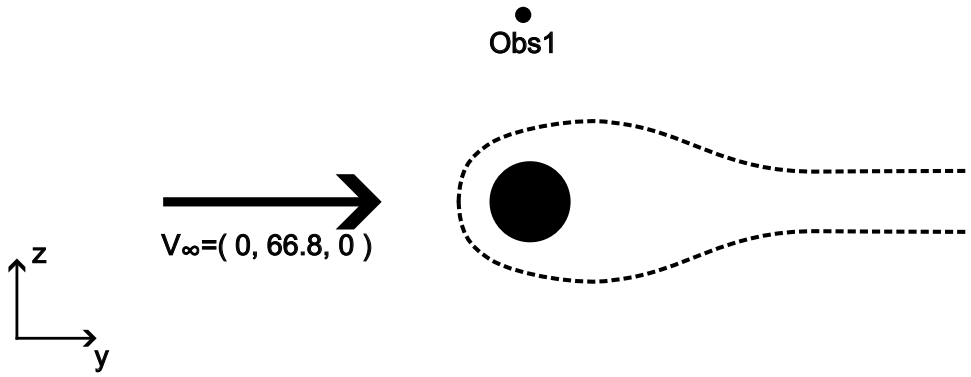


Fig. 1.1: FWH surface and *Obs1* motions - CFD frame of reference

To use the *run_fwh* utility we need three inputs - a *zCFD control file*, a *reference condition* specification and an *observers .csv file*.

The zCFD control file is simply the control file for the zCFD simulation we used to create the FWH surface data - in this case, *cylinder_fwh.py*. *run_fwh* reads *cylinder_fwh.py* and can see that we have a single FWH surface, with data stored at *'cylinder_fwh_OUTPUT/ACOUSTIC_DATA/fwh_surface1_FWHData.h5'*.

Next, we have to define a *reference condition*. In external flow simulations of this type, the *run_fwh reference condition* is the freestream conditions, defined as a zCFD *reference condition block* under `solver.reference_conditions`. In this case, the freestream conditions are already defined in *cylinder_fwh.py* to define the farfield boundary condition, so we can simply use *'IC_1'* for the *run_fwh* reference condition.

Lastly, we need to define the name and location of our observers in a `.csv` file. The centre of the cylinder in the CFD mesh is at (0.25,0,0), and we want *Obs1* to always be (0,0,20) metres away from the cylinder centre during the FWH simulation. The file `Observers.csv` (included in the tutorial *.zip file*) therefore contains the following:

```
Obs1, 0.25, 0.0,  20.0
```

Having decided on our inputs, we run the *run_fwh* utility by simply typing the following command within the *zCFD command-line environment*.

```
run_fwh -c cylinder_fwh.py --reference_condition IC_1 --observer_locations Observers.
↪csv
```

Running this command prints output to screen and to the *cylinder_fwh.fwh.log* file, and outputs the FWH calculated pressure history at *Obs1* to *cylinder_fwh.fwh.Obs1.csv*. In the case where there are multiple noise sources, the contribution of noise to each source is broken out, with the total observed noise also reported. Suggested post-processing is introduced *below*.

Note

The `run_fwh` command-line interface takes a zCFD control file, a reference-condition name and an observers `.csv` file, whatever the control file's `config_version`. The FWH tools read the control dictionary's raw parameters directly, rather than through the same schema-validated path as the solver: the `run_fwh` helper module has not been updated for the current control-file format and references a class that no longer exists in `zutil.fileutils`. Check your local installation's release notes for the current status of these tools.

Using the fwh python module

As can be seen above, the *run_fwh* utility is very easy to use and is applicable in most simple FWH calculations. However, it has limitations - namely the fact that all observers and surfaces must be in the same *translational reference frame*, and that using it when more than one FWH surface encloses a noise generating region will *lead to double accounting*.

Where the *run_fwh* utility is not applicable, the *fwh python module* can be used instead. In this section, we will use the *fwh python module* to calculate observer pressure histories at both *Obs1* and *Obs2*, where *Obs2* is a 'flyby' observer whose pressure history cannot be calculated using the *run_fwh* utility.

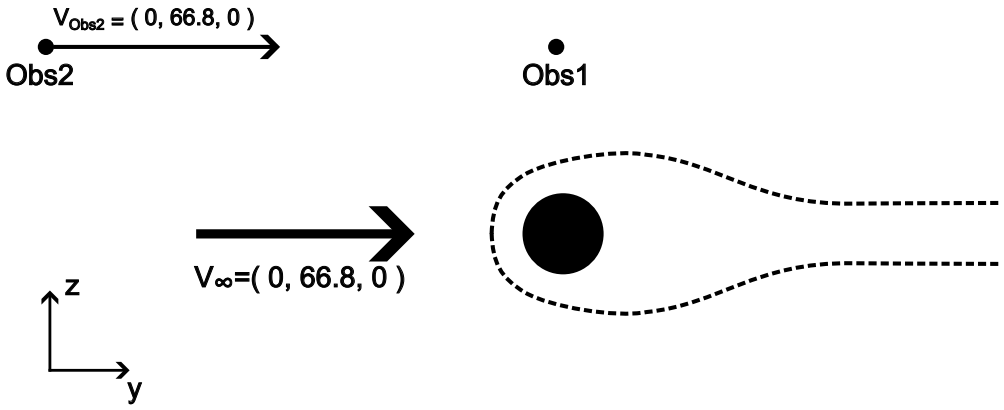


Fig. 1.2: FWH surface and observer motions - CFD frame of reference

FWH Observer and surface motions

When the FWH equations are derived, we make an important assumption - that of a 'quiescent medium' (i.e. still air, with a free-stream velocity of zero). Clearly, with a freestream velocity of *(0,66.8,0)*, the flow in our CFD simulation does not satisfy this condition. In order to use the FWH solver, we therefore have to perform a co-ordinate transformation, as in Fig. 1.3. In the *run_fwh* utility, this co-ordinate transformation is performed automatically, whereas in the *fwh python module* we set up the surface and observer motions through the FWH quiescent medium manually.

In this 'quiescent medium' co-ordinate system, the cylinder and *'Obs1'* are 'flying' through the air. In this co-ordinate system, we can also see that *'Obs2'* represents a microphone fixed to the ground, as the cylinder flies overhead.

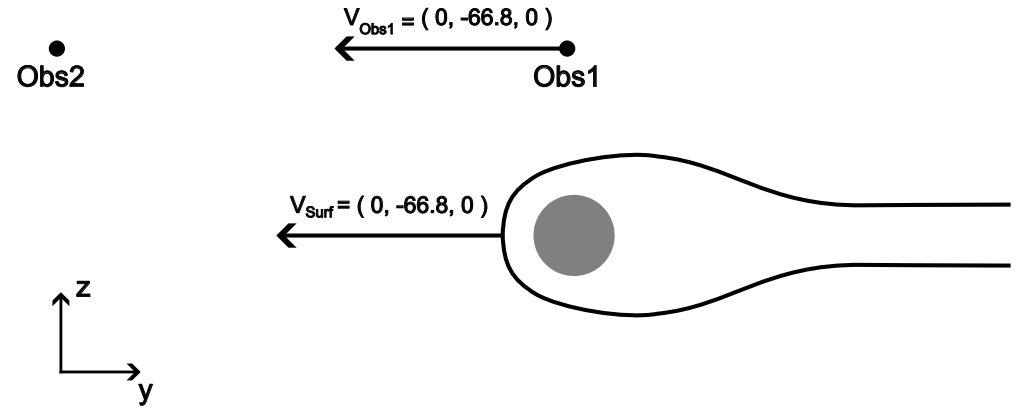


Fig. 1.3: FWH surface and observer motions - FWH (quiescent medium) frame of reference

Setting up the FWH solver inputs

Now that we know the motions of our observers and surface, we can run the FWH solver. The python code required to run the FWH solver is in the file *'run_fwh_solver.py'*, and also written out below. We define the surfaces, observers and the solver settings, then use `fwh.solve()` to actually run the FWH solver.

```
from fwh import fwh, motion
import json
import math

# FWH surface motion
c  = math.sqrt(1.4 * 277.7777 * 287)
v_surf = [0, - 0.2 * c, 0] # = [0,-66.8,0]
surfaceCentrePoint = motion.ConstantVelocityPoint([0.0, 0.0, 0.0], v_surf)
surfaceMotion = motion.NonrotatingSurface(surfaceCentrePoint)

# obs1 motion - [0,0,20] above cylinder centre
obs1_CFD_frame_posn = [0.25,0,20]
obs1Motion = motion.ConstantVelocityPoint(obs1_CFD_frame_posn, v_surf)

# obs2 motion - stationary, intersects with obs1 at t=2.5
obs2_posn = []
for i in range(3):
  obs2_posn.append( obs1_CFD_frame_posn[i] + 2.5* v_surf[i])
obs2Motion = motion.StationaryPoint(obs2_posn)

solverSettings = {
    "c": c,
    "dt": 0.002,
    "rho0": 101325.0 / (277.7777 * 287),
    "p0": 101325.0,
}

surfaces = {
```

(continues on next page)

(continued from previous page)

```
    "surf1": {
        "motion": surfaceMotion,
        "fileName": "./cylinder_fwh_OUTPUT/ACOUSTIC_DATA/fwh_surface1_FWHData.h5",
    }
}
observers = {"Obs1": obs1Motion, "Obs2": obs2Motion}

pOut, tOut = fwh.solve(surfaces, observers, solverSettings)

dataForJson = {"p": pOut, "t": tOut}

with open("./FWH_data.json","w") as f:
    json.dump(dataForJson,f)
```

More details on FWH solver setup can be found *here*, but the most important concept is that we use *'motion classes'* to define observer and surface motions. The available *point (observer) motions* are *'OriginPoint'*, *'StationaryPoint'* and *'ConstantVelocityPoint'*. The available *surface motions* (which rely on a *point motion* to define the motion of their centre) are *'NonrotatingSurface'* and *'RotatingSurface'*.

The velocities and fluid properties used in the FWH simulation are taken from the *'cylinder.py'* zCFD control file. `solverSettings['dt']` has been set to be the same as the timestep between outputs in the FWH surface data (0.002 seconds, see *above*).

In general, we can consider the motion of a point on an FWH surface to be defined by $\mathbf{x}(t) = \mathbf{x}_0 + \mathbf{V}_{surf}\ t$ in the FWH co-ordinate frame. $\mathbf{V}_{surf}$ is defined by the user in the *surface motion class* (called *surfaceMotion* above) and $\mathbf{x}_0$ is the position defined by the FWH surface definition file (*'fwh_surface1.stl'*, see *above*).

The centre of the cylinder in the CFD mesh is at (0.25,0,0), and we want *Obs1* to always be (0,0,20) metres away from the cylinder centre during the FWH simulation. Since the cylinder 'travels with' the FWH surface (see Fig. 1.3), we can therefore say that the motion of the cylinder centre in the FWH co-ordinate frame is $\mathbf{x}(t) = ((0.25,0,0) + (0,0,20)) + \mathbf{V}_{surf}\,t$. We therefore set the motion of *Obs1* to be $\mathbf{x}(t) = (0.25,0,20) + \mathbf{V}_{surf}\,t$. We set *Obs2* to be stationary in the FWH reference frame, and to be coincident with *Obs1* at *t=2.5*. We save the outputs to a json file called *FWH_data.json*.

Running the FWH solver

The python code above is replicated in *'run_fwh_solver.py'*. To run the FWH solver, we simply run the command below from within the *zCFD command-line environment*.

```
python3 run_fwh_solver.py
```

This will output various information to the screen, including the 'valid min and max times' for each surface / observer combination.

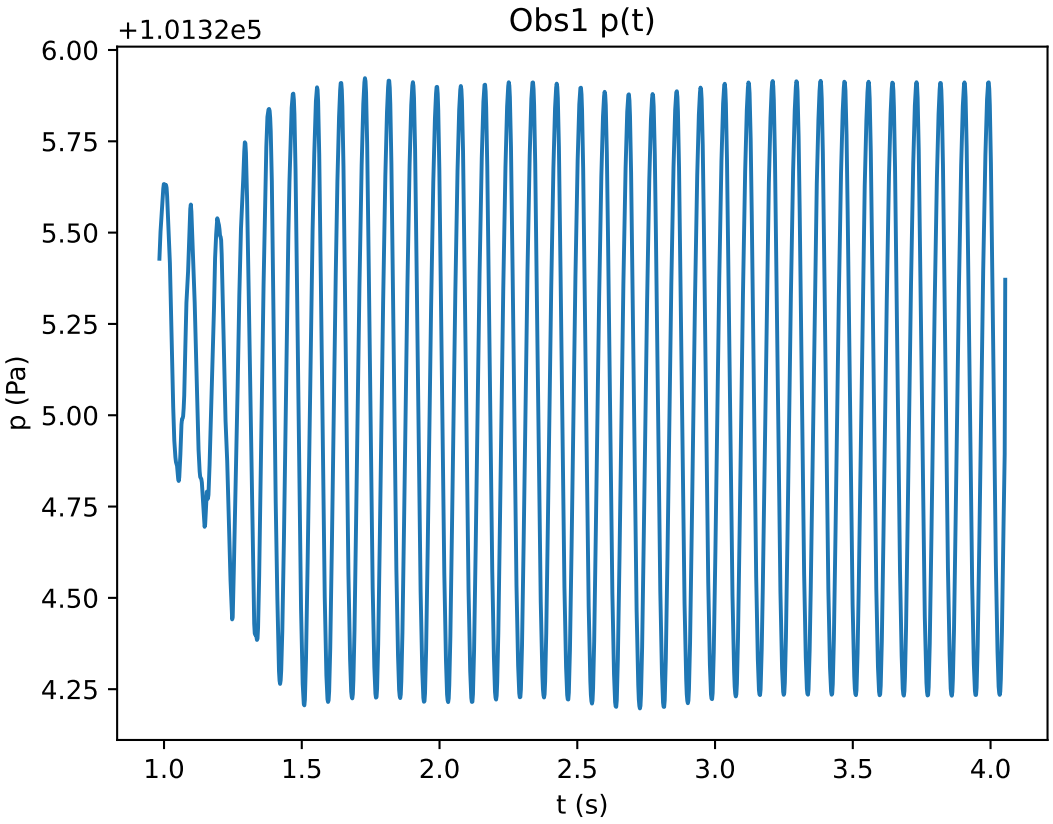
The valid min and max times denote the time window during which an observer receives data from all points on the FWH surface. A pressure signal emitted from a point on the FWH surface at time *t* takes *r/c* seconds to arrive at an observer, where *r* is the distance from the surface point to observer and *c* is the speed of sound. Therefore the valid observer time window will always be later than the FWH surface output time window (in our case, 0.002:1.2 seconds) due to the distance from the FWH surface to the observer. The valid observer time window will be narrower if the FWH surface is large, since the larger the surface the larger the time delay between signals from different points on the surface reaching the observer becomes.

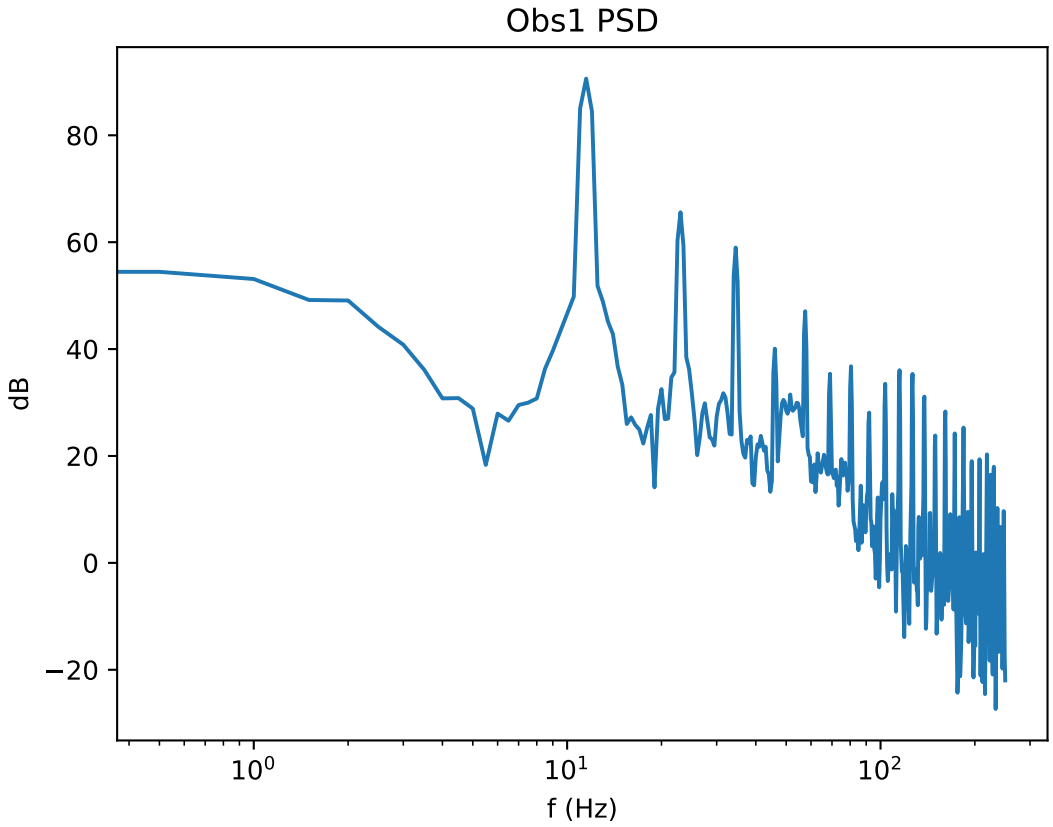
Post-processing

The *jupyter notebook* *postprocess_fwh_module_data.ipynb* from the *.zip file* can be used to post-process the FWH data. *postprocess_fwh_module_data.ipynb* uses the data from the *fwh python module section* of this tutorial, but an equivalent notebook which uses the data from the *run_fwh utility section* is available in *postprocess_run_fwh_data.ipynb*.

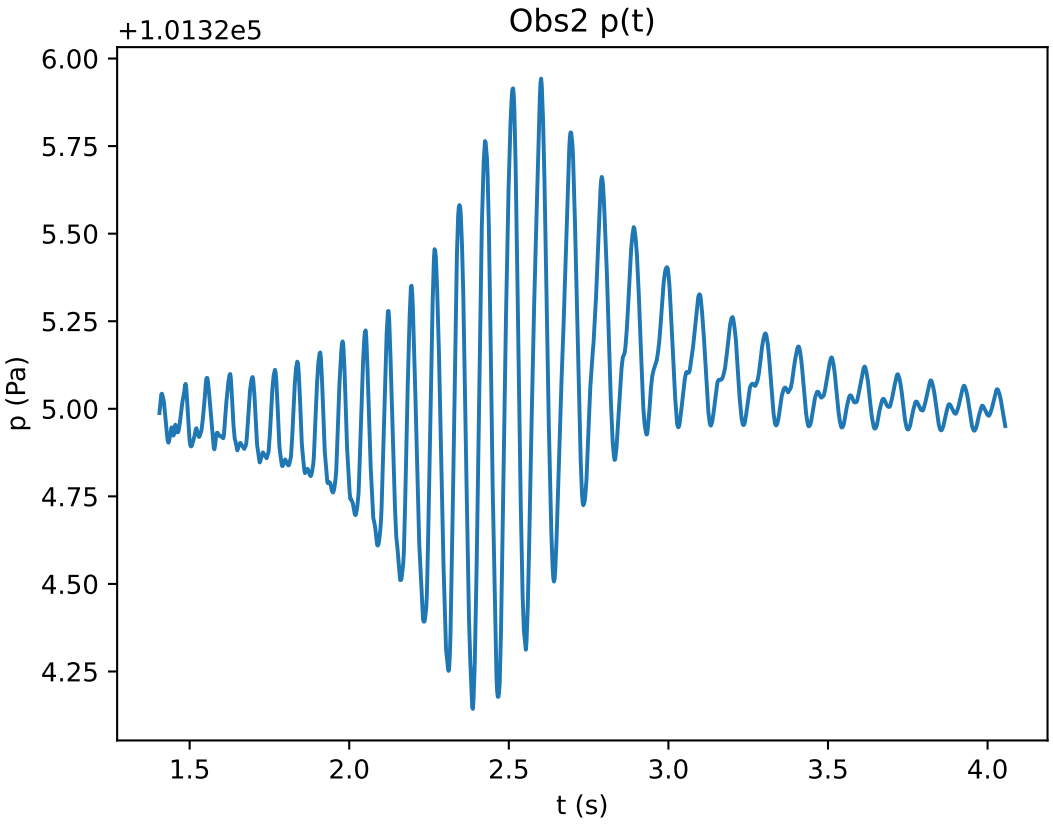
These notebooks contain a number of *zutil* functions which may be useful for users wishing to perform acoustic post-processing. The notebooks plot the pressure history *p(t)* and power spectral density (PSD) at both observers, as below.

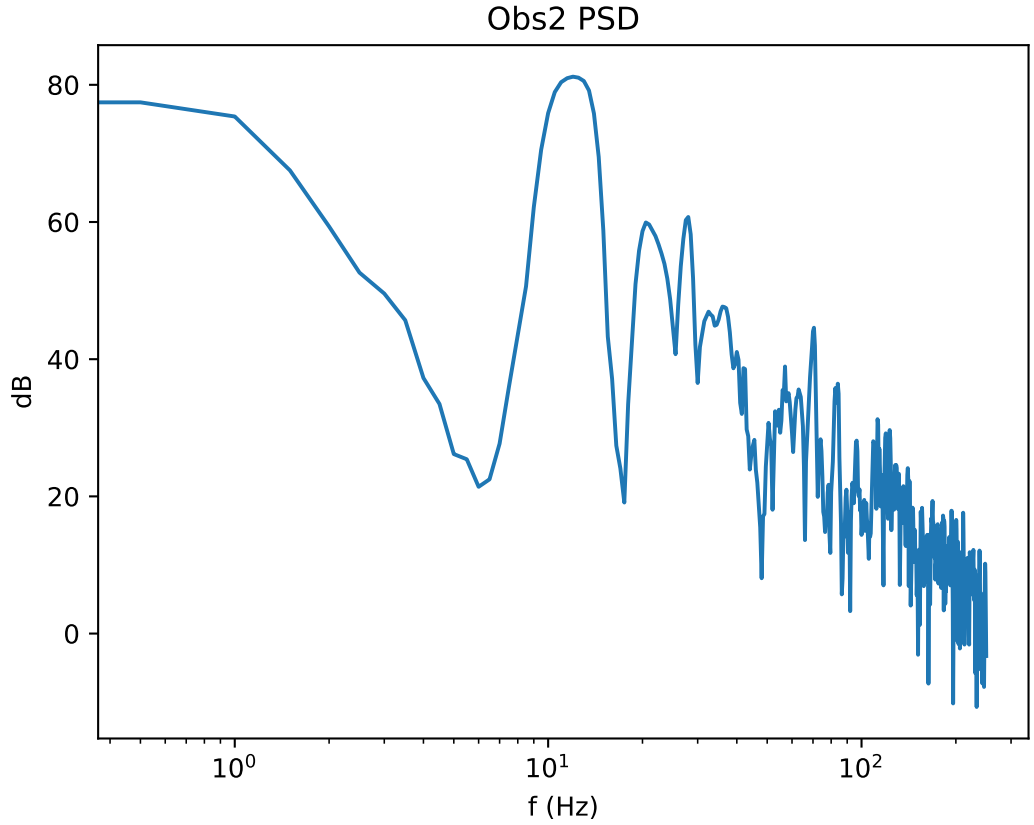
Obs1 moves through the FWH medium at a constant displacement from the cylinder centre (see Fig. 1.3). We see from our post-processing that the noise experienced by *Obs1* is dominated by a constant 10 Hz component, which is the shedding frequency of the cylinder. Various harmonics of the 10 Hz signal are present in the power spectral density plot for *Obs1*.





During the FWH simulation run in the *fwh python module section* of this tutorial, the cylinder gets closer to *Obs2* from 0 to 2.5 seconds, before getting further away from *Obs2* from 2.5 seconds onwards. We can see in the *Obs2 p(t)* plot that the peaks are closer together from 0 to 2.5 seconds than they are after 2.5 seconds. This is due to Doppler shift. We can also see the effect of Doppler shift in the power spectral density, which has a more rounded 10 Hz peak than that seen in *Obs1*.


Obs2 p(t)
+1.0132e5
6.00
5.75
5.50
5.25
5.00
4.75
4.50
4.25
p (Pa)
1.5
2.0
2.5
3.0
3.5
4.0
t (s)


Obs2 PSD
80
60
40
20
0
dB
10^0
10^1
10^2
f (Hz)

postprocess_fwh_module_data.ipynb also creates *.wav* files, which allow you to listen to the sound experienced by *Obs1* and *Obs2*.

Extensions

To become more comfortable with the FWH solver, various extensions to this exercise are possible. Here are some suggestions.

1. Re-run the zCFD solver, but this time extract FWH data on the cylinder *wall* instead of *fwh_surface1.stl*. Run the FWH solver (using the wall data as an *impermeable* FWH surface) and compare the results to those achieved using the *fwh_surface1* permeable FWH surface.
2. Re-run the zCFD solver, but this time also extract FWH data on the *fwh_surface2.stl* and *fwh_surface3.stl* surfaces. These are like *fwh_surface1.stl* but slightly larger. Run the FWH solver with these surfaces and investigate the effect of using different surfaces on the FWH results. You will have to do this using the fwh python module to avoid *double accounting*.
3. Add extra observers of your choice to the FWH solver.
4. Use the file utility *writeFWHSurfaceFrequenciesFile* to visually inspect the spatial distribution of different frequencies in the FWH surface.
5. Use the file utility *writeBlankedOffFwhSurfaceFile* to 'blank off' all FWH surface cells more than 5 cylinder diameters downstream of the cylinder's centre. Run the FWH solver using this modified surface, and based on this estimate how much of the shedding noise originates more than 5 diameters downstream of the cylinder.
6. Use the *solver.fwh_helper_functions* module to rewrite *run_fwh_solver.py* so that the *solverSettings* dictionary is automatically set based on the zCFD control file name and reference condition which were used in the *run_fwh* utility.

Note

Since the cylinder CFD simulation is '2.5D' (i.e. the mesh only has a single cell in the x direction), the distance scaling of $p(t)$ at observers will be non-physical.

Citations

1. Mitchell, B., Lele, S., & Moin, P. (1999). Direct computation of the sound generated by vortex pairing in an axisymmetric jet. Journal of Fluid Mechanics, 383, 113-142. 10.1017/S0022112099003869
2. Ricciardi, T. R. , Wolf W. R, & Spalart P. R. (2022). On the Application of Incomplete Ffowcs Williams and Hawkings Surfaces for Aeroacoustic Predictions. AIAA Journal, 60:3, 1971-1977. 10.2514/1.J061285

1.4.5 Tutorial 5: Overset Meshes

There are various situations in CFD modelling when it is useful to be able to place one mesh over the top of another one, effectively blanking out the cells in the lower level mesh in favour of the mesh on top, while maintaining solution accuracy across the interface.

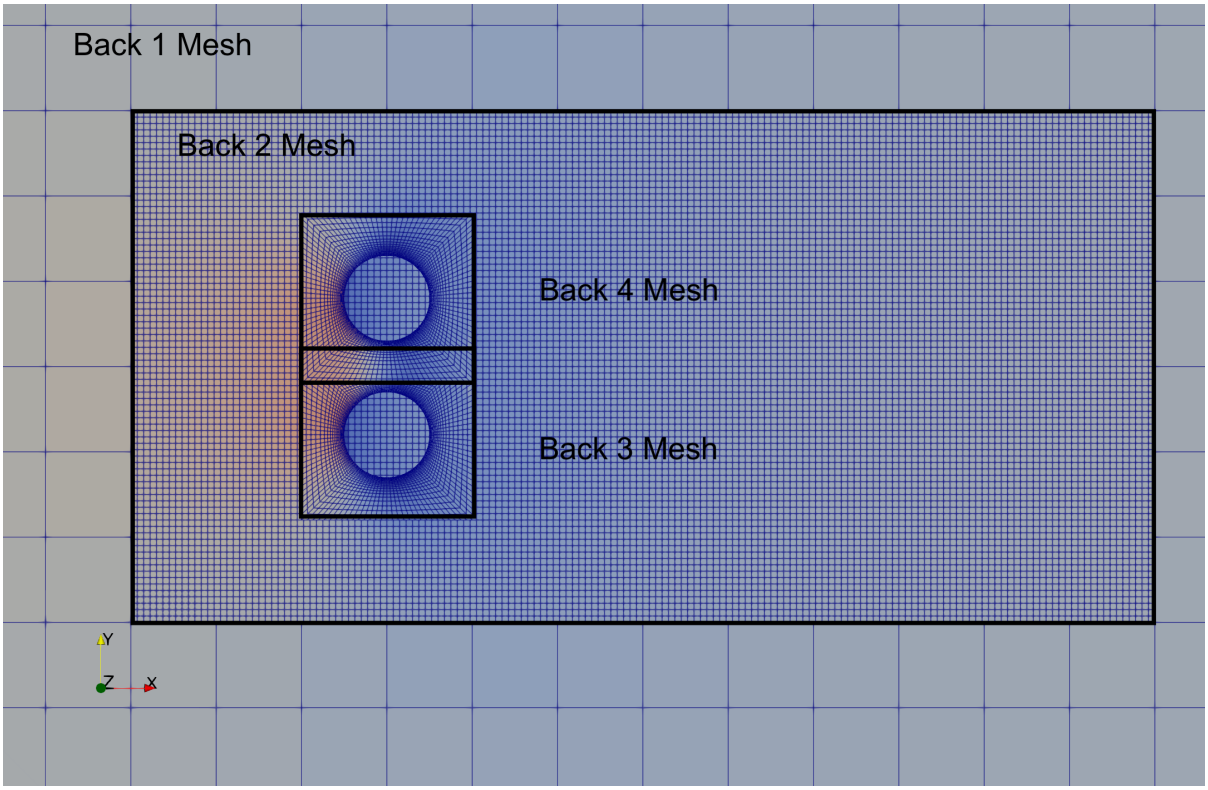
- We may want a higher order of solution accuracy in a region of the original finite volume mesh. The overset mesh would be designed for a higher order simulation than elsewhere in the flow domain.
- We may want to include a component in a simulation that is not part of the original mesh. For example, we may want to add a rotating component to a static mesh. The overset mesh would be designed for the rotating component, and would be placed over the static mesh.
- A specific mesh may already exist for a rotating component (such as a propeller) that we want to be able to easily add to models of different aircraft without having to re-create the propeller mesh. The propeller meshes (there may be several) are overset on the aircraft mesh, allowing for a fully coupled solution.
- There may be components in relative motion, such as a train entering a tunnel. In this case the mesh for the train can overset the background mesh for the tunnel, and the train can move into the tunnel

without the need for any new mesh generation. The solver takes care of the changing mesh mapping across the boundary as the train moves, including preservation of the solution accuracy with a region of overlapping meshes.

- There may already be a high quality mesh for an aircraft or a cityscape, and we want to include a new component or a new building without regenerating the entire mesh. It may be simpler to include any changes into a new mesh of a small region, and apply it like a patch as an overset mesh.

This is called an overset mesh technique, and is supported by zCFD. In all cases, the flow solutions are calculated separately on each mesh, with the solver managing the interfaces and interpolation accuracy. zCFD will support different solution methods, different orders of spatial accuracy on each mesh, and arbitrarily large numbers of overset meshes in each simulation. Each overset boundary condition can name the mesh(es) it takes donor data from, and the solver uses this to work out which mesh is active in each part of the flow domain. zCFD automatically manages the data exchange between all meshes, including parallel partitions.

Example 1 - Multiple Overset Cylinder Test Case



There are 4 meshes involved in this case. A single control dictionary called “oversetmulticylinder.py” declares one named entry per mesh under its top-level `model` key, each entry carrying its own `mesh` path:

```
"model": {
    "background": {"mesh": "back1.h5", ...},
    "refinement_region": {"mesh": "back2.h5", ...},
    "cylinder_2": {"mesh": "back3.h5", ...},
    "cylinder_1": {"mesh": "back3.h5", ...},
}
```

Note that in this case, the “back3.h5” mesh defining a cylinder has been used twice, once for “cylinder_1” and once for “cylinder_2”. This is OK, because the “cylinder_1” and “cylinder_2” model entries each apply a different transformation (down and up along the y-axis respectively). The solver knows that the meshes are used for different solutions, in different parts of the flow domain. Also note that the translated mesh for “cylinder_1” overlaps the translated mesh for “cylinder_2” (the mesh between the two cylinders). This is also OK because each model entry’s overset boundary condition explicitly names its own background mesh - the solver builds “cylinder_1” and “cylinder_2” oversetting “refinement_region”, which oversets “background”. In

each case, the active solution is the one on the top-level oversetting mesh.

The files for running the case are available for download here.

For each of the meshes, a transformation has been defined under that mesh's `transforms` key, for example the "background" entry (the "back1.h5" mesh) is moved according to the transformation defined by the following lines:

```
"transforms": {
    "transform matrix": zutil.transform.translate(
        [0.0, 0.0, 0.0], [0.0, 0.0, -1.5]
    )
},
```

This tells the solver to use the inbuilt transformation function in zutil to translate the mesh down in the z-axis by 1.5 units. The lowest level mesh (i.e. that does not overset any other meshes) does not require any other modification, providing that the order of solution spatial accuracy supported by the meshes that overset it are that same as its own accuracy. For all of the meshes that overset at least one other mesh, a special boundary condition must be applied to any zone (boundary) where information is to be exchanged between meshes. For example, in the "cylinder_2" entry the following line is used:

```
"BC_2": {"zones": [13], "type": "overset", "background": ["refinement_region"]},
```

For the "cylinder_2" model (the "back3.h5" mesh, translated down in y), zone 13 (comprising boundary faces excluding symmetry planes and the cylinder surface) is designated as an overset boundary, and its `background` field names "refinement_region" as the donor mesh it exchanges data with. Naming the background mesh explicitly like this is the preferred form: it tells the solver exactly which mesh pairs need a mapper built, rather than falling back to checking every preceding mesh in the model dictionary for overlap. The software automatically calculates the necessary mapping and interpolation coefficients to preserve the spatial accuracy of the overset solution.

The "Multiple Overset Cylinder" test case runs in a few minutes on a GPU, with the simple command:

```
run_zcfd -c oversetmulticylinder.py
```

The usual command line arguments specifying the mesh are not required (and are ignored if given) - all four meshes are read from the `mesh` key of each entry under `model` in "oversetmulticylinder.py".

When complete, the following outputs should be produced:

```
oversetmulticylinder_OUTPUT/background/
oversetmulticylinder_OUTPUT/refinement_region/
oversetmulticylinder_OUTPUT/cylinder_1/
oversetmulticylinder_OUTPUT/cylinder_2/
oversetmulticylinder.log
oversetmulticylinder_status.yaml
```

Each of the four model subdirectories under "oversetmulticylinder_OUTPUT" contains that mesh's own report, checkpoint and volume/surface solution files (e.g. "cylinder_1/cylinder_1_report.csv", "cylinder_1/cylinder_1.vtkhdf"), named after the model's key in the control dictionary rather than the mesh filename. From a post-processing perspective, overset grids offer a significant advantage: independent solutions that only exchange boundary data during computation. This independence can save post-processing time and memory, as not all solution files are needed to analyse specific regions. However, a separate step is necessary to combine these independent solutions for domain-wide processing, such as extracting cross-sections. ParaView can be used to illustrate this combination process.

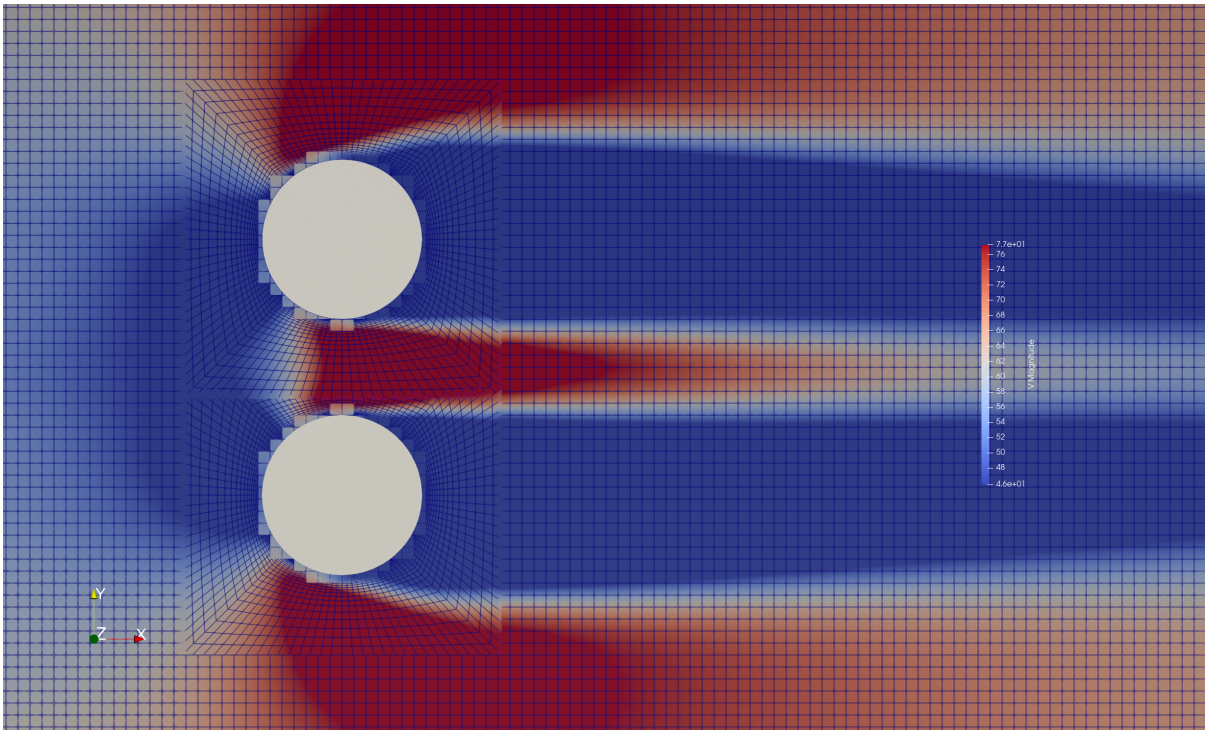
Post-processing using ParaView

To combine the 4 overset meshes into a single solution that can be operated on as though it were a single object, we use the "Append Datasets" filter in ParaView.

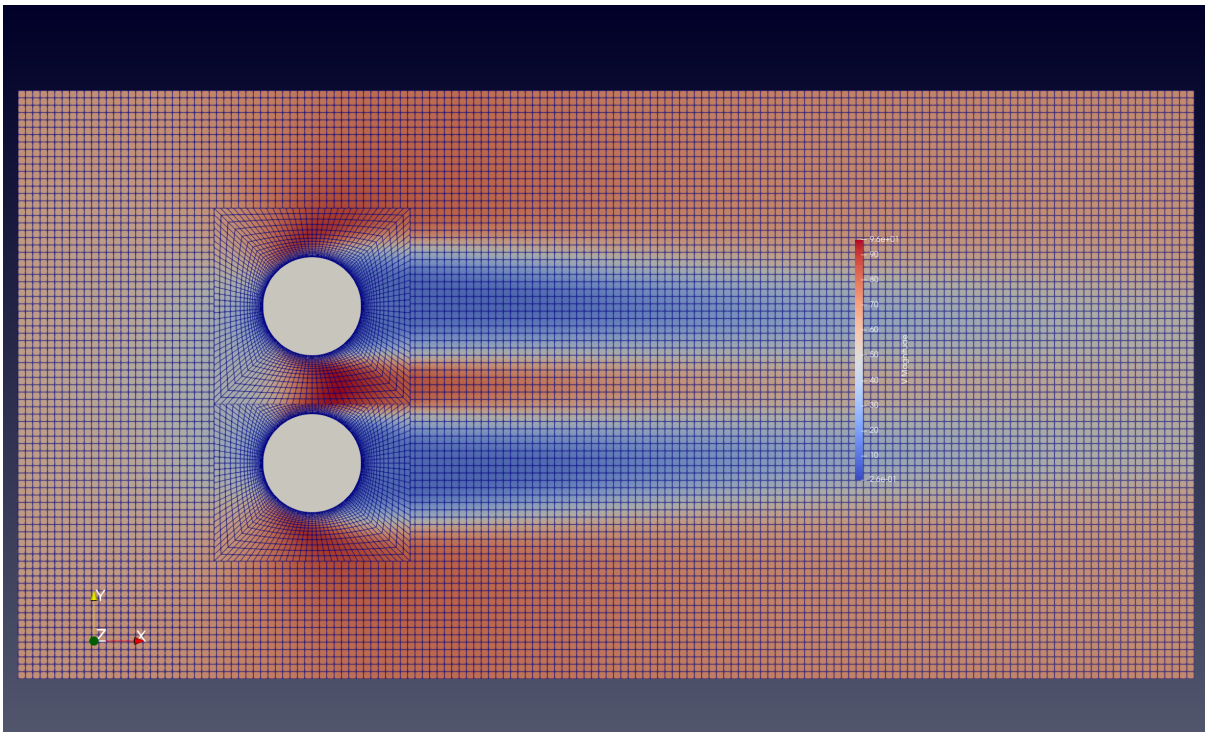
- Launch ParaView so that you can load the solution files (locally or remotely).
- Load the 4 files "oversetmulticylinder_OUTPUT/background/background.vtkhdf", ".../refinement_region/refinement_region.vtkhdf", ".../cylinder_1/cylinder_1.vtkhdf" and ".../cylinder_2/cylinder_2.vtkhdf".
- Highlight all 4 files on the navigation tree (on the left) and select "Filters > Append Datasets". This will create a new item in the navigation tree called "AppendDatasets1" that acts on all four datasets at once.
- The volumetric data output for this case contains a variable called "overset" which is 0 or 1 for any cell in the meshes. 0 means that the cell has been overset by another mesh, and 1 means that the cell is the active cell in that location. Use the "Filters > Threshold" filter to select only the cells in the "AppendDatasets1" combined dataset that have an "overset" value of 1 (do this by changing the lower threshold value from 0 to 1). This will create a new item in the navigation tree called "Threshold1" excluding any cells that have been overset by cells in another mesh.
- Use the "Filters > Cell Data to Point Data" filter to interpolate the cell data from "Threshold1" to the mesh nodes. This makes subsequent interpolation operations in ParaView more accurate. This will create a new item in the navigation tree called "CellDatatoPointData1".
- Use the "Filters > Slice" filter to extract the solution from "CellDatatoPointData1" onto a plane with normal in the "Z" direction. Note that this contains data from all 4 meshes.
- Colour the new "Slice1" object by "V" "Magnitude", selecting "Surface With Edges" in the visualisation toolbar.

Other post-processing steps such as streamlines or animations will also work with overset meshes in the same manner. If the meshes are in relative motion throughout an unsteady simulation, ParaView will automatically update their positions.

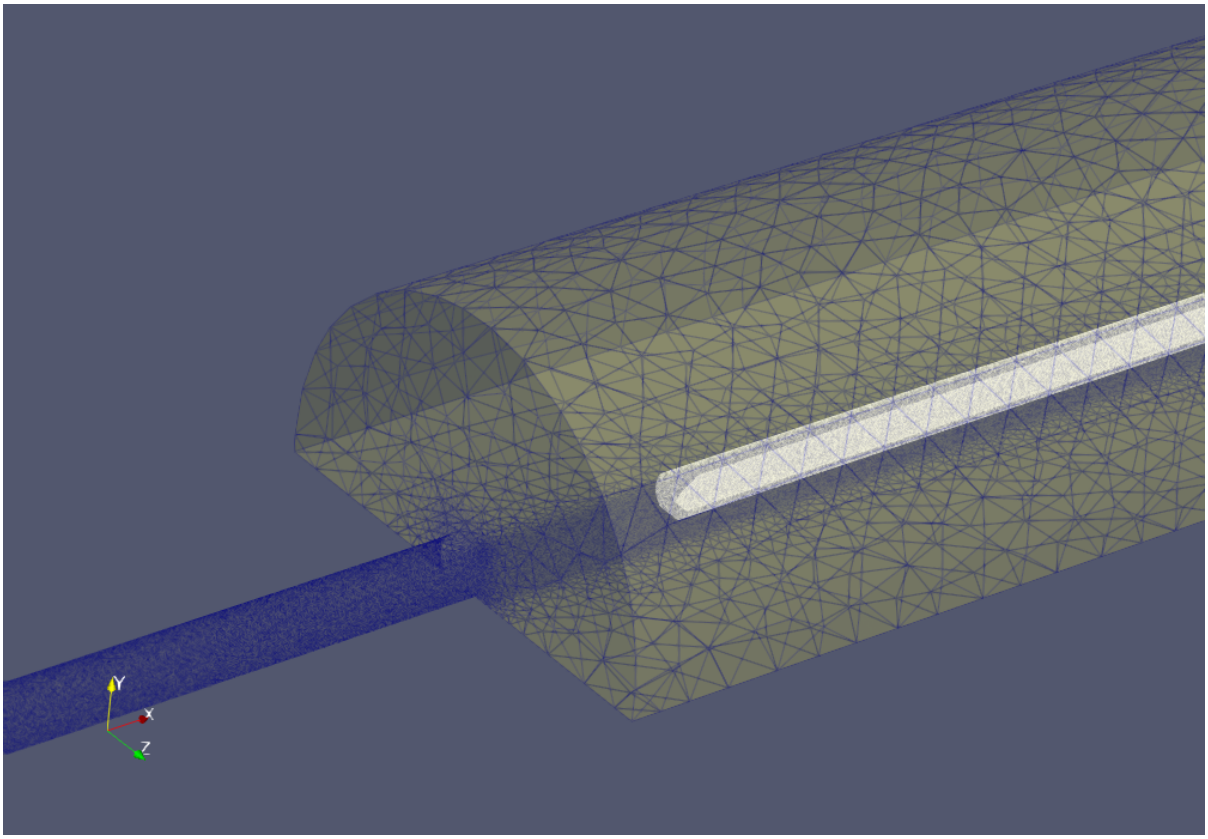
- [Optional] The "refinement_region" solution will show through behind the "cylinder_1" and "cylinder_2" overset solutions because of the holes where the cylinders are. Any solution is non-physical, and an easy way to mask this effect is to insert some simple geometry to show the location of the cylinders. Use "Sources > Cylinder" to create a unit diameter cylinder at [0,0,0] and rotate it 90 degrees about the x-axis, then translate it in the y-axis by -0.8 in y to put it into place. Repeat, but with a translation of +0.8 for the other cylinder.
- [Optional] The above steps will produce the image below:



To create the image in the validation page, you can force ParaView to bring the overset meshes to the foreground by treating each of the overset meshes independently, and adding a small incremental offset normal to the slice plane to each cylinder mesh (0.0, 0.001, 0.0015, 0.002). This will better reflect what the solver will actually see, as the overset cells in the background meshes will not be active.



Example 2 - Train Tunnel Test Case



This test case demonstrates the zCFD overset capabilities to predict the micro pressure wave generated as a train enters a tunnel, and is a good example of a practical application of overset meshing. The simulation set up follows that described in Section 7.6 of Railway applications — Aerodynamics — Part 5: Requirements and test procedures for aerodynamics in tunnels. In order to avoid transients, the train is linearly accelerated from 0 to 250 km/hr over 1.5 seconds.

For the reference train entering the reference tunnel at 250 km/hr, the maximum entry pressure gradient dp/dt should be in the range of 8800 Pa/s to 9500 Pa/s. These values are used to validate the simulation.

The input control dictionary for the train in this case also shows how to programmatically define the motion of a mesh in time:

```
def linear_accel(**kwargs):
    t = kwargs["time"]
    dt = kwargs["time_step"]
    u = [-69.4, 0, 0]
    if t < 1.5:
        u[0] = -(t / 1.5) * 69.4
    return {"velocity": tuple(u)}
```

This function returns the velocity to be applied to a fluid zone, which is then defined under the "train" mesh's `model` entry by:

```
"FZ_1": {
    "type": "translating",
    "zones": [6],
    "velocity": [-69.4, 0.0, 0.0],
    "translation function": linear_accel,
    "moving mesh": True,
},
```

The fluid zone's `velocity` is a single vector in m/s - there is no separate `vector/mach` split for a translating zone. The `moving_mesh` field is set directly on the fluid zone; setting it is only permitted when the same mesh also declares an overset boundary condition (as the "train" mesh does, for the zone where it exchanges data with the tunnel background mesh), since a moving mesh with nowhere to overset is meaningless. Setting `moving_mesh` to `True` forces a recalculation of the physical location of the mesh, including the recalculation of any overset mappings, every real time step.

The unsteady time-marching settings that apply to the whole case live under the control dictionary's top-level `solver` key, split between `time_settings` (what is being marched) and `convergence_control` (how each step converges):

```
"time settings": {
    "type": "unsteady",
    "total time": 3.6,
    "time step": 0.001,
    "order": 1,
    "start": 0,
},
"convergence control": {
    "scheme": {"name": "implicit euler", "stage": 1},
    "cfl": 30,
    "cycles": 5,
},
```

The meshes and control files for the train and the tunnel can be downloaded from here.

The train tunnel case is larger than the previous case in this tutorial: the train mesh has 1.31m cells and the tunnel has 7.35m cells. To run this case in implicit mode on a GPU will require approximately 48GB RAM.

The case can be run with the command

```
run_zcfd -c traintunnel.py
```

Post-processing

To plot the entry pressure gradient over time, copy the following Python code into a file 'create_plot.py'. The pressure monitor is defined on the "background" (tunnel) mesh, so its report is written to "traintunnel_OUTPUT/background/background_report.csv":

```
# Compute maximum entry pressure gradient dp/dt
import matplotlib.pyplot as plt

ts = 0.001
file_name = "traintunnel_OUTPUT/background/background_report.csv"
p_data = []
with open(file_name) as fp:
    line = fp.readline().split()
    count = 0
    val = 0
    while line:
        line = fp.readline().split()
        if len(line) > 0:
            c = int(line[0])
            if c != count:
                count = count + 1
                val = float(line[7])
                p_data.append(val)

    p_grad_data = []
```

(continues on next page)

(continued from previous page)

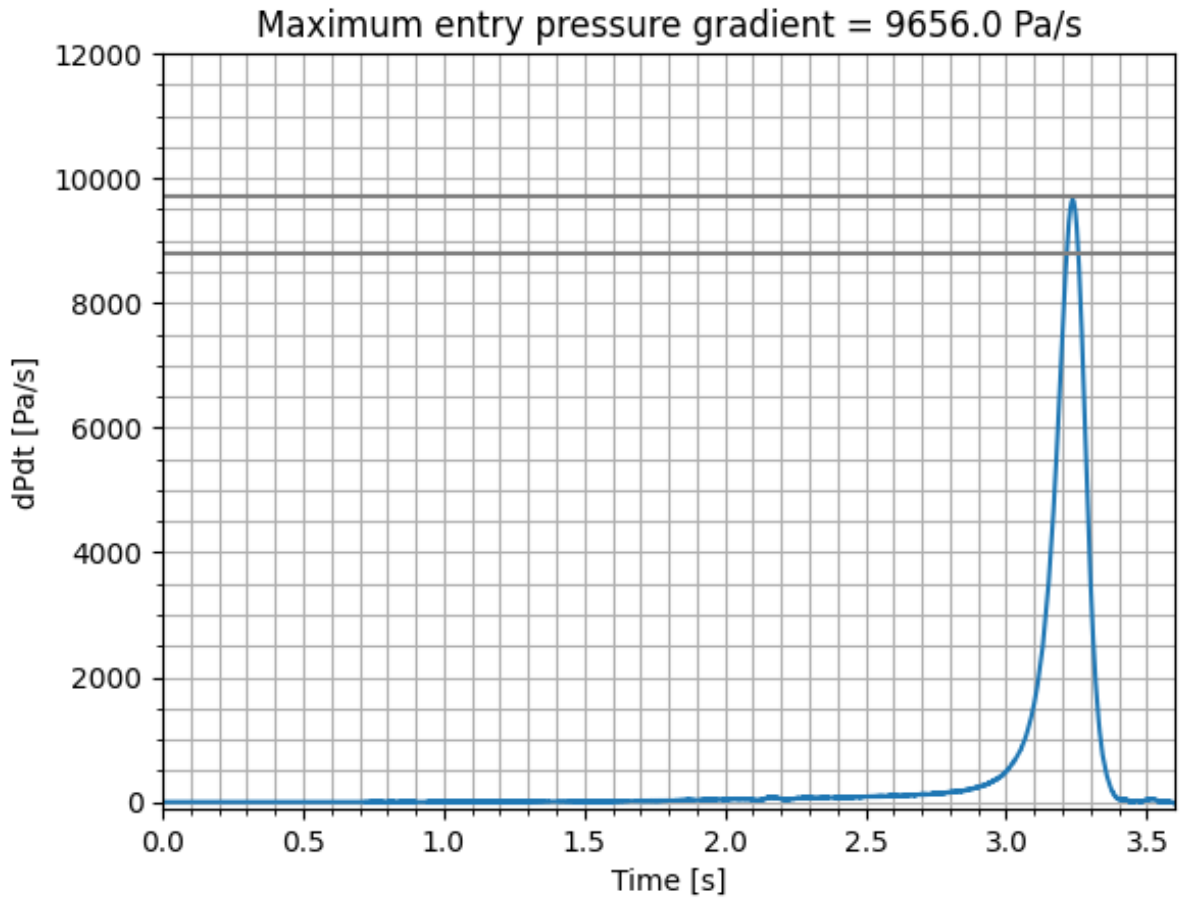
```
    time = []
    for i in range(len(p_data)-1):
       if ts*i < 4:
            p_grad_data.append((p_data[i+1] - p_data[i]) / ts)
            time.append(ts * i)

plt.minorticks_on()
plt.grid(which="both")
plt.title('Maximum entry pressure gradient = '+str(round(max(p_grad_data),6))+' Pa/s')
plt.xlabel("Time [s]")
plt.ylabel("dPdt [Pa/s]")
plt.xlim(0, 3.6)
plt.ylim(-100, 12000)
plt.plot(time, p_grad_data, label="250km/hr @ 15oC")
plt.plot([time[0],time[-1]],[8800,8800],label='min valid dP/dT',c='grey')
plt.plot([time[0],time[-1]],[9700,9700],label='max valid dP/dT',c='grey')
plt.show()
plt.savefig("EntryPressureGradient.png")
plt.close()
```

Then run the code in the same directory as the solution files with the command:

```
python create_plot.py
```

This will create the following plot, showing the maximum entry pressure gradient as a function of time. The maximum entry pressure gradient is 9656.0 Pa/s, which is within the range of values specified [8800.0, 9700.0] in the standard.



Citations

1. EN 14067-5:2006 - Railway applications - Aerodynamics - Part 5: Requirements and test procedures for aerodynamics in tunnels, *See <https://standards.iteh.ai/catalog/standards/cen/ea6cb49e-c40a-4b8b-a059-c6debdb91dc7/en-14067-5-2006>*

1.5 Reference Guide

The zCFD control file is a python file that is executed at runtime. This provides a flexible way of specifying key parameters, enabling user defined functions and leveraging the rich array of python libraries available.

A python dictionary called `parameters` provides the main interface to controlling zCFD:

```
parameters = {
    "config_version": 2,
    "solver": {....},
    "model": {....},
}
```

`config_version` is a required integer identifying the schema the rest of the dictionary is validated against. `solver` holds the settings shared across the whole simulation; `model` holds one entry per mesh.

zCFD validates the `parameters` dictionary at run time against a pydantic schema for each key. Values that should be integers and floats are coerced to the correct type where possible; values that should be strings, booleans, lists or dictionaries raise an error if given the wrong type. Which keys are valid depends on the equation set and solver type chosen. A script is provided to check the validity of a control file before submitting a job — see *Input Validation*.

This reference guide documents the keys available in a zCFD control dictionary and the valid values for each.

```
parameters = {
    "config_version": 2,
    "solver": {
        "solver_settings": {...},
        "equations": {...},
        "fluid_properties": {...},
        "reference_conditions": {"IC_1": {...}},
        "initialisation": {...},
        "time_settings": {...},
        "convergence_control": {...},
        "numerical_scheme": {...},
        "output_settings": {
            "report": {...},
            "solution": {...},
        },
    },
    "model": {
        "<mesh_name>": {
            "mesh": "<path.h5>",
            "boundary_conditions": {"BC_1": {...}},
            "fluid_zones": {"FZ_1": {...}},
            "transforms": {...},
        },
    },
}
```

1.5.1 Reference Settings

In control files using the nested schema (`config_version: 2`), these settings live under the `solver` block (global defaults, shared by every mesh) and, where noted, can be overridden per mesh under `model.<meshname>`. Each subsection below states exactly where its keyword sits in that tree, matching *zutil/zutil/validate/schemas/initialisation_schemas.py*, *solver_settings_schemas.py* and *transform_schemas.py*, which are the source of truth the C++ solver reads.

Reference state

Location: `solver.initialisation.reference`

Keyword	Required	Default	Valid values
'reference'	No	Falls back to 'initial_conditions'	Name of an entry in *solver.reference_conditions* (str)

```
parameters["solver"]["initialisation"] = {
    "reference": "IC_1",
    "initial_conditions": "IC_1",
}
```

Sets the *reference condition* used to provide the reference quantities when non-dimensionalisation is applied to *force reporting*. "IC_1" (or whichever name is given) must be a valid entry defined in `solver.reference_conditions`.

When 'reference' is omitted it falls back to the name given by 'initial_conditions'. It must be set explicitly when 'initial_conditions' is a python callable (a *'driving function'*), since a callable has no static name to fall back on — a missing 'reference' in that case is a validation error, not a silent default.

If the reference condition named here has zero velocity (for example a quiescent cavity case), set 'reference' to point at a *different* reference condition that carries the physical velocity/Reynolds number, so the solver has something non-zero to non-dimensionalise against.

Initial State

Location: `solver.initialisation.initial_conditions` (global default), overridable per mesh at `model.<meshname>.initialisation.initial_conditions`.

Keyword	Required	Default	Valid values
'initial_conditions'	Yes, unless 'restart' is True	–	Name of an entry in *solver.reference_conditions* (str), or a python callable.

```
# set initial state to IC_1
parameters["solver"]["initialisation"] = {
    "initial_conditions": "IC_1",
}
```

Sets which entry of *solver.reference_conditions* is used to provide the initial flow field conditions for the simulation. If a python callable is provided instead of a name, it is called to provide the initial flow field variables (see the *'driving function'* form), and 'reference' must then be set explicitly (see *Reference state* above).

Note

There is no implicit fallback: 'initial_conditions' must be set (to a name or a callable) unless 'restart' is True. Omitting it on a fresh (non-restart) run is a validation error.

Any mesh in the `model` block can override the solver-level defaults for initialisation and restart by supplying its own `initialisation` dict — useful in a multi-mesh (e.g. overset) case where only one mesh should restart, or where meshes start from different named conditions.

```
# Per-mesh override: this mesh restarts, others use the solver default
parameters["model"]["wing"]["initialisation"] = {
    "initial_conditions": "IC_1",
    "restart": True,
}
```

Scale mesh

Location: `model.<meshname>.transforms.scale`

Keyword	Required	Default	Valid values
'scale'	No	Not set (no scaling applied)	[scaleX, scaleY, scaleZ] : Any list of positive floats with length 3

Before being loaded into the solver, the mesh's [x,y,z] location data is multiplied by this vector. The scale is folded into the mesh's *transform matrix* (see below) before the mesh is read, so scaling and an explicit transform matrix compose correctly when both are supplied — the scale is applied first, then the transform matrix.

Example usage:

```
# Scale from mm to metres
parameters["model"]["wing"]["transforms"] = {"scale": [0.001, 0.001, 0.001]}
```

```
# Scale from inches to metres
parameters["model"]["wing"]["transforms"] = {"scale": [0.0254, 0.0254, 0.0254]}
```

```
# Scale the mesh to be 50 times larger in x
parameters["model"]["canopy"]["transforms"] = {"scale": [50.0, 1.0, 1.0]}
```

Note

The 'scale' parameter applies to the mesh only — not interpolated surface outputs etc.

Note

A top-level 'scale' key directly on a model block (e.g. `'wing': {'scale': [...], ...}`) is not a valid schema field. Even where such a key were read, `Mesh::readMesh` — the only Python-bound mesh-loading entry point — always calls `Mesh::partitionMesh` with `useBoundingBox` hard-coded to `false`, so the bounding-box-scaling code path is never invoked. The working capability — scaling mesh coordinates on load — lives at `transforms.scale` as shown above, which *is* read on the live mesh-loading path.

Mesh Transform Matrix

Location: `model.<meshname>.transforms.transform_matrix`

The mesh can be transformed using an affine transform (Wikipedia). Here the upper left 3 x 3 matrix describes a rotation, reflection, scaling or shear (or any combination of) and the 4th column a translation. This 4 x 4 matrix must be supplied as a 4 x 4 numpy array (or an equivalent nested list). Any number of transforms can be concatenated together to give the final position. Helper functions for rotations, translations and scalings are provided in `zutil.transform`. An example is given below:

```
import zutil.transform

A2B = zutil.transform.rotate([0.0, 1.0, 0.0], 10.0)
# If a matrix is passed as the last parameter to zutil.transform.* then the transform␣
↪is concatenated to it.
B2C = zutil.transform.scale([0.1, 0.1, 0.1], A2B)
C2D = zutil.transform.translate([0.0, 0.0, 0.0], [-1.0, 0.0, 0.0], B2C)

parameters["model"]["wing"]["transforms"] = {"transform_matrix": C2D}
```

When using the zutil.transform helper functions transforms are applied in the order in which the user calls the functions. Rotations are considered using the same conventions as pytransform3d and are interpreted as active and applied using the pre-multiplication convention. For more complex transformations pytransform3d provides a comprehensive collection of python functions and can be easily interfaced with zCFD by simply passing the resulting transformation matrix to the solver.

Note

`transforms` also accepts a `transform_func` key (a python callable intended to describe dynamic rigid mesh motion) in the schema. It is accepted at validation time but currently has no C++ consumer anywhere in the mesh or solver code — nothing reads it back out. Do not rely on it; use `transform_matrix` for initial placement as shown above.

Mesh Quality Remediation

Location: `solver.solver_settings.mesh_quality_remediation_angle_threshold`

Keyword	Required	Default	Valid values
'mesh_quality_remediation	No	Not set	0 - 180 (degrees)

zCFD has the ability to detect poor quality cells in the mesh and fix them as first order in space. A poor quality cell is defined as one which has a face where the angle between the cell centre to cell centre vector and the face centre to cell centre vector is greater than that set using this key. When convergence issues are encountered due to mesh quality, setting the angle threshold to around 75 degrees initially may allow the solver to run on the mesh. Ideally the mesh should be improved so the solution can be fully second order over the entire domain.

Note

The schema field exists and validates, and the underlying C++ method (`Mesh::qualityCheck`) exists and is bound through to Python — but no current Python entry point calls it with this value. Setting this key validates cleanly today but has no observable effect on a solve; it is an open question whether to wire up a caller or remove the field.

Partitioner

Location: `solver.solver_settings.partitioner`

Keyword	Required	Default	Valid values
'partitioner'	No	'scotch'	'metis' | 'scotch'

The partitioner is used to split the computational mesh up so that the job can be run in parallel.

```
parameters["solver"]["solver_settings"] = {
    "type": "compressible",
    "partitioner": "metis",
}
```

Note

Valid values are 'metis' and 'scotch'; the default is 'scotch'. Decks that need 'metis' specifically should set `"partitioner": "metis"` explicitly rather than relying on the default.

Restart

Location: `solver.initialisation.*` (global default), overridable per mesh at `model.<meshname>.initialisation.*`. 'solution_smooth_cycles' is a system setting rather than a restart-specific one, and lives separately at `solver.solver_settings.solution_smooth_cycles`.

Keyword	Required	Default	Valid values
'restart'	No	False	True/False
'restart_casename'	No	Current casename	Optionally supply the name of a case to restart from that case, if not supplied then the current casename will be used.
'restart_ignore_history'	No	False	True/False: True to ignore the cycle history from restart file and begin the solve as specified in current control file.
'interpolate_restart'	No	False	True/False: True to interpolate restart results from a different mesh. Requires 'restart_meshname' and 'restart_casename' to both be set.
'restart_meshname'	When 'interpolate_restart' is True	–	The name of the mesh to interpolate the restart from. A trailing '.h5' is stripped automatically if supplied.
'solution_smooth_cycles'	No	0	Number of Laplace smoothing cycles to smooth initial mapped solution. Set at `solver.solver_settings.solution_smooth_cycles`, not under 'initialisation'.

Restart allows you to load a solution from a previous run and continue the solver from that point. By default the solver will look for a `<casename>_results.h5` file to load the restart; this can be overridden with 'restart_casename'. The cycle history from the restart case can be ignored through 'restart_ignore_history'. To begin the current simulation using the results file specified in the current control dictionary but ignore the previous cycle history, set this option to True.

The default restart assumes that the same mesh is used. Alternatively, a solution from another mesh can be mapped onto a new mesh using 'interpolate_restart' — this requires both 'restart_meshname' and 'restart_casename' to be set, and the schema enforces that pairing at validation time, rather than leaving a missing name to fail later at solve time.

Example usage:

```
# Restart from a previous solution, from case name 'cylinder_steady_start_init'
parameters["solver"]["initialisation"] = {
    "initial_conditions": "IC_1",
    "restart": True,
    "restart_casename": "cylinder_steady_start_init",
}
```

```
# Restart from a previous solution on a different mesh
parameters["solver"]["initialisation"] = {
    "initial_conditions": "IC_1",
    "restart": True,
    "interpolate_restart": True,
    "restart_meshname": "different_mesh",
    # solution_smooth_cycles lives on solver_settings, not initialisation
}
parameters["solver"]["solver_settings"]["solution_smooth_cycles"] = 0
```

Restarting on a different number of tasks

A restart appends to the VTKHDF output it finds rather than replacing it, so a restarted case carries on one continuous time series instead of starting a second one. That holds only while the partition count does: a VTKHDF series stores its sizes as one entry per partition per step, so a run appending at a different task count cannot extend the series, and in practice the earlier output is lost.

When a restart is about to run on a different number of tasks than the solution it is restarting from, the `.vtkhdf` files already in the output directory are therefore moved into a subdirectory named for the partition count that produced them, before the solver opens its output:

```
plate_coarse_OUTPUT/background/P2/background.vtkhdf
plate_coarse_OUTPUT/background/P2/background_wall.vtkhdf
```

The new run then starts its own series at the new count, and the old output stays where it is and stays readable. The move is reported in the log:

```
Restarting from a solution generated on 2 partition(s) with 1 task(s):
moved 4 VTKHDF file(s) aside so the existing output is not written over.
plate_coarse_OUTPUT/background/P2
```

Points worth knowing:

- Only the current case's own output directory is swept. With `'restart_casename'` pointing at another case, that case's output is left alone — this run does not write into it.
- Each file is labelled from its own recorded partition count, so output left behind by an earlier run of the same case at a third count is labelled correctly rather than by the restart source's count.
- A file already written at the new task count is left in place, so a restart on the same number of tasks as before still extends its series as it always has.
- Passing through the same count twice (2, then 1, then 2, then 1) archives into `P2` and then `P2.1`; an existing archive is never written over.
- The archive is a subdirectory of the model directory, which post-processing does not look inside — `zutil` finds models one level below `<case>_OUTPUT` and lists output non-recursively — so an archive is never mistaken for a mesh or picked up as live output.
- Checkpoint (`_results.h5`), report and `VISUALISATION` files are not moved. Only `.vtkhdf` output is affected.

Advanced Restart

It is possible to restart the SA-neg solver from results generated using the Menter SST solver and vice versa; by default the turbulence field(s) are reset to zero. Generating an approximate initial field for SA-neg from Menter-SST results is not currently possible from a control file.

> **Note**
>
> The C++ read site for this capability is still live (`SolverBase.cxx`, reading `approximate_sa_from_sst_results` from `equations.turbulence`), but no field for it exists on the turbulence schema, which forbids extra keys — so there is currently no way to set this from a valid control file. Adding an `approximate_sa_from_sst_results` field to the turbulence model would restore it.

Safe Mode

Location: `solver.solver_settings.safe`

Keyword	Required	Default	Valid values
'safe'	No	False	True/False

Safe mode turns on extra checks and provides diagnosis in the case of solver stability issues.

Example usage:

```
parameters["solver"]["solver_settings"] = {
    "type": "compressible",
    "safe": True,
}
```

1.5.2 Boundary Conditions

Boundary conditions are a dict of named blocks nested under the mesh's model entry, at `model.<mesh_name>.boundary_conditions`. Each block is keyed by an arbitrary name (conventionally `BC_1`, `BC_2`, ...) and is itself a dict describing one boundary condition.

```
parameters = {
    "config_version": 2,
    "solver": {...},
    "model": {
        "naca0012": {
            "mesh": "naca0012_0449.h5",
            "boundary_conditions": {
                "BC_1": {"zones": [2, 3], "type": "symmetry"},
                "BC_2": {"zones": [4], "type": "wall", "kind": "no slip"},
                "BC_3": {
                    "zones": [1],
                    "type": "farfield",
                    "condition": "IC_1",
                    "kind": "riemann",
                },
            },
        }
    },
}
```

Every boundary condition block requires a **type** (which BC kernel handles these zones) and **zones** (which mesh zones it applies to). Most types also require a **kind**, which selects between variants of that type with their own extra fields — for example a wall's `kind` chooses between `slip`, `no slip` and `wall function`; the fields available below `kind` change with the choice.

zCFD supports the following boundary condition types:

BC Type	Description
Wall	Wall boundaries
Farfield	Farfield boundaries with automatic inflow/outflow detection
Inflow	Pressure, condition-driven or massflow inflow boundaries
Outflow	Pressure, massflow or radial-pressure-gradient outflow boundaries
Mass Flow	Standalone mass-flow boundary condition
Symmetry	Mirror boundary condition
Overset	The boundary of a mesh to be overset on a background mesh
Periodic	Repeating boundary condition, with flow from one periodic boundary mapped to the other.

Zones

Every boundary condition block requires a **zones** keyword: a list of integer zone IDs from the mesh that this BC applies to. The exact zone numbering is determined by the mesh format and mesh generation tool used — inspect the mesh (for example with the tool that generated it) to determine which zone numbers correspond to which physical boundary.

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to

Wall

Wall boundaries in zCFD can be either slip walls, no-slip walls, or use automatic wall functions. No-slip walls are appropriate when the mesh is able to fully resolve the boundary layer (i.e. $y^+ \leq 1$), otherwise automatic wall functions should be used. This behaviour is chosen by setting `'kind'` in the wall specification below. Optionally it is possible to set wall velocity, roughness, temperature and an FSI modal model.

`'type'` allows you to choose between a standard `'wall'` definition or `'immersed wall'` boundaries. Immersed wall boundaries are *only* supported with meshes created using the *zM3* mesh generator. The zM3 mesh generator will determine vectors to represent the underlying geometry within the 3D Cartesian mesh. It will also generate additional information such as locations where data is to be sampled in order to define conditions at the geometry surface, and the subsequent halo boundary values.

Note

If `'slip'` conditions are chosen, there will be zero momentum normal to the wall.

If `'no slip'` conditions are chosen, the no-slip condition is applied directly (mesh must resolve down to $y^+ \sim 1$). For an immersed wall, values computed near the wall are linearly extrapolated from the donor (sample) point location associated with that point — this can potentially cause sensitivity and early flow separation in strong pressure gradients, alleviated to some extent with mesh refinement at additional computational cost.

If `'wall function'` conditions are chosen, any values computed near the wall will be based on the Musker wall model, which closely represents the flow along a flat plate in zero pressure gradient. As such, there are fundamental limitations on the accuracy of this model with highly curved surfaces and strong pressure gradients, alleviated to some extent with mesh refinement at additional computational cost.

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'wall'` or `'immersed wall'`	Specifies a solid wall boundary, adiabatic unless a temperature or a heat flux is specified
`kind`	No	`'no slip'`	`'slip'`, `'no slip'`, `'wall function'`	Specifies the wall model. slip - zero-normal-momentum wall. no slip - viscous wall requiring $y^+ \sim 1$ mesh resolution. wall function - viscous wall with automatic handling of mesh resolution between $y^+ \sim 1$ and $y^+ > 100$
`immersed_wall_normal_t`	No	`'STL'`	`'STL'` or `'immersed boundary vector'`	Only for `'immersed wall'`. Specifies how the geometry unit normals are determined (directly from the STL, or approximated by the vector from the face centre to the nearest surface point)
`roughness`	No		See *Roughness*	Surface roughness model. If not specified the wall is perfectly smooth
`velocity`	No		See *Velocity*	Wall velocity specification, for surfaces in relative motion
`temperature`	No		See *Temperature*	Wall temperature. If not specified the wall is treated as adiabatic
`heat flux`	No		See *Temperature*	Prescribed wall heat flux, `'wall function'` walls only. Mutually exclusive with `temperature`
`fsi`	No		See *FSI*	FSI modal model coupling this wall to a structural solve

Example no-slip wall:

```
"BC_2": {"zones": [4], "type": "wall", "kind": "no slip"},
```

Example immersed wall with a slip condition:

```
"immersed_wall": {"zones": [7], "type": "immersed wall", "kind": "slip"},
```

Example immersed wall using a wall function:

```
"aircraft": {"zones": [7], "type": "immersed wall", "kind": "wall function"},
```

Roughness

Keyword	Required	Default	Valid values	Description
`type`	Yes		`'height'` or `'length'`	How the roughness magnitude is being specified
`scalar`	No (one of `scalar`/`field` required)		Positive float	Value of roughness length or height, in mesh units
`field`	No (one of `scalar`/`field` required)		Valid filename	Name of a VTP file containing a node array called `Roughness`, used to set the value by nearest-point lookup

Exactly one of `scalar` or `field` must be set — neither both nor neither is accepted.

Roughness **length** is the height at which the mean velocity profile of the flow is zero — beyond this distance from the surface, the flow is considered effectively smooth and the velocity distribution approaches a logarithmic law.

Roughness **height** is the average height or magnitude of surface roughness elements on a solid boundary — the scale of irregularities or protrusions on the surface that disrupt the smooth flow and create additional drag or turbulence.

Example:

```
parameters["model"]["background"]["boundary_conditions"]["BC_3"]["roughness"] = {
    "type": "height",
    "scalar": 0.001,
}
```

Wall Function

An adiabatic `'wall function'` wall uses an incompressible law of the wall: the velocity profile is solved with the density and viscosity of the first cell off the wall, and no heat is transferred. Giving the wall a `temperature` or a `heat flux` switches that wall to the compressible wall function: the Van Driest II / White-Christoph velocity transformation coupled to the Crocco-Busemann thermal law of the wall, which evaluates y^+ and the wall shear stress with the wall-state density and viscosity and produces the wall heat flux.

With a `heat flux` the wall temperature is recovered; with a `temperature` the heat flux is. Both are available as the `heatflux` and `twall` surface output variables.

A constant-heat-flux wall is singular where the boundary layer is thin, for example at a plate leading edge: the modelled layer cannot carry an arbitrarily large flux there and the wall temperature will rise until it is limited internally. Check the `twall` output if a prescribed flux is large.

The compressible wall function is not supported in combination with *roughness*, on immersed walls, or in the incompressible solver, and these combinations are rejected at input validation.

Velocity

The `velocity` keyword applies a surface velocity, used to model cases with surfaces in relative motion, e.g. a car moving over a road surface or a rotating tyre. The wall boundary condition supports either a **linear** velocity or a **rotating** one, selected by its own `type` field.

The options for the **linear** velocity are:

Keyword	Required	Default	Valid values	Description
`type`	Yes		`'linear'`	Selects a linear (translational) wall velocity
`vector`	Yes		`[x, y, z]`	Direction of the translation velocity
`mach`	No		Number	Overrides the magnitude of the translation velocity vector

The options for the **rotating** velocity are:

Keyword	Required	Default	Valid values	Description
`type`	Yes		`'rotating'`	Selects a rotational wall velocity
`axis`	Yes		`[x, y, z]`: unit vector	Axis of rotation
`center`	Yes		`[x, y, z]`	Origin (centre point) of the rotation axis
`omega`	Yes		Number	Angular velocity of the rotation, in radians per second

Example linear wall velocity:

```
parameters["model"]["background"]["boundary_conditions"]["BC_3"]["velocity"] = {
    "type": "linear",
    "vector": [34.72, 0.0, 0.0],
}
```

Example rotating wall velocity:

```
"moving_wheel": {
    "zones": [14, 20],
    "type": "wall",
    "kind": "wall function",
    "velocity": {
        "type": "rotating",
        "axis": [0.0, -0.9988, 0.0498],
        "center": [0.0, -0.6593, 0.2533],
        "omega": rot_rate,
    },
},
```

Temperature

A wall is adiabatic unless it is given a `temperature` or, on a `'wall function'` wall, a `heat flux`. Both take the same value: a number for a uniform wall, or a dictionary with a single `field` key naming a VTP file for a value that varies over the wall. The file must carry a point array named `Temperature` (Kelvin) or `HeatFlux` (W/m^2), and each wall face takes the value at the nearest point, or the interpolated value when the file carries cells. Heat flux is **positive into the fluid**, i.e. a heated wall; negative values remove energy.

Keyword	Required	Default	Valid values	Description
`temperature`	No	unset (adiabatic)	Number (Kelvin) or `{"field": "<file>.vtp"}`	Wall temperature
`heat flux`	No	unset (adiabatic)	Number (W/m^2) or `{"field": "<file>.vtp"}`	Prescribed wall heat flux. `'wall function'` walls only; mutually exclusive with `temperature`

Examples:

```
"BC_1": {"zones": [1], "type": "wall", "kind": "no slip", "temperature": 310.0},
"BC_2": {"zones": [2], "type": "wall", "kind": "wall function", "temperature": 290.0},
"BC_3": {"zones": [3], "type": "wall", "kind": "wall function", "heat flux": 5.0e3},
"BC_4": {"zones": [4], "type": "wall", "kind": "no slip", "temperature": {"field":
↪"hot_patch.vtp"}},
```

FSI

The `fsi` keyword attaches a fluid-structure-interaction modal model to a wall boundary (any `kind`), coupling the wall's motion to a structural solve. The full set of FSI model parameters is documented separately in *Fluid-structure interaction*; this section only shows how it attaches to a wall BC.

Example:

```
"BC_FSI": {
    "zones": fsi_zones,
    "type": "wall",
    "kind": "slip",
    "fsi": {
        "type": "modal_model",
        "file_format": "nastran",
        "nastran_casename": "panel_nm_0.04m",
        "transform_type": "rbf_multiscale",
        "base_point_fraction": 1.0,
        "support_radius": 0.02,
        "scale": [1.0, 1.0, 1.0],
        "mode_mapping_max_distance": 0.01,
        "fluid_force_scaling": 0.08 / 0.000166667,
        "mode_list": [0, 4, 10, 13],
        "modal_damping": [0.0, 0.0, 0.0, 0.0],
    },
},
```

Farfield

Farfield boundary conditions are typically used for external flow simulations and automatically switch between inflow and outflow. This boundary condition cannot be used with the incompressible solver. The farfield boundary automatically determines whether the flow is locally an inflow or an outflow by solving a Riemann equation (for `kind: 'riemann'` and `'preconditioned'`). The conditions on the farfield boundary are set by referring to a *reference condition* block.

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'farfield'`	Boundary condition type
`kind`	No	`'riemann`	`'riemann'`, `'pressure'`, `'supersonic'` or `'preconditioned'`	**Model used for the farfield boundary:** • *riemann* uses Riemann invariants for a non-reflecting inflow with a pressure outflow. • *pressure* uses a pressure boundary condition for both inflow (downstream pressure) and outflow (user-specified pressure). • *supersonic* uses upstream conditions for both inflow and outflow. • *preconditioned* is a preconditioned variant of the Riemann boundary condition.
`condition`	Yes		String reference to a reference condition	Reference flow condition block for these boundaries — see *reference conditions*
`sponge_layer_damp`	No		See *Sponge Layer Damping*	Optional sponge layer damping applied near this boundary
`driver`	No		See *Force Driver*	Optional automatic driver that adjusts the inflow to target a force coefficient

Example farfield with `kind: 'riemann'`:

```
"BC_3": {"zones": [1], "type": "farfield", "condition": "IC_1", "kind": "riemann"},
```

Example farfield with `kind: 'pressure'`:

```
parameters["model"]["naca0012"]["boundary_conditions"]["BC_3"] = {
    "zones": [1],
    "type": "farfield",
    "condition": "IC_2",
    "kind": "pressure",
}
```

`kind: 'supersonic'` and `kind: 'preconditioned'` are valid schema values but no current testcase exercises either — both are constructed the same way, just with `"kind": "supersonic"` or `"kind": "preconditioned"` substituted above.

Sponge Layer Damping

Keyword	Required	Default	Valid values	Description
`distance`	Yes		Number	Distance over which damping is applied
`damping_factor`	Yes		Number	Damping factor magnitude
`condition`	Yes		String reference to a reference condition	Reference condition the sponge layer damps toward

Example:

```
"BC_2": {
    "zones": [12],
    "type": "farfield",
    "kind": "riemann",
    "condition": "IC_1",
    "sponge_layer_damping": {
        "distance": 20.0,
        "damping_factor": 2.037495,
        "condition": "IC_1",
    },
},
```

Force Driver

Drives a farfield boundary's inflow conditions to target a specific force coefficient by adjusting a control variable (angle of attack or velocity magnitude). Declare *what* you want (target, value, control mode); the runtime controller handles the feedback loop.

Keyword	Required	Default	Valid values	Description
`target`	Yes		String	Report column name to target (e.g. `'wall_Ftz'`)
`target_value`	Yes		Number	Target value for the force component
`force_report`	Yes		String	Name of the force report block (e.g. `'FR_1'`) to read forces from
`control`	Yes		`'angle_of_attack'` or `'velocity_magnitu`	Control variable to adjust
`initial_value`	No	0.0	Number	Starting value for the control variable (AoA: degrees, velocity: Mach)
`max_increment`	No	2.0	Number > 0	Maximum change per update
`relaxation_factor`	No	0.75	0 < Number <= 2.0	Under-relaxation factor for updates
`update_period`	No	200	Integer > 0	Cycles between control variable updates
`settling_period`	No	500	Integer >= 0	Initial cycles before the first update
`convergence_tolerance`	No	1.0e-4	Number > 0	Target error tolerance for convergence
`smoothing_window`	No	20	Integer > 0	Moving-average window for force smoothing

Example:

```
parameters["model"]["naca0012"]["boundary_conditions"]["BC_3"] = {
    "zones": [2, 3, 5],
    "type": "farfield",
    "condition": "IC_2",
    "kind": "riemann",
    "driver": {
        "target": "wall_Ftz",
        "target_value": 1.0778,
        "force_report": "FR_1",
        "control": "angle_of_attack",
    },
}
```

Note

The native `driver` block validates and registers correctly, but the angle-of-attack control mode does not engage at runtime for at least one known case: it reports zero update every cycle instead of converging alpha onto the target. This is a known solver-wiring gap, not a documentation error — do not rely on `driver` for angle-of-attack control until it is fixed.

Inflow

zCFD supports three kinds of inflow boundary condition, selected using the `'kind'` key: `'default'`, `'pressure'` and `'massflow'`. All three refer to a *reference condition* via the `reference` keyword (**not** `condition` — that keyword is farfield's). By default the flow direction is normal to the boundary; the `'pressure'` kind can override this with an explicit direction vector.

Note

The massflow rate needs to be specified in kg/s and can only be used if the mesh is in metres. Massflow ratio is nondimensional, where A_{in} is defined in mesh units: $massflowratio = \rho_{in} U_{in} A_{in} / (\rho_{ref} U_{ref})$.

Common fields:

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'inflow'`	Boundary condition type
`kind`	No	`'default`	`'default'`, `'pressure'` or `'massflow'`	Selects how the inflow state is specified
`reference`	No (required in practice — see kinds below)		String reference to a reference condition	Reference condition supplying the base state for ratio fields

`kind: 'default'` (condition-driven inflow — the incompressible solver's velocity inlet; carries no extra fields beyond the common ones above):

Example:

```
"BC_3": {"zones": [1], "type": "inflow", "kind": "default", "reference": "IC_1"},
```

`kind: 'pressure'` — velocity and density are derived from a total pressure ratio and total temperature ratio relative to `reference`:

Keyword	Required	Default	Valid values	Description
`total_pressure_ra`	No	1.0	Number	Total pressure ratio relative to `reference`
`total_temperature`	No	1.0	Number	Total temperature ratio relative to `reference` (setting this — even at its default — requires `reference` to be set)
`static_pressure_r`	No		Number	Static pressure ratio (requires `reference` if set)
`direction`	No		`[x, y, z]`: unit vector	Overrides the default surface-normal flow direction

Example:

```
parameters["model"]["pipe"]["boundary_conditions"]["BC_2"] = {
    "zones": [3],
    "type": "inflow",
    "kind": "pressure",
    "total_pressure_ratio": inlet_total_p_ratio,
    "reference": "IC_1",
    "total_temperature_ratio": 1.0,
}
```

`kind: 'massflow'` — the velocity is set from a mass flow rate or ratio:

Keyword	Required	Default	Valid values	Description
`mass_flow_rate`	No (exactly one of `mass_flo`		Number (kg/s)	Direct mass flow rate
`mass_flow_ratio`	No (exactly one of `mass_flo`		Number	Mass flow ratio relative to `reference` (requires `reference`)

Example using a rate:

```
"kind": "massflow", "mass_flow_rate": massflow, "reference": "IC_1",
```

Example using a ratio:

```
"kind": "massflow", "mass_flow_ratio": inlet_mass_flow_ratio, "reference": "IC_1",
↪"total_temperature_ratio": 1.0,
```

Outflow

Pressure, massflow, or radial-pressure-gradient outflow boundaries, specified in a similar manner to *Inflow*. All refer to a *reference condition* via `reference`.

> **Note**
>
> The massflow rate needs to be specified in kg/s and can only be used if the mesh is in metres. Massflow ratio is nondimensional, where A_{out} is defined in mesh units: $massflowratio = \rho_{out} U_{out} A_{out} / (\rho_{ref} U_{ref})$.

Common fields:

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'outflow'`	Boundary condition type
`kind`	Yes		`'pressure'`, `'massflow'` or `'radial pressure gradient'`	Selects how the outflow state is specified
`reference`	No		String reference to a reference condition	Reference condition supplying the base state for ratio fields

`kind: 'pressure'` — the exit pressure is set from a static pressure ratio relative to `reference`, or directly via `pressure`:

Keyword	Required	Default	Valid values	Description
`static_pressure_r`	No	1.0	Number	Static pressure ratio relative to `reference`
`pressure`	No		Number > 0	Direct exit pressure, overriding the ratio
`pressure_ramp`	No		Dict	Pressure ramping parameters
`total_temperature`	No		Number	Total temperature ratio (requires `reference` if set)

Example:

```
"BC_3": {
    "zones": [2],
    "type": "outflow",
    "kind": "pressure",
    "static_pressure_ratio": outlet_static_p_ratio,
    "reference": "IC_1",
},
```

`kind: 'massflow'` — scales the local velocity so the integrated mass flow across all zones matches the requested value:

Keyword	Required	Default	Valid values	Description
`mass_flow_rate`	No (exactly one of `mass_flo`		Number (kg/s)	Direct mass flow rate
`mass_flow_ratio`	No (exactly one of `mass_flo`		Number	Mass flow ratio relative to `reference` (requires `reference`)
`total_temperature`	No		Number	Total temperature ratio (requires `reference` if set)

Example using a rate:

```
"kind": "massflow", "mass_flow_rate": massflow, "reference": "IC_1",
```

Example using a ratio:

```
"kind": "massflow", "mass_flow_ratio": inlet_mass_flow_ratio, "reference": "IC_1",
```

`kind: 'radial pressure gradient'` — like `'pressure'`, but the static pressure ratio is defined at a specified radius rather than uniformly:

Keyword	Required	Default	Valid values	Description
`reference_radius`	Yes		Number	Radius at which the static pressure ratio is defined
`total_temperature`	No		Number	Total temperature ratio (requires `reference` if set)

No current testcase exercises this kind; the following is a schema-accurate illustration rather than a real deck excerpt:

```
"BC_4": {
    "zones": [5],
    "type": "outflow",
    "kind": "radial pressure gradient",
    "reference_radius": 1.0,
    "reference": "IC_1",
},
```

Mass Flow

A standalone boundary condition (`type: 'mass flow'`) that sets a mass flow rate directly, with no `kind` and no reference condition — distinct from the inflow/outflow `kind: 'massflow'` variants above.

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'mass flow'`	Boundary condition type
`mass_flow`	Yes		Number	Mass flow rate

No current testcase uses this BC type; the following is a schema-accurate illustration rather than a real deck excerpt:

```
"BC_2": {"zones": [3], "type": "mass flow", "mass_flow": 10.0},
```

Note

This is a deliberate spelling duplicate of the inflow and outflow `kind: 'massflow'` variants above — the same underlying concept has two different names: `'mass flow'` as a top-level `type` here, and `'massflow'` as a `kind` there. Both spellings are valid; unifying them would require a coordinated schema and C++ change.

Symmetry

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'symmetry'`	Boundary condition type

Example:

```
"BC_1": {"zones": [2, 3], "type": "symmetry"},
```

Periodic

This boundary condition needs to be specified in pairs with opposing transformations. The transforms specified should map each zone onto the matching zone.

Note

Each periodic boundary needs to provide the correct transform to the opposite matching boundary.

Note

The incompressible solver (`solver_settings.type: 'incompressible'`) only supports the **'linear'** (translational) periodic kind. **'rotated'** periodic boundaries are only supported by the compressible solver.

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'periodic'`	Boundary condition type
`kind`	Yes		`'rotated'` or `'linear'`	Selects the periodic transformation type (see note above on incompressible-solver support)
`transform`	Yes		See below	The periodic transformation, shape depends on `kind`

The options for the **'linear'** `transform` are:

Keyword	Required	Default	Valid values	Description
`vector`	Yes		`[x, y, z]`	Translation vector mapping this zone onto the matching periodic zone

Example linear periodic boundary condition:

```
"BC_3": {
    "zones": [11],
    "type": "periodic",
    "kind": "linear",
    "transform": {"vector": [0.0, 0.0, 1.0]},
},
```

The options for the **'rotated'** `transform` are (compressible solver only — see note above):

Keyword	Required	Default	Valid values	Description
`axis`	Yes		`[x, y, z]`: unit vector	Axis of rotation mapping this boundary onto the matching periodic zone
`origin`	Yes		`[x, y, z]`	Origin of the axis of rotation
`theta`	Yes		Number	Angle of rotation about the axis, in radians

Example rotated periodic boundary condition:

```
"BC_4": {
    "zones": [5],
    "type": "periodic",
    "kind": "rotated",
    "transform": {
        "theta": math.radians(120.0),
        "axis": [1.0, 0.0, 0.0],
        "origin": [0.0, 0.0, 0.0],
    },
},
```

Overset

The overset boundary condition should be applied to the boundaries of a mesh to be overset on top of a larger background mesh.

Keyword	Required	Default	Valid values	Description
`zones`	Yes		List of integers	Mesh zone IDs this boundary condition applies to
`type`	Yes		`'overset'`	Boundary condition type
`background`	No	unset (falls back to all-pairs connectivity)	Model name, list of model names, or unset	Model(s) whose mesh provides donor data at this boundary. Used to derive the blanking priority order — models are topologically sorted so a model always appears after the model(s) it names as background. When unset, the solver falls back to all-pairs connectivity against every preceding model in the topological order; explicit specification is preferred, as it avoids unnecessary mapper creation for non-overlapping mesh pairs
`idw_from_cells_or`	No	0	Integer >= 0	Interpolation order for IDW mapping (0 = standard, 1 = use neighbour info)
`additional_active`	No	0	Integer >= 0	Number of additional active cell layers near the blanking boundary
`moving_mesh_halo_`	No	0	Integer >= 0	Halo fringe layers around the hole when the overset mesh moves (0 = treat every blanked cell as a halo; N > 0 is sufficient when the hole is re-cut every real time step)

Example without an explicit background:

```
"BC_2": {"zones": [7], "type": "overset"},
```

Example with an explicit background model:

```
"BC_2": {"zones": [10], "type": "overset", "background": ["background"]},
```

1.5.3 Time Marching

zCFD is capable of performing both steady-state and time dependent simulations. Steady-state solutions are marched in pseudo time towards convergence. Time dependent simulations can be either globally time-stepped or use local pseudo time-stepping within a dual time-stepping iteration in physical time.

Two sibling dictionaries under `solver` control this:

- `time_settings` — the physical time layout: is the simulation steady, globally time-stepped, or dual-time-stepped, and (for unsteady runs) the total time, physical time step and physical time order.
- `convergence_control` — how each pseudo-time step marches towards convergence: the number of cycles, the pseudo-time scheme (explicit or implicit), and the CFL number/ramp.

Important

`convergence_control` only applies to **steady** and **dual time stepping** runs, because both drive an inner pseudo-time convergence loop. **Global time stepping has no pseudo-time loop** — it marches physical time directly — so `convergence_control` must not be set at all when `time_settings.kind` is `"global timestepping"`; a deck that sets both will fail validation. Global time stepping instead carries its own `scheme` directly inside `time_settings` (see *Global time stepping* below).

```
parameters = {
    "config_version": 2,
    "solver": {
        ...
        "time_settings": {...},
        "convergence_control": {
            "cycles": 5000,
            "scheme": {...},
            "cfl": 30.0,
        },
        ...
    },
    ...
}
```

Time Settings

`time_settings` selects steady or unsteady operation via `type`, and for unsteady runs, `kind` selects global or dual time stepping.

Keyword	Required	Default	Valid values	Description
`type`	Yes	–	`"steady"` | `"unsteady"`	Selects steady or time-dependent operation.
`kind`	Yes, if `type` is `"unsteac`	–	`"global timestepping"` | `"dual time stepping"`	Selects the unsteady time-marching mode. Not present (and not valid) when `type` is `"steady"`.
`total_time`	Yes, if `type` is `"unsteac`	–	Positive float	Total physical time to simulate.
`time_step`	Yes, if `type` is `"unsteac`	–	Positive float	Physical time step for the simulation.
`order`	No	`"second"`	`"first"` | `"second"` | `1` | `2`	Order of physical time integration.
`start`	No	`0`	Non-negative integer	How many steady-state pseudo-time cycles to run to initialise the solution before physical time-stepping begins.
`scheme`	Yes, if `kind` is `"global timeste`	–	`{...}` — see *Global time stepping*	Only present under global time stepping; dual time stepping and steady runs configure their scheme in `convergence_control` instead.

Note

`type: "steady"` takes no further keys — a steady run has no physical time layout at all; all of its pacing comes from `convergence_control`.

Running steady state

For a steady-state simulation, set `type` to `"steady"`. No `total_time`, `time_step` or `kind` is needed or accepted.

Example usage:

```
# Run a steady state simulation
"time_settings": {
    "type": "steady",
},
```

Note

Setting `total_time` equal to `time_step` under dual time stepping is rejected outright (`total_time == time_step` raises a validation error) — use `{"type": "steady"}` for a steady-state run instead.

Global time stepping

Global time stepping (GTS) advances the solution in physical time only, using the same physical time step in every cell — there is no pseudo-time convergence loop, so `convergence_control` must not be set for a GTS run. In cases where there are very small cells it would be impractical to use global time stepping, as every cell would have to march at the pace of the smallest.

Because GTS drives physical time directly, only the explicit pseudo-time schemes apply: `scheme.name` may be `"euler"` or `"runge kutta"`. `"implicit euler"` and `"lu-sgs"` are pseudo-time-only machinery and cannot drive a physical explicit march — setting either under a GTS `scheme` is rejected.

Example usage:

```
# Run a globally time-stepped unsteady simulation
"time_settings": {
    "type": "unsteady",
    "kind": "global timestepping",
    "total_time": 0.2,
    "time_step": 1.0e-5,
    "scheme": {
        "name": "runge kutta",
        "stage": 3,
    },
},
```

> **Note**
>
> Global time stepping restricts all cells to the smallest time step and should only be used for unsteady simulations.

Dual time stepping

Dual time stepping (DTS) advances the solution in physical time, with an inner pseudo-time convergence loop run at each physical time step to converge that step before advancing. Time integration is based on the solution at the current physical time step and the contribution of previous physical time steps, represented by a Backward Difference Formula; `order` controls the order of this formula.

`start` provides the option of initialising a DTS simulation with a number of steady-state pseudo-time cycles before physical time-stepping begins. These run at real-time cycle 0 with no physical time derivative, so they are the same solve as a `"steady"` run of the same length; physical time stepping begins at real-time cycle 1, where the time integration order builds up from first order to `order`. Restarting a DTS case from the results of a steady run (or from a results file written at the end of the start phase) skips real-time cycle 0 and begins time-accurate stepping at once, using the restart field as the time history.

The pseudo-time scheme, CFL and cycle count for each physical step are configured in `convergence_control`, exactly as for a steady run — see *Convergence Control* below.

Example usage:

The unsteady laminar cylinder test case sheds a vortex street with a specific Strouhal number. We want to run the simulation for 4 seconds of physical time, with a small enough time step to capture the unsteady shedding phenomenon, and 20 pseudo-time cycles of multistage Runge-Kutta per physical step to converge each step:

```
# Run an unsteady simulation for 4 seconds with a physical time step of 0.002 seconds
"time_settings": {
    "type": "unsteady",
    "kind": "dual time stepping",
    "total_time": 4.0,
    "time_step": 0.002,
    "order": 2,
    "start": 0,
},
"convergence_control": {
    "cycles": 20,
    "scheme": {"name": "runge kutta", "stage": 5},
    "cfl": {
        "cfl": 2.5,
        "cfl ramp": [{"type": "exponential", "factor": 1.05, "initial": 0.1}],
    },
},
```

The example below uses an implicit pseudo-time scheme so far fewer cycles are needed per physical step, and starts with 500 steady-state pseudo-time cycles before the unsteady simulation begins:

```
"time_settings": {
    "type": "unsteady",
    "kind": "dual time stepping",
    "total_time": tt,
    "time_step": tt / 400,
    "order": "second",
    "start": 500,
},
"convergence_control": {
    "cycles": 500,
```

(continues on next page)

(continued from previous page)

```
    "scheme": {"name": "implicit euler", "stage": 1},
    "cfl": {
        "cfl": 15,
        "cfl ramp": [{"type": "exponential", "factor": 1.02, "initial": 1.0}],
    },
},
```

Convergence Control

`convergence_control` paces the inner pseudo-time loop towards convergence: how many cycles to run, which scheme drives each pseudo-step, and — on the compressible solver — the CFL number sizing it. It is required for steady and dual-time-stepping runs, and must be absent for global time stepping (see the note at the top of this page).

The rest of this section describes the compressible block. `cycles` is common to both solvers; the incompressible block replaces `scheme` with the pressure-velocity coupling scheme and takes no `cfl` — see *Incompressible convergence control*.

```
"convergence_control": {
    "cycles": 5000,
    "scheme": {"name": "implicit euler"},
    "cfl": 30.0,
},
```

Cycles

The total number of pseudo time-steps to use in a *steady-state* simulation, or the number of pseudo time-steps to use in each physical time-step of a *dual time stepping* simulation.

Keyword	Required	Default	Valid values
`cycles`	Yes	–	Any positive integer.

Example usage:

```
# A good starting value for most explicit, steady state cases. Implicit simulations␣
↪can use a higher CFL and therefore require fewer cycles - around 100 - 500 cycles␣
↪is a good starting point for an implicit steady state case.
"cycles": 5000,
```

Scheme

There are four pseudo-time integration schemes available in zCFD: two explicit (`"euler"`, `"runge kutta"`) and two implicit (`"implicit euler"`, `"lu-sgs"`). These can be used for steady runs and for dual-time-stepped unsteady runs. Global time stepping is restricted to the two explicit schemes (see *Global time stepping* above).

Keyword	Required	Default	Valid values	Description
`name`	No	`"runge kutta"`	`"euler" | "runge kutta" | "implicit euler"|"lu-sgs"`	Name of the pseudo-time scheme. A `scheme` dict that omits `name` defaults to `"runge kutta"`.
`stage`	No	–	`1|3|4|5|"rk third order tvd"`	(`"runge kutta"`, `"euler"` and `"implicit euler"` only) Number of Runge-Kutta stages. Has no effect under `"euler"` or `"implicit euler"` (both are single-stage by construction); accepted on those two for backward compatibility with decks that carry it as boilerplate.

Note

The `"euler"` scheme has a single stage and is the safest option for new or problematic cases.

name

The solution is integrated in pseudo-time using a scheme that is either explicit (variables driving the change are evaluated at the previous step) or implicit (variables driving the change are evaluated at the current step). `"euler"` and `"runge kutta"` are the explicit schemes; `"implicit euler"` and `"lu-sgs"` are the implicit options.

A larger CFL — and hence larger pseudo-time step — can be taken with an implicit scheme, since the method is more stable than an explicit scheme.

Note

The trade-off in using implicit time integration is an increase in memory requirement, typically 3-5 times more than for an explicit scheme.

stage

The number of Runge-Kutta stages in each pseudo-time step affects the order of temporal convergence and the maximum CFL that can be used.

Note

For most test cases using Runge-Kutta integration, 5 stages are applied. This is a typical industry standard and has been shown to be robust for most test cases.

Example usage:

```
# A 5-stage Runge-Kutta explicit scheme
"convergence_control": {
    "scheme": {"name": "runge kutta", "stage": 5},
    ...
},
```

implicit euler

The default implicit scheme, solving the full linearised system with the *Petsc or AMGX linear solver* each pseudo-time step.

```
# An implicit scheme with a ramped CFL, typical for a RANS steady-state case
"convergence_control": {
    "scheme": {"name": "implicit euler"},
    "cfl": {
        "cfl": 100.0,
        "cfl ramp": [{"type": "exponential", "factor": 1.1, "initial": 0.05}],
    },
    "cycles": 3000,
},
```

lu-sgs

A matrix-free, point-implicit LU-SGS/DP-LUR scheme. It avoids assembling and inverting the full linear system, trading some convergence rate for lower memory use and, often, faster wall-clock time per cycle than `"implicit euler"`.

Keyword	Required	Default	Valid values
`sub_iterations`	No	–	Any positive integer. Number of LU-SGS/DP-LUR sweeps per pseudo-time step.
`over_relaxation`	No	–	Any positive float. Diagonal over-relaxation factor for the LU-SGS/DP-LUR sweeps.
`turbulence_solver`	No	`"point-implicit"`	`"amg"` | `"point-implicit"`. `"amg"` keeps the assembled AMG solve for the turbulence equations; `"point-implicit"` uses the matrix-free sweep instead.

```
# LU-SGS scheme with a CFL ramp
"convergence_control": {
    "scheme": {"name": "lu-sgs"},
    "cfl": {
        "cfl": 20.0,
        "cfl ramp": [{"type": "exponential", "factor": 1.1, "initial": 0.05}],
    },
    "cycles": 30000,
},
```

CFL Control

The Courant-Friedrichs-Lewy (CFL) number controls the local pseudo time-step that the solver uses to reach a converged solution. The larger the CFL number, the faster the solver will run but the less stable it will be. `cfl` accepts either a plain number, or a dictionary for finer-grained control:

Keyword	Required	Default	Valid values
`cfl`	Yes	`1.0`	A positive float, or a dictionary (below) for detailed control.
`cfl.cfl`	Yes (within the dict form)	–	Any positive floating point number.
`cfl.cfl_turbulence`	No	–	Any positive floating point number. Working CFL for the turbulence transport equations; has no effect when no turbulence model is active.
`cfl.cfl_coarse`	No	Current CFL number	Any positive floating point number. Only valid when `numerical_scheme.multigrid` is set (see *Multigrid*).
`cfl.cfl_ramp`	No	–	List of ramp specifications, or a callable — see *cfl_ramp* below.
`cfl.cfl_viscous_factor`	No	–	Any positive floating point number.

Note

A bare number, e.g. `"cfl": 30`, is shorthand for `"cfl": {"cfl": 30}`. Any of the other keys can be added alongside it; zCFD hoists them into the equivalent dictionary form automatically.

cfl

Sets the CFL number used in the mass, momentum and energy equations.

Example usage:

```
# Typical value used for a RANS simulation using explicit time marching
"cfl": 1
```

```
# Typical value used for a RANS simulation using implicit time marching
"cfl": 30
```

Note

Besides mesh quality, CFL is the most important parameter affecting simulation stability. If you find your simulation does not converge, consider reducing CFL.

cfl_turbulence

Sets the working CFL number used to update the turbulent transport equations (for example k and ω).

```
# Use cfl_turbulence to promote solver stability
"cfl": {"cfl": 30, "cfl_turbulence": 20},
```

Note

If you find your simulation does not converge and reports high turbulence equation residuals which increase with successive cycles, consider setting this to a value lower than **cfl**.

cfl_coarse

For simulations with multigrid acceleration, `cfl_coarse` is used as a CFL condition on the coarse meshes. For such cases `cfl` is still applied to the finest mesh. This only applies to explicit finite-volume solutions, and only when `numerical_scheme.multigrid` is set.

```
# The quality of agglomerated (multigrid) coarse meshes is typically less than the␣
↪quality of the primary (fine) mesh and so a lower CFL number may be required for␣
↪numerical stability.
"cfl": {"cfl": 2.5, "cfl_coarse": 1.0, "cfl_turbulence": 1.0},
```

cfl_ramp

This allows a relatively large target CFL number to be specified, but starts the simulation using a smaller one to provide stability as large non-physical transients are eliminated from the solution. Starting a simulation with a large CFL number can cause instability that crashes the solver; a more gentle start reduces the impact of non-physical transients.

`cfl_ramp` is always a **list**. Each entry names its shape with a `type` key and describes one ramp; a single ramp is a list of one:

```
# Start cfl at 0.1 and grow at a rate 1.1 until the target cfl of 30 is reached
"cfl": {
    "cfl": 30,
    "cfl_ramp": [{"type": "exponential", "factor": 1.1, "initial": 0.1}],
},
```

The available shapes match the ramp builders in `zutil.control.ramp`:

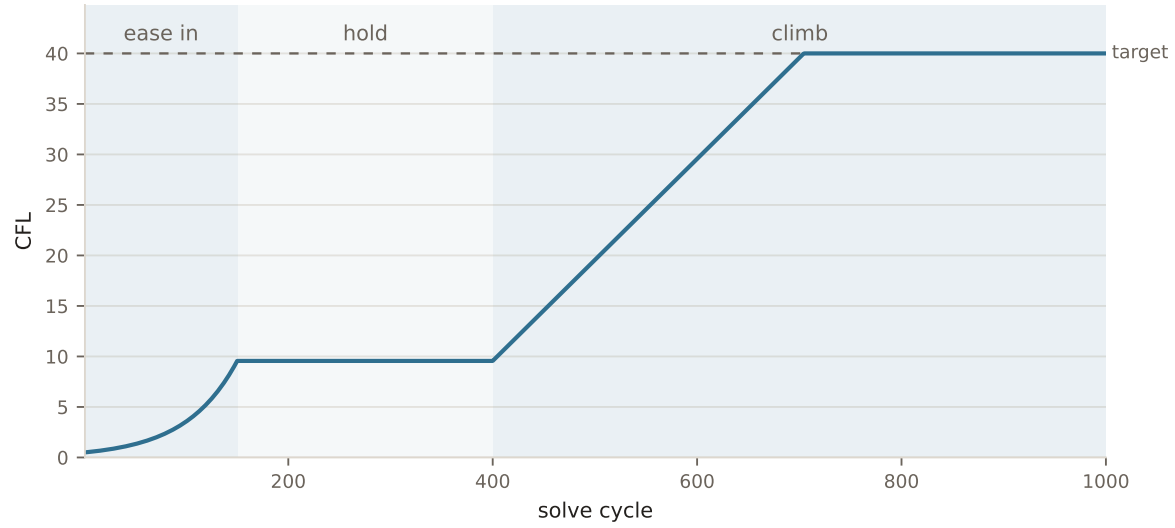
type	Extra keys	Description
`"linear"`	`increment`, `start_cycle`, `end_cycle`	Adds `increment` to the CFL every cycle between `start_cycle` and `end_cycle`.
`"stepped_linear"`	`increment`, `step_cycles`, `start_cycle`, `end_cycle`	Adds `increment` every `step_cycles` cycles.
`"exponential"`	`factor`, `start_cycle`, `end_cycle`, `max_allowed`	Multiplies the CFL by `factor` (>= 1.0) each cycle, capped at `max_allowed`.

Several entries run one after another. Every entry is called on every cycle, each acting on the previous entry's output, so an entry confines itself to its own stretch of the run with `start_cycle` and `end_cycle`:

```
# Ease in exponentially for 200 cycles, then climb steadily to the target
"cfl": {
    "cfl": 30,
    "cfl_ramp": [
        {"type": "exponential", "factor": 1.1, "initial": 0.1, "end_cycle": 200},
        {"type": "linear", "increment": 0.5, "start_cycle": 201},
    ],
},
```

Holding the CFL steady for a stretch is a `"linear"` entry with an `increment` of `0`. This is useful when a case needs time at a low CFL to shed its startup transients before being pushed harder — ease in, hold while the flow settles, then climb to the target:

```
"cfl": {
    "cfl": 40,
    "cfl_ramp": [
        # ease in gently from 0.5
        {"type": "exponential", "factor": 1.02, "initial": 0.5, "end_cycle": 150},
        # hold there while the startup transients wash out
        {"type": "linear", "increment": 0.0, "start_cycle": 151, "end_cycle": 400},
        # then climb steadily until the target is reached
        {"type": "linear", "increment": 0.1, "start_cycle": 401},
    ],
},
```



Cycles no entry covers hold the CFL where it is, so the middle entry above could be left out entirely and the profile would be the same. Writing the hold makes it visible to the next person reading the control dictionary.

Every shape takes `initial`, the CFL the ramp starts from. It is returned at the ramp's `start_cycle`, so the ramp sets its own starting point:

```
"cfl": {
    "cfl": 30,
    "cfl_ramp": [
        {
            "type": "exponential",
            "factor": 1.1,
            "initial": 0.1,
            "max_allowed": 30,
        }
    ],
},
```

A Python callable can be supplied instead of the list — see the *ramp function* reference. The callable is invoked as `func(cycle, current_cfl)` and returns either the new CFL, or a dictionary mapping any of `cfl`, `cfl_turbulence` and `cfl_coarse` to drive those alongside it (keys not returned keep their static value); a three-argument `(solve_cycle, real_time_cycle, cfl)` callable is not accepted.

cfl_viscous_factor

Factor applied to the diffusive (viscous) time step when applying the CFL condition. As viscous effects are rarely the cause of solver instability, the contribution of the viscous terms to the time step evaluation may be reduced by setting `cfl_viscous_factor` to a number less than 1.

```
# For a low Reynolds number case, viscous effects are dominant but do not contribute␣
↪to instability.
# A value of 1e-6 effectively discounts viscosity in the determination of the time␣
↪step when applying the CFL condition.
"cfl": {
    "cfl": 200,
    "cfl_viscous_factor": 1.0e-6,
    "cfl_ramp": [{"type": "exponential", "factor": 1.05, "initial": 0.1}],
},
```

Inner cycle convergence

Choosing `cycles` for an unsteady simulation means guessing how many pseudo time iterations each real time step needs at the chosen `cfl`. The `inner_convergence` dictionary lets the solver decide instead: each real time step ends once its inner iteration has converged, and `cycles` becomes an upper bound rather than a fixed count.

It applies only to *dual time stepping* runs — there is no inner iteration to converge in a steady or globally time-stepped run, and a deck that sets it for one fails validation. Omit the dictionary entirely to run the full `cycles` on every real time step. Both solvers honour the same dictionary; what `target_orders` is measured on differs between them and is set out under *Choosing a target*.

Keyword	Required	Default	Valid values	Description
`target_orders`	No	1.0	Float	Orders of magnitude the judged measure must fall within the current real time step: `InnerRatio` for the compressible solver, `InnerDrop` for the incompressible solver. Scale free in both, so one value carries between cases. Set to 0 to disable.
`target_ratio`	No	unset (off)	Float	Compressible solver only. Absolute floor on `InnerRatio` itself, ORed with `target_orders`. Not portable between cases — measure it from a run first. Unset or 0 disables it; an incompressible deck that sets it fails validation.
`min_cycles`	No	5	Positive integer	Never end a real time step before this many inner cycles.
`check_frequency`	No	5	Positive integer	How often the criterion is tested. A check cycle is also a reporting cycle.
`variables`	No	`"mean flow"`	`"mean flow"` | `"all"` | list	Which residuals the criterion is judged on; the worst of them governs. `"mean flow"` is density, momentum and energy for the compressible solver, momentum and pressure for the incompressible coupled scheme, and momentum alone for SIMPLE. `"all"` adds the turbulence transport equations. A list names report columns directly.
`stall_window`	No	10	Non-negative integer	Give up if the reduction improves by less than `stall_tolerance` over this many checks. 0 disables stall detection.
`stall_tolerance`	No	0.05	Float	Orders of magnitude that count as progress for stall detection.
`start_real_time_st`	No	0	Non-negative integer	Real time step from which the criterion may end a step early. Before it, `cycles` is used unchanged. The diagnostics are reported throughout either way.

Example usage:

```
# Let each real time step run until the mean flow residual has dropped two
# orders, but never more than 50 inner cycles
"time_settings": {
    "type": "unsteady",
    "kind": "dual time stepping",
    "total_time": 1.7,
    "time_step": 0.002,
    "order": "second",
},
"convergence_control": {
```

(continues on next page)

(continued from previous page)

```
    "cycles": 50,
    "cfl": 2.0,
    "inner_convergence": {
        "target_orders": 2.0,
        "min_cycles": 5,
        "check_frequency": 5,
    },
},
```

Note

The criterion is not applied to the first real time step. That step runs the `start` cycles from a uniform initial field while the CFL number is still ramping, so its residual history is not comparable with the rest of the simulation.

Note

A real time step that converges early still runs one further inner cycle, on which it writes its report row, checkpoint and visualisation output.

Choosing a target

`target_orders` is measured on `InnerRatio`: the largest, across the selected equations, of each residual divided by *its own* physical rate of change. Two properties matter. Dividing each equation by its own rate makes the comparison dimensionless, so taking the largest across equations is meaningful where taking the largest raw residual would not be. And taking the *largest* tracks the equation with the most work still to do.

Contrast `InnerDrop`, which is the smallest drop across the same equations and is reported as a diagnostic only. Residuals are not comparable between equations, so one can start well below the others simply because that is its scale, having almost nothing left to resolve; the smallest drop then picks exactly that equation and stays near zero however well the rest of the field converges. In a representative IDDES case the momentum equations dropped 2.2 orders while density dropped 0.07 and governed, so a criterion on `InnerDrop` could never fire. `InnerRatio` on the same case fell a consistent 1.8 orders per real time step.

Because `target_orders` is a reduction in a ratio rather than an absolute level, one value carries between cases. To choose it, run once with the criterion off and look at `InnerRatioDrop` through a real time step. Most of the reduction happens in the first handful of inner cycles at the chosen `cfl` and the curve then has a long flat tail — on that IDDES case 0.7 orders by cycle 5, 1.0 by cycle 10 and 1.8 by cycle 50, so five cycles bought some four fifths of the actual reduction in residual and the remaining forty-five bought the rest. Set `target_orders` near the knee with `min_cycles` at the knee's cycle number.

Use `target_ratio` only when an absolute floor is wanted as well; it is ORed with `target_orders` and its value has to be measured per case.

The incompressible measure

The incompressible solver has no pseudo-time residual to form a ratio from: its report carries the nonlinear residual of the outer iteration, already normalised by the matrix diagonal and by each variable's own range, so it is scale free per equation before anything divides it. `target_orders` is therefore judged on `InnerDrop` — the orders the worst selected residual has fallen since the first inner cycle of the step — and `InnerRatio` and `InnerRatioDrop` are reported as zero. `target_ratio` has nothing to test and is rejected.

Under the coupled scheme `"mean flow"` is momentum and pressure; the pressure row is the continuity constraint and is the equation that most often governs. Under SIMPLE the pressure entry in the report is the residual left by the pressure-correction solve, bounded by the linear solver's tolerance rather than by the flow, so it is not judged and `"mean flow"` is momentum alone.

The default of 1.0 was set on the coupled scheme, where a developed cylinder wake ends the median real time step after 10 of 50 inner cycles and the cycles given up are worth about a quarter of an order. SIMPLE converges each step less far — on the same flow its momentum residual falls about 0.8 orders in the first five cycles and then holds — so it needs a lower target, 0.5 in the shipped regression case, and meets it on most steps rather than all. Read `InnerDrop` through a few steps with the criterion off before choosing, exactly as for `InnerRatioDrop` above.

Note

The equation governing `InnerRatio` can change from one inner cycle to the next, so `InnerRatioDrop` compares the worst ratio now against the worst at the first cycle rather than following a single equation. It reads as how far the bottleneck has improved, and flattens when a slower equation takes over — which is the signal, not an artefact.

Note

A solution restarted from a steady state and left to develop — RANS into DES, or a wake that has yet to break down — looks converged on every real time step for as long as the unsteadiness is still small: the residual starts low, barely moves, and sits far below the physical rate of change. Ending those steps early would cut the inner iteration short exactly while the instability that has to grow is being resolved. Use `start_real_time_step` to leave that phase alone and let the criterion take over once the flow is established.

Reading the inner convergence

Whether or not an automatic criterion is set, a dual time-stepping run reports its inner cycle convergence to the *report file* in the `InnerCycle`, `InnerDrop`, `InnerRatio` and `InnerRatioDrop` columns, joined by `InnerCells` on the compressible solver only — see the *per-column table* for which columns carry meaning on each solver. `InnerDrop` is the orders of magnitude the worst mean flow residual has dropped since the first inner cycle of the current real time step, which is the quantity to judge `cycles` by.

The global residual is a poor guide here for two reasons. It is an absolute value that is never normalised per real time step, so its magnitude says nothing about how far the current inner iteration has come; and it is an RMS over cells, so a handful of cells whose limiter switches between iterations can hold the whole norm up at a noise floor while the bulk of the field is still converging cleanly.

To see the convergence of each real time step separately:

```
from zutil.plot import zCFD_Plots
plots = zCFD_Plots("cylinder.py")
plots.plot_inner_convergence()
```

This plots `InnerDrop` against the inner cycle index, one curve per real time step. Once the startup transient has passed the curves collapse onto each other, and the cycle at which they flatten is where `cycles` should be set. At the end of the run the solver also logs the median drop achieved and whether `cycles` looks too high or too low.

If the curves flatten well short of the target, see *troubleshooting* — a stall caused by limiter switching is fixed by freezing the limiter, not by adding cycles.

Incompressible convergence control

The incompressible solver marches each pseudo-step with an implicit solve rather than a CFL-limited time step, so its `convergence_control` carries `cycles`, the pressure-velocity coupling `scheme`, and two startup ramps. There is no `cfl` key.

```
"convergence control": {
    "cycles": 5000,
    "scheme": {"name": "coupled", "implicit relax": 0.9},
},
```

scheme

The coupling scheme decides how velocity and pressure are solved for together. `scheme.name` selects it, and the relaxation driving it lives in the same block.

Keyword	Required	Default	Valid values	Description
`name`	No	`"coupled"`	`"coupled"` | `"simple"`	`coupled` solves a single block-4 [u,v,w,p] system per pseudo-time step. `simple` uses a segregated SIMPLE momentum predictor followed by a pressure correction.
`implicit relax`	No	0.5	Float, 0.1 to 1.0	Under-relaxation factor (alpha_u) applied to the implicit momentum, pressure and energy updates. Lower is more stable and slower.
`turbulence relax`	No	0.5	Float, > 0.0 to 1.0	Under-relaxation factor on the turbulence transport update. Has no effect without a turbulence model.
`pin pressure`	No	`False`	`True` | `False`	Pin the pressure at a reference cell with a large diagonal instead of the default constant-null-space plus mean-removal gauge fix. Needed on a fully closed domain (no fixed-pressure boundary), where the pressure is otherwise determined only up to a constant and the matrix is singular. The pin degrades AMG conditioning, hence the default.

The coupling scheme applies to the whole run — it selects the solver itself — so unlike the rest of `convergence_control` it cannot be overridden per mesh.

Under `"simple"` three further keys configure the pressure-correction solve. They are rejected under `"coupled"`, which performs no such solve:

Keyword	Required	Default	Valid values	Description
`pressure relax`	No	0.3	Float, > 0.0 to 1.0	Explicit pressure under-relaxation factor (alpha_p) applied as `p += alpha_p * p'`.
`pressure regularisation`	No	–	Float, >= 0.0	Diagonal regularisation of the pressure-correction Poisson operator, lifting its near-zero constant mode so a solver without null-space projection returns a bounded correction. Vanishes at convergence, so the steady solution is unbiased.
`pressure correction limit`	No	–	Float, > 0.0	Limit factor applied to the pressure correction.

The PETSc options for the SIMPLE momentum and pressure-correction solves are separate — see *Linear solver*.

```
"convergence control": {
    "cycles": 5000,
    "scheme": {
        "name": "simple",
        "implicit relax": 0.7,
        "pressure relax": 0.3,
    },
},
```

Startup ramps

Both ramps ease an impulsive start and are active only during the first real-time step. Each takes `initial` (the value on the first pseudo cycle) and `cycles` (how many pseudo cycles it takes to reach 1).

Keyword	Required	Default	Description
`viscosity ramp`	No	`{"initial": 100.0, "cycles": 300.0}`	Scales the effective viscosity, decaying log-linearly from `initial` to 1, so the start is more diffusive (a lower effective Reynolds number). An `initial` of 1 or less disables it.
`relaxation ramp`	No	`{"initial": 1.0, "cycles": 300.0}`	Scales the under-relaxation factors above, rising linearly from `initial` to 1, so the solver starts more relaxed. An `initial` of 1 or more (the default) disables it.

```
# Start heavily relaxed and diffusive, easing to the configured values over 500␣
↪cycles
"convergence control": {
    "cycles": 5000,
    "scheme": {"name": "coupled", "implicit relax": 0.9},
    "viscosity ramp": {"initial": 100.0, "cycles": 500.0},
    "relaxation ramp": {"initial": 0.2, "cycles": 500.0},
},
```

Multigrid

Multigrid is configured on `solver.numerical_scheme` (spatial discretisation), and only applies for compressible cases running on the explicit pseudo-time advance path: it requires the pseudo-time loop to exist (steady or dual time stepping, never global time stepping) and the pseudo-time `scheme` to be explicit (`"euler"` or `"runge kutta"`, or unset). Setting it under the incompressible solver, under global time stepping, or alongside an implicit scheme, is rejected.

Keyword	Required	Default	Valid values
`levels`	Yes	–	Integer, 1 to 10.
`cycles`	No	100000000	Any positive integer.
`multigrid_ramp`	No	–	Multigrid ramp function or dict.
`prolong_factor`	No	–	Float. Prolongation factor.
`prolong_transport_fac`	No	–	Float. Prolongation factor for the turbulence/transport equations.

To accelerate an explicit finite volume solution, coarse meshes are generated from the original (fine) mesh by agglomerating cells together to create larger cells. `levels` sets the number of successive times to apply this operation, each iteration resulting in a coarser mesh with fewer cells. By solving on the hierarchy of coarser meshes as well as the fine mesh, information is moved faster through the flow domain and a converged solution is reached more efficiently. The multigrid scheme is designed so that the acceleration does not (in theory) change the result that would be obtained just by solving on the finest mesh alone, though in practice the use of multigrid does ultimately stall convergence on most meshes.

Use multigrid acceleration for a specified number of `cycles` and then revert to the finest mesh only. This makes use of the more efficient scheme at the start, and avoids convergence stall due to multigrid at the end of a solution process.

Example usage:

```
# 3 levels of multigrid is sufficient in most cases to reduce the number of cycles␣
↪required by an order of magnitude.
# If this causes instability, a reduction to 2 may solve the problem.
"numerical_scheme": {
    ...
    "multigrid": {"levels": 3},
    ...
},
```

```
# After 20000 explicit cycles, multigrid stall prevents convergence to the correct␣
↪drag value.
# By turning it off after that point, the correct value is obtained on the finest␣
↪mesh.
"numerical_scheme": {
    ...
    "multigrid": {"levels": 3, "cycles": 20000},
    ...
},
```

> **Note**
>
> Multigrid only applies to explicit finite volume solutions on the compressible solver.

Moving Mesh

Mesh motion is controlled by a per-fluid-zone `moving_mesh` flag on a rotating or translating fluid zone, and requires an overset boundary condition to be present on the same mesh (mesh motion needs an overset assembly to move within; a rotating or translating zone in a non-overset case uses an ALE reference frame instead, so `moving_mesh` has no effect there and is rejected).

Keyword	Required	Default	Valid values
`moving_mesh`	No	`False`	`True` | `False`. Only settable on a rotating or translating fluid zone whose mesh has an overset boundary condition.

Example usage:

```
"fluid_zones": {
    "FZ_1": {
        "type": "rotating",
        "axis": [0.0, 0.0, 1.0],
        "origin": [0.0, 0.0, 0.0],
        "omega": 10.0,
        "moving_mesh": True,
    },
},
```

For simulations where there is no relative movement of interest, `moving_mesh` should be left at its default of `False` and the zone runs in its own (rotating or translating) reference frame. Setting `moving_mesh` to `True` forces a recalculation of the physical location of the mesh every step, including the recalculation of the overset mapping.

1.5.4 Linear Solver Settings

There are several situations where the options selected in zCFD require the solution of a linear system. These are:

- When the convergence-control *scheme* is set to “implicit euler” or “lu-sgs”.
- When RBFs are used for mesh motion.
- When the incompressible solver is selected.

When running on CPUs zCFD uses the Petsc linear solver library and when running on NVidia GPUs zCFD uses the AMGX linear solver library. Both libraries offer a wide array of solver and preconditioner choices and altering the settings can have a significant impact on the performance and convergence of the solver. Options can be passed to Petsc via the control dictionary and AMGX options are set using a json file which is read at runtime.

These settings live under `solver.solver_settings.linear_solver_options`:

```
parameters = {
    "config_version": 2,
    "solver": {
        ...
        "solver_settings": {
```

(continues on next page)

(continued from previous page)

```
            "type": "compressible",
            "linear_solver_options": {
                "flow": {...},
                "turbulence": {...},
                "rbf": {...},
                "double_precision": False,
            },
        },
        ...
    },
    ...
}
```

Petsc

Details of the available options for the Petsc KSP linear system solvers used by zCFD are given here. These options are passed through to Petsc via the `linear_solver_options` dictionary. The keys are:

Keyword	Required	Default	Valid values
`flow`	No	See below	Dictionary of PETSc option flags (as strings) for the mean flow linear system.
`turbulence`	No	See below	Dictionary of PETSc option flags for the turbulence linear system.
`rbf`	No	See below	Dictionary of PETSc option flags for the RBF mesh-motion linear system.
`double_precision`	No	`False` (compressible) / `True` (incompressible)	`True` | `False`. Controls whether the mean flow linear system is solved in single or double precision when running on GPUs with AMGX (see below).

Each of `flow`, `turbulence` and `rbf` is a plain dictionary of PETSc command-line-style flags (key/value pairs, both strings). Where a PETSc option doesn't take a value, an empty string must be supplied. Most settings detailed in the Petsc documentation can be passed through this way. The compressible defaults are:

```
"linear_solver_options": {
    "flow": {
        "-ksp_min_it": "2",
        "-ksp_type": "fgmres",
        "-ksp_rtol": "1.0e-3",
        "-ksp_monitor": "",
        "-ksp_converged_reason": "",
    },
    "turbulence": {
        "-ksp_min_it": "2",
        "-ksp_type": "fgmres",
        "-ksp_rtol": "1.0e-5",
        "-ksp_monitor": "",
        "-ksp_converged_reason": "",
    },
    "rbf": {
        "-ksp_type": "fgmres",
        "-ksp_rtol": "1.0e-5",
        "-ksp_monitor": "",
    },
```

(continues on next page)

(continued from previous page)

```
    "double_precision": False,
}
```

The Hypre library of algebraic multigrid methods is available via Petsc PCHYPRE and the associated settings can be supplied via the same dictionaries. Using Hypre's boomeramg package as a preconditioner has shown good performance with the incompressible solver:

```
"linear_solver_options": {
    "flow": {
        "-ksp_type": "fgmres",
        "-ksp_rtol": "1.0e-3",
        "-ksp_monitor": "",
        "-ksp_converged_reason": "",
        "-pc_type": "hypre",
        "-pc_hypre_type": "boomeramg",
    },
    ...
}
```

Further Hypre options are detailed in the Petsc PCHYPRE documentation.

Incompressible solver

When `solver_settings.type` is `"incompressible"`, `linear_solver_options` defaults to tighter tolerances and double precision, since the pressure-Poisson system is more sensitive to linear-solver accuracy:

```
"linear_solver_options": {
    "flow": {
        "-ksp_min_it": "2",
        "-ksp_type": "fgmres",
        "-ksp_rtol": "1.0e-7",
        "-ksp_atol": "1.0e-16",
        "-ksp_max_it": "20",
        "-ksp_converged_reason": "",
        "-ksp_monitor": "",
    },
    "turbulence": {
        "-ksp_min_it": "2",
        "-ksp_type": "fgmres",
        "-ksp_rtol": "1.0e-12",
        "-ksp_atol": "1.0e-16",
        "-ksp_converged_reason": "",
        "-ksp_monitor": "",
        "-ksp_max_it": "20",
    },
    "rbf": {},
    "double_precision": True,
}
```

Two further keys are available, but only when the incompressible solver uses the SIMPLE coupling scheme (`solver.convergence_control.scheme.name == "simple"`); they have no effect and are rejected under the coupled scheme:

Keyword	Required	Default	Valid values
`pressure_correct`	No	Falls back to `turbulence`	Dictionary of PETSc options for the SIMPLE pressure-correction (block-1 Poisson) solver.
`momentum`	No	Falls back to `flow`	Dictionary of PETSc options for the SIMPLE momentum predictor (block-3) solver.

AMGX

The AMGX settings for the mean flow, turbulence and RBF linear systems are read from json files (found in ZCFD_HOME/amgx.json, ZCFD_HOME/turbamgx.json and ZCFD_HOME/RBF_amgx.json) rather than from the control dictionary. These files are AMGX's own native configuration format and are unrelated to the zCFD control file's `config_version` key — the `"config_version": 2` in the example below is AMGX's own file-format version, not zCFD's. The mean flow settings are shown below:

```
{
  "config_version": 2,
  "determinism_flag": 1,
  "solver": {
    "preconditioner": {
      "error_scaling": 0,
      "print_grid_stats": 0,
      "max_uncolored_percentage": 0.05,
      "algorithm": "AGGREGATION",
      "solver": "AMG",
      "smoother": "MULTICOLOR_GS",
      "presweeps": 0,
      "selector": "SIZE_8",
      "coarse_solver": "NOSOLVER",
      "max_iters": 1,
      "postsweeps": 3,
      "min_coarse_rows": 32,
      "relaxation_factor": 0.75,
      "scope": "amg",
      "max_levels": 40,
      "matrix_coloring_scheme": "PARALLEL_GREEDY",
      "cycle": "V"
    },
    "use_scalar_norm": 1,
    "solver": "FGMRES",
    "print_solve_stats": 1,
    "obtain_timings": 1,
    "max_iters": 10,
    "monitor_residual": 1,
    "gmres_n_restart": 5,
    "convergence": "RELATIVE_INI_CORE",
    "scope": "main",
    "tolerance": 1e-3,
    "norm": "L2"
  }
}
```

In general these settings provide good performance for most cases. If issues are encountered converging the linear system, reducing the "tolerance" may help. Otherwise altering the multigrid aggregation selection algorithm "selector": "SIZE_8" to "SIZE_4" or "SIZE_2" will build smaller aggregates and hence increase the number of coarse levels - improving convergence at the expense of memory use. Increasing the number of "gmres_n_restart" may also improve convergence at the expense of increased memory use.

There are too many AMGX options to detail here but example configurations for different solvers are given here.

When running on NVidia GPUs and using AMGX, the only linear-solver-related option that still comes from the zCFD control dictionary is `double_precision`, on `linear_solver_options`:

```
"linear_solver_options": {"double_precision": False}
```

The `double_precision` option controls whether the mean flow linear system is solved in single or double precision. Solving in single precision brings a reduction in memory use and a speed up to the solver. However, double precision may be required for more challenging cases.

1.5.5 Equations

The `equations` block lives under the top-level `solver` key and selects the equation set to be solved:

```
parameters = {
  "config_version": 2,
  "solver": {
    "equations": {
      "type": "rans",
      "turbulence": {"model": "sst"},
    },
    # ... numerical_scheme, fluid_properties, reference_conditions, ...
  },
}
```

`type` is case-insensitive at input (`"RANS"`, `"rans"` and `"Rans"` all validate the same way), but its canonical, stored form is always lowercase.

Whether the *compressible* or *incompressible* solver is used is a separate choice, made by `solver.solver_settings.type` (`"compressible"` or `"incompressible"`), not by `equations.type` itself. The same `equations.type` values (`"viscous"`, `"rans"`, `"les"`) are shared between both solvers — the incompressible solver simply accepts a handful of extra keys alongside them (see *Incompressible Viscous*). `"euler"` is the one exception: the incompressible solver has no Euler equation set, since inviscid, incompressible flow with no pressure-velocity coupling mechanism is not a physically meaningful system to solve.

Solver	Description	Compressible	Incompressible
Euler	Use the Euler flow equations in the simulation. These are compressible, inviscid flow equations.	*"euler"*	–
Viscous	Use the viscous flow equations. These are compressible, viscid flow equations which do not model turbulence.	*"viscous"*	*"viscous"*
RANS	Use the RANS (Reynolds-Averaged Navier-Stokes) equations in the simulation. These are compressible, viscid flow equations with additional terms included to account for the effect of turbulence on the flow without the expense of simulating the turbulent flow structures themselves.	*"rans"*	*"rans"*
LES	Use the Large Eddy Simulation (LES) equations.	*"les"*	*"les"*

Common Settings

Every equation set shares two fields: `type` (above) and `precondition`.

Keyword	Required	Default	Valid values	Description
`type`	Yes	–	`"euler"` | `"viscous"` | `"rans"` | `"les"`	Selects the equation set.
`precondition`	No	`False`	`True` | `False`	Enable low Mach number preconditioning.

precondition

Low-speed Mach preconditioning modifies the pseudo-time system to restore good conditioning at low Mach number, so it only has meaning where a pseudo-time loop exists: steady runs and dual time stepping. Under pure global time stepping there is no pseudo-time loop to precondition, so `precondition` is not available there. `False` is always inert, so it never conflicts with a time-marching choice.

Turbulence

The `turbulence` key is required for the *"rans"* equation set. Its shape depends on the equation set: RANS uses the model documented in this section; LES uses a different, much smaller model documented under *"les"*.

Keyword	Required	Default	Valid values	Description
`model`	Yes	–	`"sst"` | `"sas"` | `"sa-neg"` | `"sst-transition"` | `"spalart-allmara`	RANS turbulence model.
`hybrid`	No	–	`"DES"` | `"DDES"` | `"IDDES"` | `"SAS"`	Hybrid RANS/LES approach layered on top of `model`.
`limit_mut`	No	`True`	`True` | `False`	Limit eddy viscosity in the SST model.
`limit_gradient_k`	No	–	Float, 0.01 to 1.0	Gradient limiter coefficient for turbulent kinetic energy.
`qcr`	No	–	2000 | 2020 | 2021	Activates the non-linear QCR correction to the turbulent stress.
`rotation_correct`	No	`False`	`True` | `False`	Adds the rotation correction term to SST or SA-neg.
`relax`	No	`0.5`	Float, > 0.0 to 1.0	Implicit relaxation factor for the incompressible turbulence solver.
`production`	No	–	`"vorticity"` | `"strain-incompre` | `"strain-compress` | `"kato-launder"`	Production term model for SST. See *Production Term Table* below. **Currently broken end-to-end — see note below.**
`les_cfl_limit`	No	–	Float, >= 0.0	CFL shield for the DES/DDES/IDDES hybrid branch. See below.
`les_cfl_width`	No	–	Float, > 0.0	Width of the blend above `les_cfl_limit`. See below.

model

Selects the RANS turbulence model.

Setting	Notes
'sst'	The Menter Shear Stress Transport Turbulence Model (https://turbmodels.larc.nasa.gov/sst.html)
'spalart-allmaras'	Spalart-Allmaras turbulence model (https://turbmodels.larc.nasa.gov/spalart.html)
'sa-neg'	Negative Spalart-Allmaras turbulence model (https://turbmodels.larc.nasa.gov/spalart.html)
'sas'	Scale-Adaptive Simulation, run as a standalone RANS model
'sst-transition'	The Langtry-Menter 4-equation Transitional SST Model (https://turbmodels.larc.nasa.gov/langtrymenter_4eqn.html)

hybrid

Selects the hybrid RANS/LES model layered on top of the base model selected with `model`. Omit the key entirely to solve pure RANS with no hybrid blending.

Setting	Notes
'DES'	Detached Eddy Simulation
'DDES'	Delayed Detached Eddy Simulation
'IDDES'	Improved Delayed Detached Eddy Simulation
'SAS'	Scale-Adaptive Simulation run as the hybrid branch of `model`

les_cfl_limit

Convective CFL number at or below which the LES branch of a DES/DDES/IDDES model is left untouched. Omitted (the default), the CFL shield is disabled entirely.

The LES branch replaces the modelled RANS stress with resolved eddies of the size of the local filter width Δ, but those eddies are only genuinely resolved if the time step can carry them, that is if the convective CFL number is of order one. Where the CFL is larger the model still drops the RANS stress, while the temporal discretisation cannot supply the resolved stress that should replace it. The eddy viscosity shielding function f_d does not protect against this because it is built from r_d, a purely spatial ratio.

Setting a limit adds a second shield built from the cell CFL number,

$$\mathrm{CFL} = \frac{|\mathbf{u} - \mathbf{u}_{grid}|\,\Delta t}{\Delta}, \qquad f_{CFL} = \mathrm{smoothstep}\left(\mathrm{clamp}\left(\frac{\mathrm{CFL} - \mathrm{limit}}{\mathrm{width}}, 0, 1\right)\right)$$

which is combined with f_d so that the more protective of the two wins. At $f_{CFL} = 1$ the hybrid length scale is exactly the RANS length scale, so the LES branch is fully suppressed. Because f_{CFL} is exactly zero at or below the limit, a simulation whose CFL number is everywhere within the limit is unaffected.

The shield is only active for time-accurate runs (`solver.time_settings` of type `"unsteady"` — either dual time stepping or global time stepping), since a steady run has no meaningful real time step. It applies to the `'DES'`, `'DDES'` and `'IDDES'` settings only, not to `'SAS'` or `'wale'`. Use the **lescfl** output variable to find where the CFL number exceeds the limit before enabling it.

les_cfl_width

Width of the blend above `les_cfl_limit`. RANS is fully enforced at a CFL number of `limit + width`. A wider blend gives a more gradual return to RANS, which is usually preferable since the shielded cells also revert to full upwind dissipation.

qcr

Activates the non-linear QCR correction to the turbulent stress. Applicable to both SST and SA-neg. QCR has been shown to improve the prediction of separated corner flows, particularly on highly loaded wings. Three versions of the QCR model are available: QCR 2000, QCR 2020 and QCR 2021.

Example RANS turbulence usage:

```
parameters["solver"]["equations"] = {
    "type": "rans",
    "precondition": True,
    "turbulence": {
        "model": "sa-neg",
        "hybrid": "IDDES",
    },
}
```

Advanced parameters for turbulence models

The following parameters can be used in the `turbulence` dictionary to allow tuning of the turbulence models in zCFD. These are advanced parameters and shouldn't normally need to be adjusted.

Keyword	Required	Default	Valid values	Description
`betastar`	No	0.09	Positive number	Constant in the dissipation term on the k equation in the SST model
`cdes_kw`	No	0.78	Positive number	The $k-\omega$ part of the blended C_{DES} in SST
`cdes_keps`	No	0.61	Positive number	The $k-\epsilon$ part of the blended C_{DES} in SST
`cd1`	No	20.0	Positive number	Constant in the f_{dt} blending function in DDES and IDDES
`cd2`	No	3	Positive integer	Constant in the f_{dt} blending function in DDES and IDDES
`cw`	No	0.15	Positive number	Constant in the length scale calculation in IDDES
`a1`	No	0.31	Positive number	Constant in the eddy viscosity limiter in SST
`cdes`	No	0.65	Positive number	C_{DES} constant used for SA-neg based hybrid RANS/LES models
`ca1`	No	2.0	Positive number	Constant in the production of the gamma equation in the transition model
`ca2`	No	0.06	Positive number	Constant in the destruction term for the gamma equation in the transition model
`ce1`	No	1.0	Positive number	Constant in the production of the epsilon equation in the transition model
`ce2`	No	50.0	Positive number	Constant in the destruction term for the epsilon equation in the transition model
`cthetat`	No	0.03	Positive number	Constant in the Re-theta transition correlation in the transition model
`sigmagamma`	No	1.0	Positive number	Constant in the transition model
`sigmathetat`	No	2.0	Positive number	Constant in the transition model
`separation_correction`	No	`True`	`True`|`False`	Activates the separation correction for the transition model

production can have the following values:

Setting	Notes
`"vorticity"`	The vorticity based production term (SST-V) $P_m = \mu_t \Omega_v^2 - \frac{2}{3}\rho k \delta_{ij} \frac{\partial u_i}{\partial x_j}$
`"strain-incompressible"`	The strain based production term for incompressible flows $P_m = \mu_t S^2$
`"strain-compressible"`	The strain based production term for compressible flows $P_m = \mu_t S^2 - \frac{2}{3}\rho k \delta_{ij} \frac{\partial u_i}{\partial x_j}$
`"kato-launder"`	The Kato-Launder source term $P_m = \mu_t S \Omega_v - \frac{2}{3}\rho k \delta_{ij} \frac{\partial u_i}{\partial x_j}$

When `production` is omitted, the solver build chooses for itself: `"kato-launder"` for `sst-transition`,

`"vorticity"` otherwise.

> **Warning**
>
> Setting `production` explicitly does not currently work end-to-end. The schema accepts and validates these string values, and also accepts integer values 0-3, which it normalises to the equivalent string. The C++ SST solver, however, reads `production` as an integer, so a deck that sets this field does not reach the solver as intended. This is a known, unresolved mismatch between the schema and the solver, not a documentation gap — do not rely on `production` selecting anything other than the build's own default until the solver is updated to read the string names.

Euler

Compressible Euler flow is inviscid (no viscosity and hence no turbulence). The compressible Euler equations are appropriate when modelling flows where momentum significantly dominates viscosity - for example at very high speed. The computational mesh used for Euler flow does not need to resolve the flow detail in the boundary layer and hence will generally have far fewer cells than the corresponding viscous mesh would have.

Euler only takes the *common* `type` and `precondition` fields — there is no equation-specific configuration beyond that.

Example usage:

```
parameters["solver"]["equations"] = {
    "type": "euler",
    "precondition": True,
}
```

Viscous

The viscous (laminar) equations model flow that is viscous but not turbulent. The Reynolds number of a flow regime determines whether or not the flow will be turbulent.

The computational mesh for a viscous flow has to resolve the boundary layer so will generally be larger than that used for an Euler simulation. A viscous simulation will run faster than a RANS simulation using the same mesh since fewer equations are being modelled.

Compressible Viscous only takes the *common* `type` and `precondition` fields.

Example usage:

```
parameters["solver"]["equations"] = {
    "type": "viscous",
    "precondition": True,
}
```

Incompressible Viscous

Selecting the incompressible solver (`solver.solver_settings.type == "incompressible"`) adds the following keys alongside `type` and `precondition` wherever `equations.type` is `"viscous"`, `"rans"` or `"les"`.

Keyword	Required	Default	Valid values	Description
`correction`	No	`"over_re`	`"none"` | `"orthogonal"` | `"minimum"` | `"over_relaxed"`	Non-orthogonal correction method.
`correction_limit`	No	0.5	Float, 0.0 to 1.0	Limiter for the non-orthogonal correction. 1.0 applies the full correction, 0.0 turns off the correction.
`gradient_limiter`	No	`"cellmd"`	`"cellmd"` | `"cell"`	Type of gradient limiter. `cellmd` is more robust on highly non-orthogonal meshes; `cell` is less dissipative.
`gradient_limiter`	No	0.5	Float, 0.0 to 1.0	Coefficient for the gradient limiter. 1.0 is the most dissipative, 0.0 turns off the limiter.
`max_second_order`	No	90.0	Float, 0.0 to 90.0	Maximum non-orthogonal angle (degrees) at which second order discretisation is used. Above this angle first order discretisation is used.
`central_blend`	No	0.8	Float	Coefficient of blending between first and second order terms.
`solve_energy`	No	`False`	`True` | `False`	Solve the energy (temperature) transport equation.
`energy_variable`	No	`"temperа`	`"temperature"` | `"potential temperature"`	Variable transported by the energy equation. Potential temperature $\theta = T(p_0/p)^{R/c_p}$ is the natural choice for atmospheric cases.
`base_state`	No	Not set	Dictionary, see below	Hydrostatic base state of a stratified (anelastic) atmosphere. Setting it turns stratification on: the density becomes the base-state $\bar{\rho}(z)$ and buoyancy is $g(\theta - \bar{\theta})/\bar{\theta}$. Requires `solve_energy: True` and `energy_variable: "potential temperature"`.
`theta_perturbati`	No	Not set	Dictionary, see below	Initial potential-temperature perturbation $\theta'(x, z)$ added to the base state. Seeds the initial condition only; the base state itself is unchanged. Requires `base_state`.
`buoyancy_diagnos`	No	`False`	`True` | `False`	Log the buoyancy source against the momentum diagonal every cycle. Costs four host syncs per cycle, so for debugging only. Requires `base_state`.

`base_state` describes a hydrostatic column in SI units. Height z is measured along $-\mathbf{g}$ from the reference condition's location.

Keyword	Required	Default	Valid values	Description
`kind`	Yes	—	`"isothermal"` | `"constant lapse rate"` | `"constant brunt vaisala"`	Temperature profile; selects which profile parameter applies. `constant brunt vaisala` gives uniform stratification $\theta = \theta_0 \exp(N^2 z/g)$.
`reference`	No	The initialisation reference	String	Reference condition supplying the surface temperature and pressure (e.g. `"IC_1"`).
`surface_temperat`	No	From the reference condition	Float (K)	Temperature at the reference location; overrides the reference condition's value.
`surface_pressure`	No	From the reference condition	Float (Pa)	Pressure at the reference location; overrides the reference condition's value.
`lapse_rate`	No (kind `"constar lapse rate"` only)	0.0065	Float (K/m)	$-dT/dz$ for `constant lapse rate`. $g/c_p \approx 0.0098$ is the neutral dry adiabat.
`brunt_vaisala_fr`	No (kind `"constar brunt vaisala'` only)	0.01	Float (1/s)	N for `constant brunt vaisala`.

`theta_perturbation` is the small θ' the gravity-wave and convection benchmarks add to the base state. x is the mesh x coordinate and z the height, both measured from the reference condition's location.

Keyword	Required	Default	Valid values	Description
`kind`	Yes	–	`"inertia gravity wave"` | `"bubble"`	`inertia gravity wave`: $\theta' = A\sin(\pi z/H)/(1+((x-x_c)/a)^2)$ (Skamarock & Klemp 1994). `bubble`: $\theta' = A\cos^2(\pi r/2)$ inside the ellipse $r = \lvert((x-x_c)/r_x, (z-z_c)/r_z)\rvert \le 1$, zero outside.
`amplitude`	Yes	–	Float (K)	A. Negative for a cold bubble.
`centre`	No	`[0.0, 0.0]`	`[x_c, z_c]` (m)	Perturbation centre. z_c is used by `bubble` only.
`half_width`	No	5000.0	Float (m)	a for `inertia gravity wave`, or the bubble x radius r_x.
`half_height`	No	2000.0	Float (m)	Bubble z radius r_z.
`depth`	No	10000.0	Float (m)	H in $\sin(\pi z/H)$ for `inertia gravity wave`.

The pressure-velocity coupling scheme, the relaxation factors that drive it and the SIMPLE pressure knobs live in `convergence_control` — see *Incompressible convergence control*.

Example usage:

```
parameters["solver"] = {
    "solver_settings": {"type": "incompressible"},
    "equations": {
        "type": "viscous",
    },
    # ... numerical_scheme, fluid_properties, reference_conditions, ...
}
```

RANS

The fully turbulent (Reynolds Averaged Navier-Stokes) equations.

As well as the *common* settings, the `"rans"` equation set requires the additional `turbulence` dictionary documented in *Turbulence* above.

Example usage:

```
parameters["solver"]["equations"] = {
    "type": "rans",
    "precondition": True,
    "turbulence": {
        "model": "sst",
        "rotation_correction": False,
        "limit_gradient_k": 0.5,
        "qcr": 2020,
    },
}
```

Incompressible RANS

Incompressible RANS equations combine the *Incompressible Viscous* fields with the `turbulence` dictionary from *Turbulence* above. Note that the turbulence model's own `relax` can be set independently of the main equation block's `relax`.

Example usage:

```
parameters["solver"] = {
    "solver_settings": {"type": "incompressible"},
    "equations": {
        "type": "rans",
        "precondition": True,
        "turbulence": {"model": "sst"},
    },
    # ... numerical_scheme, fluid_properties, reference_conditions, ...
}
```

LES

Large Eddy Simulation resolves the energy-containing turbulent scales directly and only models the sub-grid scales below the mesh filter width. Unlike RANS, a non-hybrid LES run solves no additional transport equation for turbulence at all — the sub-grid-scale model is a runtime kernel inside the viscous solver library, not a separate set of turbulence equations. For this reason the `"les"` equation set is handled by the same solver code path as `"viscous"`: choosing LES does not add turbulence-transport equations to the system being solved, it only switches on an SGS closure kernel within the viscous solve.

Hybrid RANS-LES methods are the exception, since they *do* carry a transport-equation RANS model underneath their LES branch (SST-IDDES, for example, still solves the SST k and ω equations). Those are configured through the *RANS* equation set's `turbulence.hybrid` field (`"DES"` / `"DDES"` / `"IDDES"` / `"SAS"` on top of `model`), not through `"les"`.

The `turbulence` dictionary for the `"les"` equation set is therefore a much smaller, LES-specific model — it selects and configures the sub-grid model, and is unrelated to the RANS `turbulence` schema described above.

Keyword	Required	Default	Valid values	Description
`les`	Yes	–	`"none"` | `"wale"`	Sub-grid-scale model. `none` is implicit LES (no explicit SGS model); `wale` is the Wall-Adapting Local Eddy-viscosity model.
`wall_damping`	No	–	`True` | `False`	Enable wall damping for the LES model.
`eps`	No	0.41	Positive number	von Kármán constant for WALE near-wall scaling.
`cw`	No	0.325	Positive number	WALE model constant (subgrid viscosity coefficient).

The whole `turbulence` dictionary is optional for LES; omitting it is equivalent to implicit LES with no SGS model.

Example usage:

```
parameters["solver"]["equations"] = {
    "type": "les",
    "turbulence": {
        "les": "wale",
    },
}
```

Incompressible LES

Incompressible LES equations combine the *Incompressible Viscous* fields with the LES-specific `turbulence` dictionary described in *LES* above.

Example usage:

```
parameters["solver"] = {
    "solver_settings": {"type": "incompressible"},
    "equations": {
        "type": "les",
        "turbulence": {"les": "wale"},
    },
    # ... numerical_scheme, fluid_properties, reference_conditions, ...
}
```

Numerical scheme

`solver.numerical_scheme` holds the spatial discretisation applied to the equation set: the order of the reconstruction and how many cycles to hold it at first order, the limiter, the gradient method, the inviscid flux scheme and its low-dissipation sensor, and multigrid.

limiter

`limiter` selects the slope limiter applied to the second order reconstruction. It has no effect when `order` is `"first"`, nor during the cycles covered by `first_order_cycles`.

Two families are available. The first four values are **one-dimensional** functions $\psi(a,b)$ of two directional differences, where a is the jump across the face and b a backward difference reconstructed from the cell gradient. They are applied independently to every reconstructed variable and to each side of every face, and are regularised by a threshold $\varepsilon = 0.001 \cdot \frac{1}{2}(u_L + u_R) + 10^{-8}$. Note that this threshold is proportional to the *magnitude* of the variable, so for a variable that changes sign - a velocity component, for example - it becomes small and the limiter then acts on correspondingly smaller variations.

Value	Description
`"van albada"`	Default. $\psi = \max\left(0, \frac{2ab+\varepsilon^2}{a^2+b^2+\varepsilon^2}\right)$. Smooth, symmetric, bounded in $[0,1]$. The recommended general purpose choice.
`"van leer"`	$\psi = \max\left(0, \frac{4ab+\varepsilon^2}{(a+b)^2+\varepsilon^2}\right)$. Slightly more compressive than van Albada.
`"symmetric ratio"`	$\psi = \max\left(0, \min\left(1, \frac{4\min(\sigma a, \sigma b)+\varepsilon}{\lvert a+b\rvert+\varepsilon}\right)\right)$ with $\sigma = \text{sign}(a+b)$. The least dissipative of the three. **This value was previously called** `"barth jespersen"`.
`"none"`	$\psi = 1$. Unlimited reconstruction: second order everywhere, but not monotonicity preserving. Appropriate only for smooth flows with no shocks or strong gradients.

> **Warning**
>
> `"barth jespersen"` **changed meaning.** It previously selected the one dimensional function now called `"symmetric ratio"`, which takes no minimum or maximum over the cell's neighbours and so was never the Barth-Jespersen limiter. It now selects the real thing, described under *cell-based limiters*. A deck that wants the previous behaviour must be changed to `"symmetric ratio"`.

Writing $r = b/a$ for the ratio of the two differences, `"symmetric ratio"` is $\psi = \min(1, 4\min(a,b)/(a+b))$: zero for $r \le 0$, rising as $4r/(1+r)$, **exactly 1 across the whole plateau** $r \in [1/3, 3]$, then decaying as $4/(1+r)$. It satisfies $\psi(1/r) = \psi(r)$. That wide plateau is why it is the least dissipative of the one dimensional limiters - `"van albada"` and `"van leer"` reach $\psi = 1$ only at $r = 1$ exactly:

r	`"symmetric ratio"`	`"van albada"`	`"van leer"`
0.1	0.364	0.198	0.331
1/3	1.000	0.600	0.750
1	1.000	1.000	1.000
3	1.000	0.600	0.750
10	0.364	0.198	0.331

cell-based limiters

`"barth jespersen"`, `"michalak gooch"` and `"venkatakrishnan"` are **multidimensional**: they bound the reconstructed face value by the range of values over the cell's whole set of face neighbours, which the one dimensional limiters above do not. They are evaluated per cell in their own pass, which the reconstruction then only reads.

For cell i, with $u_{\max}$ and $u_{\min}$ taken over the face neighbours and $\Delta_f = \nabla u_i \cdot (x_f - x_i)$ the unlimited extrapolation to face f, define $y_f = (u_{\max} - u_i)/\Delta_f$ when $\Delta_f > 0$ and $(u_{\min} - u_i)/\Delta_f$ when $\Delta_f < 0$. Then $\phi_i = \min_f P(y_f)$, with:

Value	$P(y)$	Notes
`"barth jespersen"`	$\min(1, y)$	Barth & Jespersen (1989) as published. Bound preserving, but only C^0: the kink at $y = 1$ makes the limiter value flicker between cycles and can stall the residual. It has no smoothness deactivation, so it limits wherever the bound binds.
`"michalak gooch"`	$y - \frac{4}{27} y^3$ for $y < 3/2$, else 1	Michalak & Ollivier-Gooch. C^1, and $P(y) \leq \min(1, y)$, so never less monotonicity preserving than Barth-Jespersen. Deactivates in smooth flow via a grid-scaled threshold, so formal order survives refinement. The recommended choice.
`"venkatakrishnan"`	$\min\left(1, (y^2 + 2y)/(y^2 + y + 2)\right)$	Smooth, but exceeds 1 for $y > 2$, so it needs the clip, which reintroduces a C^0 kink at $y = 2$. Provided as a familiar cross-check.

ϕ scales the gradient term of the reconstruction only; the face-jump term needs no limiting because the result is then a convex combination of two states that are individually inside the neighbour bounds. The $\kappa = 1/3$ accuracy of the reconstruction is therefore retained. One consequence is that $\phi = 0$ does not recover pure first order - use `order`, `first_order_cycles` or a coarser scheme if that is what is wanted.

limiter_coefficient

K in the grid-scaled smoothness threshold $\varepsilon^2 = (Kh/L_{ref})^3$ used by `"michalak gooch"` and `"venkatakrishnan"` to deactivate limiting in smooth flow, where h is the cell length scale and L_{ref} is `reference_length`. Default 5.0; larger values limit more widely. The cube falls faster than the $O(h^2)$ reconstruction error, so under refinement the limiter switches itself off in smooth regions. Ignored by the other limiters.

freeze_limiter_cycle

If set the values of the limiter used in the MUSCL reconstruction are frozen at the end of the specified cycle and held fixed for the rest of the pseudo time solve. This can be useful in the latter stages of a solve if noise from the limiter is causing the residual convergence to stall: a small number of cells whose limiter switches between iterations can hold the residual at a noise floor that more cycles will never clear.

The cycle is counted within the **current pseudo time solve**:

Time marching	The pseudo time solve is	Limiter is
Steady	the whole run	frozen from cycle N to the end
Dual time, the `start` cycles	the start-up cycles at real time cycle 0	frozen from cycle N to the end of that phase
Dual time, each real time step	that real time step	live for cycles 1 to N, frozen for the rest of the step
`kind: "global timestepping"`	–	never frozen; the setting is ignored

For dual time stepping the limiter is therefore released and re-computed at the start of every real time step, so each step's snapshot reflects that step's solution and mesh position. **N must be less than** `cycles` or the snapshot is taken too late to influence that step's solution; a warning is issued if it is not.

Global time stepping advances one physical time step per cycle, so there is no pseudo time solve for the freeze to be scoped to and no later opportunity to refresh the snapshot. The setting is ignored and a warning is issued.

The cycle count is stored in the restart file, so a steady run restarted at or beyond the given cycle freezes its limiter again on the first cycle of the restarted run. With a face-based limiter the snapshot is held per face and is not stored in the restart file; it is re-computed from the restored solution instead, so a restarted simulation is not bitwise identical to an uninterrupted one. A *cell-based limiter* is stored – see below.

Freezing is deferred while the spatial discretisation is still first order, for example during `first_order_cycles`, because a first order reconstruction has no limiter values to store.

With one of the *cell-based limiters* the values are held per cell rather than per face, but freezing behaves the same way: the per-cell pass simply stops running and the stored ϕ stands.

A frozen cell limiter **is** written to the restart file, so a restart resumes holding the same ϕ the run was using rather than recomputing it from the restored solution. Only the owned cells of each partition are stored and the halo values are re-derived by a halo exchange on read, so the restart is still free to use a different number of partitions. The stored values are used only if the restarting control dictionary still sets `freeze_limiter_cycle` – the input deck stays the authority on the numerics, so removing the setting gives a live limiter back. Under dual time stepping the restored snapshot is released at the top of the next real time step like any other, so a restart mid-step resumes on the phi that step was using and the following steps recompute it. If the restart uses a different turbulence model the components the file does not carry are left unlimited.

`moving_mesh` is supported: the mesh only moves at the end of a real time step and the limiter is re-computed after the move, so a snapshot is never used at a mesh position it was not taken at. A `modal_model` FSI coupling is the exception, because it deforms the mesh from within the pseudo time loop; a warning is issued if both are set.

1.5.6 Fluid-structure interaction

zCFD supports two fluid-structure interaction (FSI) coupling schemes: a built-in **modal model** driven by mode shapes imported from a Nastran modal analysis, and a **generic** loosely-coupled scheme for driving an external FEA solver through a user-supplied `genericFSI.py` script. Both schemes use the same underlying mesh-deformation machinery (RBF, RBF multiscale, or IDW - see *Mesh Transformation*) to propagate the motion of the coupled surface into the volume mesh.

Where FSI settings live

An FSI coupling is attached to the wall boundary condition whose surface is the moving, coupled structural boundary, via that boundary condition's `'fsi'` key:

```
"model": {
    "<case_name>": {
        "boundary_conditions": {
            "BC_FSI": {
                "zones": [1, 2, 3],
                "type": "wall",
                "kind": "slip",
                "fsi": {
                    "type": "modal_model",  # or "generic_fsi"
                    ...
                },
            },
            ...
        },
    }
},
```

The `'fsi'` block is discriminated by its `'type'` key: `'modal_model'` selects the built-in Nastran modal-model coupling, `'generic_fsi'` selects the generic external-solver coupling. For the modal model, the moving zones are **not** repeated inside the `'fsi'` block - they are inherited from the owning boundary condition's own `'zones'` list. Generic FSI is the exception: it carries its own `'zone'` key (singular, unlike every other zone list in zCFD, which is why it stands out below).

Restarting an FSI simulation

FSI restart behaviour is controlled by `'restart'` in the `'initialisation'` block - the same flag that governs restarting the flow solution (see *Initial Conditions*). The FSI coupling resolves this flag through the same model-block-then-solver-block precedence used everywhere else in zCFD: if the model that owns the FSI-coupled boundary condition sets its own `'initialisation'` block (i.e. under `model.<case_name>.initialisation`), the `'restart'` value there takes priority; only when the model block does not set `'initialisation'` does the coupling fall back to the top-level `solver.initialisation.restart`.

This determines whether the modal model resumes its structural state (modal displacements and velocities, and, for the RBF transform, the previously built base-point set) from a previous run, so a per-model `'initialisation'` block, when present, is always the one that decides FSI restart - not the solver-level one.

Modal model

The modal model reduces the structural response to a small set of Nastran mode shapes, integrates the modal equations of motion (via a Newmark-beta scheme) forced by the aerodynamic pressure load, and maps the resulting modal deformation back onto the fluid surface and, via RBF or IDW, into the volume mesh.

Keyword	Require	Default	Valid values	Description
'type'	No	'modal_mo	'modal_model'	Discriminator selecting the modal-model FSI driver.
'file_format'	Yes		'nastran'	Structural file format the mode shapes are read from.
'nastran_casename'	Yes		String	Nastran case name, without file extension.
'mode_mapping_max_d	Yes		Float	Maximum distance allowed when mapping structural mode shapes onto the fluid surface.
'fluid_force_scaling'	Yes		Float	Scaling applied to the fluid pressure forces before they drive the modal equations of motion.
'mode_list'	Yes		List of int	Which structural modes (by Nastran mode ID) to include in the coupling.
'modal_damping'	Yes		List of float	Modal damping ratio for each entry in 'mode_list', in the same order.
'transform_type'	Yes		'rbf_multiscale' | 'idw'	Selects the mesh-deformation scheme used to propagate the modal deformation into the volume mesh - see *RBF multiscale* and *IDW* below.
'support_radius'	No	1.0	Float > 0	RBF/IDW support radius. Also sets the scale used to normalise mode shapes during mesh morphing.
'scale'	No		List of 3 floats	Scaling factors [x, y, z] applied to the mode shapes read from the Nastran file.
'forcing'	No		Dict	Optional external forcing applied to the modal equations of motion - see *External forcing* below.
'mode_mapping_power'	No	1.0	Float > 0	Inverse-distance power used when mapping structural mode shapes onto the fluid surface.
'mode_mapping_pure_i	No	True	True/False	Use pure inverse-distance weighting (rather than a blended scheme) for the mode-shape mapping.
'mode_mapping_n_nea	No	4	Positive integer	Number of nearest structural points used to map each fluid surface point's mode shape.
'recalculate_rbfs_alpha_fraction	No	0.0	Float >= 0	Deformation fraction of the RBF support radius at which the RBF base-point set is rebuilt. 0 never rebuilds it.
'newmark_alpha'	No	0.25	Float	Alpha parameter of the Newmark-beta time integrator used to march the modal equations of motion.
'newmark_delta'	No	0.5	Float	Delta parameter of the Newmark-beta time integrator used to march the modal equations of motion.

Note

Unlike generic FSI, the modal model does not accept a `'user_variables'` dictionary - modal-model behaviour is controlled entirely by the keywords above.

RBF multiscale transform

Set `'transform_type': 'rbf_multiscale'` to deform the volume mesh using the multiscale RBF scheme (see *Interpolation method* for how RBF and RBF multiscale work).

Keyword	Require	Default	Valid values	Description
'basis'	No	'c2'	'c2' | 'c4' | 'c0' | 'tps' | 'gaussian'	RBF basis function used for the mesh deformation.
'base_point_fraction'	Yes		Float, (0, 1]	Fraction of surface points included in the reduced RBF base set.

IDW transform

Set `'transform_type': 'idw'` to deform the volume mesh using inverse distance weighting (see *Interpolation method* for the IDW weighting formulae).

Keyword	Require	Default	Valid values	Description
'power'	Yes		Float, or list of 3 floats	IDW power(s) for the near-field and far-field regions, and the blend between them.
'n_nearest'	No	100	Positive integer	Number of nearest volume points used per surface point.
'deformation_distance'	No	1.0	Float > 0	Maximum region of influence for the IDW mapping.
'blending_stiffness'	No	0.5	Float, [0, 1]	Blending stiffness for the IDW interpolation.
'fixed_zones'	No		List of zone ID integers	Zones excluded from mesh deformation, such as an interface, periodic boundary, or symmetry plane.
'stencil_size'	No		Positive integer	Alternative to 'support_radius' for bounding the IDW stencil by node count instead of distance.

Note

'support_radius' and 'stencil_size' are alternative ways of bounding the IDW stencil - setting both explicitly is rejected as ambiguous.

External forcing

An optional `'forcing'` block applies an additional external force to the modal equations of motion, independent of the aerodynamic pressure load.

Keyword	Require	Default	Valid values	Description
'location'	Yes		[x, y, z]	Location at which the forcing is applied.
'force'	Yes		[Fx, Fy, Fz], or a Python callable	Constant force vector, or a callable returning one as a function of time.
'frequency'	Yes		Float	Forcing frequency.

Modal model example

RBF multiscale coupling of a panel to a Nastran modal basis:

```
"boundary_conditions": {
    "BC_FSI": {
        "zones": [1, 2, 3],
        "type": "wall",
        "kind": "slip",
        "fsi": {
            "type": "modal_model",
            "file_format": "nastran",
            "nastran_casename": "panel_nm_0.04m",
            "transform_type": "rbf_multiscale",
            "base_point_fraction": 1.0,
            "support_radius": 0.02,
            "scale": [1.0, 1.0, 1.0],
            "mode_mapping_max_distance": 0.01,
            "fluid_force_scaling": 0.08 / 0.000166667,
            "mode_list": [0, 4, 10, 13],
            "modal_damping": [0.0, 0.0, 0.0, 0.0],
        },
    },
},
```

The same panel coupled with an IDW transform instead, with a set of surrounding zones held fixed:

```
"boundary_conditions": {
    "BC_FSI": {
        "zones": [1, 2, 3],
        "type": "wall",
        "kind": "slip",
        "fsi": {
            "type": "modal_model",
            "file_format": "nastran",
            "nastran_casename": "panel_nm_0.04m",
            "transform_type": "idw",
```

(continues on next page)

(continued from previous page)

```
            "power": [3.0, 8.0, 10.0],
            "deformation_distance": 0.05,
            "blending_stiffness": 0.0,
            "n_nearest": 350,
            "fixed_zones": [4, 5, 6, 7, 8, 9],
            "scale": [1.0, 1.0, 1.0],
            "mode_mapping_max_distance": 0.01,
            "fluid_force_scaling": 0.08 / 0.000166667,
            "mode_list": [0, 4, 10, 13],
            "modal_damping": [0.0, 0.0, 0.0, 0.0],
        },
    },
},
```

Generic FSI

Generic FSI is a loose (explicit) coupling to an external FEA program. Out of the box, `genericFSI.py` performs no changes - it is a hook to be implemented to exchange pressures, displacements and node coordinates with your own external solver.

Keyword	Require	Default	Valid values	Description
'type'	No	'generic_fsi	'generic_fsi'	Discriminator selecting the generic FSI driver.
'zone'	Yes		List of zone ID integers	Zones coupled to the external solver. Singular key name - unlike every other zone list in zCFD (including the modal model, where the zone list lives on the owning boundary condition), this matches the C++ read site exactly.
'fixed_zones'	No		List of zone ID integers	Zones excluded from mesh deformation.
'transform_type'	Yes		'rbf_multiscale' | 'rbf' | 'idw'	Mesh-deformation transform used to propagate the coupled surface's displacements into the volume mesh.
'support_radius'	No	1.0	Float	RBF support radius. Only applies to 'rbf' and 'rbf_multiscale'.
'basis'	No	'c2'	'c4' | 'c2' | 'c0' | 'tps' | 'gaussian'	RBF basis function. Only applies to 'rbf' and 'rbf_multiscale'.
'base_point_fraction'	No	0.1	Float	Fraction of points used as the reduced RBF base set. Only applies to 'rbf' and 'rbf_multiscale'.
'tol'	No	0.01	Float	Target error tolerance for the RBF interpolation. Only applies to 'rbf'.
'max_error'	No	0.1	Float	Maximum initial error tolerance for the RBF interpolation. Only applies to 'rbf'.
'n_nearest'	No	100	Positive integer	Number of nearest points used for IDW interpolation. Only applies to 'idw'.
'deformation_distance'	No	1.0	Float	Maximum support radius for IDW interpolation. Only applies to 'idw'.
'blending_stiffness'	No	0.5	Float	Blending stiffness for the IDW interpolation. Only applies to 'idw'.
'power'	No	[3.0, 5.0, 10.0]	List of 3 floats	IDW power for the near-field and far-field boundaries, and the power between them. Only applies to 'idw'.
'user_variables'	No		Dict	Arbitrary variables passed through to `genericFSI.py` (for example time-marching or FSI control settings) - genuinely open-ended and not otherwise validated.

Generic FSI example

```
"boundary_conditions": {
    "BC_FSI": {
        "zones": [1, 2, 3],
        "type": "wall",
        "kind": "slip",
        "fsi": {
            "type": "generic_fsi",
```

(continues on next page)

(continued from previous page)

```
            "zone": [1, 2, 3],
            "fixed_zones": [4, 5, 6],
            "transform_type": "idw",
            "n_nearest": 100,
            "deformation_distance": 1.0,
            "blending_stiffness": 0.5,
            "power": [3.0, 5.0, 10.0],
            "user_variables": {"restart_file": "structure_state.dat"},
        },
    },
},
```

1.5.7 Material Specification

`fluid_properties` sets the material properties of the fluid being modelled, nested under the top-level `solver` key:

```
parameters = {
    "config_version": 2,
    "solver": {
        "fluid_properties": {....},
        ..
    },
    ..
}
```

The default values are those of air, but the dictionary can be modified to model any ideal, compressible, perfect gas.

Which fields are accepted depends on the equation set (*equations*): the inviscid Euler equations only need `gamma`/`gas_constant` to close the equation of state, while viscous, RANS and LES equations also need `sutherlands_const` and the Prandtl numbers to model diffusion of momentum and heat.

Keyword	Required	Default	Valid values	Equations
`material`	No	`"ideal gas"`	String	All. A descriptive label only — it does not select a named sub-block.
`gamma`	No	1.4	Float, > 1.0	All
`gas_constant`	No	287.0	Float, > 0	All
`sutherlands_const`	No	110.4	Float, > 0	Viscous, RANS, LES
`prandtl_no`	No	0.72	Float, > 0	Viscous, RANS, LES
`turbulent_prandtl_no`	No	0.9	Float, > 0	Viscous, RANS, LES
`gravity`	No		[X, Y, Z] vector, m/s^2	All. `None`/omitted means gravity is off.
`thermal_expansion_coe`	No		Float, > 0	Incompressible only, and only once `solve_energy` is set true in *equations* — Boussinesq buoyancy coefficient β (1/K).
`boussinesq_buoyancy_r`	No		Float, >= 0	Incompressible only
`latitude`	No		Float, degrees, -90 to 90	Compressible only. Latitude for the Coriolis force; omitted means no Coriolis.

Note

`gravity`, `thermal_expansion_coeff`, `boussinesq_buoyancy_reference_density` and `latitude` are body-force/buoyancy fields that live on `fluid_properties` because that is where the solver reads them (`Reference::set` takes all four from `parameters.fluidProperties()`). A dedicated per-solver buoyancy block is the intended eventual home for these fields; `fluid_properties` is where they are set in the meantime.

Example usage:

```
parameters = {
    "config_version": 2,
    "solver": {
        ..
        "fluid_properties": {
            "material": "air",
            "gamma": 1.4,
            "gas_constant": 287.0,
            "sutherlands_const": 110.4,
            "prandtl_no": 0.72,
            "turbulent_prandtl_no": 0.9,
        },
        ..
    },
    ..
}
```

sutherlands_const

This option controls the variation of the viscosity of the fluid with temperature.

Sutherland's law: $\mu = \mu_{ref}\left(\frac{T}{T_{ref}}\right)^{3/2}\frac{T_{ref}+S}{T+S}$, where S is Sutherland's constant.

In zCFD, T_{ref} and μ_{ref} are given by the *reference* initial condition dictionary, where T is specified directly and μ is specified either directly via **'viscosity'** or indirectly via **'reynolds_no'**.

Typically for air $\mu_{ref} = 1.716e-5$ and $T_{ref} = 273.15$

For more information: https://www.cfd-online.com/Wiki/Sutherland%27s_law

prandtl_no

This option sets the Prandtl number, a non-dimensional number which is the ratio of momentum and thermal diffusivities in a fluid. In zCFD, where μ and c_p are already set, setting the Prandtl number has the effect of setting the thermal conductivity, k, of the fluid.

$\text{Pr} = \frac{\text{momentum diffusivity}}{\text{thermal diffusivity}} = \frac{c_p\mu}{k}$

This can also be considered as $\text{Pr} = \frac{Thickness\ of\ thermal\ boundary\ layer}{Thickness\ of\ fluid\ boundary\ layer}$

gas_constant

This changes R, the gas constant of the fluid. Different fluids have different molecular masses, and therefore different gas constants. 287.1 is appropriate for air.

Constant pressure and constant volume specific heats, c_p and c_v, are set by $gamma$ and R according to the following relationships:

$c_p = \frac{\gamma-1}{\gamma}R$

$\gamma = \frac{c_p}{c_v}$

gamma

This changes γ, the ratio of specific heats of the fluid. 1.4 is appropriate for air, but a value of 1.33 is often used for combustion exhaust gases.

turbulent_prandtl_no

This changes the turbulent Prandtl number of the fluid being modelled. Only meaningful for viscous, RANS and LES equations — omitted entirely for Euler.

In simulations which model turbulence viscosity (RANS, LES and other scale resolving simulations), the eddy viscosity, μ_t accounts for the additional diffusivity of momentum due to turbulence relative to the equivalent laminar flow. There is likewise additional diffusivity of heat due to turbulence relative to the equivalent laminar flow, and the turbulent prandtl number accounts for this.

In an eddy viscosity based simulation, $Pr_t = \frac{c_p \mu_t}{k_t}$

gravity

This sets the direction in which gravity acts in your simulation. For most industrial and aerospace flows, buoyancy effects are not important and therefore gravity should not be set. However, certain flows (e.g. plumes) do require buoyancy effects for accurate modelling.

gravity is specified as an [X, Y, Z] vector, for example `[0, 0, -9.81]`. Setting `gravity` alone is deliberately valid on the incompressible solver: without `thermal_expansion_coeff` set, gravity affects nothing (no buoyancy source is built), so it is not gated on the Boussinesq fields the way they are gated on it.

thermal_expansion_coeff and boussinesq_buoyancy_reference_density

These configure Boussinesq buoyancy on the incompressible solver. `thermal_expansion_coeff` (β, 1/K) is only accepted once `equations.solve_energy` is true — the buoyancy source acts on the temperature field, so it needs the energy equation solved. `boussinesq_buoyancy_reference_density` sets the reference density the buoyancy approximation is linearised about, and (like `latitude`) is rejected on the compressible solver, which has its own body-force path.

latitude

Sets the latitude (degrees, -90 to 90) used to compute the Coriolis force on the compressible solver. Omit for no Coriolis force. Not accepted on the incompressible solver.

1.5.8 Initial Conditions

A reference-condition entry defines a fluid state. These entries live in the `solver.reference_conditions` dict, each keyed by a name (conventionally "IC_1", "IC_2", ...):

```
parameters["solver"]["reference_conditions"] = {
    "IC_1": {...},
    "IC_2": {...},
}
```

The same dict, and the same entry shape, serves three different roles: there is no structural split between "the initial condition" and "a reference condition":

- *'initial_conditions'* on `solver.initialisation` names the entry used to fill the flow field at the start of the simulation (the entry used is whichever name is given there — there is no hard-coded "IC_1" default, it must be named explicitly).
- *'reference'* on `solver.initialisation` names the entry used to non-dimensionalise *force reporting*.
- A boundary condition's own `condition` field (see *boundary conditions*) names the entry used to set flow entering (inflow/farfield) or leaving (outflow) the domain.

Each entry is validated against a different pydantic model depending on `solver.equations.type` — 'euler' entries reject viscosity/turbulence fields outright, 'viscous' entries add viscosity, and 'rans'/'les' entries add the full turbulence set. This routing is automatic; the deck just supplies the fields relevant to its equation type and the schema picks the right model.

One entry can also inherit from another by including a 'reference' key naming the base entry — this is resolved before validation (the fields are merged and overridden, then 'reference' is discarded), so it never reaches the C++ dict. This is how a lean, ratio-only entry (used for boundary conditions specified relative to a base flow state) is expressed without repeating every field. See *'reference conditions'* for the ratio-specific keywords ('total_pressure_ratio', etc.) available on every entry.

Another way of prescribing an initial condition is by defining a dynamic *'driving function'*, a python callable which on evaluation returns a reference-condition dict; when 'initial_conditions' is set to a callable, 'reference' must be set explicitly to name a static entry for non-dimensionalisation.

Keyword	Required	Default	Valid values	Description
'pressure'	Yes		Float	Static pressure in Pascals. Used to compute density and set Total Energy in the Compressible solver.
'temperature'	Yes		Float	Static temperature in Kelvin. Used to compute density and for dynamic viscosity scaling using Sutherlands law.
'v'	No	[0.0, 0.0, 0.0]	Dict	See *Velocity*. Optional — a zero-velocity entry is legal (e.g. a quiescent cavity's initial state); pair it with a different *reference* entry for non-dimensionalisation in that case.
'reference_length'	No	1.0	Float	Reference length used to calculate viscosity from the Reynolds number. Only meaningful when 'reynolds_no' is set (viscous/RANS/LES equation types).
'reynolds_no'	When 'viscosity' not set (viscous/RANS/LE only)		Float	Used to calculate viscosity at the supplied temperature. Not valid for 'euler' equations.
'viscosity'	When 'reynolds_no' not set (viscous/RANS/LE only)		Float	Sets the dynamic viscosity at the temperature, with scaling provided by Sutherlands law defined in *material* section. Not valid for 'euler' equations. Exactly one of 'reynolds_no'/'viscosity' must be given for viscous/RANS/LES equations, never both.
'turbulence_intensity'	Yes (RANS/LES only)		Float	Turbulence intensity (fraction, e.g. 0.01 = 1%). Only valid for 'rans'/'les' equation types.
'eddy_viscosity_ratio'	Yes (RANS/LES only)		Float	Ratio of eddy viscosity to molecular viscosity used to set the turbulence quantities. Only valid for 'rans'/'les' equation types.
'ambient_turbulence_intens	No (RANS/LES only)	1e-20	Float	Value of background turbulence intensity to maintain in the fluid.
'ambient_eddy_viscosity_rat	No (RANS/LES only)	1e-20	Float	Value of background turbulence eddy viscosity ratio to maintain in the fluid.
'profile'	No	Not set	Dict	See *Velocity Profile*.
'monitor'	No	Not set	Dict	Update/terminate/prefix contract used only when this entry's parent dict is itself a python *driving function* — not applicable to a static entry.
'total_pressure_ratio' / 'total_temperature_ratio' / 'static_pressure_ratio'	No		Float	Ratio relative to a base entry named by that entry's own 'reference' key. See *'reference conditions'*.

Note

'turbulence_intensity' and 'eddy_viscosity_ratio' are required (no default) for 'rans'/'les' equation types. 'ambient_turbulence_intensity' defaults to 1e-20, matching 'ambient_eddy_viscosity_ratio' (also 1e-20 by default). Field names are snake_case throughout ('turbulence_intensity', not 'turbulence intensity'; 'v', not 'V'; etc.).

Example usage:

```
parameters["solver"]["reference_conditions"] = {
    "IC_1": {
        "v": {
            "vector": [1.0, 0.0, 0.0],
            "mach": 0.15,
        },
        "pressure": 101325.0,
        "temperature": 273.15,
        "reynolds_no": 6.0e6,
        "reference_length": 1.0,
        "turbulence_intensity": 5.2e-2,
        "eddy_viscosity_ratio": 1.0,
        "ambient_turbulence_intensity": 5.2e-2,
        "ambient_eddy_viscosity_ratio": 1.0,
    },
}
```

Velocity

Velocity can be specified in two ways: either a bare vector, or a vector and a Mach number. When a Mach number is specified the velocity direction is provided by the vector and the velocity magnitude is computed from the Mach number and the speed of sound at the pressure and temperature provided.

For example to set a velocity of 50 mesh units/sec in the X direction:

```
..
"v": {"vector": [50.0, 0.0, 0.0]}
..
```

Or to set a velocity of mach 0.2 in the X direction:

```
..
"v": {"vector": [1.0, 0.0, 0.0], "mach": 0.2}
..
```

Viscosity

Dynamic (shear, absolute or molecular) viscosity should be defined at the static temperature previously specified. This can be specified either as a dimensional quantity or by a Reynolds number and reference length. Both are only valid for 'viscous'/'rans'/'les' equation types — omit both for 'euler'.

```
# Dynamic viscosity in dimensional units
"viscosity": 1.83e-5,
```

or

```
# Reynolds number
"reynolds_no": 5.0e6,
```

(continues on next page)

(continued from previous page)

```
# Reference length
"reference_length": 1.0,
```

Note

Reynolds number is defined as: $Re = \frac{density * V * referencelength}{viscosity}$

Exactly one of 'reynolds_no'/'viscosity' must be set; setting both, or neither, is a validation error.

Turbulence intensity & Eddy viscosity

Turbulence intensity is defined as the ratio of velocity fluctuations u' to the mean flow velocity. A turbulence intensity of 1% is considered low and greater than 10% is considered high. Both fields below are required for 'rans'/'les' equation types (no default) and not valid for 'euler'/'viscous'.

```
# Turbulence intensity %
"turbulence_intensity": 0.01,
# Turbulence intensity for sustainment %
"ambient_turbulence_intensity": 0.01,
```

The eddy viscosity ratio (μ_t/μ) varies depending on the type of flow. For external flows this ratio varies from 0.1 to 1 (wind tunnel 1 to 10).

For internal flows there is greater dependence on Reynolds number as the largest eddies in the flow are limited by the characteristic lengths of the geometry (e.g. the height of the channel or diameter of the pipe). Typical values are:

Re	3000	5000	10,000	15,000	20,000	> 100,000
eddy	11.6	16.5	26.7	34.0	50.1	100

```
# Eddy viscosity ratio
"eddy_viscosity_ratio": 0.1,
# Eddy viscosity ratio for sustainment
"ambient_eddy_viscosity_ratio": 0.1,
```

Both ambient fields default to **1.0e-20** — a near-zero numerical floor, not the freestream `turbulence_intensity`/`eddy_viscosity_ratio` value. There is no fallback to the freestream turbulence level when these are unset; sustaining the ambient level at the freestream value requires setting them explicitly to the same value:

```
"reference_conditions": {
    "IC_1": {
        ..
        "turbulence_intensity": 5.2e-2,
        "eddy_viscosity_ratio": 1.0,
        # Sustain ambient turbulence at the freestream level, rather
        # than the 1.0e-20 default floor:
        "ambient_turbulence_intensity": 5.2e-2,
        "ambient_eddy_viscosity_ratio": 1.0,
    }
},
```

Velocity Profile

The user can also provide functions to specify a 'wall-function' — or the turbulence viscosity profile near a boundary.

Keyword	Required	Default	Valid values	Description
'abl'	No		Dict	Specify an *ABL* (Atmospheric Boundary Layer) profile.
'field'	No		File name	Specify a field of data that is used as a lookup for setting the flow conditions.
'use_wall_distance'	No	False	True/False	Use wall distance in place of z coordinate in the field during interpolation. Can be used to set the profile of flows on a terrain mesh with a variable ground location at the boundary.

field

The file specified using the 'field' parameter should be in the ParaView/VTK VTP format. It should contain a node array with one or more of the following array names: 'Pressure', 'Temperature', 'Velocity', 'TI' and 'EddyViscosity'. zCFD will look up those values in the solution by finding the nearest point in the VTP file.

```
..
"profile": {
    "field": "inflow_field.vtp",
    # Localise field using wall distance rather than z coordinate
    "use_wall_distance": True,
},
..
```

Note

The field overrides conditions specified elsewhere in the entry; the user can specify only the conditions that differ from default.

ABL

Location: `solver.reference_conditions.<name>.profile.abl`

Keyword	Required	Default	Valid values	Description
'roughness_length'	Yes		Float > 0	A measure of the height at which the mean velocity of the wind becomes zero, due to the frictional effects between the air and the surface.
'friction_velocity'	No		Float > 0	Specify the friction velocity at the surface. At least one of 'friction_velocity'/'surface_layer_height' should be set.
'surface_layer_height'	No (compressible solver only)		Float > 0	The surface layer height, also known as the aerodynamic or boundary layer height.
'monin_obukhov_length'	No (compressible solver only)		Float, nonzero	The Monin-Obukhov length. Negative values enable an unstable-stratification correction to the log-law profile; there is currently no stable-stratification (positive L) correction. Omit for neutral stability.
'tke'	No (compressible solver only)		Float >= 0	Reference turbulent kinetic energy. Only applied when 'friction_velocity' is also set; otherwise it is ignored (with a warning) and TKE is derived from wall distance instead.
'up'	No (compressible solver only)	[0,0,1]	Unit vector	Direction of "up", used with 'ground_level' to compute height above ground. This is independent of the fluid's gravity vector — nothing cross-checks that the two are anti-parallel.
'ground_level'	No (compressible solver only)	0.0	Float >= 0	Datum elevation of the ground, measured along 'up', subtracted before the log-law height is computed. This is unrelated to the aerodynamic roughness length ('roughness_length' above).

Note

'ground_level' is the canonical spelling; 'z0' is a legacy spelling accepted for backward compatibility and normalised to 'ground_level' automatically (the older name collided with the unrelated roughness-length z_o notation). 'up' is an independent vertical direction vector, unrelated to the gravity vector set on the fluid model.

Note

'surface_layer_height', 'monin_obukhov_length', 'tke', 'up' and 'ground_level' are only accepted when the solver is compressible (or 'solver.solver_settings.type' is unset). The incompressible inflow boundary condition reads only 'roughness_length' and 'friction_velocity' from this profile; setting any of the other five on an incompressible deck is a validation error, since they would silently have no effect on that solver's inflow.

roughness_length

Is a measure of the height at which the mean velocity of the wind becomes zero, due to the frictional effects between the air and the surface.

The roughness length is typically denoted by the symbol "z_o" and is defined as the vertical distance between the Earth's surface and the height at which the logarithmic wind profile intersects the surface. The logarithmic wind profile is a mathematical relationship that describes the variation of wind speed with height in the atmospheric boundary layer.

The roughness length depends on the surface characteristics of the terrain or objects present on the Earth's surface. Different surfaces, such as forests, urban areas, or bodies of water, have different roughness lengths. For example, a smooth surface like open water would have a small roughness length, while a densely forested area would have a larger roughness length.

The wall boundary mesh is assumed to be at the height of the roughness length rather than ground level and therefore the first cell height should not be smaller than the roughness length.

friction_velocity

Friction velocity, often denoted as "$u\star$" (pronounced "u-star"), is another important parameter in the study of atmospheric boundary layers. It represents the characteristic velocity of turbulent motions in the surface layer of the atmosphere, generated by the shear stress between the air and the Earth's surface.

Friction velocity is defined as the square root of the shear stress divided by the air density:

$u\star = \sqrt{\frac{\tau}{\rho}}$

where:

- $u\star$ is the friction velocity,
- τ is the shear stress between the air and the surface, and
- ρ is the density of the air.

The shear stress, τ, arises from the momentum transfer between the moving air and the surface roughness elements. It is related to the wind speed and the roughness length through a logarithmic wind profile, described by the following equation:

$u(z) = (\frac{u\star}{\kappa}) * \ln \frac{z}{z_o}$

where:

- $u(z)$ is the wind speed at height z above the surface,
- κ is the von Kármán constant (approximately 0.4), and
- z_o is the roughness length.

The friction velocity provides a measure of the intensity of turbulence near the Earth's surface. It influences several atmospheric processes, including heat and moisture exchange, pollutant dispersion, and the formation of atmospheric boundary layer structures. The magnitude of the friction velocity depends on surface roughness, wind speed, and atmospheric stability, among other factors.

surface_layer_height

The surface layer height, also known as the aerodynamic or boundary layer height, refers to the vertical extent of the atmospheric boundary layer near the Earth's surface. It represents the region where the physical properties, such as wind speed, temperature, humidity, and turbulence, are influenced by the underlying surface. The height of the surface layer is often estimated based on empirical relationships or atmospheric stability conditions. It can vary from a few meters to a few hundred meters above the ground, depending on factors such as surface roughness length, wind speed, and atmospheric stability. Additionally, the surface layer height can change diurnally, with variations due to daytime heating and nighttime cooling effects.

monin_obukhov_length

The Monin-Obukhov length, often denoted as "L", is a parameter used in the study of atmospheric boundary layers to characterise the stability of the air near the Earth's surface. It is named after the Russian scientists A. S. Monin and A. M. Obukhov, who made significant contributions to the understanding of turbulence and boundary layer physics.

The Monin-Obukhov length is defined as the ratio of the potential temperature difference to the buoyancy flux. It is given by the following equation:

$L = -\frac{u\star^3 * \theta_v}{\kappa * g * w\theta}$

where:

- L is the Monin-Obukhov length,
- $u\star$ is the friction velocity,
- θ_v is the virtual potential temperature,
- κ is the von Kármán constant (approximately 0.4),
- g is the acceleration due to gravity, and
- $w\theta$ is the vertical flux of virtual potential temperature.

The Monin-Obukhov length is an indicator of atmospheric stability. It quantifies the relationship between the mechanical production of turbulence due to wind shear and the buoyancy effects caused by vertical temperature gradients. Positive values of L indicate stable atmospheric conditions, while negative values indicate unstable conditions. A neutral atmosphere corresponds to L = 0, but L = 0 itself is rejected as an input value — omit the key for neutral stability instead.

The Monin-Obukhov length influences various atmospheric processes, such as turbulence structure, heat and moisture exchange, and dispersion of pollutants. It plays a significant role in determining the behaviour of the atmospheric boundary layer, including the vertical mixing of air masses and the development of turbulence. The value of L is influenced by factors such as surface properties, wind speed, temperature, and humidity gradients.

1.5.9 Reference Conditions

A *reference-condition entry* can be a complete, self-contained flow state, or it can instead be a lean entry that states only a ratio relative to another entry. This page covers the ratio form; for the full entry shape (temperature, pressure, velocity, viscosity, turbulence, ...) see *Initial Conditions*.

A ratio entry gives a `reference` key naming the base entry the ratios are relative to, plus one or more of the ratio fields below — no other field is required. It has no meaning on its own; something must point at it, typically a *boundary condition*'s own `condition/reference` field, for the ratios to be resolved against the flow field:

```
parameters["solver"]["reference_conditions"] = {
    "IC_1": {
        "temperature": zutil.to_kelvin(540.0),
        "pressure": 101325.0,
        "v": {"vector": zutil.vector_from_angle(alpha, 0.0), "mach": 0.15},
        "reynolds_no": 6.0e6,
        "reference_length": 1.0,
        "turbulence_intensity": 5.2e-2,
        "eddy_viscosity_ratio": 1.0,
    },
    "IC_2": {
        "reference": "IC_1",
        "total_pressure_ratio": 1.05,
    },
}
```

Keyword	Required	Default	Description
`reference`	Yes (for a ratio entry)		Name of the base entry these ratios are relative to.
`total_pressure_ratio`	No		Total pressure relative to the base entry's total pressure.
`total_temperature_rat`	No		Total temperature relative to the base entry's total temperature.
`static_pressure_ratio`	No		Static pressure relative to the base entry's static pressure.

These three ratio fields are part of every reference-condition entry regardless of equation type — set directly on a full entry, or on a lean entry pointing at one via `reference`.

Note

`mass_flow_rate`/`mass_flow_ratio` are not reference-condition fields — they are set directly on an inflow or outflow *boundary condition*, which resolves them against its own `condition` entry rather than through a ratio entry here.

1.5.10 Fluid Zones

Fluid zones live at `model.<mesh_name>.fluid_zones` — a dictionary of named blocks, keyed the same way as *Boundary Conditions*:

```
'model': {
    '<mesh_name>': {
        'mesh': '<mesh_name>.h5',
        'fluid_zones': {
            'FZ_1': {'type': '...', ...},
            'FZ_2': {'type': '...', ...},
        },
        'boundary_conditions': {...},
    }
}
```

Each block's `type` key selects the zone kind and determines which other keys are valid and required for that block; an unrecognised key anywhere in a fluid zone is rejected at validation time rather than silently ignored.

Note

Incompressible solvers. All of the source-term zone types below – `momentum`, `porous`, `canopy`, `disc` and `abl` – are available to the incompressible solver (`solver_settings type 'incompressible'`) as well as the compressible one. Points to be aware of:

- The incompressible momentum equation is solved per unit mass (density is divided out), so a `momentum` zone's `func` must return a force **per unit mass** rather than per unit volume. The porous, canopy and actuator disc models handle this internally, so their input values (`alpha`, `C2`, `cd`, the disc geometry and model dictionaries) are identical for both solvers.
- A **strong** `porous` or `canopy` zone under the segregated SIMPLE scheme (`scheme name 'simple'` in `convergence_control`) needs the scheme's `pressure_regularisation` set. Their drag adds to the momentum diagonal, so in a heavily blocked cell every pressure-correction face coefficient collapses towards zero and that cell's row of the p' Laplacian becomes near-empty, which the conjugate-gradient solver reports as an indefinite matrix before the run diverges. A value around `1e-2` restores stability, though a near-solid block still converges more slowly and less far than under the coupled scheme – prefer the `'coupled'` scheme for strong drag. Weak drag such as a typical canopy converges under either scheme with the default of 0 and the two agree to discretisation level. `momentum`, `disc` and `abl` are explicit body forces, never touch the momentum diagonal, and are unaffected either way.
- The `canopy` turbulence source is only defined for the `sst` model. Under Spalart-Allmaras or LES the canopy applies its momentum sink only, and the solver logs a warning.

Selecting cells

Most zone types select the cells they act on with `zones`: a list of the mesh's numbered cell zones (the same zone-numbering convention used by `boundary_conditions`). A smaller set of types — `momentum`, `porous`, `disc` and `canopy` — additionally accept `definition`, the path to a closed-surface VTP file describing the region, as an alternative to `zones`. For `momentum`, `porous` and `canopy` zones, exactly one of the two is enforced by validation; for `disc` zones, both are optional at the schema level even though the solver needs one of them to select cells in practice. Rotating, translating and ABL zones only accept `zones` today — they have no VTP-region equivalent.

Note

A few rough edges in the current schema (`zutil/zutil/validate/schemas/fluid_zones_schemas.py`) are worth knowing about before they surprise you:

- The zone-type enum used for validation also lists `rbf`, `rbf_multiscale`, `idw` and `fsi` as if they were fluid zone kinds. They are not — these are mesh-motion/coupling mechanisms, not physical fluid zones, and none of them corresponds to a working `type` value: the mesh-deformation zone types that do work are spelled `'rbf transform'` and `'idw transform'` (below), and there is no `fsi` or `rbf_multiscale` fluid zone at all. Writing `'type': 'rbf'` (or `idw`/`fsi`) in a fluid zone block fails validation.
- `ramp` (available on rotating and translating zones) is accepted as an arbitrary, untyped dictionary — its internal shape is not validated — and nothing in the current solver reads it, so setting it has no effect on the run today.
- `rotation_direction` (actuator disc zones) is a free-form string, not a `'clockwise'` / `'anticlockwise'` enum, so a misspelling is only caught at solve time, if at all, rather than at validation time.
- Actuator disc and translating zones are defined as their own standalone models rather than through the common base every other zone type shares, which is why their field lists are spelled out individually below rather than inheriting a common `zones`/`type` pair.

Fluid zone kinds

Type	Description
Rotating	Rotating reference frame or moving rotating region (MRF / overset)
Translating	Linearly translating region (overset)
Porous Media	Darcy/Forchheimer momentum sink representing a porous obstruction
Momentum Source	Arbitrary momentum source supplied by a Python function
Actuator Disc	Simple or blade-element (BET) turbine/propeller disc
Canopy	Vegetation drag/turbulence source for terrain wind modelling
Atmospheric Boundary Layer (ABL)	ABL-zone placeholder (see note below — not yet wired to a source term)
Mesh-deformation transform zones	Affine, IDW or RBF transforms applied to a subset of the mesh

Rotating

Keyword	Required	Default	Valid values
`type`	Yes	–	`'rotating'`
`zones`	Yes	–	List of mesh cell-zone IDs
`axis`	Yes	–	`[x, y, z]` rotation axis (normalised automatically)
`origin`	Yes	–	`[x, y, z]` point the axis passes through
`omega`	Yes	–	Angular velocity, rad/s
`moving_mesh`	No	`False`	`True` | `False` — only settable if this mesh has an `overset` boundary
`ramp`	No	–	Untyped dict (currently unread — see note above)

By default (`moving_mesh: False`) a rotating zone applies a rotating reference-frame momentum source without actually moving the mesh — the classic MRF treatment. Setting `moving_mesh: True` instead moves the zone's cells for real, which requires the same mesh to also declare an `overset` boundary condition; setting it without one is rejected at validation time.

```
'fluid_zones': {
    'FZ_1': {
        'zones': [17],
        'type': 'rotating',
        'axis': [0.0, -0.9988, 0.0498],
        'origin': [0.0, -0.6593, 0.2533],
        'omega': rot_rate,  # rad/s
    },
},
```

For a genuinely moving (overset) rotating zone:

```
'fluid_zones': {
    'rotating_zone': {
        'type': 'rotating',
        'zones': [9],
        'axis': [0, 0, 1],
        'origin': [0, 0, 0],
        'omega': 17.453292519943297,
        'moving_mesh': True,
    }
},
```

(continues on next page)

(continued from previous page)

```
'boundary_conditions': {
    'overset_boundaries': {'zones': [13], 'type': 'overset'},
    ...
},
```

Translating

Keyword	Required	Default	Valid values
`type`	Yes	–	`'translating'`
`zones`	Yes	–	List of mesh cell-zone IDs
`velocity`	Yes	–	`[x, y, z]` translation velocity, mesh units/sec
`translation_func`	No	–	Python function returning a time-dependent velocity, overriding `velocity`
`moving_mesh`	No	`False`	`True` | `False` — only settable if this mesh has an `overset` boundary
`ramp`	No	–	Untyped dict (currently unread — see note above)

```
'fluid_zones': {
    'FZ_1': {
        'type': 'translating',
        'zones': [10],
        'velocity': [1.0, 0.0, 0.0],
    },
},
```

Porous Media

Keyword	Required	Default	Valid values
`type`	Yes	–	`'porous'`
`zones`	One of zones/de	–	List of mesh cell-zone IDs
`definition`	One of zones/de	–	Closed-surface VTP file selecting the porous region
`alpha`	No	`1.0e12`	Permeability (Darcy term), ≥ 0
`C2`	No	`0.0`	Inertial resistance factor (Forchheimer term), ≥ 0
`porosity`	No	–	`0 \le porosity \le 1` (accepted but not yet read by the solver)
`direction`	No	–	`[x, y, z]` preferred flow direction (accepted but not yet read by the solver)

```
'fluid_zones': {
    'FZ_1': {
        'type': 'porous',
        'zones': [10],
        'alpha': 10e-7,
        'C2': 500,
    },
},
```

Note

The model constants need to be provided in mesh units. The examples below assume a mesh in metres (Units of alpha is $1/m$ and C2 is $1/m^2$)

The momentum source term for porous media is given by:

$$S_i = -\left(\frac{\mu}{\alpha} V_i + \frac{1}{2}\rho C_2 |V| V_i\right)$$

where S_i is the source term in the i-th direction, μ is the dynamic viscosity, α is the permeability, ρ is the fluid density, C_2 is the inertial resistance factor, and V_i is the velocity component.

In laminar flow using Darcy's power law:

$$\Delta P = \frac{\mu}{\alpha} V$$

where ΔP is the pressure drop, μ is the dynamic viscosity, α is the permeability, and V is the velocity.

`alpha` defaults to 10^{12} — effectively no resistance — so a porous zone does nothing until you lower it or raise `C2`.

Table 1.2: Example Permeability Values

Material	Permeability α (m^2)
Fence	1×10^{-7}
Car radiator	1×10^{-9}
Sand	1×10^{-11}

Table 1.3: Example Inertial Resistance Factor (C2) Values

Material	C2 (1/m)
Open cell foam	1000
Gravel	500
Sand	100
Fine mesh	10000

Momentum Source

An arbitrary momentum source, supplied as a Python function of the cell centres.

Keyword	Required	Default	Valid values
`type`	Yes	–	`'momentum'`
`zones`	One of `zones/de`	–	List of mesh cell-zone IDs
`definition`	One of `zones/de`	–	Closed-surface VTP file selecting the source region
`func`	No	–	Python function: `f(cell_centre_list) -> [(fx, fy, fz), ...]`
`source`	No	–	`[x, y, z]` fixed momentum source vector, used instead of `func`
`kind`	No	`'fixed'`	`'fixed'` | `'ramp'` | `'func'`
`frequency`	No	`1`	Update frequency (see note below)

Note

`func` is currently sampled once, when the zone is set up, not re-evaluated periodically during the solve — unlike actuator disc zones, where `update_frequency` genuinely gates a repeated re-sample. `kind` and `frequency` are accepted by the schema but have no corresponding behaviour for a plain `momentum` zone today.

```
def momentum_drive(cell_centre_list):
    """Return a constant +z body force (fx, fy, fz) per cell."""
    return [(0.0, 0.0, body_force) for _ in cell_centre_list]

'fluid_zones': {
    'FZ_1': {
        'type': 'momentum',
        'definition': 'pipe_periodic_region.vtp',
        'func': momentum_drive,
    }
},
```

Actuator Disc

Actuator disc zones model the thrust/torque of a wind turbine or propeller disc (see also *Actuator Disc Modelling* for a detailed overview). The `model.type` key selects between a `'simple'` disc (fixed or curve-driven thrust/power coefficients) and a `'bet'` (blade-element theory) disc.

Keyword	Required	Default	Valid values
`type`	Yes	–	`'disc'`
`zones`	No *	–	List of mesh cell-zone IDs
`definition`	No *	–	Closed-surface VTP file selecting the disc's momentum-source cells
`name`	Yes	–	Zone name (used in logging/output)
`geometry`	Yes	–	`{normal, centre, up, inner_radius, outer_radius}`
`discretisation`	Yes	–	`{type, number_of_elements}`
`model`	Yes	–	`{type: 'simple'|'bet', ...}` — see below
`controller`	Yes	–	`{type: 'none'|'tsr curve'|'fixed'|'schedule', ...}`
`rotation_directi`	Yes	–	Free-form string (conventionally `'clockwise'`/`'anticlockwise'` — not validated, see note above)
`reference_plane`	No	`False`	`True` | `False`
`reference_point`	No	–	`[x, y, z]` location used to sample reference flow conditions for thrust calculations
`u_ref`	No	–	Reference velocity (m/s) override, e.g. for hover cases with near-zero freestream
`update_frequency`	No	`1`	Cycles between re-evaluating the disc's momentum source
`verbose`	No	`False`	`True` | `False` — print turbine model/controller details at load

* Unlike momentum and canopy zones, nothing in the schema enforces that a disc zone sets `zones` or `definition` — both are individually optional, so a block setting neither still passes validation but has no way to select cells at solve time.

The `model` block for a `'simple'` disc:

Keyword	Required	Default	Valid values
`type`	Yes	–	`'simple'`
`kind`	No	–	`'propellor'` | `'turbine'`
`thrust_coefficient`	No	–	Fixed thrust coefficient
`thrust_coefficient_cu`	No	–	Thrust coefficient vs. tip-speed-ratio curve, as `[x, y]` pairs
`power_coefficient`	No	–	Fixed power coefficient (propellor model)
`power`	No	–	Fixed power (W)
`power_model`	No	–	`'curve'` | `'fixed'`
`power_curve`	No	–	Power vs. tip-speed-ratio curve, as `[x, y]` pairs

```
'fluid_zones': {
    'FZ_1': {
        'type': 'disc',
        'discretisation': {'type': 'disc', 'number of elements': 24},
        'model': {
            'type': 'simple',
            'power model': 'fixed',
            'kind': 'propellor',
            'thrust coefficient': 0.06,
            'power coefficient': 0.048,
        },
        'controller': {'type': 'fixed', 'omega': 314.1592653589793, 'pitch': 0.0},
        'rotation direction': 'clockwise',
        'geometry': {
            'normal': [0.0, 0.0, -1.0],
            'centre': [0.0, 0.0, 1e-06],
            'up': [1.0, 0.0, 0.0],
            'inner radius': 0.039025,
            'outer radius': 0.175,
        },
        'name': 'R01',
        'reference plane': True,
        'verbose': True,
        'update frequency': 10,
        'definition': './propellor_vtp/R01.vtp',
    },
},
```

Note

Keys may be written with spaces (`'inner radius'`, `'tip speed ratio'`) or underscores (`inner_radius`, `tip_speed_ratio`) — both forms are normalised to the same field before validation.

The `model` block for a `'bet'` disc adds the blade/aerofoil description:

Keyword	Required	Default	Valid values
`type`	Yes	–	`'bet'`
`kind`	No	–	`'propellor'|'turbine'`
`blade_chord`	No	–	`[span_fraction, chord_length]` pairs
`blade_twist`	No	–	`[span_fraction, twist_degrees]` pairs
`number_of_sections`	No	–	Number of circumferential sections
`number_of_blades`	No	–	Number of blades
`aerofoil_positions`	No	–	`[[span_fraction, aerofoil_name], ...]`
`aerofoils`	No	–	`{aerofoil_name: {'cd': [[deg, cd], ...], 'cl': [[deg, cl], ...]}}`
`tip_loss_correction`	No	–	`'elliptic'|'acos-fit'|'acos shift-fit'|'f-fit'|'none'|'rstar'`
`tip_loss_correction_r`	No	–	Span fraction beyond which the tip-loss correction is applied

```
'fluid_zones': {
    'FZ_1': {
        'type': 'disc',
        'discretisation': {'type': 'disc', 'number of elements': 24},
        'model': {
            'type': 'BET',
            'kind': 'propellor',
            'blade chord': [[0.2333, 0.1468], [0.2667, 0.1681], ..., [1.0, 0.0581]],
            'blade twist': [[0.2333, 28.6202], [0.2667, 25.5228], ..., [1.0, 0.0]],
            'number of sections': 24,
            'number of blades': 2,
            'aerofoil positions': [[0.0, 'aerofoil1'], [1.0, 'aerofoil1']],
            'aerofoils': {
                'aerofoil1': {
                    'cd': [[-60.0, 1.4552], ..., [60.0, 2.3542]],
                    'cl': [[-60.0, -0.7978], ..., [60.0, 1.3818]],
                },
            },
            'tip loss correction radius': 0.8,
            'tip loss correction': 'rstar',
        },
        'controller': {'type': 'fixed', 'omega': 314.1592653589793, 'pitch': 0.0},
        'rotation direction': 'clockwise',
        'geometry': {
            'normal': [-0.6597, -0.4356, -0.6124],
            'centre': [0.0, 0.0, 1e-06],
            'up': [0.0474, 0.7891, -0.6124],
            'inner radius': 0.039025,
            'outer radius': 0.175,
        },
        'name': 'R01',
        'reference plane': True,
```

(continues on next page)

(continued from previous page)

```
        'update frequency': 10,
        'definition': './propellor_trans_vtp/R01.vtp',
    }
},
```

A `'tsr curve'` controller (target tip-speed ratio driving a torque controller from a lookup curve) adds `tip_speed_ratio`/ `tip_speed_ratio_curve` to the `controller` block:

```
'controller': {
    'type': 'tsr curve',
    'tip speed ratio': 9.9485,
    'tip speed ratio curve': [[0.0, 0.0], [2.9991, 0.7055], ..., [41.0, 0.0]],
},
```

Canopy

For tree canopies in terrain wind models.

Keyword	Required	Default	Valid values
`type`	Yes	–	`'canopy'`
`zones`	One of zones/de	–	List of mesh cell-zone IDs
`definition`	One of zones/de	–	Closed-surface VTP file defining the canopy region
`field`	No	–	VTP file of surface points with a node array named `'Height'`, used as an alternative canopy-height lookup
`function`	No	–	Python function computing leaf area density (LAD) from cell centres
`cd`	No	`0.25`	Drag coefficient, ≥ 0
`beta_p`	No	`0.17`	Turbulence production parameter, ≥ 0
`beta_d`	No	`3.37`	Turbulence dissipation parameter, ≥ 0
`Ceps_4`	No	`0.9`	Turbulence parameter, ≥ 0
`Ceps_5`	No	`0.9`	Turbulence parameter, ≥ 0

where `cd`, `beta_p`, `beta_d`, `Ceps_4` and `Ceps_5` are defined in the literature (for example Desmond, 2014).

```
def lad_function(cell_centre_list):
    lai = 5.0  # for example following Da Costa 2007
    h_can = 20.0
    lad_list = []
    for cell in cell_centre_list:
        wall_distance = cell[3]
        lad = (np.interp(wall_distance, a_z[0] * h_can, a_z[1]) * lai / h_can,)
        lad_list.append(lad)
    return lad_list

'fluid_zones': {
    'canopy_zone': {
        'type': 'canopy',
        'definition': 'canopy_dacosta.vtp',
        'function': lad_function,
        'cd': 0.25,
        'beta_p': 0.17,
        'beta_d': 3.37,
```

(continues on next page)

(continued from previous page)

```
        'Ceps_4': 0.9,
        'Ceps_5': 0.9,
    },
},
```

and the variation in leaf area density is a function of height:

```
a_z = (
    np.asarray([0.0, 0.43, 0.1, 0.45, 0.2, 0.56, 0.3, 0.74, 0.4, 1.10,
                0.5, 1.35, 0.6, 1.48, 0.7, 1.47, 0.8, 1.35, 0.9, 1.01,
                1.0, 0.00]).reshape(11, 2).T
)
```

Note

When using `field` instead of `definition`, the height of the canopy at each cell is set by finding the nearest point to the cell centre on the supplied VTK file, with the height value looked up in a node-based scalar array named `'Height'`.

Atmospheric Boundary Layer (ABL)

Keyword	Required	Default	Valid values
`type`	Yes	–	`'abl'`
`zones`	Yes	–	List of mesh cell-zone IDs
`roughness_length`	No	–	Surface roughness length, > 0
`friction_velocit`	No	–	Friction velocity, > 0
`reference_height`	No	–	Reference height, > 0
`wind_direction`	No	–	`[x, y, z]` wind direction (normalised automatically)

```
'fluid_zones': {
    'abl_zone': {
        'type': 'abl',
        'zones': [10],
        'roughness_length': 0.03,
        'friction_velocity': 0.45,
        'reference_height': 10.0,
    },
},
```

Note

These fields are validated but not yet consumed by the solver: an ABL zone currently reduces at runtime to a bare momentum-source kernel over the selected cells, with no source term derived from `roughness_length`/`friction_velocity`/`reference_height`/ `wind_direction` wired in. Declaring one has no physical effect on the solve today.

Mesh-deformation transform zones

Three zone types apply a geometric transform to a subset of the mesh rather than a physical source term: an affine (rigid) transform, and two mesh deformation/interpolation schemes (RBF and IDW) typically used to propagate a moving boundary's displacement into the surrounding volume mesh. All three require `zones` and a required `func` that supplies the boundary/point displacement driving the deformation.

RBF transform (`'rbf transform'`)

Full RBF when `base_point_fraction` is `1.0` (the default); multiscale RBF when it is below `1.0`.

Keyword	Required	Default	Valid values
`type`	Yes	–	`'rbf transform'`
`zones`	Yes	–	List of mesh cell-zone IDs
`func`	Yes	–	Python function supplying the boundary displacement
`support_radius`	No	`1.0`	Support radius for interpolation, > 0
`basis`	No	`'c2'`	`'tps'` | `'thin_plate_spline'` | `'multiquadric'` | `'inverse_multiquadric'` | `'gaussian'` | `'linear'` | `'cubic'` | `'quintic'` | `'c0'` | `'c2'` | `'c4'` | `'c6'`
`base_point_fract`	No	`1.0`	`0 < base_point_fraction \le 1;` < `1.0` selects multiscale RBF

```
'fluid_zones': {
    'TR_1': {
        'type': 'rbf transform',
        'zones': [4],
        'func': my_transform,
        'support_radius': 200.0,
        'basis': 'c2',
    }
},
```

IDW transform (`'idw transform'`)

Keyword	Required	Default	Valid values
`type`	Yes	–	`'idw transform'`
`zones`	Yes	–	List of mesh cell-zone IDs
`func`	Yes	–	Python function supplying the boundary displacement
`n_nearest`	No	`100`	Number of nearest points, ≥ 1
`deformation_dist`	No	`1.0`	Maximum deformation distance, > 0
`power`	No	`[3.0, 5.0, 10.0]`	IDW power parameters
`fixed_zones`	No	–	List of mesh cell-zone IDs held fixed during deformation
`blending_stiffne`	No	–	`0 \le blending_stiffness \le 1`
`support_radius`	No	`1.0`	Alternative to `stencil_size` for bounding the stencil — set at most one of the two
`stencil_size`	No	–	Alternative to `support_radius` for bounding the stencil — set at most one of the two

```
'fluid_zones': {
    'TR_1': {
        'type': 'idw transform',
        'zones': [4],
        'func': my_transform,
        'fixed_zones': [1],
        'n_nearest': 500,
        'power': [4.0, 8.0, 10.0],
        'deformation_distance': 100.0,
```

(continues on next page)

(continued from previous page)

```
        'blending_stiffness': 0.5,
    }
},
```

Affine transform (`'affine transform'`)

Keyword	Required	Default	Valid values
`type`	Yes	–	`'affine transform'`
`zones`	Yes	–	List of mesh cell-zone IDs
`func`	Yes	–	Python function supplying the boundary displacement
`transform_matrix`	No	–	4x4 transformation matrix (non-singular 3x3 rotation/scaling submatrix)
`transform_func`	No	–	Python function returning a time-dependent transform, as an alternative to a fixed `transform_matrix`
`moving_mesh`	No	`False`	`True` | `False` — only settable if this mesh has an `overset` boundary

```
'fluid_zones': {
    'TR_1': {
        'type': 'affine transform',
        'zones': [4],
        'func': my_transform,
        'transform_matrix': [
            [1.0, 0.0, 0.0, 0.0],
            [0.0, 1.0, 0.0, 0.0],
            [0.0, 0.0, 1.0, 0.0],
            [0.0, 0.0, 0.0, 1.0],
        ],
    }
},
```

1.5.11 Mesh Transformation

zCFD has three distinct, independently-configured mechanisms for moving or deforming a mesh, all set up per model (i.e. per mesh) under `model.<name>`:

- **Initial placement** — a single static rotate/translate/scale of the whole mesh, applied once when the mesh is loaded. Set under `model.<name>.transforms`.
- **Dynamic rigid-body motion** — a zone of cells rotating or translating relative to the rest of the mesh (a rotor, a moving overset component). Set as a `'rotating'` or `'translating'` entry in `model.<name>.fluid_zones`.
- **Mesh morphing** — a known deformation of a surface (e.g. a wing or blade deflecting under load) propagated into the surrounding volume mesh using RBF or IDW interpolation. Set as an `'rbf transform'` or `'idw transform'` entry in `model.<name>.fluid_zones`.

These are covered in turn below.

Initial placement

`model.<name>.transforms` sets a single static affine transform applied to the whole mesh when it is read, before the solver starts.

Keyword	Required	Default	Valid values	Description
`transform_matrix`	No	`None`	4x4 list of lists, or numpy array	Affine transformation matrix applied to the mesh at load time.
`scale`	No	`None`	List of 3 floats	Per-axis `[x, y, z]` scale factors. Folded into `transform_matrix` before the solver runs (see note below); an identity scale (`[1, 1, 1]`) is a no-op.

Note

`scale` and `transform_matrix` are not two independent transforms applied by the mesh loader — only `transform_matrix` is ever read at mesh-load time. If `scale` is given, it is composed into `transform_matrix` (on top of any matrix already given) before the solver runs. Setting a non-identity `scale` with no `transform_matrix` produces a pure diagonal scale matrix; giving both composes the scale on top of the existing matrix, it does not replace it.

Build a rotation or translation matrix with the `zutil.transform` helpers (`rotate`, `translate`, `scale`), which each return/compose a 4x4 numpy array, and can be chained:

```
import zutil.transform

# Two 90-degree rotations about x, then 10 degrees about y, composed in order
mesh_transform = zutil.transform.rotate([1, 0, 0], 90.0)
mesh_transform = zutil.transform.rotate([1, 0, 0], 90.0, mesh_transform)
mesh_transform = zutil.transform.rotate([0, 1, 0], 10.0, mesh_transform)

parameters["model"]["naca0012"]["transforms"] = {"transform_matrix": mesh_transform}
```

A plain anisotropic mesh stretch needs only `scale`:

```
parameters["model"]["canopy_dacosta"] = {
    "transforms": {"scale": [50.0, 1.0, 1.0]},
    "mesh": "canopy_dacosta.h5",
    ...
}
```

Note

The schema also defines a `transform_func` field on `transforms`, intended as a Python-callable hook for whole-mesh dynamic rigid motion. It is not currently read anywhere in the solver or launcher — no control deck in this repository sets it, and setting it has no effect. For dynamic rigid-body motion, use the `'rotating'`/`'translating'` fluid zone types below instead.

Dynamic rigid-body motion

A zone of cells can be rotated or translated relative to the rest of the mesh by adding a `'rotating'` or `'translating'` entry to `model.<name>.fluid_zones`. This is used both for a simple rotating reference frame (e.g. a spinning cylinder or rotor in its own non-overset mesh) and, when combined with an `'overset'` boundary condition and `moving_mesh: True`, for a mesh that is physically re-cut and re-mapped against its background mesh every time step.

Keyword	Required	Default	Valid values	Description
`type`	Yes	–	`'rotating'` | `'translating'`	Selects rigid rotation or rigid translation of the zone.
`zones`	Yes	–	List of zone ID integers	Surface/volume zones the motion applies to.
`axis`	Yes (`'rotati` only)	–	Normalised 3-vector	Rotation axis.
`origin`	Yes (`'rotati` only)	–	3-vector	Point the rotation axis passes through.
`omega`	Yes (`'rotati` only)	–	Float	Angular velocity in rad/s.
`velocity`	Yes (`'transl` only)	–	3-vector	Translation velocity vector.
`translation_func`	No	`None`	Python function	Overrides `velocity` with a time-dependent translation velocity function (`'translating'` only).
`moving_mesh`	No	`False`	`True` | `False`	Physically moves the mesh (requires this model to also declare an `'overset'` boundary condition — see *overset*). Without an overset boundary, the zone still rotates/translates as a reference frame, but `moving_mesh` has no additional effect.
`ramp`	No	`None`	Dict	Optional ramping of the motion at start-up. Not a validated sub-schema — no control deck in this repository sets it; check the C++ read site before relying on a specific shape.

A rotating reference frame, with the mesh itself placed by an initial rotation (no overset boundary, so `moving_mesh` is left at its default):

```
import zutil.transform

parameters["model"]["background"] = {
    "mesh": "cylinder.h5",
    "transforms": {"transform_matrix": zutil.transform.rotate([0, 1, 0], -90.0)},
    "fluid_zones": {
        "rotating_zone": {
            "type": "rotating",
            "zones": [3],
            "axis": [0, 0, 1],
            "origin": [0, 0, 0],
            "omega": 17.453292519943297,
        }
    },
```

(continues on next page)

(continued from previous page)

```
    "boundary_conditions": {...},
}
```

An overset mesh that is actually rotated in place each time step (`moving_mesh: True`, paired with an `'overset'` boundary condition on the same model):

```
parameters["model"]["cylinder_1"] = {
    "mesh": "cylinder_origin.h5",
    "transforms": {
        "transform_matrix": [
            [1.0, 0.0, 0.0, 0.0],
            [0.0, 1.0, 0.0, 1.2],
            [0.0, 0.0, 1.0, 1.2],
            [0.0, 0.0, 0.0, 1.0],
        ]
    },
    "fluid_zones": {
        "FZ_1": {
            "type": "rotating",
            "zones": [0],
            "axis": [1.0, 0.0, -0.0],
            "origin": [0.0, 1.2, 1.2],
            "omega": 104.71975511965978,
            "moving_mesh": True,
        }
    },
    "boundary_conditions": {"BC_1": {"zones": [7], "type": "overset"}},
}
```

Mesh morphing

A known deformation of a surface in the mesh is propagated out into the volume using either radial basis function or inverse distance weighted interpolation. Deforming an aircraft wing or wind turbine blade to a static load shape are two uses of this functionality. Set up as an entry in `model.<name>.fluid_zones` with `'type': 'rbf transform'` or `'type': 'idw transform'`.

Keyword	Required	Default	Valid values	Description
`type`	Yes	–	`'rbf transform'` | `'idw transform'`	Selects the *interpolation* method used.
`zones`	Yes	–	List of zone ID integers	Surface zones to deform.
`func`	Yes	–	*Transform Function*	A python function that describes the surface deformation to be propagated into the volume mesh. Called for each node on the surfaces named in `zones`.
`support_radius`	No	1.0	Float > 0	Support radius (radius of influence) of the interpolation. Shared by both `'rbf transform'` and `'idw transform'`; for IDW it is an alternative to `stencil_size` (see note below).
`basis`	No	`'c2'`	`'tps'` | `'thin_plate_spline'` | `'multiquadric'` | `'inverse_multiquadri` | `'gaussian'` | `'linear'` | `'cubic'` | `'quintic'` | `'c0'` | `'c2'` | `'c4'` | `'c6'`	RBF function type (`'rbf transform'` only).
`base_point_fract`	No	1.0	Float, 0 < x <= 1	Fraction of points used as base points (`'rbf transform'` only). `1.0` (the default) is full RBF; `< 1.0` is multiscale RBF — plain and multiscale RBF are a single `'rbf transform'` type, distinguished by this value.
`stencil_size`	No	`None`	Positive integer	Maximum stencil size (nearest-neighbour count), an alternative to `support_radius` (`'idw transform'` only). Setting both `support_radius` and `stencil_size` explicitly is rejected as ambiguous.
`power`	No	`[3.0, 5.0, 10.0]`	List of 3 floats	Exponents used in the IDW interpolation scheme (`'idw transform'` only).
`n_nearest`	No	100	Positive integer	Number of nearest points used (`'idw transform'` only).
`deformation_dist`	No	1.0	Float > 0	Maximum deformation distance over which the surface deformation is blended into the volume mesh (`'idw transform'` only).
`blending_stiffne`	No	`None`	Float, 0 <= x <= 1	Stiffness of the blending function (`'idw transform'` only). Unlike the other IDW fields above, the C++ morpher reads this value unconditionally with no fallback, so it should always be given explicitly for an `'idw transform'` zone — there is no safe default to fall back on

Note

There is no `'tol'` field for the greedy-RBF point-addition tolerance — `RBFTransformConfig` has no tolerance field and rejects unknown keys, so this option cannot be set.

Note

The RBF C++ code also supports an optional `use_local_rbf_nodes` flag, restricting each partition's RBF solve to only the nodes it owns (a memory/performance optimisation for large parallel RBF problems). It is not part of the schema: no control deck sets it, every live RBF/IDW test case runs correctly using the full node set, and the C++ read is guarded (defaults to using all nodes when the key is absent), so leaving it unset changes nothing for any existing deck.

Interpolation method

Selects the interpolation method used to propagate the surface deformation into the volume mesh. Two `type` values are available:

'rbf transform' with `base_point_fraction` at its default of `1.0` uses the 'greedy' method of Allan et al, where the deformation is started with a subset of points and additional points are added until the surface deformation converges.

'rbf transform' with `base_point_fraction` set below `1.0` instead uses the multiscale RBF method, where a subset of surface points with a fixed support radius are solved implicitly and the remaining points are solved explicitly with a support radius determined as part of the solution process. The base set of points are assigned the `support_radius` and hence can have a wider sphere of influence than the remaining points which are used in the multiscale process.

Note

Increasing the number of points in the base set and/or increasing `support_radius` generally makes the surface deformation propagate further into the volume mesh, reducing the likelihood of generating poor quality cells.

'idw transform' uses a simple inverse distance weighted interpolation. The **power** parameter provides a list of three exponents used in the IDW scheme. The first (p1) is applied to the interpolation weight of points close to the surface, the second (p2) to points which are the deformation distance away from the surface and the third (p3) is used to blend between the two extremes.

$wt_1 = (deformation\_distance/distance)^{p1}$ $wt_2 = (deformation\_distance/distance)^{p2}$ $blend = ((deformation\_distance - distance)/deformation\_distance)^{p3}$

Then the weight is:

$wt = (wt1 * (1.0 - blend) + blend * wt2)$

Examples

RBF Example

```
import zutil

alpha = 10.0

def my_transform(x, y, z):
    v = [x, y, z]
    v = zutil.rotate_vector(v, alpha, 0.0)
```

(continues on next page)

(continued from previous page)

```
    return {"v1": v[0], "v2": v[1], "v3": v[2]}


parameters["model"]["naca0012"]["fluid_zones"] = {
    "TR_1": {
        "type": "rbf transform",
        "zones": [4],
        "func": my_transform,
        "support_radius": 200.0,
        "basis": "c2",
    }
}
```

RBF Multiscale Example

```
import zutil

alpha = 10.0

def my_transform(x, y, z):
    v = [x, y, z]
    v = zutil.rotate_vector(v, alpha, 0.0)
    return {"v1": v[0], "v2": v[1], "v3": v[2]}


parameters["model"]["naca0012"]["fluid_zones"] = {
    "TR_1": {
        "type": "rbf transform",
        "zones": [4],
        "func": my_transform,
        "support_radius": 200.0,
        "basis": "c2",
        "base_point_fraction": 0.1,
    }
}
```

IDW Example

```
import zutil

alpha = 10.0

def my_transform(x, y, z):
    v = [x, y, z]
    v = zutil.rotate_vector(v, alpha, 0.0)
    return {"v1": v[0], "v2": v[1], "v3": v[2]}


parameters["model"]["naca0012"]["fluid_zones"] = {
    "TR_1": {
        "type": "idw transform",
        "zones": [4],
        "func": my_transform,
        "fixed_zones": [1],
        "n_nearest": 500,
```

(continues on next page)

(continued from previous page)

```
        "power": [4.0, 8.0, 10.0],
        "deformation_distance": 100.0,
        "blending_stiffness": 0.5,
    }
}
```

1.5.12 Overset

zCFD is capable of automatically oversetting multiple meshes into a single simulation. An overset case is defined entirely within one control file: every mesh in the simulation is declared as its own named entry directly under the control file's top-level `model` key, each carrying its own `mesh` path, and each mesh that receives donor data from another mesh must declare an `'overset'` boundary condition on its hole-cutting surface (see *Overset boundary condition* below).

Setting up an overset case

One model entry is the background mesh, typically the largest of the meshes that the others overlap with. Each further entry is a mesh that oversets onto another entry (the background mesh, or another overset mesh), declared through an `'overset'` boundary condition whose `background` field names the donor model(s) (see below).

Example `model` structure (one background mesh, a refinement region overset onto it, and two further meshes overset onto the refinement region):

```
"model": {
    "background": {"mesh": "back1.h5", ...},
    "refinement_region": {"mesh": "back2.h5", ...},
    "cylinder_1": {"mesh": "back3.h5", ...},
    "cylinder_2": {"mesh": "back3.h5", ...},
}
```

Run the case with a plain:

```
(zCFD)> run_zcfd -c oversetmulticylinder.py
```

All meshes are read from the `mesh` key of each entry under `model`; no separate mesh/case file pairing and no command-line mesh argument is needed.

Overset boundary condition

Overset mapping options are set directly on the boundary condition that marks a mesh's overset (hole-cutting) surface, using `'type': 'overset'` inside that model's `boundary_conditions` block:

Keyword	Required	Default	Valid values	Description
`type`	Yes	–	`'overset'`	Marks this boundary as an overset (hole-cutting) surface.
`background`	No	`None`	Model name (string) or list of model names	The model(s) whose mesh provides donor data at this boundary. When omitted, the solver falls back to legacy all-pairs connectivity (mapping against every preceding model).
`idw_from_cells_orde`	No	0	0 or 1	Sets the order of accuracy for IDW mapping.
`additional_active_l`	No	0	Non-negative integer	Adds additional active cells to the overlapped mesh near the blanking boundary.
`moving_mesh_halo_la`	No	0	Non-negative integer	Halo fringe layers around the hole when the overset mesh moves (0 treats every blanked cell as a halo; a value > 0 is sufficient when the hole is re-cut every real time step).

Example usage, adapted from a real multi-mesh overset testcase (one background mesh, a refinement region overset onto it, and two further meshes overset onto the refinement region):

```
parameters = {
    ...
    "model": {
        "background": {
            "mesh": "back1.h5",
            "boundary_conditions": {
                "BC_1": {"zones": [9, 11], "type": "symmetry"},
                "BC_2": {"zones": [10], "type": "farfield", "condition": "IC_1", "kind
↪": "riemann"},
            },
        },
        "refinement_region": {
            "mesh": "back2.h5",
            "boundary_conditions": {
                "BC_1": {"zones": [9, 11], "type": "symmetry"},
                "BC_2": {
                    "zones": [10],
                    "type": "overset",
                    "background": ["background"],
                },
            },
        },
        "cylinder_1": {
            "mesh": "back3.h5",
            "boundary_conditions": {
                "BC_1": {"zones": [11, 12], "type": "symmetry"},
                "BC_2": {
                    "zones": [13],
```

(continues on next page)

(continued from previous page)

```
                    "type": "overset",
                    "background": ["refinement_region"],
                },
                "BC_3": {"zones": [10], "type": "wall", "kind": "no slip"},
            },
        },
    },
    ...
}
```

idw_from_cells_order

If **idw_from_cells_order** is set to 0, this provides a 0th order interpolation where we only utilise data from the cell a mapped point is located within. If set to 1, a first order interpolation is achieved using an IDW interpolation, using the cell a point is found in, and its neighbours.

additional_active_layers

Adds additional active cells which may have been made inactive by an overlapping mesh. If for example, we have a rotating domain, the resolution of the rotating boundary may result in rotated overset mesh boundary points, entering cells made inactive. This will result in no mapping between background and overset meshes at these points. To resolve this issue, it is possible to add additional active cells (for safety) to ensure a mapping between overset and background meshes. Whilst not necessary, this parameter may be useful if issues with mapping between meshes occur.

moving_mesh_halo_layers

Relevant only when the overset mesh is itself moving (see the rotating/translating fluid zone motion types with `moving_mesh: True` in *Mesh Transformation*). Controls how many halo fringe layers are kept around the re-cut hole each time the mesh moves; a value of 0 treats every blanked cell as a halo, which is safe but conservative, while N > 0 is sufficient when the hole is re-cut every real time step and avoids remapping cells that don't need it.

background

Names the model(s) whose mesh supplies donor data at this overset boundary, and is used to derive the blanking priority order: models are topologically sorted so that a model always appears after the model(s) it declares as its background, with root meshes (no overset boundary conditions) placed first. Explicit specification is preferred over the `None` default — it avoids unnecessary mapper creation for non-overlapping mesh pairs. Accepts either a single model name or a list, as shown in the example above.

> **Note**
>
> There is no control-file option for detailed per-mesh-pair mapper diagnostics. The underlying C++ verbose-mapping flag exists, but nothing in the current control-file schema or launcher wires a value to it, so it cannot be enabled from a control deck.

1.5.13 Write Output

write output

Controls the frequency and contents of the files output into the **output** directory. This block lives at `solver.output_settings.solution` (it can also be overridden per mesh at `model.<name>.output_settings.solution`). Like every block under `output_settings`, it rejects unknown keys outright rather than silently ignoring a mistyped one.

The **output** directory will be created in the working directory from which the solver is run and is named using the following format:

```
<case_name>_P<number of mpi ranks>_OUTPUT
```

Keyword	Required	Default	Valid values
'frequency'	No	See *"frequency"*	Dictionary; see *"frequency"*
'format'	No	'vtk'	'none' | 'vtk' | 'ensight' | 'fwensight' | 'native'
'surface_variables'	No	None	List of *"variable"* names
'volume_variables'	No	None	List of *"variable"* names
'fwh_interpolate'	No	None	List of .stl file filenames
'fwh_wall_data'	No	False	True | False
'write_restart'	No	True	True | False
'binary'	No	True	True | False
'precision'	No	'single'	'single' | 'double'
'global_node_ids'	No	True	True | False
'output_solver_cycle'	No	False	True | False
'immersed_boundary_surface'	No	None	List of dictionaries with "zone" and "stl" keys, specifying which immersed boundary zones will be mapped onto specified STL surfaces

Example usage:

```
parameters = {
  ...
  "solver": {
      ...
      "output_settings": {
          "solution": {
              "format": "vtk",
              "surface_variables": ["V", "p"],
              "volume_variables": ["V", "p"],
              "fwh_interpolate": ["fwh_surface1.stl"],
              "frequency": {
                  "volume_data": 500,
                  "surface_data": 500,
              },
              "immersed_boundary_surface": [
                  {"zone": 7, "stl": "body.stl"},
                  {"zone": 9, "stl": "hub.stl"},
              ],
          },
      },
  },
  ...
}
```

format

Selects the file format to output the data in. Using 'none' as format will output no files. 'vtk' outputs in VTK HDF format (.vtkhdf) for reading into ParaView.

> **Note**
>
> The .vtkhdf output format requires ParaView 6.1.1 or later.

surface_variables

A list of *variables* to output on the boundaries of the mesh. This is useful for visualisation of the flow variables on the wall of the mesh for post processing. Which variable names are valid depends on the equation type (and, for RANS, the turbulence model) being solved — see *Output Variables*. For the 'vtk' format the surface data is written to the *output* directory with the file name:

```
<case_name>_<boundary>.vtkhdf
```

volume_variables

A list of *variables* to output in a volumetric format. Which variable names are valid depends on the equation type (and, for RANS, the turbulence model) being solved — see *Output Variables*.

For the 'vtk' format the volume data is written to the *output* directory with the file name:

```
<case_name>.vtkhdf
```

volume interpolate

> **Warning**
>
> zCFD can interpolate volume data onto a supplied surface or volume mesh (VTK `.vtu`/`.vtp` or `.stl` files) — this is the mechanism that feeds monitor surfaces and downstream post-processing without needing to store the full solution at every timestep. The frequency at which this runs is configurable (see *'volume_interpolate'* below), but there is **no schema field to list the target files**: `SolutionModel` (`zutil/zutil/validate/schemas/output_schemas.py`) has no `volume_interpolate` key alongside `surface_variables`/`volume_variables`, even though the C++ side (`PythonExport.cxx`) reads a `volume_interpolate` file list directly from the `solution` dict. This gap is not called out in `developer_docs/input-deck.md`, so it is noted here: there is no way to set this key from a schema-valid control file.

fwh_interpolate

In addition to volume interpolation, zCFD is capable of writing volume interpolated data to HDF5 files for use in the *zCFD Ffowcs-Williams Hawkings (FWH) solver*. This parameter lets you list the filenames of surfaces to interpolate to for HDF5 file output. The files specifying the surfaces must be .STLs. The outputted files contain density, velocity and pressure data only, and are suitable for use as permeable surfaces in the zCFD FWH solver. The data is outputted to the *ACOUSTIC_DATA* sub-directory within the *output* directory.

fwh_wall_data

This parameter controls whether wall data is outputted to a HDF5 file for use in the *zCFD FWH solver*. The outputted file contains pressure data only, and is suitable for use as an impermeable surface in the zCFD FWH solver. The data is outputted to the *ACOUSTIC_DATA* sub-directory within the *output* directory.

write_restart

Whether to write *restartable* checkpoint files at all. Set to `False` to skip restart output entirely, regardless of the `'checkpoint'` frequency.

binary

Whether output files are written in binary (`True`, the default) or text form.

precision

Floating point precision used for output data: `'single'` (default) or `'double'`.

global_node_ids

Retains the local-to-global node map so VTKHDF output can stitch partitions back together without a clean-to-grid filter in ParaView. Advanced option; leave at the default (`True`) unless you have a specific reason to disable it.

output_solver_cycle

Stamps each VTKHDF output step with the solver cycle number instead of the physical time. Every step of a steady run carries the same physical time, so with the default (`False`) the appended steps are indistinguishable in ParaView's time controls; setting this to `True` makes them steppable. Has no effect on formats other than VTKHDF.

immersed_boundary_surface

If the case uses immersed wall boundary conditions then force reporting on those surfaces requires STL files to be provided per immersed wall zone. The solution is interpolated onto the points in the STL so the mesh needs to be at least as fine as the boundary otherwise the accuracy of the reported forces will be affected. Additionally the normals used to calculate output values will be calculated directly from the provided STL file. Therefore a user should either use a surrogate STL generated by *zM3*, or ensure the provided STL file is watertight and has normals pointing into the body.

The interpolated solution on the STL is also output at the same frequency as other surface data.

> **Note**
>
> Two `'write output'` keys have no schema equivalent at all and are unaccounted for in `developer_docs/input-deck.md`: `'scripts'` (the list of ParaView Catalyst script filenames for in-situ visualisation) and `'variable_name_alias'` (the dict mapping zCFD variable names to alternate output names). The C++ side (`OutputVTK.cxx`) reads both directly from the `solution` dict, but `SolutionModel` declares neither field, so a control file has no way to set them.

What a checkpoint records about the run

A `<model>_results.h5` checkpoint carries the run's own configuration alongside the flow solution, so the file explains where it came from without depending on the run directory still being intact:

Location in the file	Contents
`version` (root attribute)	Layout version of the checkpoint, currently 2. This is the file layout, not the deck's `'config version'`.
`/provenance/status`	The `<case>_status.yaml` file verbatim — zCFD version, date, host names, devices, partition count and the mesh MD5 checksums. Written once, since it cannot change during a run.
`/provenance/control/<filename>`	The source of every Python file the control-file read executed, one dataset each, with the full path and mtime as attributes. The set is the whole include chain, not just the file named on the command line, so a deck built from a base plus overrides is recorded in full.

The control deck can be edited while a run is in progress — the solver polls it and applies any change the schema marks updatable — so it is re-read on each checkpoint and rewritten whenever it has actually changed. Reading it back is a plain HDF5 string read:

```
import h5py

with h5py.File("case_OUTPUT/background/background_results.h5") as f:
    print(f.attrs["version"])
    print(f["provenance/status"][()].decode())
    for name in f["provenance/control"]:
        print("---", f[f"provenance/control/{name}"].attrs["path"])
        print(f[f"provenance/control/{name}"][()].decode())
```

Recording these is best effort: a checkpoint that could not be annotated is still a valid, restartable checkpoint, and the run reports a warning rather than stopping.

Frequency

Sets how frequently the solver should output the flow solution. A single integer value will control output of both the volume and surface data output. A dictionary value here allows for fine grained control over the frequency with which each individual output is done. If the solver is running an unsteady simulation then this is the frequency of the real time step otherwise the pseudo cycle.

Keyword	Required	Default	Valid values
'volume_data'	No	1.00E+08	Positive integer
'surface_data'	No	1.00E+08	Positive integer
'volume_interpolate'	No	1.00E+08	Positive integer
'fwh_interpolate'	No	1.00E+08	Positive integer
'fwh_wall_data'	No	1.00E+08	Positive integer
'checkpoint'	No	1.00E+08	Positive integer
'volume_data_start'	No	1	Positive integer
'surface_data_start'	No	1	Positive integer
'volume_interpolate_start'	No	1	Positive integer
'fwh_interpolate_start'	No	1	Positive integer
'fwh_wall_data_start'	No	1	Positive integer
'checkpoint_start'	No	1	Positive integer

volume_data, **surface_data**, **volume_interpolate**, **fwh_interpolate**, **fwh_wall_data** and **checkpoint** control the frequency at which the solution data of the given type is written. The frequency is specified as the solver cycle number. The **checkpoint** option writes a *restartable* checkpoint of the flow solution at that point (subject to `'write_restart'` above).

volume_data_start, **surface_data_start**, **volume_interpolate_start**, **fwh_interpolate_start**, **fwh_wall_data_start** and **checkpoint_start** specify the first cycle that the solver should start writing that particular output.

A plain integer is still accepted in place of the dictionary: it is expanded to `surface_data`, `volume_data` and `checkpoint` all set to that value, with the rest left at their defaults.

Note

On systems with slower file I/O the overhead of writing data can slow the overall solve time, reducing the output frequency lets you address this.

Example usage:

```
parameters = {
  ...
  "solver": {
      ...
      "output_settings": {
          "solution": {
              "format": "vtk",
              "surface_variables": ["V", "p"],
              "volume_variables": ["V", "p"],
              "frequency": {
                  # Output visualisation files every 3 cycles
                  "volume_data": 3,
                  "surface_data": 3,
                  "volume_interpolate": 3,
                  "checkpoint": 3,
                  # Start output every 3 cycles after cycle 1 (default = 1)
                  "volume_data_start": 1,
                  "surface_data_start": 1,
                  "volume_interpolate_start": 1,
                  "checkpoint_start": 1,
              },
          },
      },
  },
  ...
}
```

compute_average_and_rms / average_start_time_cycle

> **Warning**
>
> The schema (`OutputModel` in `output_schemas.py`) places both fields — `compute_average_and_rms` (bool, default `False`) and `average_start_time_cycle` (int, default 0) — directly on `output_settings`, as siblings of `solution` and `report`, rather than inside `solution`. Every C++ read site (`SolverData.cxx`, `ExplicitSolver.cxx`, `incomp/Coupled.cxx`) looks for `compute_average_and_rms` **inside** the `solution` dict instead, and `SolutionModel` rejects it there as an unknown key. Setting `compute_average_and_rms` at the location the schema requires (`output_settings.compute_average_and_rms`) therefore has no effect on the solve. This mismatch is not covered in `developer_docs/input-deck.md`'s "What's left", so it is noted here. `average_start_time_cycle` additionally has no C++ read site at all under any name — it has no effect regardless of where it is nested.

Schema-valid placement, for reference (validates, but see the warning above):

```
"output_settings": {
    "solution": {"format": "vtk"},
    "report": {"frequency": 10},
    "compute_average_and_rms": True,
    "average_start_time_cycle": 1000,
},
```

Enables computation of time averaged and RMS values for pressure, temperature and velocity. This only applies to *unsteady* cases. `average_start_time_cycle` is intended to control the first real time cycle from which the running average is calculated, allowing the solver to run for a number of time cycles before averaging begins.

1.5.14 Reporting

zCFD has the capability to output certain parameters to a <casename>_report.csv file as the solver progresses. These can be used to monitor the solver's convergence. By default, the solver outputs volume integrated residuals for each solution variable to the <casename>_report.csv file, but adding to the `report` block means other quantities of interest can also be monitored as the simulation progresses.

The report block lives at `solver.output_settings.report` (it can also be overridden per mesh at `model.<name>.output_settings.report`). Like every block under `output_settings`, it rejects unknown keys outright rather than silently ignoring a mistyped one.

Keyword	Required	Default	Valid values
'frequency'	Yes	–	Positive integer
'monitor'	No	–	Dictionary of monitor point configs
'forces'	No	–	Dictionary of force configs
'mass_flow'	No	–	Dictionary of mass flow configs
'scale_residuals_by_volume	No	False	Boolean
'residual_statistics'	No	False	Boolean

'frequency' sets how frequently a new line is appended to the <casename>_report.csv file. If frequency = x, the solver will output to the <casename>_report.csv every x solver cycles. For a dual time-stepping simulation, this is the inner (pseudo-time), not outer, cycles.

> **Note**
>
> When *'inner_convergence'* is set, its **'check_frequency'** cycles are also reporting cycles, so rows may appear more often than **'frequency'** alone would suggest.

'scale_residuals_by_volume' scales the reported volume-integrated residuals by cell volume. It applies to the `resvar_i` and `resshare_i` output variables as well as to the report, so the field and the norm of that field always describe the same quantity.

Residual statistics columns

Setting **'residual_statistics'** adds four columns per equation, describing the *true* residual $R(Q)$ rather than the solution update. They answer a question the RMS cannot: an RMS says "not converged" without saying that the field is converged everywhere except a handful of cells, which is the case that takes longest to diagnose.

The columns are named prefix-first (`Rinf_rho`, not `rho_Rinf`) and are written to <casename>_report.csv only - they are deliberately kept off the console residual line, which four extra columns per equation would otherwise swamp.

Column	Meaning
'Rinf_<eq>'	$\|R\|_\infty$, the largest residual anywhere in the mesh for that equation.
'Rrms_<eq>'	RMS of the true residual. Note this is *not* the equation's own residual column, which is the RMS of the solution update $\Delta Q/\Delta\tau$; the two coincide only when the linear solve is exact.
'Neff_<eq>'	Participation ratio $(\sum R^2)^2/\sum R^4$: the number of cells the residual is effectively spread over. Equal to the cell count when every cell contributes equally, and to 1 when a single cell carries everything. A small value against a large mesh means a localised stall, not a stalled field.
'Rtop_<eq>'	Fraction of $\sum R^2$ held by the single worst cell.

To find *where* those cells are, add the *resshare_i* volume output variables. `resshare_i` is each cell's share of the same $\sum R^2$, summing to one over the mesh, so thresholding it at `1/Neff_<eq>` isolates the cells carrying

more than an equal share - the cells `Neff` is counting.

> **Note**
>
> This is a different measure from *'InnerCells'*, and neither supersedes the other. `InnerCells` is a single scalar over the solution *update*, used to steer dual-time inner cycles. These are per-equation figures over the true residual, for diagnosing where a steady-state stall is sitting.

Off by default: the statistics cost an extra pass over the residual field and two MPI reductions per report.

Inner cycle convergence columns

Dual time-stepping simulations report extra columns describing the convergence of the inner (pseudo-time) iteration within each real time step. The residual columns themselves are absolute and are never normalised per real time step, so they cannot answer "has this real time step converged"; these can. The first four are always present; which of them carries meaning depends on the solver.

Column	Solver	Meaning
'InnerCycle'	both	The inner cycle index within the current real time step, counting from 1. Use this as the x axis when plotting the convergence of a single real time step.
'InnerDrop'	both	Orders of magnitude the worst selected residual has dropped since the first inner cycle of the current real time step. This is what **'target_orders'** tests on the incompressible solver; on the compressible solver it is a diagnostic only - see *Choosing a target* for why.
'InnerRatio'	compressible	Largest, across the mean flow equations, of each residual divided by its own physical rate of change. Zero on real time cycle 0, where there is no time history to compare against, and always zero on the incompressible solver, which has no pseudo-time residual.
'InnerRatioDrop'	compressible	Orders of magnitude 'InnerRatio' has fallen within the current real time step. This is what **'target_orders'** tests on the compressible solver. Always zero on the incompressible solver.
'InnerCells'	compressible	Effective number of cells carrying the residual. A small value means limiter switching in a few cells rather than a stalled field. Not written by the incompressible solver, which has no slope limiter.

See *Reading the inner convergence* for how to plot these and how to use them to choose **'cycles'**.

> **Note**
>
> With *'inner_convergence'* enabled the number of inner cycles varies between real time steps, so the **'Cycle'** column no longer advances by a fixed stride per real time step. Group on **'RealTimeStep'** rather than assuming a stride.

'monitor', **'forces'** and **'mass_flow'** are dictionaries keyed by whatever name you choose for each block; the keys are not otherwise validated, but naming them consecutively with a prefix (e.g. `MR_1`, `FR_1`, `MF_1`) keeps a large control file readable:

Reporting on	Suggested key prefix
'monitor'	MR_?
'forces'	FR_?
'mass_flow'	MF_?

Example usage:

For a simulation with 1000's of cycles, it can be unnecessary to report every time cycle, which slows the solver down. For a dual timestepping simulation with 20 inner time cycles, outputting every 10 cycles generally gives enough information without substantially slowing the solver down

```
# Report every 10 cycles
"report": {
    "frequency": 10,
},
```

A simulation with two monitor points and one forces block.

In addition to standard flow field outputs (see below), zCFD can provide information at monitor points in the flow domain, and integrated forces across all parallel partitions. Any number of monitor, forces or mass flow blocks may be specified.

```
"report": {
    "frequency": 10,
    "monitor": {
        "MR_1": {
            "point": [0.0, 0.0, 0.0],
            "variables": ["p", "V"],
        },
        "MR_2": {
            "point": [1.0, 0.0, 0.0],
            "variables": ["p", "V"],
        },
    },
    "forces": {
        "FR_1": {
            "zones": [4],
        },
    },
},
```

Monitor Points

Monitor points report the specified variable at fixed mesh locations.

Keyword	Required	Default	Valid values
'point'	Yes	–	[x,y,z] position vector specifying a valid location in the mesh
'variables'	Yes	–	List of variables to report (see *Output Variables*)
'name'	No	the block's own key	String; name that appears in the <casename>_report.csv column heading

'point' should be the x,y,z location of the point of interest in the mesh as it appears in the solver (i.e. after any mesh scaling due to parameters *scaling* parameters).

'variables' controls which variables are reported at this point. In the <casename>_report.csv the column name will be a combination of the **'name'** (or, if omitted, the block's own dictionary key) and the variable name.

'name' overrides the column heading used for variables from this monitor point. If omitted, the dictionary key you gave the block (e.g. `MR_1`) is used instead.

Example usage:

Tracking pressure and velocity is useful for determining shedding cycle behaviour, or in the case of a scale resolving simulation, to determine the turbulence spectrum which is being resolved. It may be important to track other variables (e.g. turbulence intensity, `ti`) depending on the simulation setup.

```
"report": {
    "frequency": 10,
    "monitor": {
        "MR_1": {
            "name": "monitor_1",
            "point": [180.0, 100.0, 50.0],
            "variables": ["V", "p", "T", "ti"],
        },
    },
},
```

Forces

Force reporting outputs the x,y,z total forces and moments on a surface or set of surfaces. These are presented as force coefficients (using the non-dimensionalisation convention used for aerofoil lift, drag and moment coefficients), as opposed to dimensional values. The force and moment coefficients resulting exclusively from viscous forces and exclusively from pressure forces are also given.

Keyword	Required	Default	Valid values	Description
'zones'	Yes	–	List of valid mesh zone IDs (integers)	All the mesh zone IDs for the surfaces used in this force report. The forces and moments over all the zones are summed to give the forces and moments outputted in the force report.
'name'	No	the block's own key	String	Sets the name that appears in the <casename>_report.csv column headings for variables from this force monitor. If omitted, the dictionary key you gave the block (e.g. `FR_1`) is used instead.
'transform'	No	–	An (optional) transform function.	See *Transform function*. A transformation function may be supplied to the force report block to transform the force into a different coordinate system.
'reference_area'	No	1.0	Positive number	Sets the dimensional reference area which is used to non-dimensionalise the reported force and moment coefficients
'reference_length'	No	1.0	Positive number	Sets the dimensional reference length which is used to non-dimensionalise the moment coefficients
'reference_point'	No	[0,0,0]	[x,y,z] position vector	Sets the reference point which is used to non-dimensionalise the moment coefficients
'reference_pressure'	No	0.0	Positive number	For a wetted surface, this sets the pressure on the non-wetted side of the surface. Pressure forces on the surface are calculated by integrating (p - pRef) over the surface
'dimensional'	No	False	Boolean	Reports the forces in dimensional units
'inertial_frame'	No	True	Boolean	For a rotating zone, transforms the forces and moments into the inertial (ground) frame before they are reported, rather than leaving them in the rotating (blade) frame.

Example usage:

```
"report": {
    "frequency": 10,
    # Report force coefficients in the grid axes, using a user defined transform
    "forces": {
        "FR_1": {
            "name": "force_1",
            # Required: Zones to be included
            "zones": [1, 2, 3],
            # Transformation function
            "transform": my_transform,
            "reference_area": 0.112032,
```

(continues on next page)

(continued from previous page)

```
            "reference_length": 1.0,
            "reference_point": [0.0, 0.0, 0.0],
            # Calculate forces using a specified reference pressure rather than␣
↪absolute
            "reference_pressure": 0.0,
        },
    },
},
```

Output names

The forces will be written into the <casename>_report.csv prefixed with the supplied **'name'** (or, if not given, the block's own dictionary key). The forces will be written with the following names:

Item	Reporting output name
Total forces and moments	**'name'**_F[x,y,z] and **'name'**_M[x,y,z]
Forces and moments due only to pressure forces	**'name'**_Fp[x,y,z] and **'name'**_Mp[x,y,z]
Forces and moments due only to viscous (friction) forces	**'name'**_Ff[x,y,z] and **'name'**_Mf[x,y,z]
Transformed total forces and moments (if transform function is set)	**'name'**_Ft[x,y,z] and **'name'**_Mt[x,y,z]
Transformed forces and moments due only to pressure forces (if transform function is set)	**'name'**_Ftp[x,y,z] and **'name'**_Mtp[x,y,z]
Transformed forces and moments due only to viscous forces (if transform function is set)	**'name'**_Ftf[x,y,z] and **'name'**_Mtf[x,y,z]

Note

For a rotating zone, `'inertial_frame'` (default `True`) transforms the totals above **in place** before they are written — there is no separate set of "static frame" columns. Set `'inertial_frame': False` to report the forces and moments in the rotating (blade) frame instead, still under the same `_F`/`_M` column names. A rotating zone also automatically adds `'name'`_Thrust, `'name'`_Tq and `'name'`_Tqp (thrust, torque and power about the rotation axis) — there is no keyword to configure these; they appear whenever the reported zones belong to a rotating boundary.

Additional information

Total force, $\mathbf{F}$, is simply , $\mathbf{F}_p + \mathbf{F}_v$, and likewise for moments.

Forces are non-dimensionalised into force coefficients as $C_x = \frac{\mathbf{F}_x}{q_{\text{inf}} S}$ Where q_{inf} = reference dynamic pressure and S = reference area

Moments are non-dimensionalised into force coefficients as $C_{Mx} = \frac{\mathbf{M}_x}{q_{\text{inf}} S c}$ Where c = reference length

$\mathbf{F_p} = \oint -(p - p_{ref})\, \hat{\mathbf{n}}\, \mathrm{d}A$

Where p is surface pressure and $\hat{\mathbf{n}}$ is the surface normal (pointing towards the wetted area).

$\mathbf{F}_v = \oint \mu \nabla.V \, \mathrm{d}A$

Where μ is dynamic viscosity and V is the velocity vector at the wall.

$\mathbf{M}_p = \oint -(p - p_{ref}) \hat{\mathbf{n}}\, (x - x_{ref})\, \mathrm{d}A$

$\mathbf{M}_v = \oint \mu \nabla V\, (x - x_{ref})\, \mathrm{d}A$

Note

Reference pressure p_{ref} is dynamic pressure ($\frac{1}{2}\rho V^2$), taken from the *reference state*.

Mass flow

Keyword	Required	Default	Valid values	Description
'zones'	Yes	–	List of valid mesh zone IDs (integers)	All the mesh zone IDs used in this mass flow monitor

'zones' The mass flows through all the zones are summed to give a single output value for the mass flow monitor. Flow out of the domain is considered positive massflow, flow into the domain is considered positive massflow and the massflows are dimensional (typically kg per second).

The <casename>_report.csv column is always named `<block key>_massflow` — e.g. a block keyed `MF_1` reports as `MF_1_massflow`.

Note

Unlike `'monitor'` and `'forces'`, a mass flow block has no `'name'` field to override the reported column name — only `'zones'` is a valid key here. If you need a friendlier column name than the dictionary key, choose the key itself accordingly (e.g. `"pipe_outflow"` instead of `"MF_1"`).

Example usage:

```
"report": {
    "frequency": 10,
    # Report massflow
    "mass_flow": {
        "MF_1": {
            # Zones to be included
            "zones": [1, 2, 3],
        },
    },
},
```

1.5.15 Output variables

Which variable names are valid for `surface_variables`, `volume_variables` (*write output*) and `report.monitor.*.variables` (*reporting*) depends on the equation type being solved, and — for RANS — on the turbulence model. The schema (`zutil/zutil/validate/schemas/output_variables.py`) builds each equation type's valid set incrementally: every equation type gets the base set below, then adds the variables for its physics tier. A request for a variable outside the valid set for the current equation type fails validation.

The equation types are: `euler`, `viscous`, `rans`, `les`, `incomp viscous`, `incomp rans`, `incomp les`. Each tier below is cumulative — `rans` and `les` get everything listed under Base, Euler-tier, Viscous-tier, LES-tier and RANS-tier.

Base variables

Available to every equation type.

Variable Name	Alias	Definition
temperature	T, t	Temperature in (K)
potentialtemperature		Potential temperature in (K)
pressure	p	Pressure (Pa)
gauge_pressure		Pressure relative to the reference value (Pa)
density	rho	Density (kg/m^3)
velocity	V, v	Velocity vector (m/s)
mach	m	Mach number
cp		Pressure coefficient
totalcp		Total pressure coefficient
ek		
enstrophy		
pressuregrad		Pressure gradient vector
densitygrad		Density gradient vector
velocitygrad		Velocity gradient tensor
temperaturegrad		Temperature gradient vector
velocitynd		Non-dimensional velocity
velocitynearnd		Non-dimensional velocity at the near-wall cell
walldist		Wall distance (m)
timestep		Local timestep
zone		Boundary/mesh zone number
centre		Cell centre location
cell_centre		Cell centre location
densitylimiter		Density limiter value
klimiter		Turbulence kinetic energy limiter value
omegalimiter		Turbulence eddy frequency limiter value
overset		Overset blanking/donor status
timestepratio		Local time step ratio
V_avg		Running average of the velocity vector (m/s)
V_rms		RMS of the velocity vector (m/s)
p_avg		Running average of the static pressure (Pa)
p_rms		RMS of the static pressure (Pa)
T_avg		Running average of the temperature (K)
T_rms		RMS of the temperature (K)

The `_avg` and `_rms` fields are written once *'compute_average_and_rms'* is on.

Euler-tier volume variables

Added for `euler` and every equation type above it.

Variable Name	Alias	Definition
vorticity		Vorticity vector ($1/s$)
vorticitynd		Non-dimensional vorticity
Qcriterion		Q criterion
Qcriterionnd		Non-dimensional Q criterion
helicity		Helicity
helicitynd		Non-dimensional helicity
cell_velocity		
cell_volupdate		Cell volume update rate (moving mesh)
cellzone		
cellcolour		Partition/colouring index used by the mesh partitioner
sponge		Sponge layer damping coefficient
parent		Parent cell/zone id (overset)
FailCell		Cells flagged as first-order fallback
MomentumSource		Momentum source term (fluid zone sources)
roelowdissipation		Roe low-dissipation blending factor
cellvolume		Cell volume
badcells		Cells tagged as first order when using mesh quality remediation
artificial_viscosity		Artificial viscosity applied
cellorder		Spatial order used in each cell
lambdavariable		Spectral radius of the flux Jacobian

Euler-tier surface variables

Added for `euler` and every equation type above it.

Variable Name	Alias	Definition
pressureforce		Pressure component of force on a face
pressuremoment		Pressure component of moment on a face
pressuremomentx		Component of pressure moment around x axis
pressuremomenty		Component of pressure moment around y axis
pressuremomentz		Component of pressure moment around z axis
centre		Face centre location
immersed boundary yplus		Non-dimensional wall distance on an immersed boundary surface
immersed boundary vector		Interpolated velocity vector on an immersed boundary surface
immersed boundary normal		Surface normal on an immersed boundary surface
immersed boundary friction velocity		Friction velocity on an immersed boundary surface

Viscous-tier volume variables

Added for `viscous` and every equation type above it (`rans`, `les`, and the `incomp` variants of each).

Variable Name	Alias	Definition
viscosity	mu	Dynamic viscosity ($Pa \cdot s$)
nu		Kinematic viscosity (m^2/s)

Viscous-tier surface variables

Added for `viscous` and every equation type above it.

Variable Name	Alias	Definition
yplus		Non-dimensional wall distance
ut		Friction velocity at the wall
frictionforce		Skin friction component of force on a face
frictionmoment		Skin friction component of moment on a face
frictionmomentx		Component of skin friction moment around x axis
frictionmomenty		Component of skin friction moment around y axis
frictionmomentz		Component of skin friction moment around z axis
cf		Skin friction coefficient
eddy		Eddy viscosity ($Pa \cdot s$)
viscosity	mu	Dynamic viscosity ($Pa \cdot s$)
nu		Kinematic viscosity (m^2/s)

LES-tier volume variables

Added for `les` and `rans` (both LES and RANS carry this tier; `les` and `rans` end up with the same available set — see the note at the end of this page).

Variable Name	Alias	Definition
eddy		Eddy viscosity ($Pa \cdot s$)
reynoldstress		Reynolds stress tensor

RANS-tier volume variables

Added for `rans` and `les`.

Variable Name	Alias	Definition
viscouslengthscale		Minimum distance between cell and its neighbours
lesregion		DES/DDES/IDDES region flag
lescfl		Convective CFL on the LES length scale, used by the DES CFL shield
fdshield		DES/DDES/IDDES shielding function f_d: 1 in resolved LES, 0 in the shielded RANS region
ti		Turbulence intensity
turbulencekineticenergy		Turbulence kinetic energy (J/kg)
turbulencekineticenergygrad		Turbulence kinetic energy gradient
turbulenceeddyfrequency		Turbulence eddy frequency ($1/s$)
turbulenceeddyfrequency-grad		Turbulence eddy frequency gradient
nuTilde		Spalart-Allmaras working variable
walldistancezone		Wall-distance zone id
sagradientlimiter		Spalart-Allmaras gradient limiter value
celllimiter		Cell-based reconstruction limiter value for density (1 = unlimited)
celllimiterpressure		Cell-based reconstruction limiter value for pressure (1 = unlimited)
turbulencetimestep		Local turbulence timestep
menterf1		Menter SST blending function

RANS-tier surface variables

Added for `rans` and `les`.

Variable Name	Alias	Definition
roughness		Wall roughness height
walldistancezone		Wall-distance zone id
walldist		Wall distance (m)
lesregion		DES/DDES/IDDES region flag
lescfl		Convective CFL on the LES length scale, used by the DES CFL shield
fdshield		DES/DDES/IDDES shielding function f_d: 1 in resolved LES, 0 in the shielded RANS region
turbulencekineticenergy		Turbulence kinetic energy (J/kg)
turbulenceeddyfrequency		Turbulence eddy frequency ($1/s$)
nuTilde		Spalart-Allmaras working variable

Transition-model variables

Added for `rans` and `incomp rans` when the turbulence model is `sst-transition`, which is the model that solves for them.

Variable Name	Alias	Definition
turbulenceintermittency		Transition model intermittency
transitionreynoldsnumber		Transition momentum-thickness Reynolds number

Per-equation dynamic variables

`var_i`, `resvar_i`, `resshare_i` and `vargrad_i` (solution value, residual, share of the total squared residual, and gradient of solved equation `i`) are generated for `i` from 1 up to the number of solved equations, which depends on the equation type and, for RANS, the turbulence model:

Equation type	Turbulence model	Number of equations
euler / viscous / les / incomp viscous / incomp les	–	5
rans / incomp rans	(default, e.g. sst)	7
rans / incomp rans	sa-neg	6
rans / incomp rans	sst-transition	9

For example, an `incomp rans` case using the default (7-equation) turbulence model has `var_1` through `var_7`, `resvar_1` through `resvar_7`, and `vargrad_1` through `vargrad_7` available (used for the extra turbulence equations, e.g. `var_6`/`var_7` for the two SST transport variables in the *naca0012*-style examples elsewhere in this guide).

`resvar_i` is divided by the cell volume only when *'scale_residuals_by_volume'* is set, so it always matches the reported residual norms.

`resshare_i` is that cell's share of the total squared residual for equation `i`, and the values sum to one over the mesh. Threshold it at `1/Neff_<eq>` - reported when *'residual_statistics'* is on - to isolate the cells carrying the residual, or sort the cumulative sum to answer "which cells hold the top 90%". It is exactly `resvar_i` squared and normalised, so the two always agree on which cells are the hot ones.

FSI / modal variables

When a fluid zone or boundary condition of type `fsi` is present in the configuration, these are additionally available:

Variable Name	Context
modeshapedisplacement_1 .. modeshapedisplacement_14	Surface
modeshaperotation_1 .. modeshaperotation_10	Surface
modaldeformation	Surface
meshvelocity	Surface and volume
cell_centre	Surface

Note

Some names read like variables but are not ones the solver knows, so a deck asking for them is rejected:

- `facecentre` — `centre` is the valid spelling, in both contexts.
- `velocitygradient` — only the shorter spelling `velocitygrad` is valid.
- `kinematicviscosity` — only its alias `nu` is valid; the full name is not.
- `turbulenceintensity` — only its alias `ti` is valid; the full name is not.
- `frictionvelocity` — only its alias `ut` is valid; the full name is not.
- `walldistance` — the name the wall distance is *written* under, but `walldist` is what a deck asks for.

`les` and `rans` resolve to the same valid-variable set (LES-tier variables are unioned in for both), so the tier split above does not gate anything differently between the two.

1.6 zCFD Functions

Several parameters in the zCFD control dictionary will accept functions as arguments. These functions are then evaluated by the solver when required. This allows you to customise & tune to solver as it runs.

1.6.1 Scale Mesh Function

Description	Transform the location of a supplied point.
Valid for parameter	*scale_mesh*
Parameters	[x, y, z]: Point to transform
Returns	{'coord': [x, y, z]}: Transformed coordinates

Example:

```
# Custom function to scale z coordinate by 100
def my_scale_func(point):
    point[2] = point[2] * 100.0
    return {'coord' : point}
```

```
parameters = {
 ...
 'scale': my_scale_func
 ...
}
```

1.6.2 Transform Mesh Function

Description	Transform the location of a supplied point.
Valid for parameter	*mesh transform func*
Parameters	[x, y, z]: Point to transform
Returns	{"v1": x, "v2": y, "v3": z}: Transformed coordinates

Example:

```
rotation = 10.0

def my_transform(x, y, z):
  v = [x, y, z]
  v = zutil.rotate_vector(v, rotation, 0.0)
  return {"v1": v[0], "v2": v[1], "v3": v[2]}
```

1.6.3 Ramp CFL Function

The named shapes in *cfl_ramp* cover most cases. Where the CFL needs to respond to something they cannot express, `cfl_ramp` accepts a Python callable instead of the list.

The function is called once per pseudo-time cycle as `f(cycle, current_cfl)`, and returns the CFL for that cycle. The result is clamped to the `cfl` target.

Description	Provide a dynamic CFL number
Valid for parameter	*cfl_ramp*
Parameters	solve_cycle, current_cfl
Returns	float, or a dictionary of `cfl` / `cfl_turbulence` / `cfl_coarse`

Example:

```
# Hold a low CFL until the shock has settled, then step up
def my_cfl_ramp(solve_cycle, current_cfl):
    if solve_cycle < 500:
        return min(1.0 * 1.005 ** (solve_cycle - 1), 10.0)
    elif solve_cycle < 1000:
        return 15.0
    return 30.0
```

```
parameters = {
 ...
 "solver": {
   ...
   "convergence control": {
     "cfl": {"cfl": 30.0, "cfl ramp": my_cfl_ramp},
     ...
     }
   ...
   }
 ...
}
```

Returning a dictionary drives the turbulence and coarse-level CFL targets alongside the mean-flow one. Any key left out keeps the value from the control dictionary:

```
def my_cfl_ramp(solve_cycle, current_cfl):
    cfl = min(1.0 * 1.005 ** (solve_cycle - 1), 30.0)
    return {"cfl": cfl, "cfl_turbulence": 0.5 * cfl}
```

1.6.4 Transform Function

A transformation function my be supplied to the force report block to transform the force into a different coordinate system. This is particularly useful for reporting lift and drag, which are often rotated from the solver coordinate system.

Description	Transform a supplied point - rotate, scale or translate.
Valid for parameter	*Forces transform*
Parameters	(x, y, z): floats
Returns	(x, y, z): floats

Example:

```
#zutil is a Zenotech python library providing various utilities, available within a␣
 ↪zCFD installation
import zutil

# Angle of attack
alpha = 10.0
# Transform into wind axis
def my_transform(x,y,z):
    v = [x,y,z]
    v =  zutil.rotate_vector(v,alpha,0.0)
    return  {v[0], v[1], v[2]}
```

1.6.5 Initialisation Function

Description	Provide the initial flow field variables on a cell-by-cell basis
Valid for parameter	*initial*
Parameters	kwargs dictionary containing “pressure”, “temperature”, “velocity”, “wall_distance”, “location”
Returns	dictionary containing one or more of “pressure”, “temperature” or “velocity”

```
def my_initialisation(**kwargs):
  # Dimensional primitive variables
  pressure = kwargs['pressure']
  temperature = kwargs['temperature']
  velocity = kwargs['velocity']
  wall_distance = kwargs['wall_distance']
  # Cell centre localtion [X, Y, Z]
  location = kwargs['location']

  if location[0] > 10.0:
    velocity[0] *= 2.0

  # Return a dictionary with user defined quantity.
  return { 'velocity' : velocity }
```

```
parameters = {
 ...
 'initial': {
   ...
   'func': my_initialisation,
   ...
   }
 ...
}
```

1.6.6 Driven initial condition

A 'driving function' provides another way of prescribing a farfield initial condition, which on evaluation returns an initial condition dictionary.

The driving function is re-evaluated at zCFD's reporting frequency, meaning that the condition (e.g. inlet velocity) can be programmed to change as the simulation progresses.

At startup, the function is only provided with the key word arguments RealTimeStep=0, Cycle=0. However, the driving function is subsequently provided with key word arguments containing all the data in the '_report.csv' file, evaluated at the previous timestep. For any driven initial condition, every value in the initial condition dictionary is also appended to the _report.csv file.

This means that the initial condition can be programmed to respond to the flow. An example is shown below, where the driven IC will continually adjust inlet angle of attack until a specified lift coefficient is achieved.

If the driven initial condition has been programmed to respond to changes in the flow, it is important to ensure that the flow has had enough time to propagate from the farfield through to the region of interest and settle before readjusting the driven initial condition. For simulations where the farfield is distant from the geometry, local and implicit time-stepping will likely give the fastest propagation of adjusted farfield conditions through the domain.

Note

Driven initial conditions **cannot** be used as reference conditions, to initialise the flow (e.g 'IC_1') or within a restarted simulation.

Description	Provides a farfield initial condition that can respond to changes in the flow.
Valid for parameter	*initial condition*
Parameters	kwargs dictionary containing "RealTimeStep", "Cycle" and all reporting variables
Returns	dictionary containing valid keys for an *initial condition*

```
def example_ic_func(**kwargs):
  """
  Example driver function to target a lift coefficient by varying
  angle of attack
  """
  alpha_init = 0 # Initial angle of attack
  lift_target= 0.5 # Targeted lift coefficient
  update_period = 1000 # How often alpha should be updated
  flow_settling_period = 1500

  assumed_d_cL_d_alpha = 0.1 # Used to estimate alpha which would give required lift
  relaxation_factor = 1.15 # <1: under-relaxation, >1: over-relaxation
```

(continues on next page)

(continued from previous page)

```
    if 'lift_target_alpha' in kwargs.keys(): # If timestep > 1
        alpha_current = kwargs['lift_target_alpha']
        F_xyz = [kwargs['wall_Fx'], kwargs['wall_Fy'], kwargs['wall_Fz']]
        F_LDS = zutil.rotate_vector(F_xyz, alpha_current, 0.0)
        lift_current = F_LDS[2] # Lift coefficient, having rotated to account for alpha
    else:
        lift_current = 0 # placeholder value

    if kwargs['Cycle'] < flow_settling_period:
        alpha_new = alpha_init
    else:
        if (kwargs['Cycle'] % update_period) == 0:
            d_cL_required = lift_target - lift_current
            d_alpha = relaxation_factor / assumed_d_cL_d_alpha *  d_cL_required
            alpha_new = alpha_current + d_alpha
            print("Alpha updated from {} to {}".format(alpha_current, alpha_new), flush␣
↪= True)
        else:
            alpha_new = alpha_current

    dict_out = {
        "temperature": 300,
        "pressure": 101325.0,
        'v': {
            'mach' : 0.15,
            'vector' : zutil.vector_from_angle(alpha_new, 0.0)
            },
        "reynolds no": 6.0e6,
        "eddy viscosity ratio": 1.0,
        "monitor": { # Monitor entries can be floats or lists of floats
                      "prefix": "lift_target",
                      "alpha": alpha_new,    # Extra keys can be added to a driven IC␣
↪dictionary,
                      "lift": lift_current,  # allowing monitoring of key parameters␣
↪flow in the _report.csv
                    },
    }

    return dict_out
```

```
parameters = {
 ...
 'IC_2' : example_ic_func
 ...
}
```

1.6.7 Static Pressure Ratio

The python function takes in three values (x, y, z) and returns the static pressure ratio.

Description	Returns a static pressure ratio for a given point.
Valid for parameter	*Static Pressure Ratio*
Parameters	(x, y, z): floats
Returns	float

1.6.8 Total Pressure Ratio

The python function takes in three values (x, y, z) and returns the total pressure ratio.

Description	Returns a total pressure ratio for a given point.
Valid for parameter	*Total Pressure Ratio*
Parameters	(x, y, z): floats
Returns	float

1.6.9 Total Temperature Ratio

The python function takes in three values (x, y, z) and returns the temperature pressure ratio.

Description	Returns a total temperature ratio for a given point.
Valid for parameter	*Total Temperature Ratio*
Parameters	(x, y, z): floats
Returns	float

1.6.10 Direction Vector

The python function takes in three values (x, y, z) and returns a tuple (x, y, z).

Description	Returns a direction vector for a given point.
Valid for parameter	*vector*
Parameters	(x, y, z): floats
Returns	(x, y, z): floats

```
def inflow_direction(x, y, z):
  # x, y, and z are the coordinates of the face centre
  cen = [1.0, 0.0, 0.0]
  y1 = y - cen[1]
  z1 = z - cen[2]
  rad = math.sqrt(z1*z1 + y1*y1)
  phi = math.atan2(y1,z1)
  angle = -0.24
  x2 = math.cos(angle)
  y2 = math.sin(phi) * math.sin(angle)
  z2 = math.cos(phi) * math.sin(angle)
  return (x2, y2, z2)
```

```
parameters = {
 ...
 'IC_2' : {
      'reference' : 'IC_1',
      'vector' : inflow_direction
      },
 ...
}
```

1.7 Mesh Conversion

zCFD uses unstructured meshes written in an HDF5 file format. HDF5 provides a flexible, self describing specification supporting parallel I/O that can be tuned per filesystem type (e.g NFS, GPFS, Lustre etc).

Meshes from a wide variety of sources can be easily converted into this format as follows:

1.7.1 OpenFOAM-x.x.x

We provide a range of converters for popular mesh types via a special version of the open-source CFD code OpenFOAM, available freely for download here

Once downloaded and installed, the version of OpenFOAM will convert a range of file formats to the zCFD format. Prior to running any of these, you will need to activate your OpenFOAM environment using

```
source $FOAM_INST_DIR/OpenFOAM-x.x.x/etc/bashrc
```

1.7.2 OpenFOAM Mesh Converter

To convert from an OpenFOAM format mesh to zCFD, run the following command in the MESH directory.

```
foamTozCFD
```

This will create a new folder called *zInterface/* containing the HDF5 file.

1.7.3 zCFD Mesh Converter Utility

The converter for various mesh files is a utility called *convert_mesh*. This is included in the path set by activating the zCFD command line environment.

To run the utility, use

```
(zCFD): convert_mesh <filename> <new_filename>.h5
```

The utility auto-detects the format from the file extension. You can also explicitly specify the format using the `-f` flag:

```
(zCFD): convert_mesh -f <format> <filename> <new_filename>.h5
```

The following mesh formats are supported:

Fluent Mesh Converter

The converter supports ANSYS Fluent files (*.msh, .cas, .msh.gz, .cas.gz*).

```
(zCFD): convert_mesh <filename>.msh <new_filename>.h5
```

This will produce 2 files: the zCFD HDF5 file and the *name_zone.py* python file which holds the fluid and boundary zone names.

Fluent CFF Mesh Converter

The converter also supports Fluent CFF format files (*.msh.h5, .cas.h5*).

```
(zCFD): convert_mesh <filename>.msh.h5 <new_filename>.h5
```

This produces the zCFD HDF5 file.

SU2 Mesh Converter

SU2 is an open source CFD solver (https://su2code.github.io/). Note that conversion of 2D meshes is not supported as they are not handled in zCFD.

```
(zCFD): convert_mesh <filename>.su2 <new_filename>.h5
```

This produces the zCFD HDF5 file.

CGNS Mesh Converter

This utility will convert an unstructured, non-chimera, hexahedral mesh in CGNS (ref) format. The converter will handle meshes with element types (HEXA_8):

```
(zCFD): convert_mesh <filename>.cgns <new_filename>.h5
```

This produces the zCFD HDF5 file. If you require support for additional element types please contact us: mailto:zcfd@zenotech.com.

UGRID Mesh Converter

UGRID is a NASA format for meshes. You must follow the naming convention for UGRID files, which describes the format and *endian-ness* of the file data ("ascii8", "b8", "lb8", or "r8") depending upon which language (C or FORTRAN) was used to create the file. The mapbc file is optional and if provided as an extra argument, boundary conditions will be read from it; otherwise, they default to wall.

```
(zCFD): convert_mesh <filename>.[ascii8, b8, lb8, r8].ugrid <new_filename>.h5
↪<filename>.mapbc
```

This produces the zCFD HDF5 file.

1.8 FWH solver

A Ffowcs-Williams Hawkings (FWH) solver, which is used for prediction of farfield noise, is included in the zCFD distribution. The zCFD FWH solver uses the Farrasat 1A formulation, and can accept both permeable and impermeable FWH surfaces.

1.8.1 What is an FWH solver for?

A common problem in CFD is the prediction of the farfield aerodynamic noise which results from a particular airflow. Two issues make prediction of farfield aerodynamic noise challenging. First of all, many of the sources of aerodynamic noise (turbulence, vortex shedding etc.) require high fidelity, time accurate simulations and fine CFD meshes to be properly captured. Secondly, a mesh must be very fine and the simulation must have very low numerical dissipation in order to propagate aerodynamic noise any distance within a CFD mesh.

The first issue of noise simulation can be made tractable by the use of techniques like FRPM (Fast Random Particle Method), but the second issue of far-field mesh requirements for noise propagation means that e.g. predicting the noise from a helicopter main rotor a kilometre away is simply not feasible using a conventional CFD simulation.

This issue of noise propagation in the far-field is solved by the use of the Ffowcs-Williams Hawkings (FWH) equations. These equations allow us to propagate sound to an observer an arbitrary distance away without the use of a mesh which extends all the way to the observer.

1.8.2 How does an FWH solver work?

In order to use the FWH equations to propagate noise to a far-field observer, we first run a CFD simulation and record the flow variables ($p(\mathbf{x}, t)$, $\rho(\mathbf{x}, t)$ and $\mathbf{u}(\mathbf{x}, t)$) on an 'FWH surface' in the near-field, noise generating region. Fig. 1.4 shows an FWH surface enclosing the nearfield for a simulation of an aerofoil in a wind tunnel.

The FWH surface should enclose the noise generating region, and the CFD mesh should be sufficiently fine in the near-field region to propagate noise to the FWH surface without excessive dissipation. Ideally, the FWH surface should not intersect the nearfield region of any wakes.

In some situations, it is sufficient to collect FWH surface data on the wall instead of using a separate FWH surface inside the fluid domain. FWH surfaces coincident with the wall are called *impermeable* (as opposed to *permeable*) FWH surfaces, and only require p (not p, ρ and $\mathbf{u}$) to be recorded.

Once the flow variables have been recorded on the FWH surface, we can propagate the noise to our observers using an FWH solver. In order to do so we must define the motion of our FWH surface and our observers in the

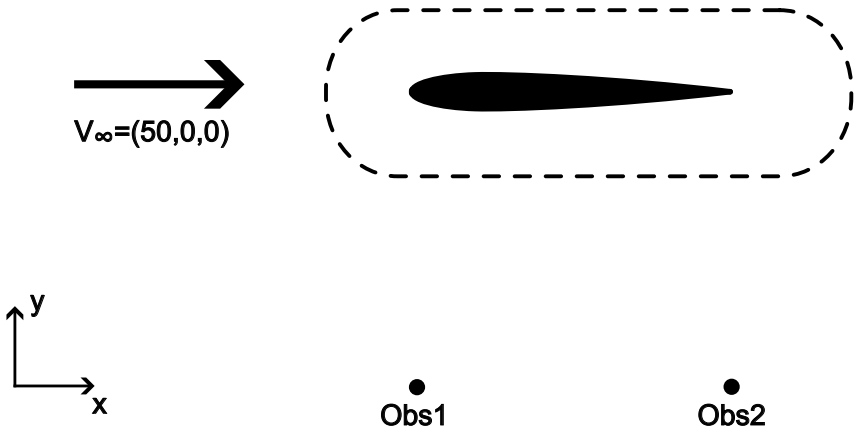


Fig. 1.4: CFD simulation of an aerofoil, including an FWH surface

FWH simulation. The FWH equations assume a quiescent medium (i.e. still air, with a free-stream velocity of zero), which means that the FWH surface and observer motions are frequently different to the motions used in the CFD simulation used to produce the FWH surface data.

In our example of an aerofoil in a wind tunnel, the CFD simulation has a freestream velocity, which does not satisfy the 'quiescent medium' condition. When using an FWH simulation to calculate the noise at the farfield observer microphone in Fig. 1.4, we therefore have to perform a co-ordinate transformation and 'fly' both the FWH surface and the observer microphone forwards through the quiescent medium, as in Fig. 1.5.

Once we have collected the FWH surface data from a CFD simulation and correctly defined the FWH surface and observer microphone settings, we simply use the FWH equations to propagate the pressure and velocity signals from the FWH surface to the our farfield observer locations. The equations used in the zCFD FWH solver are the Farrasat 1A equations, with a retarded time formulation. The retarded time is the time τ_* at which a sound wave must be emitted by a point on the FWH surface in order for it to reach an observer at time t, given the prescribed motions of both surface and observer.

Note

The Farrasat 1A equations used by the zCFD FWH solver are only valid when the FWH surface moves through the quiescent medium subsonically. The retarded time algorithm in the FWH solver will also struggle to converge as the closing speed between an FWH surface and an observer approaches the speed of sound.

1.8.3 Running the zCFD FWH solver

There are two ways of running the zCFD FWH solver - the *fwh python module* and the *run_fwh utility*. The *fwh python module* is flexible and intended for advanced users with knowledge of the python programming language. The *run_fwh utility* is a simpler interface which requires no knowledge of the python programming language and is intended for FWH data generated using *zCFD*.

This documentation actually introduces the *fwh python module* first, since doing so illustrates some mechanics of FWH solvers which all users should be aware of. The simpler *run_fwh utility* method is subsequently introduced as an alternative, more automated method to get the same results. For a quick introduction to the zCFD FWH solver which starts with the *run_fwh utility*, see the *FWH tutorial*.

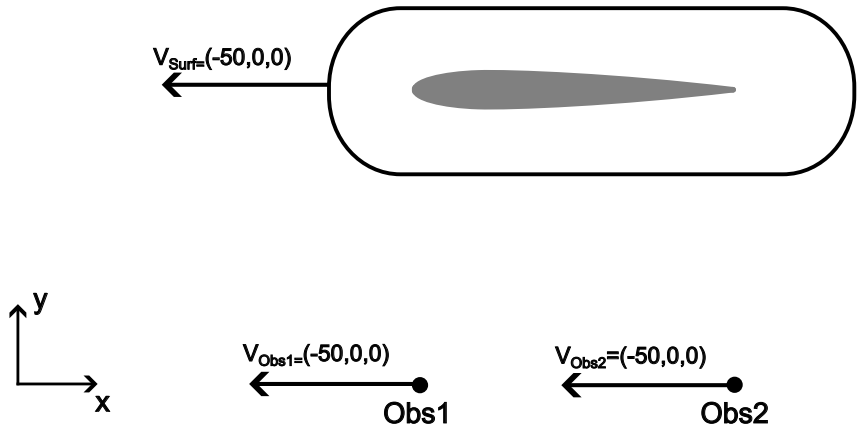


Fig. 1.5: FWH simulation of an aerofoil, with quiescent medium co-ordinate transformation applied

To use the zCFD FWH solver in either format, we must first generate FWH surface data during a zCFD simulation (see *write output* for more details). The FWH surface data is stored in a HDF5 format. While no mesh conversion tools are provided, the zCFD FWH file format can easily be interrogated and replicated with tools like *h5ls* and *h5py* should you wish to convert data from other simulation tools to this format.

Using the zCFD FWH python module

The zCFD FWH solver consists of a series of python modules, which are accessible from within the *zCFD command-line environment*. Three python dictionaries, controlling the surface, observer, and solver settings, are required as inputs to the *fwh.solve* function. This function then returns time histories of the pressures at the specified observer points, in the nested dictionaries *pOut* and *tOut*.

```
from solvers import fwh, motion

mySurfaces = { surfaces}
myObservers = { observers}
mySolverSettings = { solverSettings}

pOut,tOut = fwh.solve(mySurfaces, myObservers, solverSettings)
```

For an aerofoil in a wind tunnel (as in Fig. 1.4 and Fig. 1.5), an appropriate python script (including a small amount of post-processing) might look like this:

```
from solvers import fwh, motion
from matplotlib import pyplot as plt
import json

v = [-50,0,0]
surfaceCentreMotion = motion.ConstantVelocityPoint([0,0,0],v)
surfaceMotion = motion.NonrotatingSurface(surfaceCentreMotion)
obs1Motion = motion.ConstantVelocityPoint([1,0,0],v)
obs2Motion = motion.ConstantVelocityPoint([2,0,0],v)

mySurfaces = {
    "fwhSurf1": {
```

(continues on next page)

(continued from previous page)

```
        "motion": surfaceMotion,
        "fileName": "./zcfdSim_P2_OUTPUT/ACOUSTIC_DATA/fwhsurf1_FWHData.h5"
    }
}
myObservers = {"Obs1": obs1Motion, "Ob2": obs2Motion}
mySolverSettings = { "c": 340, "rho0": 1.2, "p0": 1e5}

pOut,tOut = fwh.solve(mySurfaces,myObservers,solverSettings)

fig,ax = plt.subplots()
ax.plot(tOut["fwhSurf1"]["Obs1"],pOut["fwhSurf1"]["Obs1"],label="Obs1")
ax.plot(tOut["fwhSurf1"]["Obs2"],pOut["fwhSurf1"]["Obs2"],lable="Obs2")
ax.set_xlabel("t (s)")
ax.set_ylabel("p (Pa)")
fig.savefig("./FWH_figure.pdf")

dataForJson = {"p": pOut, "t": tOut}
with open("./FWH_data.json","w") as f:
    json.dump(dataForJson,f)
```

When there is more than one surface, the *pOut* and *tOut* dictionaries will contain a *Surface Summation* entry in addition to the entries for each surface. This contains, for each observer, a sum of the contributions from each surface. This is useful when noise originates from several distinct regions, each covered by separate FWH surfaces (e.g. UAVs with several propellors). The *Surface Summation* entry will likely cover a shorter time period than the individual surface entries, because it only covers the time period when the observer is receiving valid noise signals from all of the FWH surfaces.

Surfaces dictionary

The zCFD FWH solver is capable of using multiple FWH surfaces in the same FWH calculation. The Surfaces dictionary therefore contains a key-value pair for each FWH surface to be used, where the key is the surface's name (to be used in the FWH solver) and the value is an *individual surface definition dictionary*. The surface name must be a string, and cannot be 'Surface Summation'.

Example usage:

```
from solvers import motion
v = [-50,0,0]
surfaceCentreMotion = motion.ConstantVelocityPoint([0,0,0],v)
surfaceMotion = motion.NonrotatingSurface(surfaceCentreMotion)

mySurfaces = {
    "fwhSurf1": {
        "motion": surfaceMotion,
        "fileName": "./zcfdSim_P2_OUTPUT/ACOUSTIC_DATA/fwhsurf1_FWHData.h5"
    }
}
```

Individual surface definition dictionary

Keyword	Required	Default	Valid values	Description
'motion'	Yes		A *surface motion class*	The motion of the surface through the FWH medium
'fileName'	Yes		A readable file path	The FWH surface data, in the FWH HDF5 format outputted by the zCFD solver
'permeable'	No	Automatically deduced from the FWH surface data file	True | False	Whether or not the surface is permeable or not (see *above*).
'flipSurfaceNormals'	No	False	True | False	Whether or not to invert the FWH surface normals before running the FWH solver. The surface normals should always point outwards towards the farfield. For FWH surface data generated using the *fwh interpolate* keyword in the zCFD solver, the surface normal direction depends on the .stl file used to define the FWH surface, and can be checked using a visualisation tool (e.g. ParaView).
'transformSurfaceVelocity'	No	True	True | False	Relevant for permeable surfaces only, and should normally be kept *True*. Setting this to *False* means that the FWH surface fluid velocity used in the FWH solver does not include the velocity resulting from the motion of the surface (as defined my the 'motion' keyword above). This option may be required when using data from other solvers, in the case where the FWH surface moves through a quiescent medium during the simulation used to create the FWH surface data.

Observers dictionary

The Observers dictionary defines a list of observer microphones at which farfield noise should be calculated. In the observers dictionary, keys are used to set the observer's name in the FWH solver output, and the value sets the motion of the observer. Observer motions must be defined using the *point motion classes*.

Example usage:

```
from solvers import motion
v = [-50,0,0]
obs1Motion = motion.ConstantVelocityPoint([1,0,0],v)
obs2Motion = motion.ConstantVelocityPoint([2,0,0],v)

myObservers = {"Obs1": obs1Motion, "Ob2": obs2Motion}
```

Solver settings dictionary

The solver settings dictionary sets the properties of the FWH medium, and the numerical settings used in the FWH solver.

Example usage:

```
mySolverSettings = { "c": 340, "rho0": 1.2, "p0": 1e5}
```

Keyword	Required	Default	Valid values	Description
'c'	Yes		Positive number	speed of sound in the far-field medium, m/s
'rho0'	Yes		Positive number	density of the far-field medium, kg/s
'p0'	Yes		Positive number	static pressure of the far-field medium, Pa
'dt'	No	set equal to the smallest timestep between outputs in all the FWH surface data files specified in the *surfaces dictionary*	Positive number	timestep between observer microphone recording times, s
'breakTolAbs'	No	'dt' set in the solver settings dictionary / 1000	Positive number	the time accuracy (s) with which we would like to predict the retarded time τ_* in the retarded time calculation.
'maxIts'	No	50	Positive integer	The maximum number of iterations to use in the algorithm used to predict retarded time
'observerStartTime'	No		Number	The simulation time (s) at which observers start recording data. If this is not set, this will be set automatically to be the earliest time when a surface/observer pair has a valid signal.
'observerEndTime'	No		Number	The simulation time (s) at which observers end recording data. If this is not set, this will be set automatically to be the last time when a surface/observer pair has a valid signal.

Point (observer) motion classes

FWH Observers are points which move through the FWH quiescent medium according to the function $x = f(t)$. In the zCFD FWH solver, we define the motion of observers using one of the following classes, which are available inside the *solvers.motion* module inside the zCFD command line environment (see *above*).

Motion Class	Inputs	Description
OriginPoint()		A point which is always on the origin, i.e. $f(t) = [0, 0, 0]$.
StationaryPoint(X0)	X0: a 3-d vector $[x0, y0, z0]$	A point which is stationary, i.e. $f(t) = [x0, y0, z0]$.
ConstantVelocityPoint(X0,dXdt)	X0: a 3-d vector $[x0, y0, z0]$ dXdt: a 3-d vector $[dxdt, dydt, dzdt]$	A point with constant velocity, i.e. $f(t) = [x0 + t * dxdt, y0 + t * dydt, z0 + t * dzdt]$

Surface motion classes

Just like FWH Observers, FWH Surfaces require their motion to be defined. There are two surface motion classes - NonrotatingSurface and RotatingSurface. Both use a *point motion class* to define the motion of their 'centre point'. In this way, complex motions like that of a propellor which is both rotating and translating through the FWH medium can be defined.

Motion Class	Inputs	Description
NonrotatingSurface(centrePoint)	centrePoint: a *point motion class*	A surface which does not rotate, and translates at the same velocity as the 'centrePoint'.
RotatingSurface(centrePoint,axis, omega)	centrePoint: a *point motion class* axis: a 3-d vector $[ax_x, ax_y, ax_z]$ omega: a number.	A surface which translates at the same velocity as the *centrePoint*, and also rotates about the centrePoint with rotation axis *axis* and rotation angular velocity (rad / s) *omega*

Using the zCFD run_fwh utility

When using the *fwh python module* defined above, we manually define FWH *surface*, *observer* and *solver settings* dictionaries which are used in *fwh.solve()*. This gives us fine grained control, but for most FWH simulations which are based on zCFD CFD simulations, the input dictionaries to *fwh.solve()* can actually be generated automatically by reading the *zCFD input parameters file*. This is what the *run_fwh* utility does. Within the *zCFD command-line environment*, the *run_fwh* utility can be called as below:

```
run_fwh -c <zcfd control file> --reference_condition <reference condition>␣
↪--observer_locations <observers .csv file>
```

The *run_fwh* utility reads the *zcfd control file* and the *observers .csv file*, and based on these, automatically runs the FWH solver. Assuming the *zcfd control file* input was `zcfdSim.py` and the *observers .csv file* specified the observers *Obs1* and *Obs2*, the *run_fwh* utility would:

1. automatically generate the inputs to the *fwh.solve()* function, setting the observer and surface motions based on the *reference condition* and using all FWH surfaces that the CFD simulation defined by `zcfdSim.py` outputs
2. Run the *fwh.solve()* function, saving the screen output to `zcfdSim.fwh.log`
3. Save the sum of the contributions of all FWH surfaces at the observers *Obs1* and *Obs2* to the *.csv* files `zcfdSim.fwh.Obs1.csv` and `zcfdSim.fwh.Obs2.csv` respectively.

For the example outlined in the *fwh python module example*, the *run_fwh* command would look something like this:

```
run_fwh -c zcfdSim.py --reference_condition IC_1 --observer_locations observers.csv
```

zCFD control file

The first input to the *run_fwh* utility is the *zCFD control file*. This is the control file for the zCFD simulation which generated the FWH surface data files which are to be used. For the example outlined in Fig. 1.4 and the *fwh python module example*, the *zCFD control file* would be called `zcfdSim.py` and would look something like the example below.

```
parameters = {
  ...
  "IC_1": {
      "temperature": 288,
      "pressure": 1e5,
```

(continues on next page)

(continued from previous page)

```
        "V": {"vector": [50,0,0],
        ...
    },
    ...
    "write output": {
        ...
        "fwh interpolate": ["fwhsurf1.stl"],
        ...
    },
    ...
}
```

The *run_fwh* utility reads the zCFD parameter file and finds the paths to all the FWH surface output files which were generated by the zCFD CFD simulation defined by `zcfdSim.py`. In this case, it would find the file `./zcfdSim_P2_OUTPUT/ACOUSTIC_DATA/fwhsurf1_FWHData.h5`.

Reference condition

In the *fwh python module example*, we have to manually define the solver settings and the motions of the observers and surfaces through the FWH quiescent medium manually. However, if all the surfaces and observers are in the same *translational reference frame*, we can set the motions and solver settings directly from a zCFD *IC block*.

Being in the same *translational reference frame* means that all the surfaces and observers translate through the FWH quiescent medium at the same velocity. In practice, this means that all observers are *static* in the 'mesh' frame of reference, i.e. do not 'fly by'. Fig. 1.4 and Fig. 1.5 show that for the example, *Obs1*, *Obs2* and the FWH surface are in the same translational reference frame, hence can be used in the *run_fwh* utility.

In the example case, we would set our reference frame to be *IC_1*. This is the freestream conditions in the *zcfdSim.py* CFD simulation. The pressure and temperature in *IC_1* set the `c`, `p0` and `rho0` in the *solverSettings dictionary*. The velocity in *IC_1* sets the motion of our surfaces and observers through the FWH quiescent medium. Since *IC_1* defines a freestream velocity of *[50,0,0]*, our surface and observers move through the FWH quiescent medium at a velocity of *[-50,0,0]*, just as in Fig. 1.4 and Fig. 1.5.

In most cases, a zCFD simulation control file will already contain an IC block suitable for use as the *run_fwh* reference condition. Since zCFD control files can contain unused *IC* blocks, simply add a suitable *IC* block to the zCFD control file if one does not already exist.

Which *translational reference frame* an FWH surface is in is independent of whether the FWH surface is rotating or not - see *below* for more details.

Observer .csv file

In the Observers .csv file, we define the locations of the observers (in the 'mesh fixed' frame of reference). To match the observers shown in Fig. 1.4 and Fig. 1.5, the *observers .csv file* should contain the following:

```
Obs1, 1.0,  0.0,  0.0
Obs2, 2.0,  0.0,  0.0
```

Advanced use

The *run_fwh* utility allows and accounts for *overset* zCFD simulations and FWH surfaces which exist in zCFD *rotating fluid zones*. This means that even complex simulations like drones with multiple rotating propellors can be simulated with the *run_fwh* utility.

Where the input *zCFD control file* is an overset *'override' file*, FWH surfaces from all control files specified in the override dictionary are used and the reference condition (e.g. *IC_1*) is taken from the first control file specified in the override dictionary. Where FWH surfaces exist in *rotating fluid zones*, the surfaces remain in

the same *translational reference frame* as the observers, but also rotate as specified in the rotating fluid zone specification.

The *run_fwh* utility does not allow translating meshes or translational boundary conditions, since both would introduce ambiguity into what the correct *translational reference frame* for the FWH surfaces and observers should be. As explained *above*, 'fly-by' observers are also not possible in the *run_fwh* utility.

In the *fwh python module*, *fwh.solve()* outputs the contribution from each individual FWH surfaces to each observer pressure history, as well as the sum of all contributions. The *run_fwh* utility only records the sum of contributions from all FWH surfaces in the output *.csv* files. This means that in the *run_fwh* utility, all noise generating noises in the zCFD simulation are implicitly assumed to be enclosed by single FWH surfaces. This means that, for instance, including both *fwh interpolate* and *fwh wall data* FWH surface data output in a zCFD control file is likely to lead to double accounting in the *run_fwh* utility.

Where the *run_fwh* utility is not suitable, it is still possible to use components of the *run_fwh* utility to automate parts of a *fwh python module* workflow. A simplified summary of what happens within the *run_fwh* executable is detailed below, showing the use of the *fwh_helper_functions* which can be used in the *fwh python module* to automate workflows.

```
from solvers import fwh_helper_functions
rho0, p0, c, freestreamV = fwh_helper_functions.get_zcfd_ref_conditions(
    "zcfdSim.py", "IC_1"
)
surfaces = fwh_helper_functions.get_zcfd_surfaces(
    "zcfdSim.py", freestreamV
)
observers = fwh_helper_functions.get_csv_observers(
    "observers.csv", freestreamV
)
solverSettings = {"rho": rho0, "p0": p0, "c": c}
pOut, tOut = fwh.solve(surfaces, observers, solverSettings)
fwh_helper_functions.write_fwh_data_to_zcfd_csv("zcfdSim.py", tOut, pOut)
```

1.8.4 FWH file utilities

As well as the FWH solver itself, two FWH file utilities are provided - *writeFWHSurfaceFrequenciesFile* and *writeBlankedOffFWHSurfaceFile*. Users may find these utilities useful for investigating the sources of noise within their CFD simulation.

writeFWHSurfaceFrequenciesFile

This python function takes an FWH surface file as input, and creates a file where the FWH surface data can be viewed in the frequency, instead of time, domain.

This can be useful for visually indicating where particular frequencies originate from in an FWH observer pressure signal. To generate the FWH surface frequencies file, an FFT (using a Hann window) of the FWH data is calculated at each surface face. The data stored in the surface frequencies file at a particular surface face and chosen frequency is then the magnitude of that FFT's amplitude, at the chosen frequency. The data can be viewed by opening the *.xdmf`* file corresponding to the output file in ParaView - the 'time' chosen in ParaView corresponds to the frequency being viewed. The data at frequency = 0 corresponds to the time average of the FWH surface data.

Example usage:

```
from solvers import fileManipulation
fileManipulation.writeFWHSurfaceFrequenciesFile("./myFWHSurface.h5","./myFWHSurface_
↪Frequencies.h5")
```

writeBlankedOffFWHSurfaceFile

This python function writes a copy of an FWH surface file where specified faces are ignored by the FWH solver.

Sometimes it is useful to 'blank off' certain areas from an FWH surface output file before the FWH solver is run - for example, if CFD setup constraints mean the noise from that region of the FWH surface is non-physical and should therefore be ignored when calculating FWH noise data. This function takes an input FWH surface file, copies it to an output FWH surface file, then sets to zero the face area of any face in the output file where the python function shouldBlankOff(x,y,z) returns 'True' for the face's (x,y,z) co-ordinates. When the face area of a face has been artificially set to 0, that face will not contribute to the observer location pressures calculated by the FWH solver. The region which has been set to have 0 face area can be inspected visually by opening the *.xdmf* corresponding to the output file in ParaView

Example usage:

```
from solvers import fileManipulation

def isPositiveX(x: float, y: float, z: float):
    if x > 0:
        return True
    else:
        return False

fileManipulation.writeBlankedOffFWHSurfaceFile("./myFWHSurface.h5","./myFWHSurface_
↪IgnorePositiveX.h5", isPositiveX)
```

1.9 zM3

zM3 or "zMesh Cubed" is a mesh generator that is included in the zCFD distribution. It is a general purpose meshing tool that can rapidly create cut cell cartesian, or immersed boundary meshes for zCFD's finite volume solver. As a user you define the domain and can then include geometry in the form of STL files to include in the domain. Mesh refinement can be controlled in a number of ways including explicit refinement regions as well as refinement based on the size of the STL triangulation.

1.9.1 Execution

zM3 is included in the standard zCFD installation, to run it simply activate your zCFD environment in the normal way and the *zm3* executable will be added to your path.

```
zm3
```

To see the command line options run:

```
zm3 -help
```

1.9.2 Options

Option	Default	Description	Example
–meshName	mesh	String defining the output mesh name	*–meshName output.h5*
–base	0 0 0	Three real numbers defining the (min_x, min_y, min_z) corner	*–base 0.0 0.0 0.0*
–dims	1.0 1.0 1.0	Three real numbers defining the x, y, z extent of the domain	*–dims 100.0 100.0 100.0*
–globalSpacing	1.0	Real number defining largest cell size in the mesh	*–globalSpacing 5.0*
–stlFiles		List of STL files containing surfaces for meshing	*–stlFiles file1.stl file2.stl*
–stlSpacing		Target cell size for each STL surface file loaded, the the same order as specified in *stlFiles*	*–stlSpacing 1.0 0.5*
–relativeSpacings	0 (for each STL)	Integer for each STL file specified, 1 = True, 0 = False. If True then the spacings defined in *stlSpacing* are treated as a factor multiplying the local STL size to get the spacing value at that location.	*–relativeSpacings 0 1*
–minSurfaceSpacing		Minimum surface spacing when using *relativeSpacings*	*–minSurfaceSpacing 0.5*
–maxSurfaceSpacing		Maximum surface spacing when using *relativeSpacings*	*–maxSurfaceSpacing 0.5*
–generateSurrogateSTL		Integer list specifying which STL files should be have a surrogate STL generated on them. Indexing begins at 0.	*–generateSurrogateSTL 0 2*
–includeProximitySpacings	0 (for each STL)	Integer for each STL file specified, 1 = True, 0 = False. If True reduce cell sizes to account for the proximity of other triangles nearby.	*–includeProximitySpacings 0 1*
–refineAnisotropicSTLTris	0 (for each STL)	Integer for each STL file specified, 1 = True, 0 = False. If True triangles will be sub-divided until all edges approach the smallest edge length for that STL.	*–refineAnisotropicSTLTris 0 1*
–includeCutCells	0	Integer specifying if cells intersected by STL are to remain in mesh, 1 = True, 0 = False	*–includeCutCells 1*
–seedPoints		List of three real numbers defining a point in the mesh to be kept, from which flood filling occurs. Multiple seed points can be defined.	*–seedPoints 50.0 50.0 50.0*
–solidPoints		Three real numbers defining a point in the mesh, within a region which must be deleted. Defines non-fluid regions. Multiple solid points can be defined.	*–solidPoints 10.0 5.0 3.0*
–boxRefinement		Lists of 7 real numbers defining a refinement box in the order x_min y_min z_min x_max y_max z_max spacing.	*–boxRefinement 20.0 20.0 20.0 50.0 50.0 50.0 7.5*
–surfaceBoxRefinement		Similar to *boxRefinement* but only STL elements within the box will be given the target refinement.	*–surfaceBoxRefinement 20.0 20.0 20.0 50.0 50.0 50.0 7.5*
–rotatedTranslatedRefinement		A rotated and translated box refinement region. The box is rotated around 0,0,0 and then trans-	*–rotatedTranslatedRefinement 10.0 10.0 10.0 0.0 0.0 1.0 10.0 10.0 10.0 7.5 75*

1.10 Visualisation

zCFD by default will output data in VTK HDF format (.vtkhdf). This format can be read directly by many post-processing tools including ParaView by Kitware.

> **Note**
>
> The .vtkhdf output format requires ParaView 6.1.1 or later.

Output in other formats is also possible, including TECPLOT and EnSight as follows:

1.10.1 VTK HDF Output

By default, zCFD will output data in VTK HDF format (.vtkhdf). To explicitly force VTK HDF output, use the following option in the *'write output'* part of the control dictionary:

```
'write output' : {
                    # Output format
                    'format' : 'vtk',
```

1.10.2 EnSight Output

To output in EnSight format, use the following option in the *'write output'* part of the control dictionary:

```
'write output' : {
                    # Output format
                    'format' : 'ensight',
```

1.10.3 ParaView Catalyst Scripts

ParaView provides a powerful yet lightweight interface that allows parallel processing of simulation data during programme execution called ParaView Catalyst. This is driven by scripts (Python files) that can be called when data is output:

```
'write output' : {
                    # Output format MUST be VTK
                    'format' : 'vtk',
                    # The scripts should be in the same directory as the input files
                    'scripts' : ['paraview_catalyst1.py','paraview_catalyst2.py']
```

1.10.4 ParaView Client

To download the free ParaView software for visualising VTK HDF format data on the local device, visit the ParaView download page. ParaView 6.1.1 or later is required for .vtkhdf file support.

1.10.5 ParaView Client-Server

In addition to running entirely locally on a device, ParaView can interface to a remote server. This allows users to run ParaView to access data that is held (and processed) on a different computer, potentially running larger simulation tasks that would be possible on the local machine. This also means that there is no need to transfer the large output files back to a local machine for post-processing. The remote machine can easily be set up to run a ParaView Server in parallel - meaning that very large post-processing jobs can be run quickly.

ParaView Server

To download and install ParaView Server, you can either use the version on the main ParaView website and follow the official documentation.

Alternatively, use the version bundled with zCFD. The 'pvserver' included in the zCFD distribution includes plugins for reading the zCFD HDF5 mesh format directly.

To start a ParaView Server on your computer within the zCFD environment run:

```
> pvserver
```

ParaViewConnect

ParaViewConnect is a helper utility for connecting to remote ParaView servers. It reduces the complexity of setting up the required ssh connections and works well with zCFD. More information and installation instructions can be found on the Github page.

1.10.6 ParaView Post-Processing Best Practices for Parallel zCFD Data

When post-processing solution data generated in parallel by zCFD using ParaView, it is important to understand the differences between various ParaView filters and their impact on data interpolation. This section provides guidance on recommended pipelines for accurate results.

CleantoGrid vs MergeBlocks

Two commonly used ParaView filters for processing parallel datasets are **CleantoGrid** and **MergeBlocks**. Each has distinct characteristics:

CleantoGrid Filter:

- **Purpose**: Merges points that are exactly coincident and removes degenerate cells
- **Interpolation**: Performs minimal interpolation, preserving original data values at cell centres and vertices
- **Use case**: Recommended when data accuracy is critical and you want to preserve the original computed values
- **Characteristics**: May result in slightly fragmented visualisations at block boundaries but maintains data fidelity

MergeBlocks Filter:

- **Purpose**: Combines multiple blocks into a single unstructured grid
- **Interpolation**: May perform interpolation across block boundaries to create smooth transitions
- **Use case**: Better for smooth visualisations and streamline generation across block boundaries
- **Characteristics**: Creates visually smoother results but may alter original computed values through interpolation

Recommended Pipelines

For **quantitative analysis** and **data extraction** (probes, line plots, force calculations):

```
# Recommended pipeline for quantitative analysis
import paraview.simple as pvs

# Load data
reader = pvs.OpenDataFile('solution.vtkhdf')

# Use CleantoGrid to preserve data accuracy
clean_data = pvs.CleantoGrid(Input=reader)
```

(continues on next page)

(continued from previous page)

```
# Convert to point data if needed
point_data = pvs.CellDatatoPointData(Input=clean_data)

# Proceed with analysis (probes, calculators, etc.)
```

For **visualisation purposes** (streamlines, contours, volume rendering):

```
# Pipeline for smooth visualisations
import paraview.simple as pvs

# Load data
reader = pvs.OpenDataFile('solution.vtkhdf')

# Use MergeBlocks for smoother visualisations
merged_data = pvs.MergeBlocks(Input=reader)

# Convert to point data for smooth contouring
point_data = pvs.CellDatatoPointData(Input=merged_data)

# Apply visualisation filters (streamlines, contours, etc.)
```

Important Considerations

1. **Data Accuracy**: Always use CleantoGrid when extracting quantitative data for analysis or validation
2. **Visualisation Quality**: MergeBlocks may provide better visual results for presentations and general visualisation
3. **Performance**: CleantoGrid is typically faster as it performs less processing
4. **Block Boundaries**: Be aware that different filters may show artefacts differently at parallel decomposition boundaries
5. **Regression Testing**: When comparing results across different runs, ensure consistent filter usage

Example Notebook Usage

The following pattern is recommended in Jupyter notebooks when processing zCFD parallel data:

```
import paraview.simple as pvs

# For data extraction and analysis
def load_for_analysis(filename):
    reader = pvs.OpenDataFile(filename)
    reader.UpdatePipeline()
    clean = pvs.CleantoGrid(Input=reader)
    return clean

# For visualisation and rendering
def load_for_visualisation(filename):
    reader = pvs.OpenDataFile(filename)
    reader.UpdatePipeline()
    merged = pvs.MergeBlocks(Input=reader)
    point_data = pvs.CellDatatoPointData(Input=merged)
    return point_data
```

This ensures that your analysis maintains data integrity while your visualisations remain smooth and professional.

1.10.7 Jupyter Notebooks

We also recommend the use of Jupyter Lab for post-processing zCFD runs. This is one of the easiest ways to use Python, via an interactive notebook on your local computer. The notebook is a locally hosted server that you can access via a web-browser. Jupyter is included in the zCFD distribution and notebooks for visualising the residual data will automatically be created when you run a zCFD case. In addition there are several example notebooks available in the zPost.

To automatically start up notebook server on your computer within the zCFD environment run:

```
> start_lab
```

1.11 Utilities

zCFD is bundled with a number of utility functions to streamline specific workflows:

1.11.1 Actuator disc modelling

Overview

When simulating the effects of an aerodynamic force device (such as a wind turbine or propellor) on flow field, it is often impractical to resolve the flow down to the scale of the blades. Instead the effects of the device on the flow can be represented using an actuator disc model, which is then superimposed into the flow as a set of additional source terms.

Actuator disc definitions

Conceptually turbines and propellors behave nearly identically in terms of the actuator disc model, with the only differences being in in the coordinate systems and sign conventions typically used to define them. In zCFD an actuator disc zone is defined as a *fluid zone*, using the following keys:

```
turb_zone = {
    "FZ_1": {
        "type": "disc",
        "controller": {},
        "geometry": {},
        "discretisation": {},
        "model": {},
        "name": "turbine_name",
        "reference plane": True,
        "def": "./turbine_vtp/turbine_def.vtp",
        "reference point": [x, y, z],
        "update frequency": 1,
    }
}
```

The *controller* dictionary defines how the forces generated from the disc should be controlled. Options here are *fixed* and *tsr curve*. For a fixed controller the user must specify an angular velocity and blade pitch angle, and for a tsr curve the user must specify a tip speed ratio curve, and optionally a blade pitch curve in the case of a *BET* model.

```
"controller": {
    "type": "tsr curve",
    "tip speed ratio": tsr,
    "tip speed ratio curve": [
        [u0, tsr0],
        ...
        [un, tsrn]
```

(continues on next page)

(continued from previous page)

```
    ],
},

"controller": {
    "type": "fixed",
    "omega": omega,
    "pitch": pitch
},
```

The *geometry* dictionary contains the physical description of the disc geometry, and has the following keys:

```
"geometry": {
    "centre": [x, y, z],
    "inner radius": inner_radius,
    "normal": [x, y, z],
    "outer radius": outer_radius,
    "up": [x, y, z],
},
```

Here the *normal* direction will determine the direction of force from the disc. The convention in zCFD is that the normal should point on the direction of the force on the fluid. i.e. for a propeller the normal should point in the direction of the flow, and on a turbine it should point towards the incoming flow.

The *discretisation* dictionary contains the information needed to create the actuator disc representation of the turbine/ propellor:

```
"discretisation": {"type": "disc", "number of elements": 48},
```

The *model* dictionary contains the information needed to drive the force calculation in the actuator disc model. Here the user can specify either a *simple* turbine model, which averages the performance of the turbine over the entire disc, or *BET* to use a blade element model:

```
"model": {
"type": "simple",
"kind": "turbine",
"power model": "curve",
"thrust coefficient": thrust_coefficient,
"thrust coefficient curve": [
        [u0, tc0],
        ...
        [un, tcn]
    ],
"power": turbine_power,
"turbine power curve": [
        [u0, p0],
        ...
        [un, pn]
    ],
},

"model": {
    "type": "BET",
    "kind": "propellor",
    "blade chord": [
        [r0/R, c0],
        ...
        [1.0, cn],
```

(continues on next page)

(continued from previous page)

```
    ],
    "blade twist": [
        [r0/R, t0],
        ...
        [1.0, tn],
    ],
    "number of sections": 24,
    "number of blades": 2,
    "aerofoil positions": [[0.0, "aerofoil1"], [1.0, "aerofoil1"]],
    "aerofoils": {
        "aerofoil1": {
            "cd": [
                [a0, cd0],
                ...
                [an, cdn],
            ],
            "cl": [
                [a0, cl0],
                ...
                [an, cln],
            ],
        },
        ...
    },
    "tip loss correction radius": 0.8,
    "tip loss correction": "rstar",
},
```

1.12 Troubleshooting

CFD solvers can sometimes fail to converge or even crash when running. zCFD includes an automatic validation to ensure that the inputs are consistent and syntactically correct, and there are some common problems that can be identified even if the code has crashed.

1.12.1 Crashing

There are several common causes of code crashing:

1. Numerical instability
2. Inconsistent boundary conditions
3. Preconditioning
4. Poor mesh quality

Numerical instability

The solver will try to advance the flow solution towards convergence at the required level of spatial and temporal accuracy, subject to a user-defined limit called the *CFL* (Courant–Friedrichs–Lewy) number. The CFL number limits the update to the solution in each cycle to maintain the stability of the underlying numerical scheme.

There are formal criteria for the CFL number for given schemes, however these typically only apply to idealised (orthogonal or structured meshes with very gradual variation in cell-to-cell volume ratios). For complex meshes or where cell quality may be poor (even in only a few cells), the limit for stability may be significantly less. In explicit mode, the theoretical CFL limit is formally approximately 1.0 for most schemes - though in many cases a higher value (1.5 to 2.0) will actually work. In implicit mode the CFL limit is theoretically 1000 or more, with values between 10 and 100 quite common.

The solver will provide some feedback as it is running that can be helpful in diagnosing stability issues.

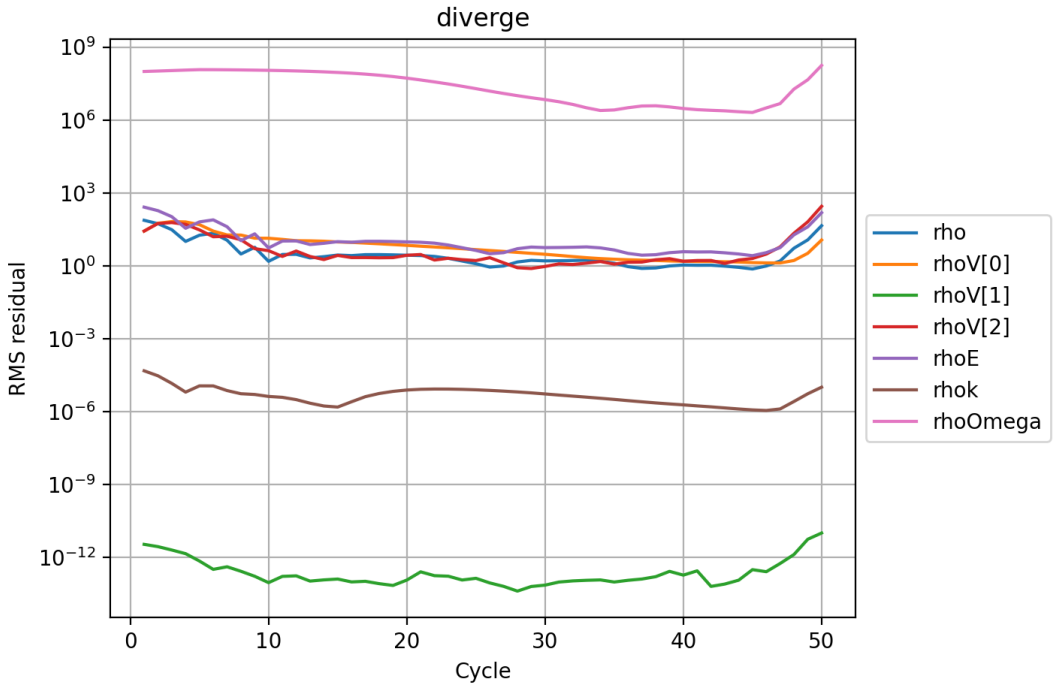


Fig. 1.6: The typical form of a divergent set of residuals. Rather than decreasing, the values all start increasing significantly from cycle 45 onwards. In this case the NACA0012 case was deliberately set with an unstable explicit CFL number of 6.0.

The residuals that are output in the _report.csv file should gradually reduce as the solver converges (per real time step if the flow is unsteady). If one or more of the reported values starts diverging, there is a good chance that the scheme has become unsteady. Note that this can also be a consequence of a particular pattern in the flow that appears during the solution process and so this divergence may occur after a period of otherwise stable convergence. For *implicit* runs using the AMGX library, reporting includes the number of cycles of the implicit linear solver and the residual value associated with each. Typically 4 to 5 cycles should be enough. If the number of iterations to achieve the desired improvement in accuracy (typically 3 to 4 orders of magnitude) starts increasing above 6, then the underlying linear system may be poorly conditioned, which will be due to numerical instability in the solution.

Listing 1.1: Example output from the AMGX implicit linear solver. In this case the turbulence equations are solved after the primary flow variables. The primary flow variables are solved in 5 cycles of the AMGX solver, achieving an order of magnitude 3 reduction in the residuals.

```
Cycle 5003 (real time cycle: 0 time: 0)
CFL 20.0 (20.0) - MG 20.0 (coarse mesh: 0)
        iter      Mem Usage (GB)       residual           rate
        ------------------------------------------------------------
            Ini             36.9899   3.037596e-06
            0             36.9899   4.565379e-07         0.1503
            1             36.9899   1.138337e-07         0.2493
            2             36.9899   3.056741e-08         0.2685
            3             36.9899   1.061114e-08         0.3471
            4             36.9899   3.308933e-09         0.3118
            5             36.9899   1.293730e-09         0.3910
```

(continues on next page)

(continued from previous page)

```
        ------------------------------------------------------------
        Total Iterations: 6
        Avg Convergence Rate:       0.2743
        Final Residual:             1.293730e-09
        Total Reduction in Residual:        4.259057e-04
        Maximum Memory Usage:       36.990 GB
        ------------------------------------------------------------
Total Time: 0.869665
    setup: 0.109454 s
    solve: 0.760211 s
    solve(per iteration): 0.126702 s
        iter      Mem Usage (GB)       residual           rate
        ------------------------------------------------------------
            Ini             36.9899   1.713350e-04
            0              36.9899   1.060694e-07    0.0006
            1              36.9899   1.898567e-10    0.0018
        ------------------------------------------------------------
        Total Iterations: 2
        Avg Convergence Rate:       0.0011
        Final Residual:             1.898567e-10
        Total Reduction in Residual:        1.108102e-06
        Maximum Memory Usage:       36.990 GB
        ------------------------------------------------------------
```

If the solver is crashing due to numerical instability, it may be worth reducing the *CFL* number to less than 1.0 (say 0.5) for explicit mode or 5.0 (or less) for implicit mode. The spatial *'order'* can also be reduced to 'first' and the *'inviscid flux scheme'* to 'Rusanov' (the most stable, though least accurate discretisation scheme).

If the solver still crashes, then the problem is likely to be due to an issue with the boundary conditions, preconditioning or the mesh quality.

Wall functions

zCFD includes a useful feature to automatically scale a turbulent wall function on a solid boundary. This is implemented via an internal iterative solution of a set of equations to match the flow speed and the wall shear stress in the cell adjacent to the wall, according to the ideal profile of a turbulent boundary layer. The number of internal iterations should be quite low (less than 20) if the solution is physically valid. zCFD reports on the time per cycle, and if this value starts climbing it may indicate that the wall function iterations are not converging. To test whether the wall functions are causing problems, switch the type of the *boundary condition* from 'wallfunction' to either 'slip' or 'noslip' depending upon which is a closer approximation to the desired behaviour.

Inconsistent boundary conditions

zCFD will automatically check whether boundary conditions BC_* in the input file are correctly specified, but it cannot determine whether a combination of boundary conditions is correct. For example a closed fluid domain (all boundary conditions set to solid wall) except for a single mass flow condition will initially start converging until the pressure either rises or falls to non-physical levels. The effects of mismatched boundary conditions can also be more subtle - especially with inflow and outflow conditions resulting in over-specification or under-specification of the solution.

A good way to diagnose inconsistent boundary conditions is to remove complexity and simplify the boundary conditions (e.g. replace wall functions with slip conditions, inflow and outflow conditions with Riemann farfield conditions). It can also be helpful to repeat the simulation, deliberately stopping the solver just before the crash in order to inspect the solution for unexpected features. Inspection of the 'temperature' values in a solution can often point to areas of the flow that are causing problems. This is particularly the case when poor quality cells are present.

Preconditioning

The timestep for zCFD running in explicit mode is limited by a local numerical wave-speed in the solver scheme. In regions of low speed flow (such as near stagnation points) this may be overly conservative. Preconditioning is used to modify the scheme to increase the local timestep subject to a local stability condition. However, the stability limits of the preconditioned scheme may be inappropriate for a given mesh, leading to flow field divergence. The easiest way to test this is to set *'precondition'* to False in the solver scheme settings.

Parallel log files

Note that for parallel runs, the main log file may not contain all of the information relating to a crash where the solution on a particular partition has caused the problem. The log files for each partition are stored in the _OUTPUT/LOGGING directory as .<partition>.log.

1.12.2 Convergence stall

More common than crashing is convergence stall - where the reported residuals initially reduce and then plateau.

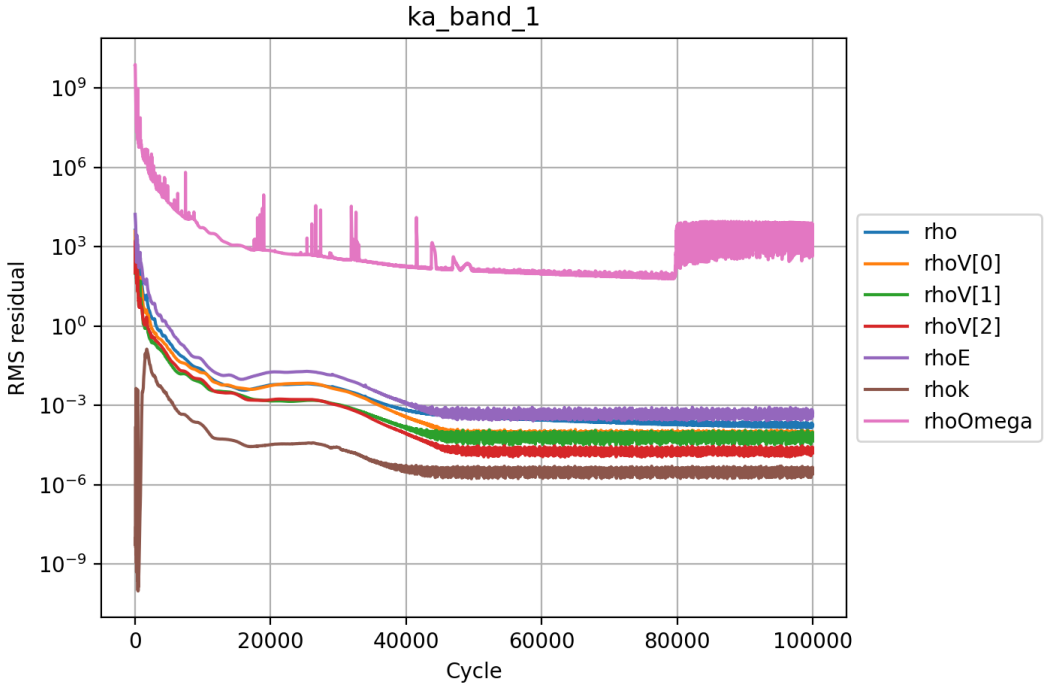


Fig. 1.7: The convergence of the steady-state explicit solver has achieved an average residual reduction of 5 orders of magnitude and then stalled. The noise in the solution from 40k cycles onwards suggests that either the flow is not fundamentally steady-state or that multigrid is adding an error to the solution. The user must determine whether the level of convergence is sufficient for their purposes and if not whether to run an unsteady simulation or degrade the spatial accuracy.

There are several common causes of convergence stall:

1. Multigrid
2. Flow unsteadiness
3. Limiter switching (within the inner cycles of an unsteady simulation)

Multigrid

For explicit simulations, the use of *multigrid* will accelerate the convergence of the solver by using a set of coarse meshes to propagate longer wavelength solution components faster through the domain than would be possible on the finest mesh, subject to the CFL condition. In theory, and for idealised meshes, the use of multigrid does not change the steady-state solution (or pseudo-steady-state solution for unsteady flows) that is reached. However for real meshes around complex geometry, multigrid does introduce an error and this can effectively limit convergence. The use of the keyword 'multigrid cycles' in the 'time marching' section of the control dictionary will turn off multigrid after a specified number of cycles to remove this error.

Flow unsteadiness

A very common cause of convergence stall - particularly for steady-state simulations is due to physical unsteadiness in the underlying flow. In such cases, the solver is attempting to find a steady-state solution to an unsteady problem and the result is that the solution keeps changing in each iterative cycle. There are two approaches to dealing with this - depending upon the flow features of interest.

If the unsteadiness is important then the unconverged steady-state flow can be used as the starting point for an unsteady simulation. If the convergence stall is within the pseudo-steady cycles of an unsteady dual time-stepping solution, then it may be that the physical *time step* specified is too large and should be reduced.

On the other hand, if the unsteadiness is not of interest then the accuracy of the solution can be deliberately degraded to provide a more 'average' steady-state flow. This might include using the Rusanov inviscid flux scheme, switching the spatial accuracy to first order and / or using a coarser mesh. Note that the implicit mode of the solver can also be used to filter the solution in time as larger time-steps can be achieved with a higher CFL number. The user should note that this approach can cause solver instability and inaccuracy in the final solution, as the 'averaging' process is non-physical.

It may also be the case that the flow residuals are stalled, but the integrated properties of interest (e.g. lift, drag, mass flow) are sufficiently converged as to be useful. This decision is left to the user.

Limiter switching

Within the inner (pseudo-time) cycles of an unsteady dual time-stepping simulation on the compressible solver, a common cause of apparent stall is the MUSCL limiter switching state from one iteration to the next in a small number of cells. Those cells keep producing a solution change that never decays, and because the reported residual is an RMS over all cells, a handful of them is enough to hold the whole norm up at a floor. The bulk of the field may be converging perfectly well behind it.

The symptom is a residual that drops cleanly for the first several inner cycles and then flattens at the same level in every real time step, while the integrated forces continue to settle. Judging this from the global residual is unreliable - use the *inner cycle convergence* diagnostics instead, which measure the drop within each real time step rather than an absolute level.

Adding inner cycles does not help, because the floor is not a convergence rate problem. The remedy is to stop the limiter changing: set `freeze_limiter_cycle` in the `solver.numerical_scheme` dictionary, which snapshots the limiter and holds it fixed thereafter. If the solution genuinely needs the limiter to keep adapting, reduce the physical *time step* instead so that less has to change within each real time step.

The incompressible solver has no slope limiter and ignores `freeze_limiter_cycle`. An inner residual that flattens there is the outer iteration's own floor: under the coupled scheme tighten the linear solver or reduce the physical time step; under SIMPLE the floor is set by the under-relaxation (`implicit relax` and `pressure relax` in the `scheme` block) and is reached within a handful of cycles, so a `target_orders` above it can never be met - set the target from a run with the criterion off rather than adding cycles.